OMNIbus Lab Guide

A comprehensive hands-on training guide for IBM Tivoli Netcool/OMNIbus event management platform. You will progress from foundational concepts through practical exercises with the ObjectServer database, probe configuration, and SNMP trap processing.

Guide Contents

1. OMNIbus Fundamentals

Overview: Introduces core OMNIbus concepts, architecture, and terminology needed for the rest of the labs.

- **1.1 OMNIbus overview** — Event management platform fundamentals, core components, and the event pipeline (sources → probes → ObjectServer → operators)
- **1.2 Core architecture and components** — Detailed look at event sources, probes, ObjectServer, gateways, client tools, and Web GUI
- **1.3 Event lifecycle in OMNIbus** — How events flow through the system, deduplication, correlation, and operator interaction
- **1.4 Key processes and terminology** — Essential vocabulary and concepts (events, alerts, probes, gateways, rules, triggers, journals, etc.)
- **1.5 Lab environment and assumptions** — Lab setup details, virtual machine components, Windows workstation requirements, and tool access

---

2. ObjectServer and Event Model

Overview: Hands-on exercises to set up the lab environment and learn basic event interaction patterns.

Exercises:

- **2.1 Environment access and tool validation** — Connect to lab server via SSH, start ObjectServer and Web GUI, configure Windows client tools (Servers Editor), validate desktop event list and Netcool/OMNIbus Administrator connectivity, confirm Web GUI access from browser
- **2.2 Basic event interaction** — Log in to Web GUI, understand OMNIbus severity levels (0-5: Clear through Critical), prioritize events, acknowledge alerts, view event details and journals, create custom journal entries, use filters and views, introduce simulated event traffic with Simnet probe, observe deduplication behavior (FirstOccurrence, LastOccurrence, Count fields), sort and filter events by severity

---

3. The OMNIbus ObjectServer Database

Overview: Database structure, user management, security, and data model fundamentals.

Exercises:

- **3.1 The ObjectServer users table** — Create new ObjectServer users, assign groups and permissions, understand ObjectServer groups (System, Administrator, Normal, Probe, Gateway, ISQLWrite, ISQL, Netcool_OMNIbus_Admin, Netcool_OMNIbus_User), assign Web GUI application roles, distinguish between ObjectServer groups and Web GUI roles, verify user login and access
- **Additional sections** — Coverage of ObjectServer database tables, alerts.status structure, field types, and how data flows through the system

---

4. Probes

Overview: Probe architecture, rules processing, and hands-on probe development.

Setup & Exercises:

- **4.0 Lab setup — Event filter and view configuration** — Create dedicated event filters for probe testing, build custom event list views focused on probe fields, configure SyslogdProbe filter to isolate events by Manager field, prepare the Web GUI for probe development exercises

- **4.1+ Probe exercises** — (Additional sections cover probe-specific topics including:)
  - Probe fundamentals and the event collection pipeline
  - Rules file structure and token processing
  - Event normalization and field mapping
  - Building custom probe rules
  - Testing and validating probe behavior in the event list

---

5. SNMP Trap Processing with MTTrapd and NcKL

Overview: SNMP trap processing using the MTTrapd probe and Netcool Knowledge Library (NcKL).

Setup & Content:

- **5.0 Lab setup** — Create SNMP Traps filter to isolate trap events from other alert sources, prepare views for trap processing
- **5.1 SNMP and MTTrapd overview** — SNMP fundamentals (SNMPv1/v2c/v3), trap concepts, OIDs and MIBs, variable bindings (varbinds), MTTrapd probe architecture and responsibilities
- **Additional sections** — (Further coverage includes:)
  - MTTrapd probe configuration and startup
  - NcKL architecture and vendor-specific rules
  - Custom MIB development and loading
  - Trap rules processing and normalization
  - End-to-end SNMP event pipeline exercises

---

6. Managing OMNIbus Processes with Process Agent and nco_pad

Overview: Configure Process Agent to manage the ObjectServer, Syslogd probe, and MTTrapd probe, then validate process-level and service-level control with `nco_pad` and the `nco_pa_*` command-line tools.

Setup & Content:

- **6.0 Process Agent fundamentals** — Understand processes, services, startup behavior, monitoring, recovery, routing, and PA-aware versus non-PA-aware processes
- **6.1 Process Agent configuration** — Add managed probe process definitions and create the automatically started `Probes` service
- **6.2+ Process and service control** — Restart Process Agent, validate automatic startup, stop and start an individual probe, control the complete `Probes` service, and review the routing configuration

---

7. Cloning an ObjectServer with nco_osreport and nco_dbinit

Overview: Export the existing EVENTS ObjectServer, review the generated SQL, initialize a clean EVENTS_BACKUP clone, register its communications endpoint, and start it as a separately managed backup ObjectServer.

Setup & Content:

- **7.0 ObjectServer cloning fundamentals** — Understand the export-review-import workflow and the roles of `nco_osreport` and `nco_dbinit`
- **7.1–7.2 Export and review** — Generate the SQL export and inspect schema, automation, security, desktop, and optional alert-data files
- **7.3–7.6 Initialize and operate the clone** — Create EVENTS_BACKUP without historical alerts, configure `omni.dat`, regenerate interfaces, add the clone to Process Agent, and validate its schema and runtime state
- **7.7 Optional alert-data import** — Understand when `-alertsdata` and `-alertsdatafile` are appropriate

---

8. Configuring a Bidirectional ObjectServer Gateway

Overview: Configure a bidirectional ObjectServer Gateway between EVENTS and EVENTS_BACKUP, verify that required custom alert fields are mapped, place the gateway under Process Agent control, and validate alert replication in both directions.

Setup & Content:

- **8.1 Verify schema compatibility** — Compare `alerts.status` schemas on both ObjectServers and confirm custom fields are present and compatible before configuring replication
- **8.2 Create a gateway working area** — Set up a dedicated configuration directory by copying the `objserv_bi` template files
- **8.3 Configure the gateway properties** — Set source and destination ObjectServer connections, credentials, map file, and startup command file
- **8.4 Map standard and custom alert fields** — Review the `CREATE MAPPING StatusMap` definition and add explicit mappings for the `CustomerID` and `CircuitID` custom fields
- **8.5 Start the gateway manually and validate operation** — Register the gateway in `omni.dat`, start it manually, and verify that a test `LinkDown` event replicates from EVENTS to EVENTS_BACKUP
- **8.6 Add the gateway to Process Agent** — Define the gateway as a managed `PaNOT_PA_AWARE` process inside the `BackupObjectServerService`, with a dependency on `BackupObjectServer`
- **8.7 Validate bidirectional replication** — Insert controlled test events from each ObjectServer using `nco_sql` and confirm they appear and update on the other side
- **8.8 Clean up test events** — Delete the test alerts and confirm that deletions also replicate bidirectionally

---

Completion Summary

A section-by-section recap of everything you built and learned, a reference table of all key components and files created, and a bridge to the Netcool/Impact training that follows.

Learning Path

You progress through the labs in sequence:

1. **Section 1** establishes conceptual foundations
2. **Section 2** validates the lab environment and introduces event management in practice
3. **Section 3** explores the ObjectServer database and user management
4. **Section 4** builds hands-on probe development and rules expertise
5. **Section 5** applies those skills to SNMP trap processing, a critical real-world use case
6. **Section 6** manages the ObjectServer and probes with Process Agent and validates individual and service-level control
7. **Section 7** clones an ObjectServer into a separately addressable backup or training environment
8. **Section 8** connects the two ObjectServers with a bidirectional gateway and validates alert replication in both directions

By the end of these labs, you will understand the full OMNIbus event pipeline from source through probe to ObjectServer, be able to interact with events in the Web GUI and desktop tools, manage users and security, develop custom probe rules, process SNMP traps from network devices using NcKL and custom MIBs, manage OMNIbus processes with Process Agent, clone ObjectServers for backup or testing, and configure a bidirectional ObjectServer Gateway to synchronize alerts between ObjectServer instances.

1. OMNIbus fundamentals

1.1 OMNIbus overview

IBM Tivoli Netcool/OMNIbus is an event management platform that collects event data from many different sources and consolidates it into a centralized alerting system. It is commonly used to reduce noise, normalize incoming event data, and give operations teams a single place to monitor and manage alerts.

The core of OMNIbus is the ObjectServer, an in-memory alert database that stores active event records and supports automation and operator interaction. Probes acquire events from source systems, apply rules to transform them into OMNIbus fields, and then forward them to the ObjectServer as alerts.

Gateways extend the platform by moving alerts between ObjectServers or integrating alert data with external systems such as databases, ticketing tools, and other management platforms. Operators and administrators then interact with alert data through desktop tools or the Web GUI, using filters, views, and updates to manage the event stream.

In practical terms, OMNIbus should be viewed as a pipeline: event source, probe, ObjectServer, automation, and operator view. The rest of this lab guide builds on that model by first explaining how alerts are stored and processed, and then showing how probes and gateways feed and move those alerts through the environment.

1.2 Core architecture and components

The OMNIbus architecture is easiest to understand as a flow of event intake, event processing, alert storage, integration, and operator access. At a high level, event sources send data to probes, probes transform that data into OMNIbus alerts, the ObjectServer stores and manages those alerts, gateways move alert data between ObjectServers or external systems, and user-facing clients connect directly to the ObjectServer to view and manage events.

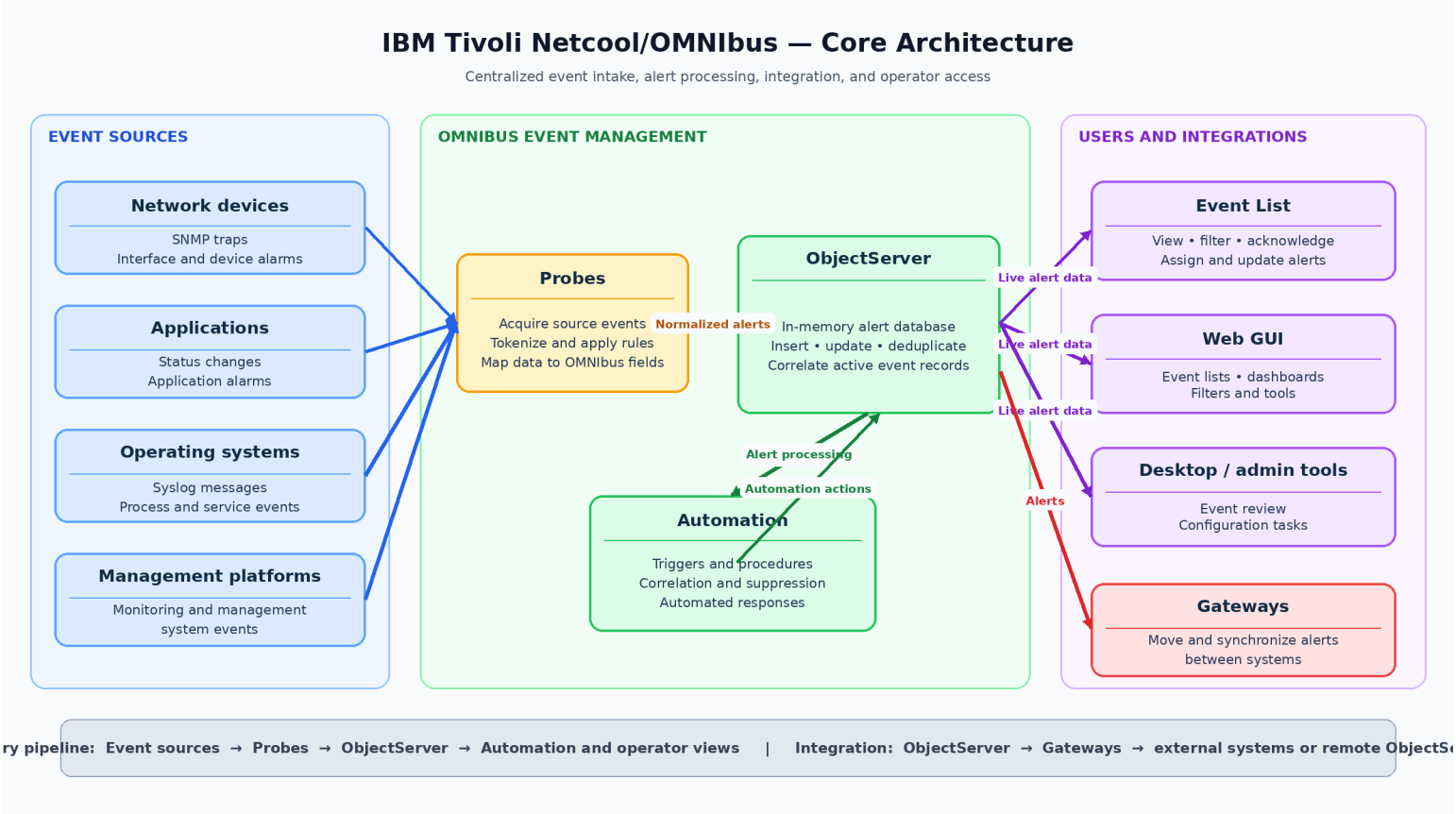


Figure 1: OMNIbus architecture diagram

Primary pipeline: Event sources → Probes → ObjectServer → Automation and operator views.

Integration path: ObjectServer → Gateways → Remote ObjectServers and external systems.

Key architecture components:

- **Event sources**: Devices, applications, management platforms, and operating systems that generate raw event data such as syslog messages, SNMP traps, process alarms, or application status changes.
- **Probes**: Collection and transformation components that receive source data, tokenize it, apply rules logic, and populate OMNIbus event fields before forwarding alerts to the ObjectServer.
- **ObjectServer**: The central in-memory alert database where events are inserted, updated, deduplicated, correlated, and made available for automation and operator action.
- **Gateways**: Components used to move alert data between ObjectServers or between OMNIbus and external systems such as ticketing tools, databases, or other management platforms.
- **Event List**: A client interface used by operators to connect directly to the ObjectServer and view, filter, acknowledge, assign, and update alerts.
- **Web GUI**: A browser-based interface that presents event lists, filters, tools, and dashboards while obtaining event data from one or more ObjectServers.
- **Operator or desktop client tools**: Administrative or operational clients that connect directly to the ObjectServer for event review, configuration tasks, and day-to-day alert handling.

1.3 Event lifecycle in OMNIbus

The OMNIbus event lifecycle begins when a monitored source generates an event that indicates a condition requiring action, such as a syslog message, SNMP trap, application alarm, or process status change. In most environments, that source data is received by a probe, which parses the input, extracts tokens, applies rules logic, and maps the result into OMNIbus alert fields before sending it to the ObjectServer.

When the alert reaches the ObjectServer, OMNIbus determines whether the incoming event should be inserted as a new alert or treated as another occurrence of an existing alert. This behavior is usually controlled by the Identifier field, which allows the ObjectServer to recognize duplicate events and update the existing alert instead of inserting a separate one.

If the event is new, it is inserted into the ObjectServer and becomes visible for display, automation, and operator action. If the event matches an existing alert, deduplication occurs, the existing alert is updated, and the recurrence count is maintained instead of creating additional duplicate rows.

Correlation logic can then further refine the lifecycle by linking related alerts, such as associating a recovery event with an earlier fault event. This helps reduce noise and makes the alert stream more meaningful from an operational perspective.

Scope-based correlation — for example, matching on fields such as `Node`, `AlertKey`, `AlertGroup`, or a custom `Identifier` — restricts correlation to a defined context so only alerts within the same scope are linked and processed together.

After the alert is stored in the ObjectServer database, operators interact and work with it through the OMNIbus client tools such as the event list and event viewer. Typical actions an operator may take on an event include acknowledging an event to indicate that the fault is being worked on, assigning the event to another user or group, changing the priority of the event to manage triage, adding notes through the use of the event journal, and other controlled updates, with all changes recorded as part of the alert history.

1.4 Key processes and terminology

OMNIbus uses a core set of terms and processes that appear throughout administration, event handling, probe development, and operator workflows. Understanding these concepts early makes it easier to follow how alerts are collected, processed, stored, and managed throughout the rest of the lab guide.

Key terms:

- **Event**: A record of something that happened in the monitored environment, such as a fault, status change, threshold violation, or system message.
- **Alert**: The OMNIbus representation of an event after it has been processed and stored for operational use in the ObjectServer.
- **ObjectServer**: The central in-memory alert database where alerts are stored, updated, deduplicated, correlated, and made available to clients and automation.
- **Probe**: A collection component that receives event data from an external source, parses it, applies rules, and forwards the resulting alert to the ObjectServer.
- **Gateway**: A component that transfers alert data between ObjectServers or between OMNIbus and external systems.
- **Rules file**: Probe logic that controls parsing, token handling, field population, event enrichment, deduplication keys, and correlation-related values.
- **Trigger**: ObjectServer-side logic that runs automatically in response to database activity or scheduled conditions and can be used for automation, enforcement, and auditing.
- **Property**: A configurable setting that controls the behavior of OMNIbus components.
- **Journal**: A record associated with an alert that captures operator or automated updates over time.
- **Details**: Supplemental information associated with an event that provides additional context beyond the primary alert fields.

Key processes:

- **Event ingestion**: The process of receiving data from a source and bringing it into OMNIbus through a probe.
- **Normalization**: The mapping of source-specific event data into standard OMNIbus fields so events from different sources can be handled consistently.
- **Deduplication**: The process of recognizing repeated occurrences of the same event and updating an existing alert rather than inserting a new one.
- **Correlation**: The process of linking related events so OMNIbus can present a more meaningful operational picture.
- **Acknowledgement**: An operator action indicating that an alert has been seen and is being handled.
- **Audit**: The recording of administrative or configuration changes so those changes can be reviewed later.

1.5 Lab environment and assumptions

This lab environment is based on pre-built virtual machines that provide the software components needed for the OMNIbus exercises. Each student will work against an assigned lab server and will use that server for most command-line administration, process control, and validation tasks.

The lab virtual machines include the following pre-installed components:

- OMNIbus core.
- Web GUI, Jazz for Service Management, and WebSphere Application Server.
- MTTrapd probe.
- Syslogd probe.
- IBM Tivoli Network Manager and Netcool/Impact for additional labs outside the OMNIbus core sections.

Each student is expected to connect to their own assigned lab server by using SSH with the IP address, username, and password provided for the lab. Students should be comfortable logging in remotely, navigating the server filesystem, editing text files, starting and stopping processes, and validating results from the command line.

In addition to the server-side lab environment, each student should have OMNIbus core installed locally on their Windows workstation. This local installation is required so the student can use administrative and support tools such as `OMNIbus Administrator` and `MIB Manager`. `OMNIbus Administrator` is a Windows graphical administration tool used to manage ObjectServer users, groups, triggers, automation, databases, and properties. On the Unix/Linux platform it is known as **nco_config** and runs as an X-Windows application.

The local Windows installation is used primarily for tooling access rather than for hosting the main lab environment. The primary ObjectServer, probes, and web components used in these exercises run on the assigned lab virtual machine, while the local workstation provides convenient access to graphical administration tools and MIB-related utilities.

2. ObjectServer and event model

2.1 Environment access and tool validation

Objective In this exercise, you will verify your access to the assigned lab environment and confirm that the primary Netcool/OMNIbus components and client tools are functioning correctly. By the end of this exercise, you will be able to:

- Connect to the lab server using SSH
- Start the ObjectServer and JazzSM Web GUI services
- Configure the Netcool Servers Editor on your Windows workstation
- Connect to the ObjectServer using the Windows native event list client
- Connect to the ObjectServer using the Netcool/OMNIbus Administrator tool
- Access and navigate the browser-based Web GUI

Prerequisites

- Assigned lab server IP address.
- SSH credentials.
- OMNIbus core installed on the local Windows workstation.
- Access to a web browser on the workstation.
- Access to Netcool/OMNIbus Administrator and the OMNIbus desktop event list on the workstation.

Background The Windows client tools — including the native Windows Event List and Netcool/OMNIbus Administrator — connect to the ObjectServer over the network using a configured server entry. Before these tools can connect, the student must add an entry to the OMNIbus Servers Editor on the Windows workstation that tells the client tools where the ObjectServer is and how to reach it.

The Servers Editor is a small Windows utility included with the OMNIbus core installation. It maintains a list of known ObjectServers and their connection details. Each entry in the list requires the following information:

- **Server name** — the logical name of the ObjectServer, which in this lab is `EVENTS`.
- **Hostname or IP address** — the hostname or IP address of the lab server where the ObjectServer is running. Either format is accepted.
- **Port** — the port on which the ObjectServer listener is running. The default listener port is `4100`, however it is not unusual to have multiple ObjectServers in an environment running on different ports.

Once the entry is saved, all OMNIbus client tools on the workstation will be able to connect to EVENTS by name, without needing to specify the hostname or port each time.

Tasks

1. Connect to the assigned lab server by using SSH as the `root` user. It is typical to install Netcool/OMNIbus software as a non-root administrative user such as `ncoadmin`, but for convenience this lab environment is installed as root and the exercises are performed as the root user.
2. Before starting the ObjectServer, set the required OMNIbus environment variables on the lab server. These are standard Netcool environment variables used by OMNIbus commands:

```
export NCHOME=/opt/IBM/tivoli/netcool
export OMNIHOME=$NCHOME/omnibus
```

3. To persist these values across new shell logins (bash), add the same lines to your `~/.bash_profile` or `~/.bashrc`:

```
echo 'export NCHOME=/opt/IBM/tivoli/netcool' >> ~/.bash_profile
echo 'export OMNIHOME=$NCHOME/omnibus' >> ~/.bash_profile
source ~/.bash_profile
```

4. Add `$OMNIHOME/bin` to your PATH for convenience. This eliminates the need to type the full path for OMNIbus commands throughout the lab:

```
export PATH=$OMNIHOME/bin:$PATH
```

   To persist this change across new shell logins, add the same line to your `~/.bash_profile` or `~/.bashrc`:

```
echo 'export PATH=$OMNIHOME/bin:$PATH' >> ~/.bash_profile
source ~/.bash_profile
```

5. Stop the nco systemd service so that we can run the ObjectServer independently for this lab:

```
systemctl stop nco
```

   Now start the Netcool Process Agent daemon (nco_pad), which is a service that manages and monitors OMNIbus processes. The Netcool Process Agent is a lightweight process supervisor that starts, monitors, and automatically restarts OMNIbus components (such as the ObjectServer and probes) if they crash or terminate unexpectedly. It provides a convenient way to manage multiple processes as a group and logs process status information for troubleshooting.

```
nco_pad -authenticate NONE
```

   Verify that the MasterObjectServer process is running by executing the following command:

```
nco_pa_status
```

   When prompted for a password, press Enter. The expected output should display the MasterObjectServer process status:
```
text [root@ibm-training-server1 ~]# nco_pa_status Login Password:  ------------------------
------------------------------------------------------------------------------------ Ser-
vice Name            Process Name            Hostname                                  User
Status            PID ---------------------------------------------------------------------
---------------------------------------- Core                    MasterObjectServer   ibm-
training-server1.dcimusic.com  root       RUNNING      432517 ------------------------------
------------------------------------------------------------------------------ [root@ibm-
training-server1 ~]#
```
   Confirm that the MasterObjectServer status shows **RUNNING**.

6. Run the following command to verify that the ObjectServer named EVENTS is available:

```
nco_ping EVENTS
```

   The nco_ping utility contacts the specified Netcool/OMNIbus server and checks whether it can accept connections. In this example, EVENTS is the ObjectServer name defined in the OMNIbus interfaces configuration. A successful response confirms not only that the ObjectServer process is running, but also that the client can locate and communicate with it using its configured host and port.

Confirm that the output includes: `NCO_PING: Server available.`

7. Ensure that the JazzSM application server is running, which hosts the Web GUI and provides the browser-based OMNIbus user interface. First, verify that the JazzSM server is running by checking its status:

```
/opt/IBM/JazzSM/profile/bin/serverStatus.sh server1 -username smadmin -password netcool
```

If the server is already running, the status command will output: `text ADMU0508I: The Application Server "server1" is STARTED` If this message does not appear or the server is not running, start it with the following command:

```
/opt/IBM/JazzSM/profile/bin/startServer.sh server1
```

Wait for startup to complete and confirm that the output includes all of the following lines:

```
ADMU0116I: Tool information is being logged in file
           /opt/IBM/JazzSM/profile/logs/server1/startServer.log
ADMU0128I: Starting tool with the JazzSMProfile profile
ADMU3100I: Reading configuration for server: server1
ADMU3200I: Server launched. Waiting for initialization status.
ADMU3000I: Server server1 open for e-business; process id is 1047344
```

8. On your Windows client workstation, open the OMNIbus Servers Editor tool:

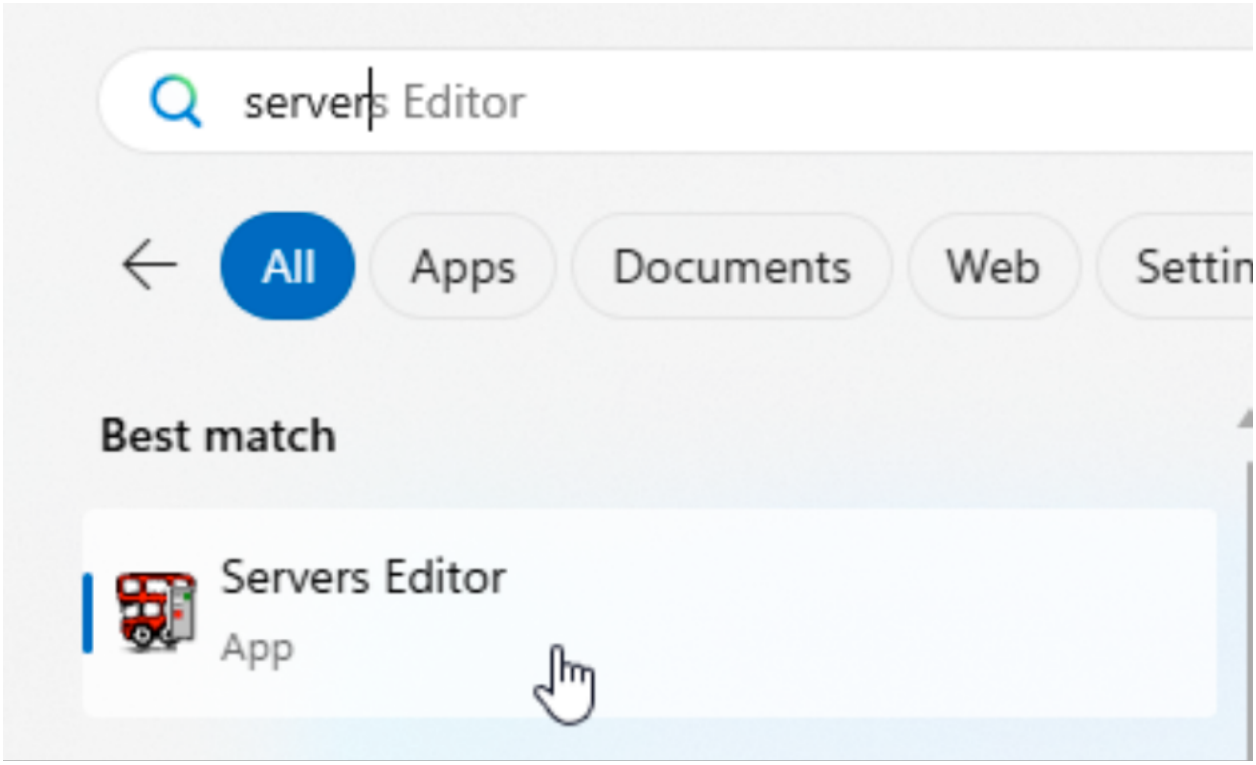


Figure 2: Servers Editor tool

Locate the existing NCOMS server entry and edit both the server and listener settings:

- **Server name**: EVENTS
- **Hostname or IP address**: the hostname or IP address of your assigned lab server as provided by the instructor
- **Port**: 4100

The server entry defines the ObjectServer logical name and connection details, while the listener setting controls which network port the OMNIbus connection is actually established on. Before saving, validate the connection by selecting the EVENTS entry and clicking the **Test** button to verify that the ObjectServer is reachable.

Confirm that the test result returns **“Server available”**, indicating that the client can successfully connect to the ObjectServer. Once validated, save the updated entry by clicking **OK** to save the updates and exit the Servers Editor tool. All OMNIbus client tools can now connect to EVENTS by name.

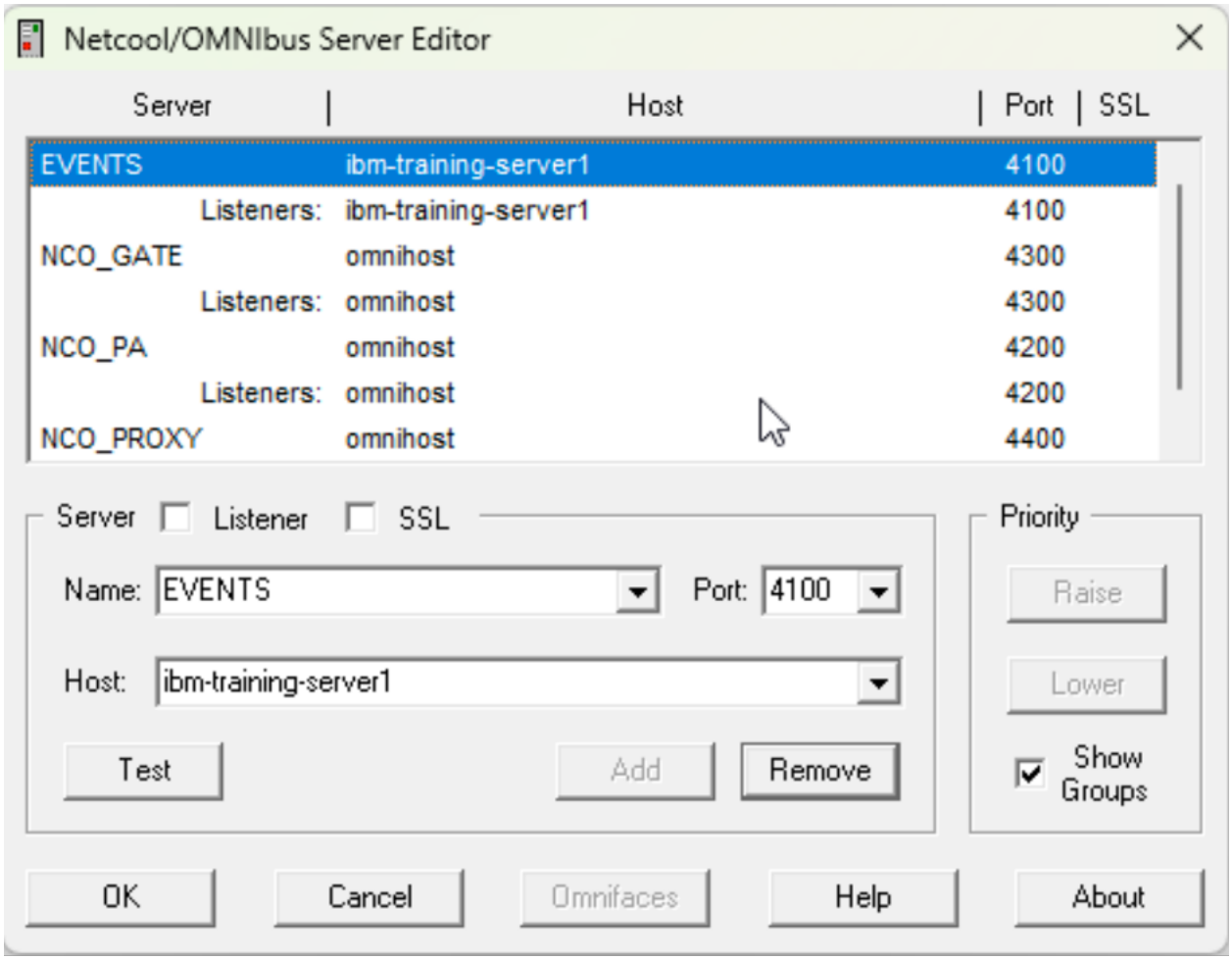


Figure 3: Netcool/OMNIbus Servers Editor

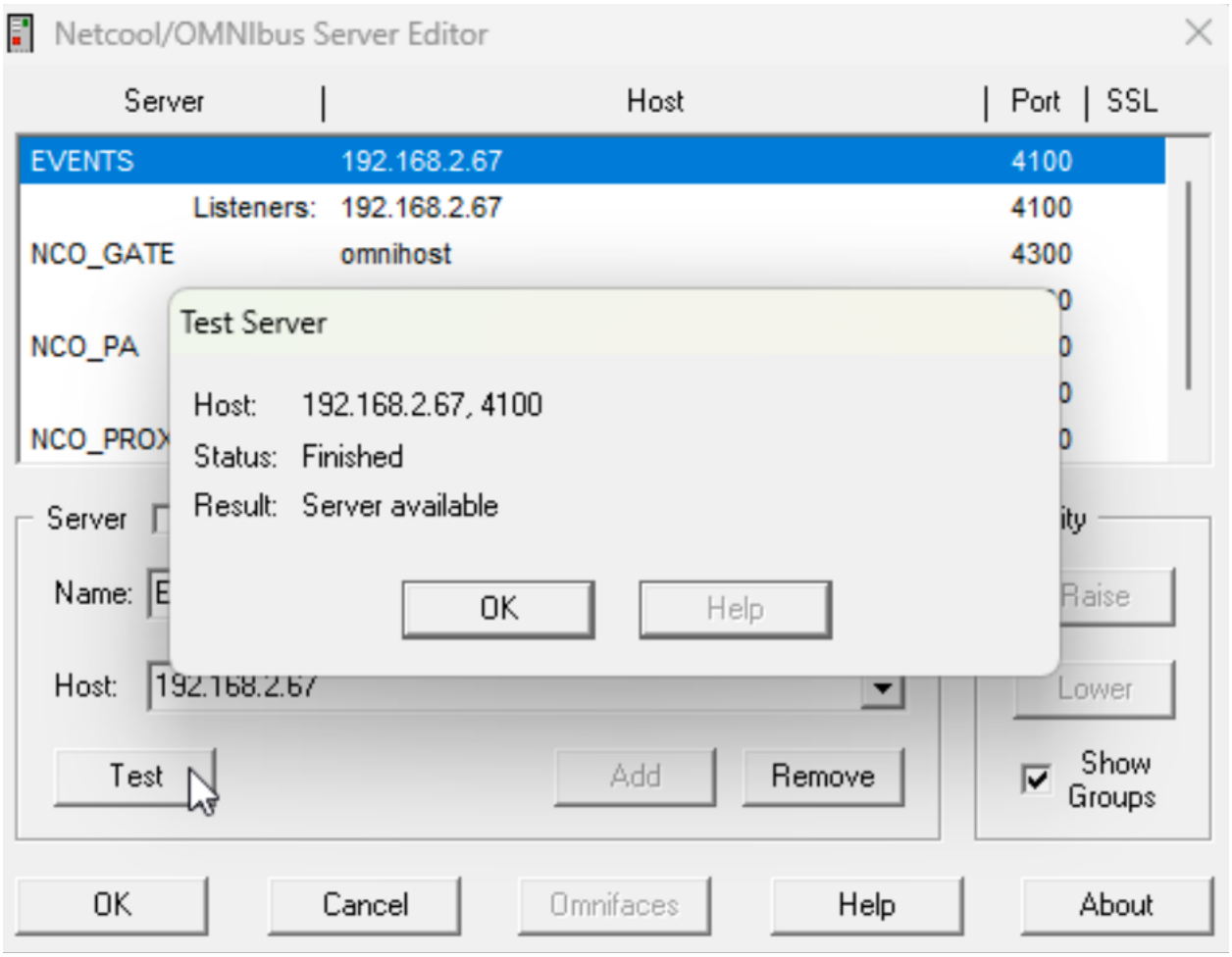


Figure 4: Test connection button

Change the remaining entries in the Server Editor from `omnihost` to the hostname of your lab server before continuing:

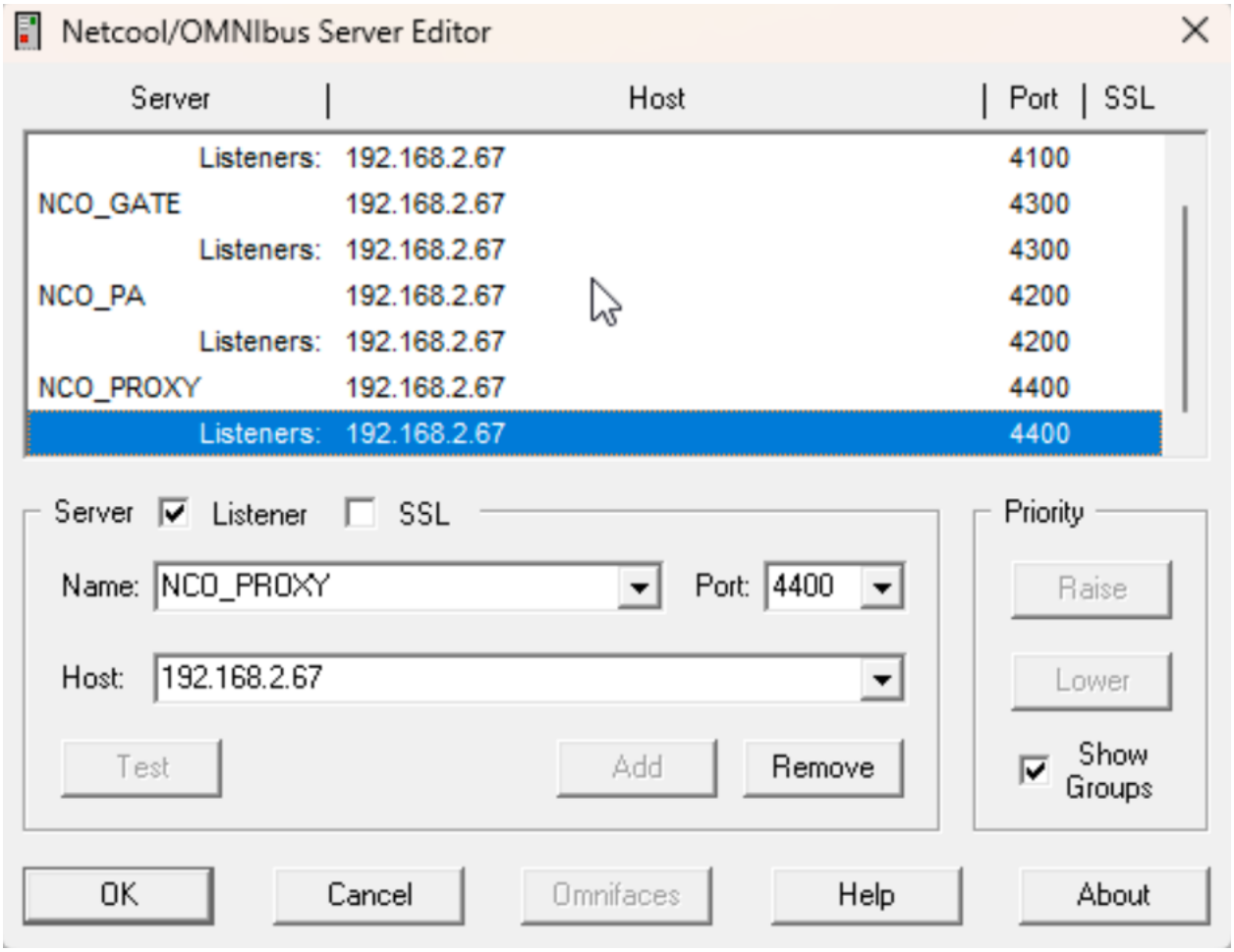


Figure 5: Test connection button

9. Locate and launch the Windows event list client:
   - Ensure that EVENTS is selected as the server and log in with the following credentials:
     - **User**: `root`
     - **Password**: `netcool`
   - Confirm that the event list opens successfully and connects to the lab ObjectServer:
10. Launch Netcool/OMNIbus Administrator tool:

- If you are asked to import the connection details (the `sql.ini` file that is saved from the Servers Editor in the previous step). Click **Yes** to load our updates.
- Since we want to import all as a first run, select **Finish** here.
- You will see two windows, one for Process Agents and the other for Object Servers. For now, find your EVENTS object server and double-click the entry or right-click and select **Connect As...**
- Enter the administrator credentials `root` and `netcool`. Note that this user is distinct from the operating system `root` user; it is the Netcool administrator user.
- You will be presented with the following screen on successful login, with menu drawers on the left you can open to configure various aspects of the solution.

11. Now that we have validated local tool access, we will verify Web GUI access from a browser:

- Open a browser and navigate to the following URL: `https://<your-server>:16311/ibm/console` If presented with a certificate warning, click **Advanced** and then **Continue to (url above)** to proceed.
- Log in to the Web GUI using the administrative account for this lab:

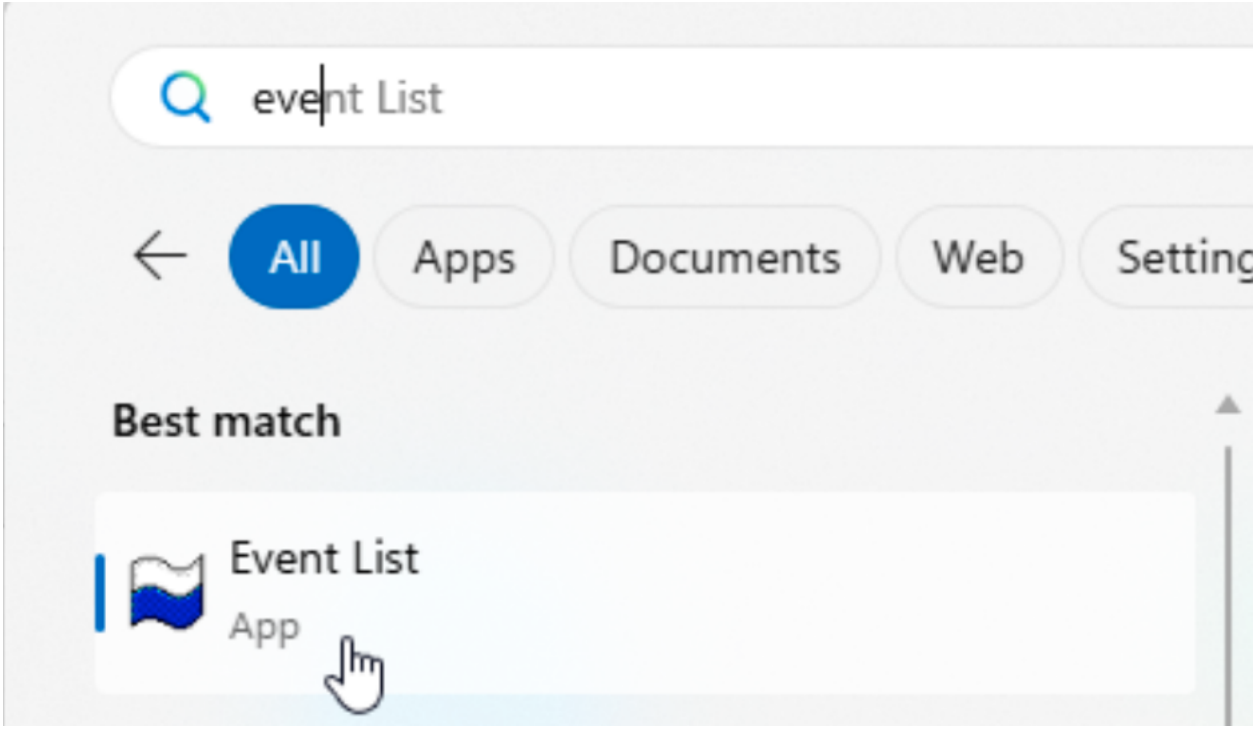


Figure 6: Event List

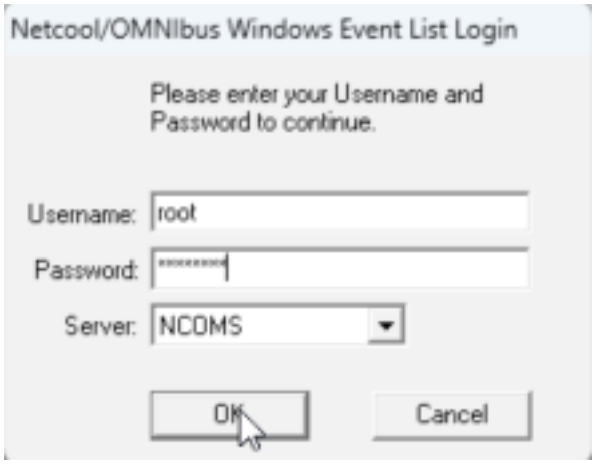


Figure 7: Event List login

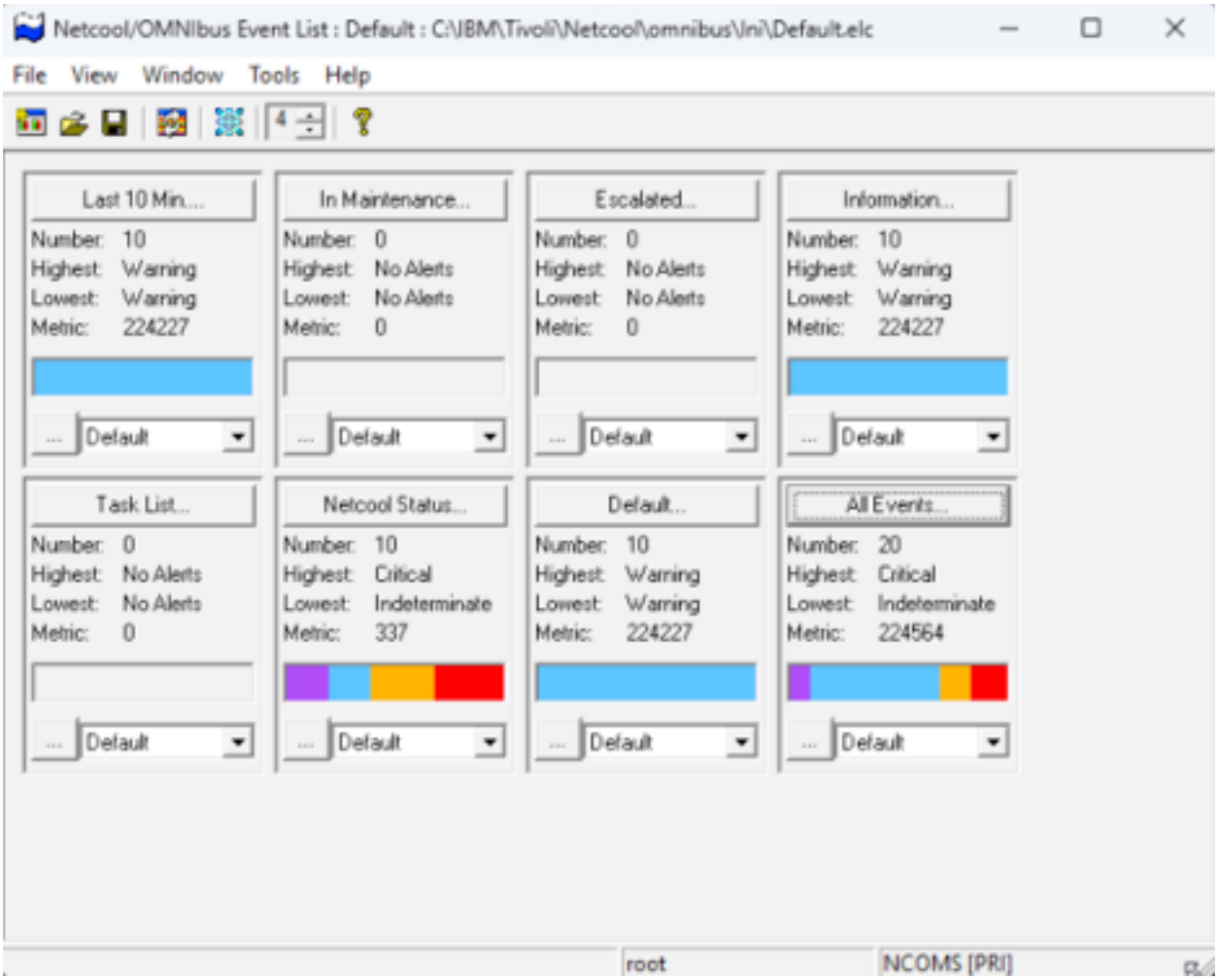


Figure 8: Event List dashboard

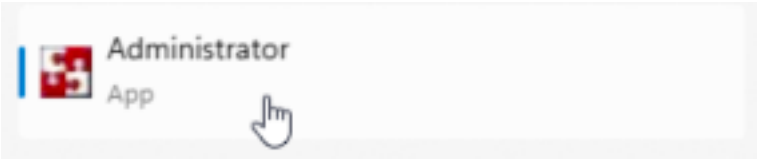


Figure 9: Netcool/OMNIbus Administrator launch

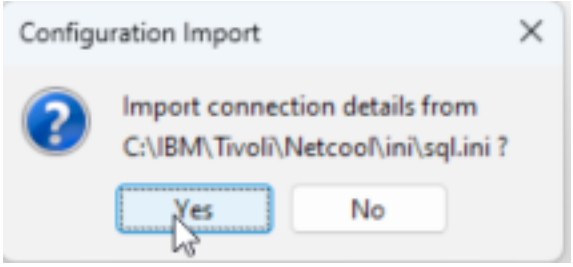


Figure 10: Import connection details



Figure 11: Import finish

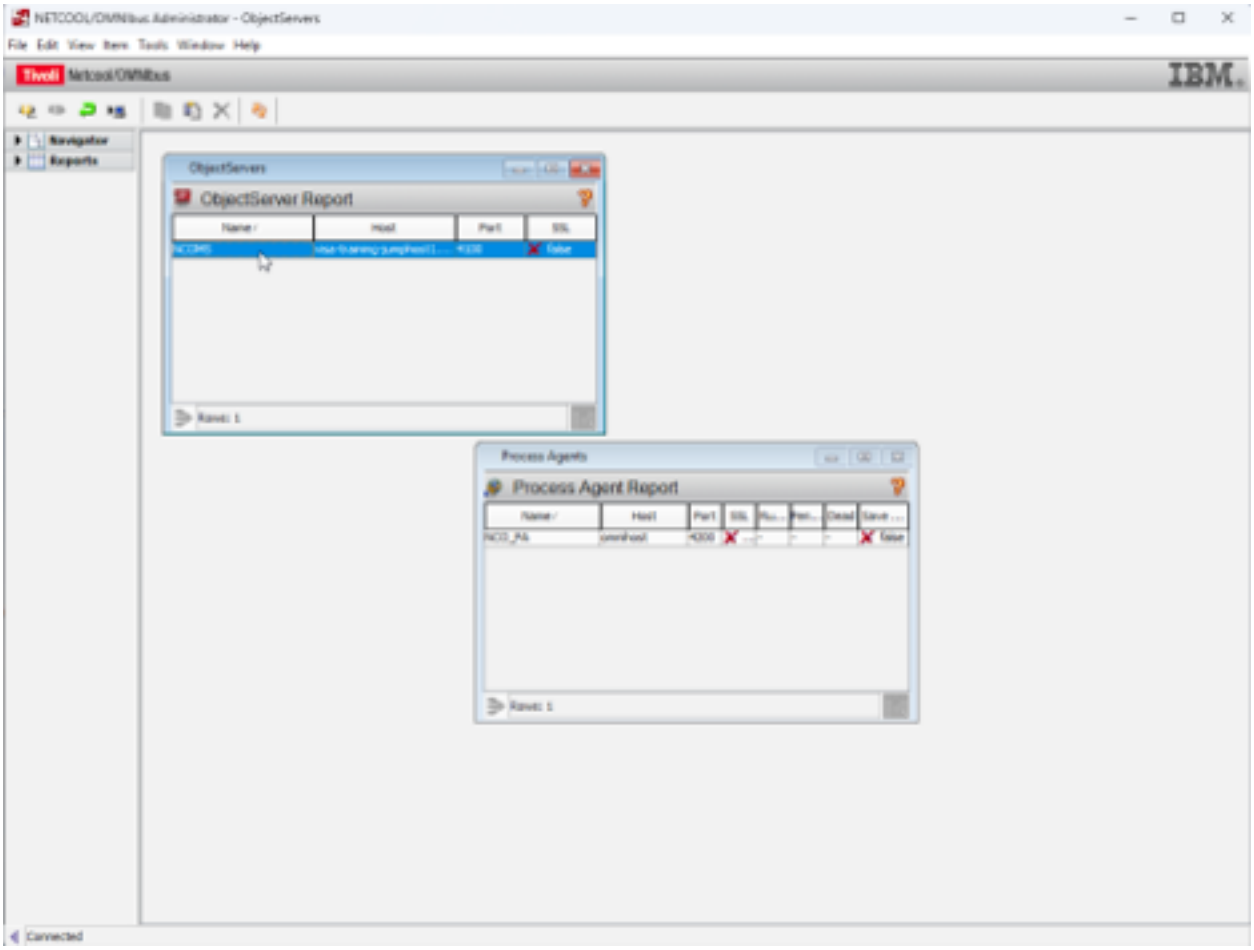


Figure 12: Connect to EVENTS

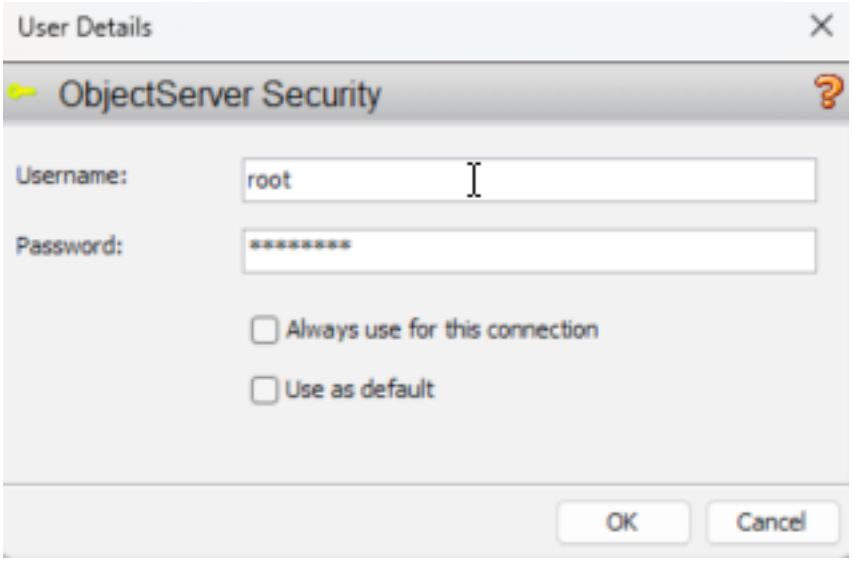


Figure 13: Enter admin credentials

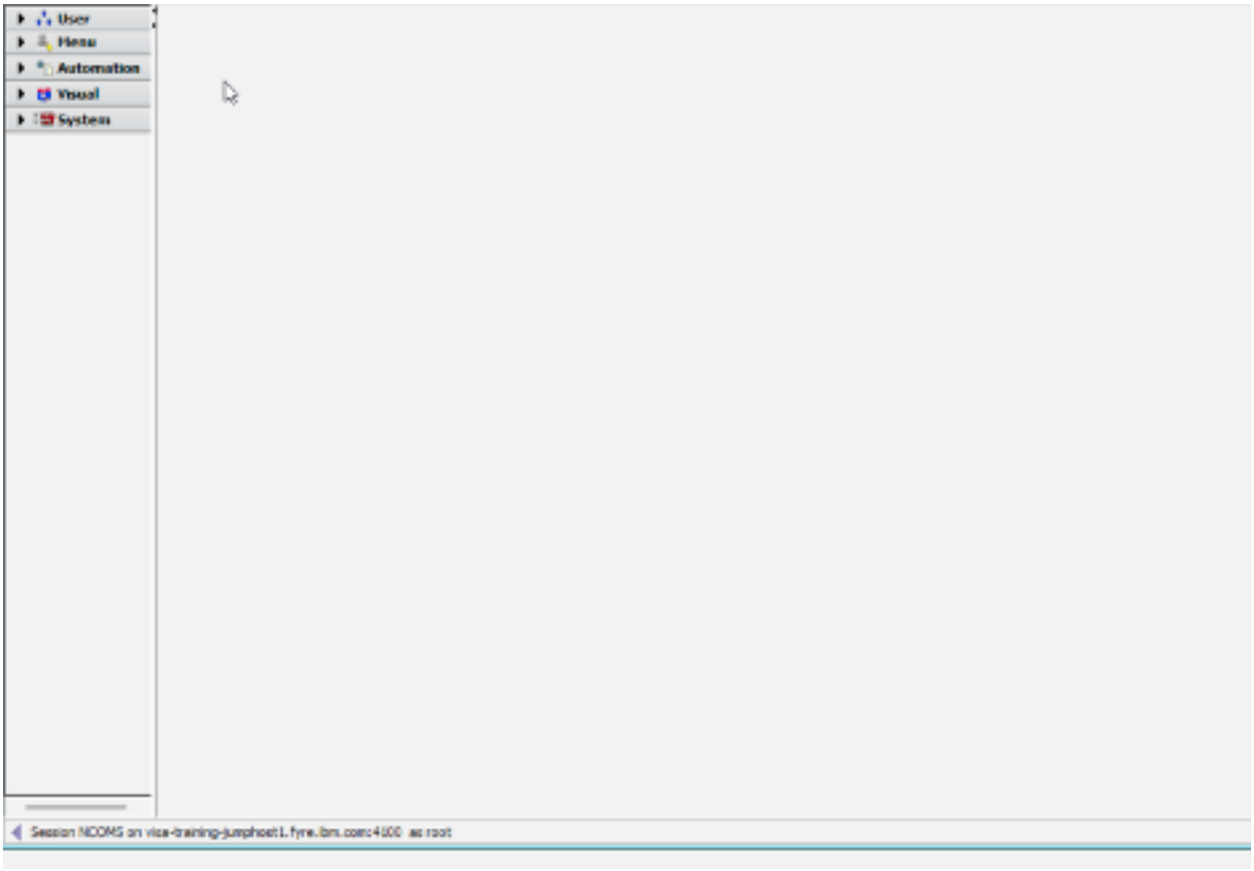


Figure 14: Admin login success

  - **User**: `ncoadmin`
  - **Password**: `netcool`

  Note: `ncoadmin` is the default administrative user for the Web GUI (this is an application account, not an operating-system account).

- After a successful login you will arrive at the “About Netcool/OMNIbus Web GUI” page and see the left-hand menu icons:

Validation The exercise is successful when all of the following are true:

- SSH login to the assigned lab server succeeds.
- `nco_ping EVENTS` returns `NCO_PING: Server available.` after starting the ObjectServer.
- The JazzSM application server reports `ADMJ3000I: Server server1 open for e-business`.
- The Windows event list opens and connects to EVENTS.
- `Netcool/OMNIbus Administrator` connects successfully to EVENTS.
- The browser-based Web GUI opens to the landing page for IBM Tivoli Netcool OMNIbus.

Expected result By completing this exercise, you have successfully established a fully functional Netcool/OMNIbus lab environment. You now have:

- **Server-side services** running and operational: the ObjectServer (EVENTS) is actively processing events and the JazzSM application server is hosting the Web GUI.
- **Verified connectivity** from multiple client interfaces: SSH terminal access to the lab server, the Windows native event list client, the Netcool/OMNIbus Administrator tool, and the browser-based Web GUI all successfully connect to your ObjectServer.
- **Confirmed network connectivity** between your workstation and lab server by testing the server connection in the Servers Editor tool.
- **A foundation for advanced exercises**: this baseline environment is now ready for the rest of Chapter 2, where you will explore event handling, the ObjectServer database, event deduplication, and automation rules.

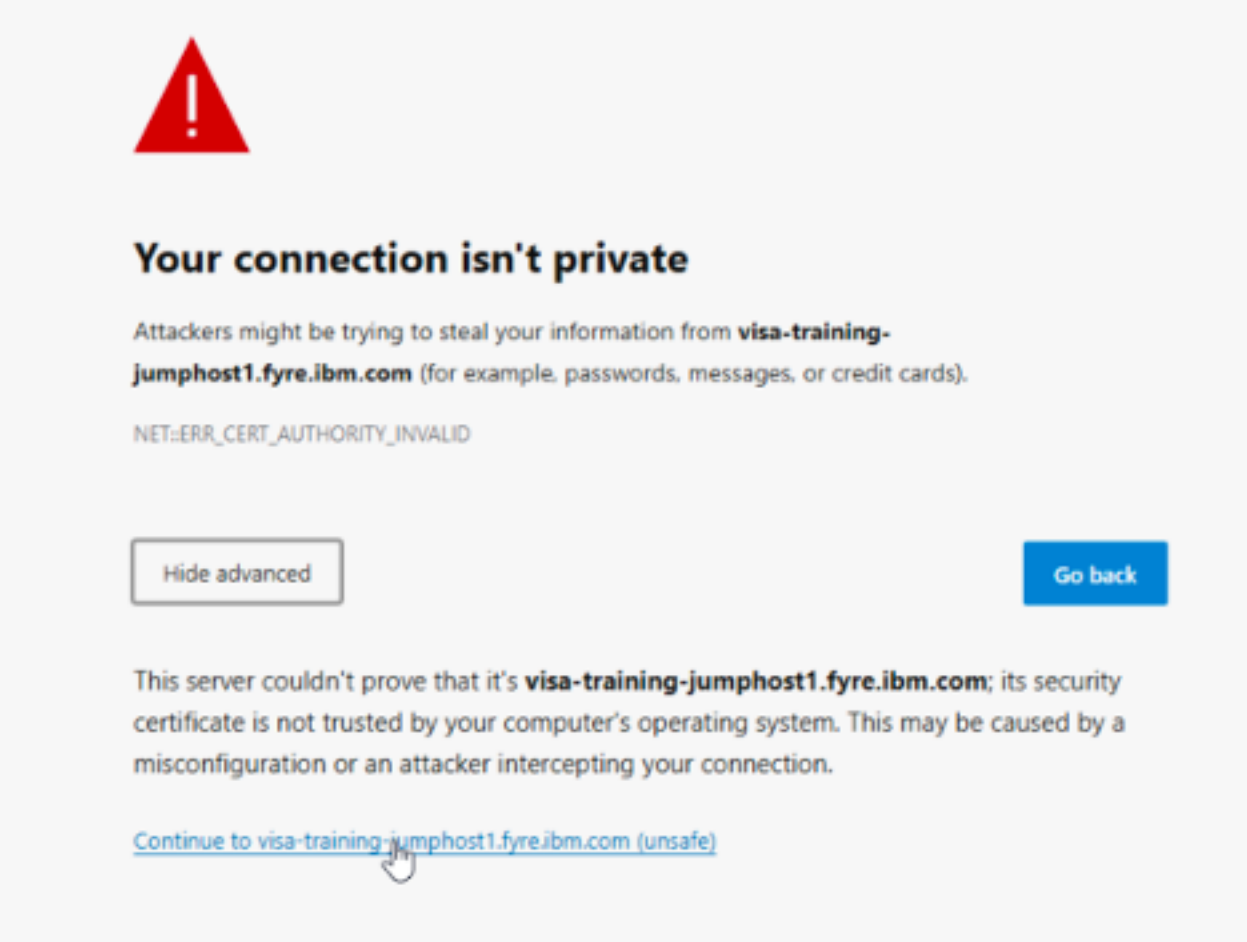


Figure 15: Web GUI certificate warning



Figure 16: Web GUI login

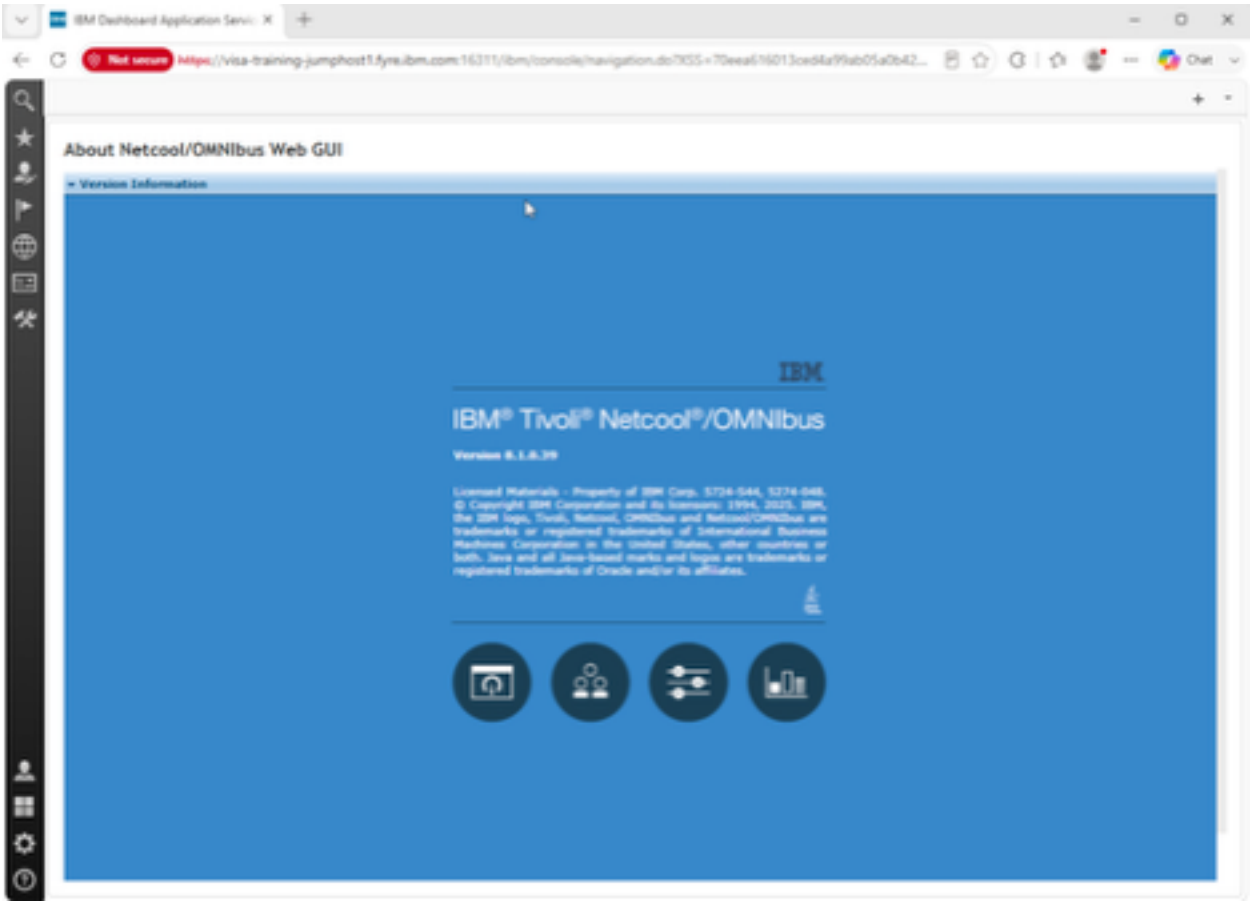


Figure 17: Web GUI about page

2.2 Basic event interaction

Objective In the following labtaskss, you will use the OMNIbus event list to locate and examine events, interpret their severity and key fields, and perform basic event-management actions. The goal is to become familiar with how an operator moves from seeing an alert to understanding the reported condition and recording or taking the appropriate response. By the end of this exercise, you will be able to:

- Inspect and examine detailed event information
- Modify event severity levels to reprioritize alerts
- Acknowledge alerts to indicate ownership and status
- Apply filters to focus on specific subsets of events
- Switch between different event view layouts
- Observe real-time event generation and deduplication behavior as events arrive from the Simnet probe

Prerequisites

- Completion of section 2.1.
- The ObjectServer is running.
- The Web GUI application server is running.
- The student can log in to the Web GUI and open an event list.

Background IBM Netcool/OMNIbus collects event information from monitored infrastructure and presents it in the event list for operator review and action. An event represents a condition detected by a monitoring source, such as a device fault, an application problem, a threshold breach, or a change in service status.

Each event includes fields that help an operator understand what happened, where it occurred, when it was detected, and how urgently it should be addressed. Common fields include the event summary, source, node, event type, timestamp, and severity. Operators use this information to assess the impact of the condition, identify related events, and determine the appropriate response.

OMNIbus uses a numeric severity scale to indicate the operational priority of an alert. The severity values used throughout the event list and in probe rules are as follows:

- **0 — Clear**: The condition has been resolved.
- **1 — Indeterminate**: The severity is unknown or has not yet been assessed.
- **2 — Warning**: A condition exists that may require attention.
- **3 — Minor**: A minor fault or degradation has occurred.
- **4 — Major**: A significant fault has occurred that is likely to affect service.
- **5 — Critical**: A severe fault has occurred that is affecting or will affect service immediately.

Severity helps operators prioritize their work. Critical and major events normally require prompt investigation because they may indicate a service-impacting condition. Warning and minor events may require review, trend analysis, or follow-up, while indeterminate events often require additional assessment before their impact can be determined. Clear events indicate that the monitored condition is no longer active.

In the event list, severity levels are typically shown using distinct colors, allowing high-priority events to be identified quickly. Severity can also be used in filters, views, automations, and probe rules to determine how events are displayed, routed, escalated, or processed.

Tasks

1. Return to the Web GUI login. If you are not already logged in as the administrative Web GUI user, open a browser and navigate to: `https://<your-server>:16311/ibm/console`

   Log in with the following credentials for this lab:

   - **User**: `ncoadmin`
   - **Password**: `netcool`

2. Locate the flag menu icon in the upper-left corner of the Web GUI and select it. In the menu that appears, choose **Event Viewer** to open the event list.

3. Observe the rows currently displayed in the event list. Note that each row in the event list represents an event stored in the ObjectServer, and that the displayed columns correspond to event fields.

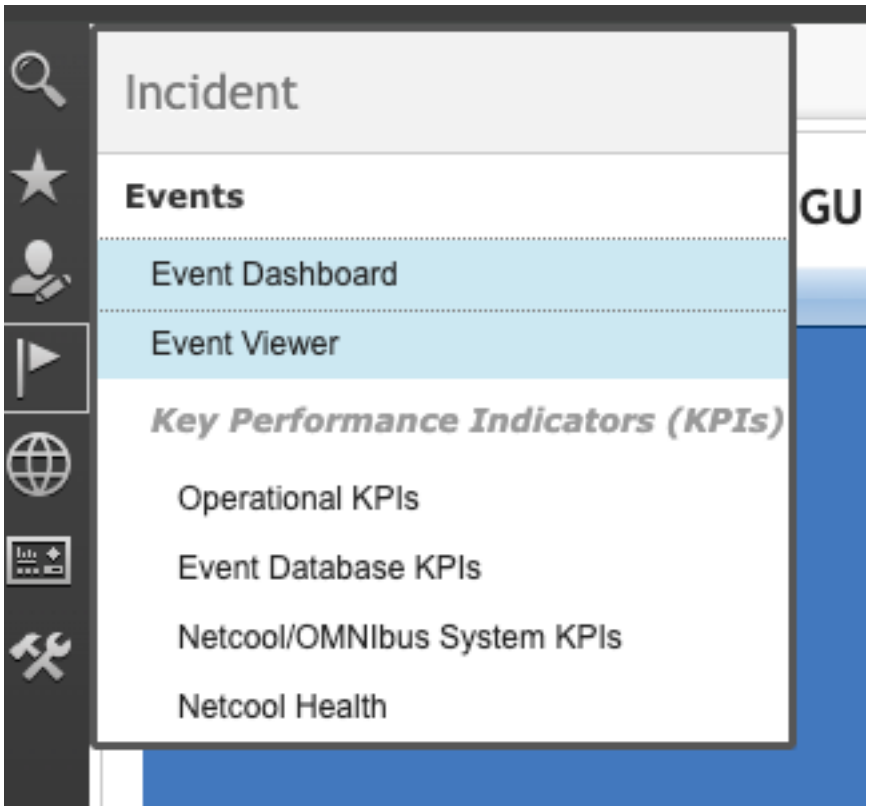


Figure 18: Event Viewer menu

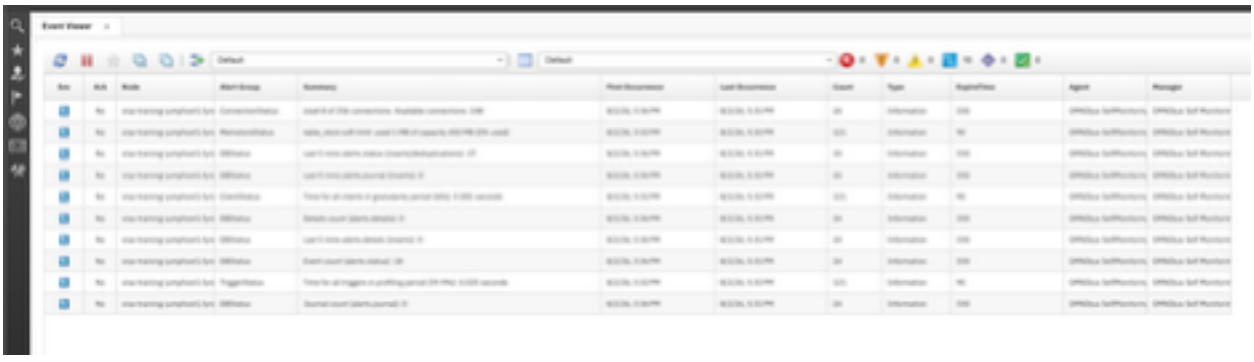


Figure 19: Event list rows

4. Right-click a single event and in the popup menu choose **Prioritize -> Critical**. Confirm that the severity value and row color change in the event list.

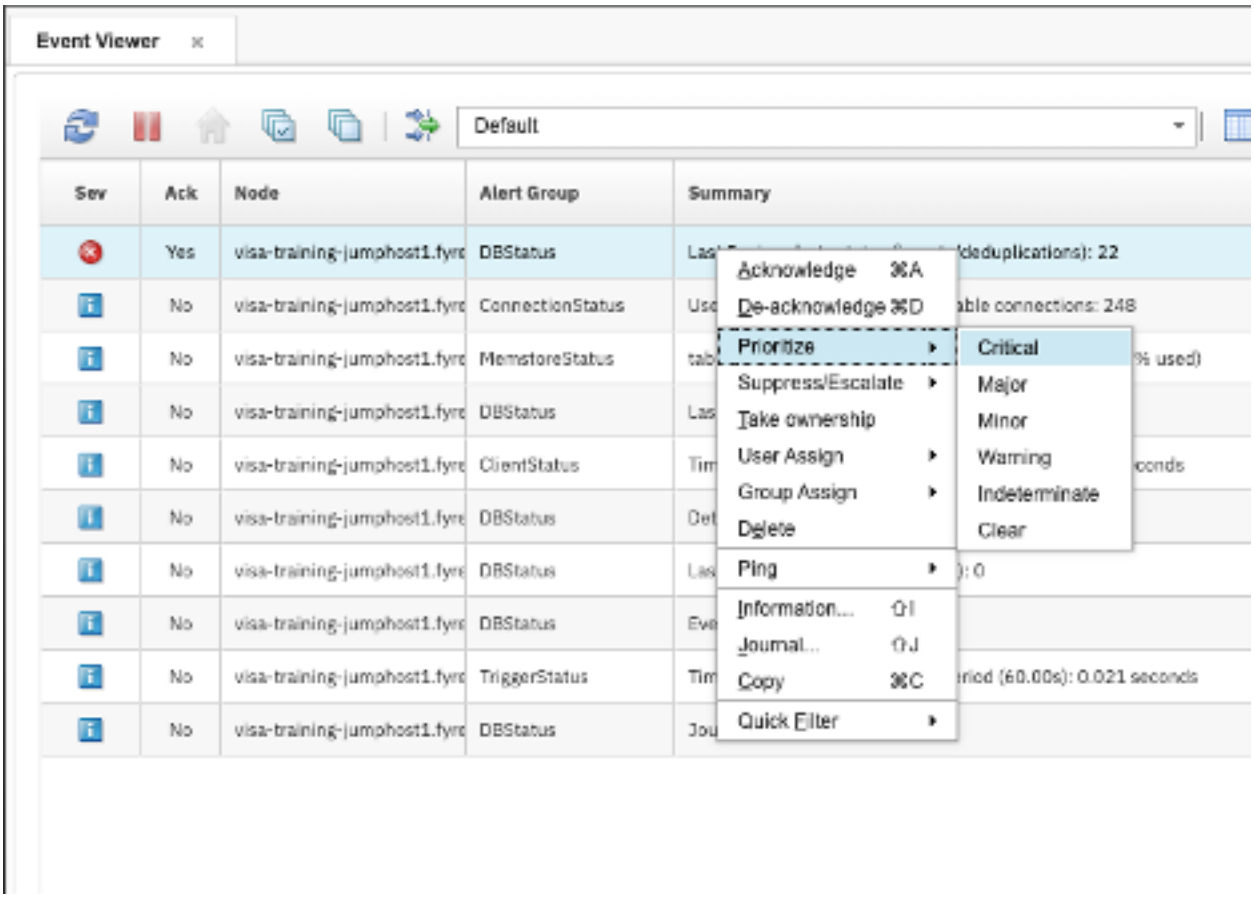


Figure 20: Prioritize -> Critical menu

5. The event list typically shows a subset of all available event fields. To see every field and its current value, either right-click the event and select **Information...** from the popup tools, or double-click the event to open the full event details.

   Note: You can create views that include or exclude any of these fields in your event list layout, and switch between different saved event layouts as needed. We will work with views and layouts in an upcoming section.

6. If the event details or Information window is still open, close it. Then right-click the same event and select **Acknowledge** from the popup menu. After acknowledging, double-click the event to re-open the details and confirm the **Acknowledged** field shows the acknowledging user and timestamp.

   Acknowledged indicates that an operator has seen the alert and is taking ownership or actively working on it; acknowledgement records the operator and time but does not clear the event.

7. In the same event properties/details window, select the **Journal** tab. Note the journal entry indicating that user `ncoadmin` acknowledged the alert at the shown time (automation tools sometimes add journal entries automatically). You can also add freeform notes: type text into the **New Journal entry** field, click **Apply**, and confirm a new journal row appears showing the user, timestamp, and entry text.

   The event journal provides a shared, time-stamped record of investigation notes, actions taken, and resolution details for an event. It helps operators coordinate work across shifts and teams, avoid duplicated troubleshooting, and maintain an auditable history that can be used for incident review and follow-up.

8. If the event details or Information window is still open, close it now. Then locate the filter selector dropdown at the top of the event list (near the icon with the three arrows) and choose the **Escalated** filter. If no events have been escalated yet, the event list will be empty.

   An **event filter** is a saved condition that limits which alerts are shown in the event list (it behaves like a SQL WHERE clause against `alerts.status`). Filters are useful because they let you instantly switch between different sets of events, for example by event type, by time window, by customer or service, or by escalation or acknowledgement state. This is especially

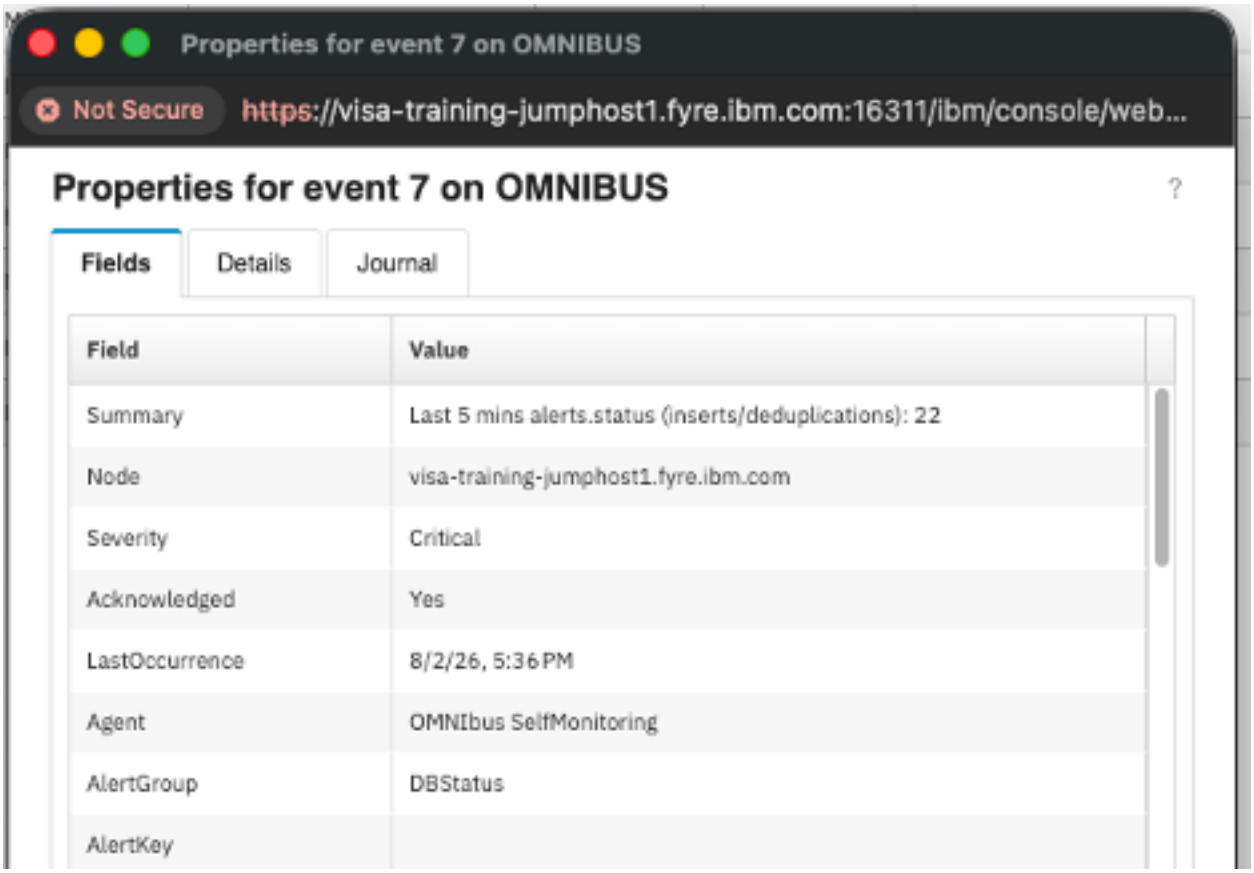


Figure 21: Event information/details

helpful in a busy operations environment where you may need to focus on only critical incident alerts, only recent events, or only alerts for a specific customer or business service. You will be working with this in a upcoming tasks.

9. Return to the **Default** filter. Right-click an event, select **Suppress/Escalate**, and choose **Escalated** from the submenu. Escalation levels can be used in several ways:

   - Manually by an operator to raise an alert's visibility and route it to a higher-level support queue or incident management workflow.
   - Automatically by rules or automation when an alert meets an operational condition, for example if a high-severity alert has not been acknowledged within 10 minutes.
   - To trigger secondary actions such as notifications, paging, or promotion to a management dashboard for elevated handling.

   This makes escalation levels useful for both immediate operator attention and timed visibility workflows.

   Once an event is escalated, the selected escalation level becomes part of the event state and can be used by filters, views, and dashboards to identify alerts requiring elevated handling. In OMNIbus, escalation levels can distinguish events that have entered different stages of attention, enabling operators to focus on conditions that have exceeded normal response thresholds or require additional coordination. Depending on the organization's configuration, automations can detect escalated events and perform follow-up workflow tasks such as notifying an on-call team, creating or updating an incident ticket, assigning the event to an appropriate support group, adding a journal entry, increasing its priority, or forwarding relevant event details to another management system. This helps ensure that important unresolved conditions receive timely visibility and are handled according to established operational procedures.

10. Return to the **Escalated** filter and confirm that the selected event now appears in the filtered event list.

11. Locate the views dropdown box. It is located to the right of the icon that looks like a spreadsheet. Select the **Self-Monitoring** view and observe how the displayed event columns change when the view changes.

    A **view** controls how event information is presented in the event list. It determines which event fields appear as columns, the order in which those columns are displayed, their width, and whether field values are left-, center-, or right-justified. Views can also define how events are sorted—for example, by severity to place the most urgent alerts first, or by last occurrence time

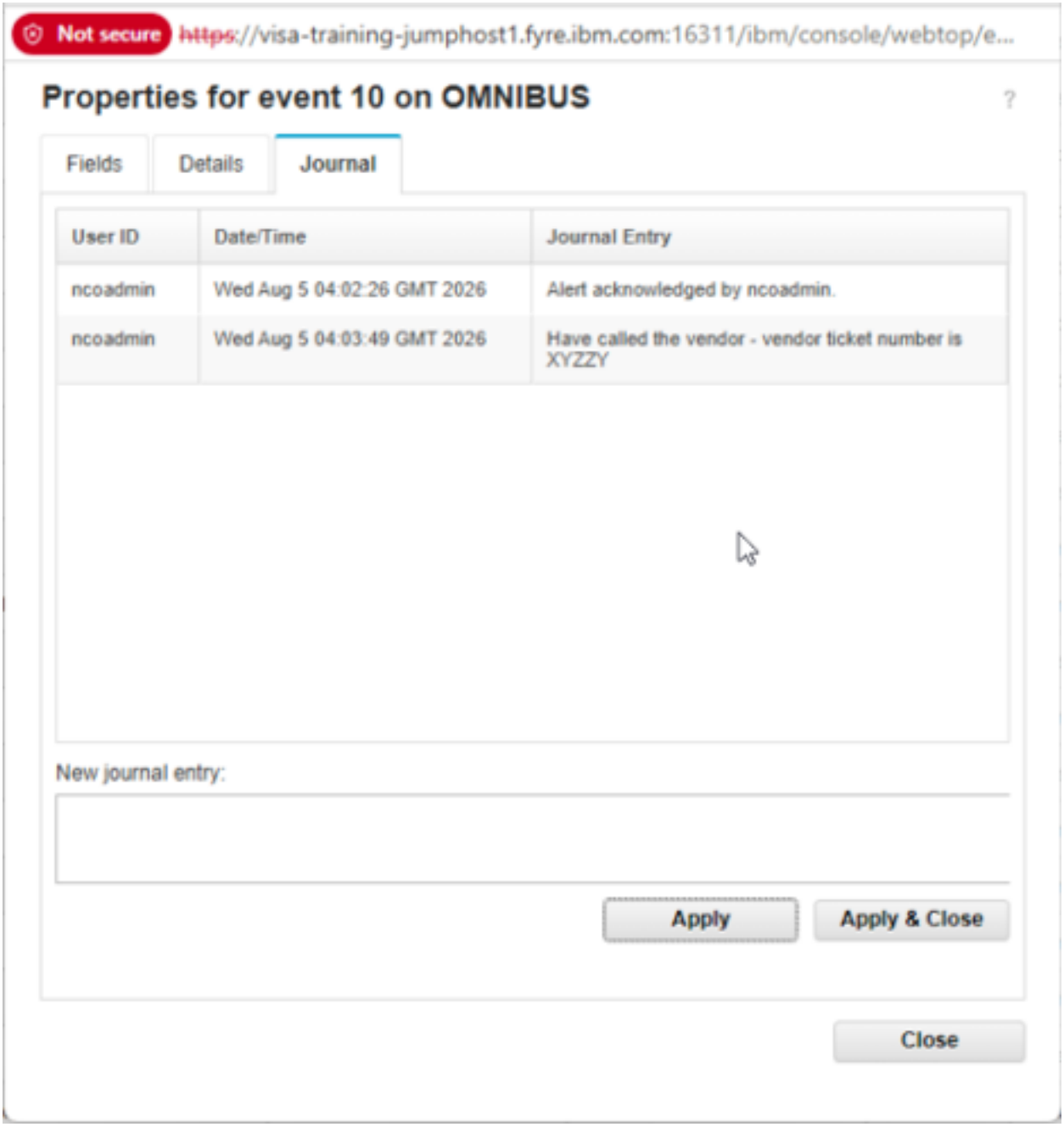


Figure 22: Journal tab entry and add new entry

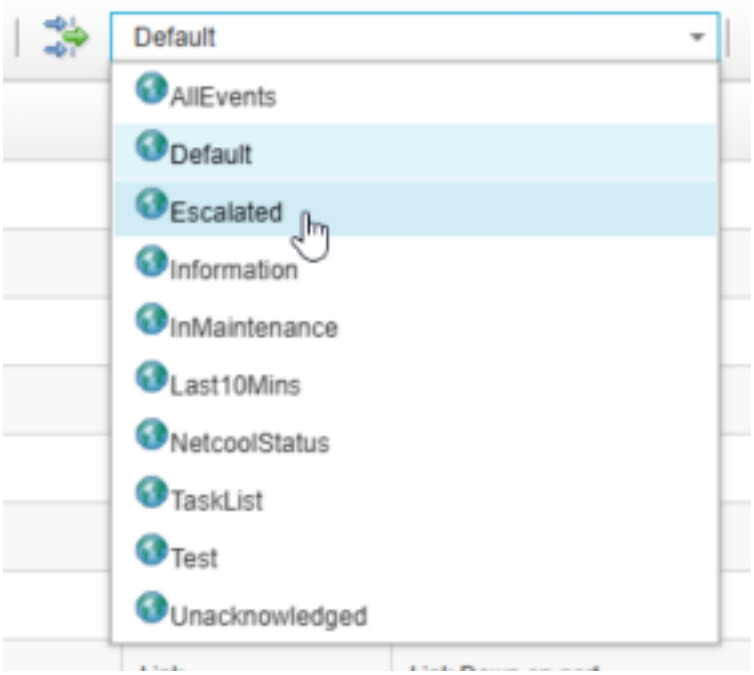


Figure 23: Escalated filter selected (empty list)

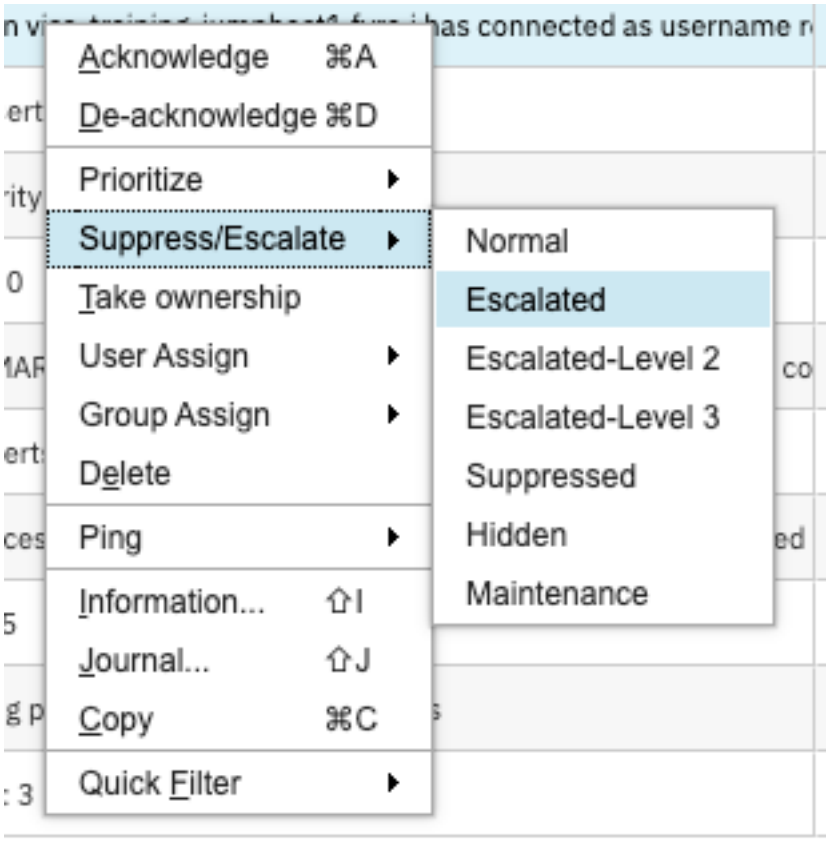


Figure 24: Escalate event to Escalated level

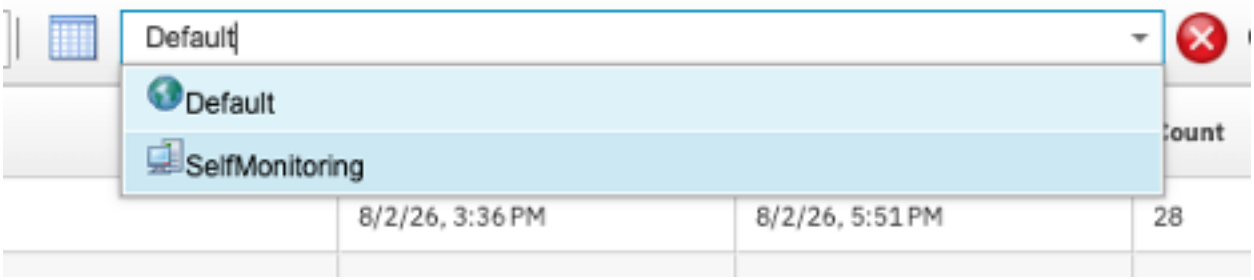


Figure 25: Select Self-Monitoring view

to show the most recently updated events. In addition, views can group related events by a common field such as location, customer, service, node, or alert group, making patterns and areas of impact easier to identify.

Different views are useful when operators need to focus on different types of information. For example:

- A troubleshooting view may show `Summary`, `Node`, `AlertKey`, `Severity`, `FirstOccurrence`, `LastOccurrence`, and `Count` so an operator can quickly understand the issue, when it appeared, and how often it has recurred.
- A service-level view may show `CustomerID`, `Service`, `AlertGroup`, and `LastOccurrence` so a manager can track impact by business service or customer.
- A network view may show `Node`, `AlertGroup`, `AlertKey`, `Summary`, `CircuitID`, and related fields that are useful when troubleshooting connectivity or infrastructure issues.

Changing a view does not change which alerts are returned by the active filter. The filter determines the set of events shown in the event list, while the view determines how the fields from those events are displayed, sorted, grouped, and organized for the operator.

12. Next, we will generate some simulated events to see what more realistic externally generated alerts might look like. The Simnet probe is included in a base install of OMNIbus and is used to generate synthetic alert traffic for testing and demonstration purposes. SSH into the lab server if you are not already connected, then start the Simnet probe with the following command:

```
$OMNIHOME/probes/nco_p_simnet -server EVENTS &
```

Verify that the Simnet probe continues to run by executing the following command:

```
ps -ef | grep simnet
```

The expected output should show the Simnet probe process running and connected to the EVENTS server:

```
root      434536  432237  1 10:23 pts/1    00:00:00
 ↪  /opt/IBM/tivoli/netcool/omnibus/platform/linux2x86/probes64/nco_p_simnet -server EVENTS
```

Confirm that the process is actively running before proceeding to the next step.

13. Return to the Web GUI event list and observe new simulated events flowing in. Note the differences in **Severity**, **Summary**, and **Node** values as the simulated events arrive.

    Pay particular attention to the FirstOccurrence, LastOccurrence, and Count fields. These fields demonstrate OMNIbus deduplication behavior, which prevents repeated reports of the same condition from creating a separate event-list row each time they are received. Instead, when an incoming alert matches an existing event according to the configured deduplication criteria, OMNIbus updates the existing event record. The FirstOccurrence field retains the time that the event was initially received, while LastOccurrence is updated to show the most recent matching occurrence. The Count field increases with each repetition, showing how often the condition has been reported. This gives operators one consolidated event to manage while still showing both the duration and recurrence of the underlying condition. The standard deduplication trigger, for example, updates LastOccurrence and increments the event tally when a matching event is reinserted.

14. Columns are sortable by clicking on the sort icon in the column heading. Sort the event list by **LastOccurrence**, and note how newly arriving Simnet events move within the list as fresh activity is received.

15. Use the quick-severity colored icons at the top of the event list to filter by severity level, then switch the event list back to the **Default** view before ending the exercise so the next lab begins from a known state.

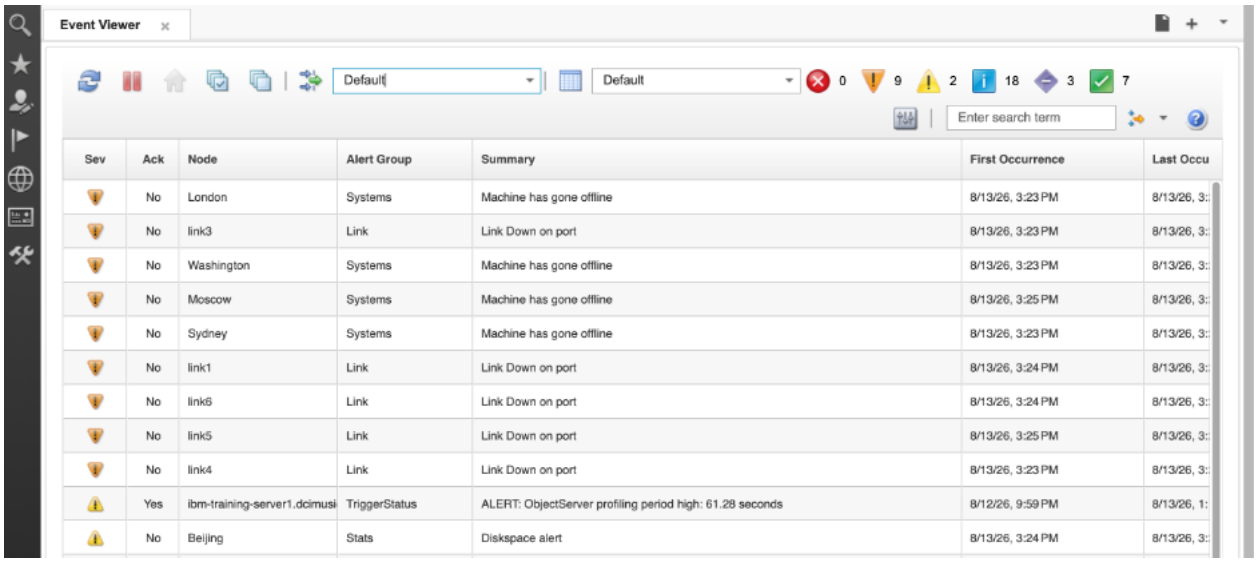


Figure 26: Simnet events arriving in event list

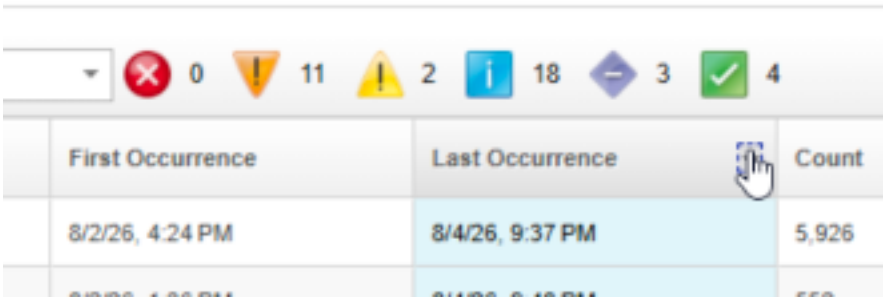


Figure 27: Sort by LastOccurrence

Validation The exercise is successful when all of the following are true:

- You are able to log in to the Web GUI and open the event list.
- A selected event can be changed to **Critical** by using the event tools.
- A selected event can be acknowledged in the Web GUI event list.
- The event details display shows the updated values.
- A journal entry can be added to an alert and displays the user, timestamp, and entry text.
- The **Escalated** filter is empty before escalation and shows the selected event after escalation.
- The **Self-Monitoring** view changes the displayed event columns.
- Starting `nco_p_simnet` causes new events to appear in the Web GUI event list.
- The student can identify **FirstOccurrence**, **LastOccurrence**, and **Count** as indicators of repeated occurrences of the same event.

Expected result By completing this exercise, you now understand how to effectively work with events in the Netcool/OMNIbus Web GUI. You have gained hands-on experience with:

- **Event inspection and modification**: viewing detailed event properties and updating event severity to prioritize alerts based on operational impact
- **Event management**: acknowledging alerts to mark ownership and adding journal entries to track investigation progress and actions
- **Event filtering and views**: using saved filters to focus on specific subsets of events and switching between different view layouts to highlight relevant information for different operational contexts
- **Event deduplication**: observing how the FirstOccurrence, LastOccurrence, and Count fields track event recurrence and consolidation, preventing alert fatigue from repeated occurrences of the same condition
- **Real-time event monitoring**: watching synthetic events arrive in real-time from the Simnet probe and understanding how new events integrate into the existing event list

You now have the practical foundation needed to manage alert traffic in a real Netcool/OMNIbus environment and are ready to explore the ObjectServer database architecture and event automation rules in the subsequent exercises.

3. The OMNIbus ObjectServer database

The OMNIbus ObjectServer database is the core data store of the OMNIbus environment. It is an in-memory relational database that receives alert information from probes and gateways, stores and manages that data in database tables, and makes it available to operators, client tools, and automation.

The database is made up of several categories of tables, including alerts tables, service tables, system catalog tables, statistics tables, security tables, and client tool support tables. In the sections that follow, the focus will be on the tables and fields that are most important for everyday event management, especially the `alerts.status` table and the event data it contains.

3.1 The ObjectServer users table

Objective In this exercise, you will use Netcool/OMNIbus Administrator to examine ObjectServer users and create a new user account. You will then use the Web GUI to assign application roles to that user and verify that the new account can log in and interact with events by using the expected tools and views.

Background ObjectServer security is based on users, groups, and roles stored in the ObjectServer database. Users are assigned to groups, and those groups carry the permissions that determine what the user can see and do within the ObjectServer. OMNIbus supports multiple authentication models: user accounts can be managed directly inside the ObjectServer security tables, or authentication can be delegated to the operating system via PAM, or to an external directory such as LDAP.

In this lab, we are working with ObjectServer-authenticated users. By default, OMNIbus environments commonly include administrative accounts such as `ncoadmin` and `root`, with `root` functioning as the primary superuser account for ObjectServer administration in this lab environment.

Note that the `root` user referred to throughout this lab is the **ObjectServer** administrative account, not the Linux `root` system account. Similarly, `ncoadmin` is an ObjectServer account, not an OS account. Although the lab server may also have OS-level accounts with the same names, these are independent credentials managed by different systems. When logging in to `nco_config`, the event list, or the Web GUI, the credentials used are ObjectServer credentials unless stated otherwise.

Tasks

1. Launch the Netcool/OMNIbus Administration tool on the Windows workstation and log in to the EVENTS ObjectServer as `root` with password `netcool`.

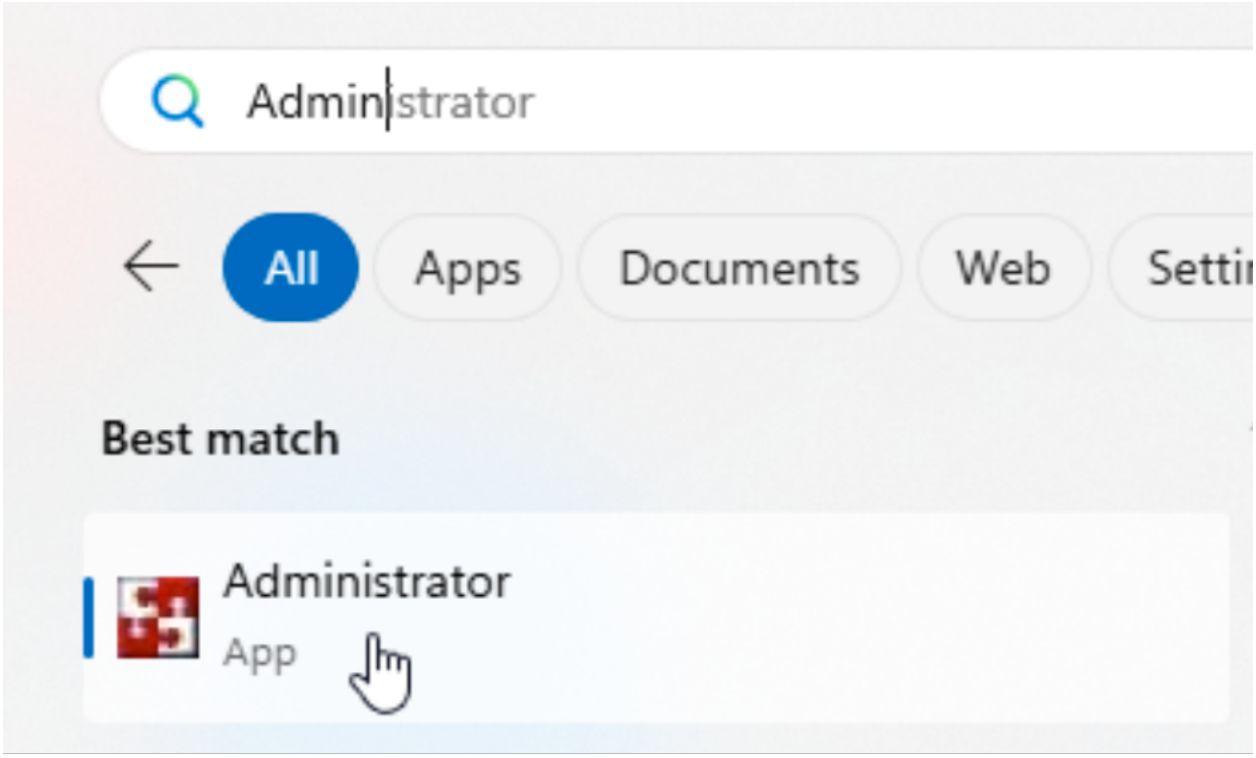


Figure 28: Netcool/OMNIbus Administrator Users window screenshot

2. In Netcool/OMNIbus Administrator, open the **User** window/drawer at the left, select **Users** icon, and review the existing users. Note that these users are stored in the ObjectServer security configuration data and are used to control access to the database and client tools.
3. Make note of the default `ncoadmin` user and the `root` user. In this lab environment, `root` is the primary superuser and main administrative user of the ObjectServer. The `ncoadmin` user is created upon installation of the WebGUI component.
4. Add a user. Right-click in the **Users** window and select **Add User**.
5. In the user details window, set:
   - **Username**: `netcool`
   - **Full name**: `Net Cool`
   - **User ID**: leave the default value, or set it explicitly if you need a specific numeric ID. The next available user ID is selected automatically when creating a user.

   The user details window also includes standard ObjectServer group options:

   - **System**: Has SuperUser role and unrestructed ObjectServer permissions. Do not assign to normal operators.
   - **Administrator**: grants full administrative access to ObjectServer configuration and management functions. Use only for OMNIbus administrators.
   - **Normal**: basic user access for standard event viewing and read-only ObjectServer operations.
   - **Probe**: rights required by probes to insert events into the ObjectServer. Not an interactive operator group.
   - **Gateway**: permissions needed by gateways to transfer and synchronize alert data between ObjectServers. Not an interactive operator group
   - **ISQLWrite**: allows command line tool (nco_sql) SQL write access to the ObjectServer for tools and scripts that modify tables.
   - **ISQL**: allows command line tool SQL read access for queries and inspection without write privileges.
   - **Netcool_OMNIbus_Admin**: broad Netcool/OMNIbus administrative permissions, typically used for users who manage the OMNIbus system.
   - **Netcool_OMNIbus_User**: standard operational access for Web GUI and event list users who need to view and manage alerts without full administrative rights.

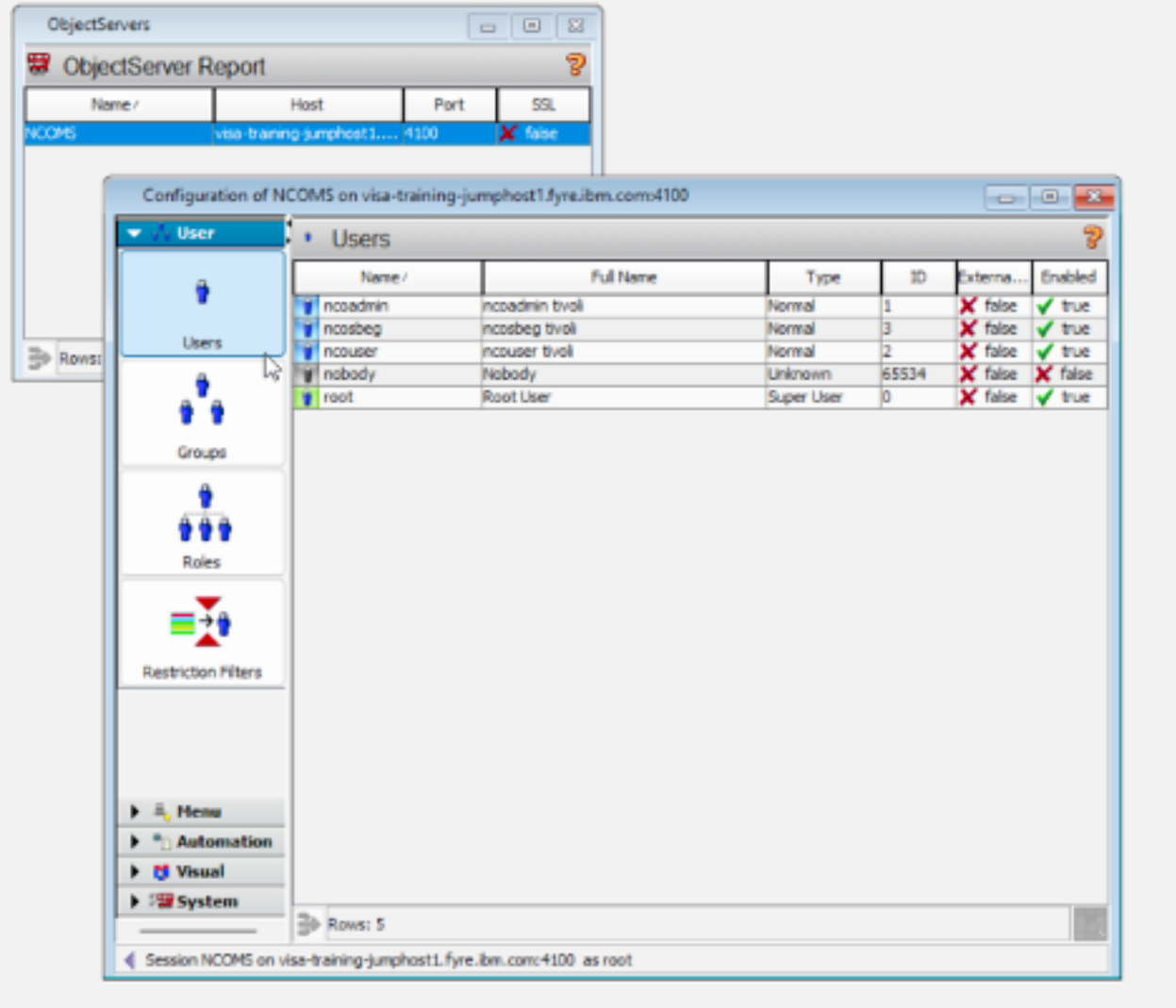


Figure 29: Netcool/OMNIbus Administrator Users window screenshot

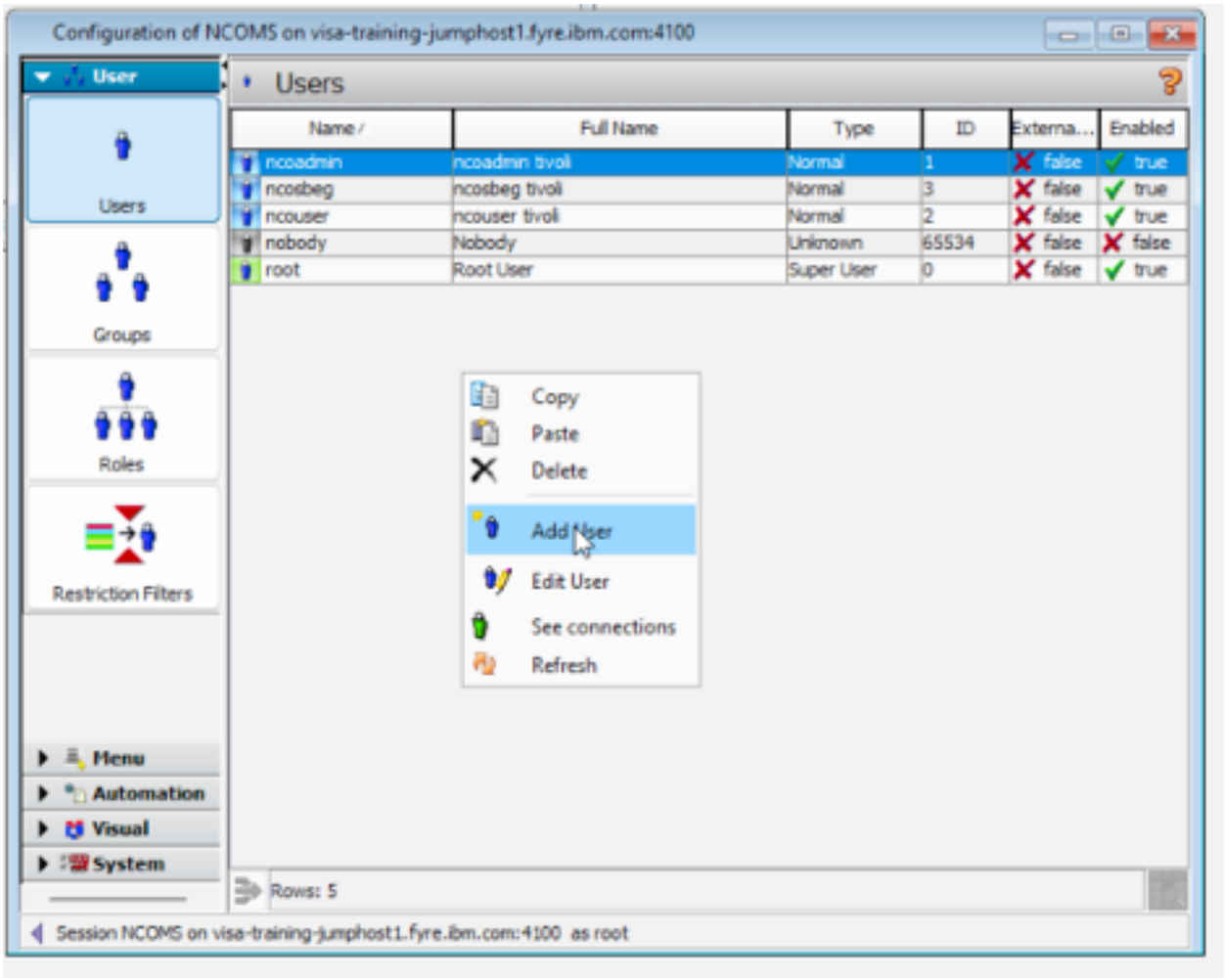


Figure 30: Add User in Netcool/OMNIbus Administrator

6. On the **Groups** tab, highlight `Netcool_OMNIbus_User` and move it to the **Assigned Groups** list by using the single right arrow button. Confirm that **Assigned Groups** now shows `Netcool_OMNIbus_User`.

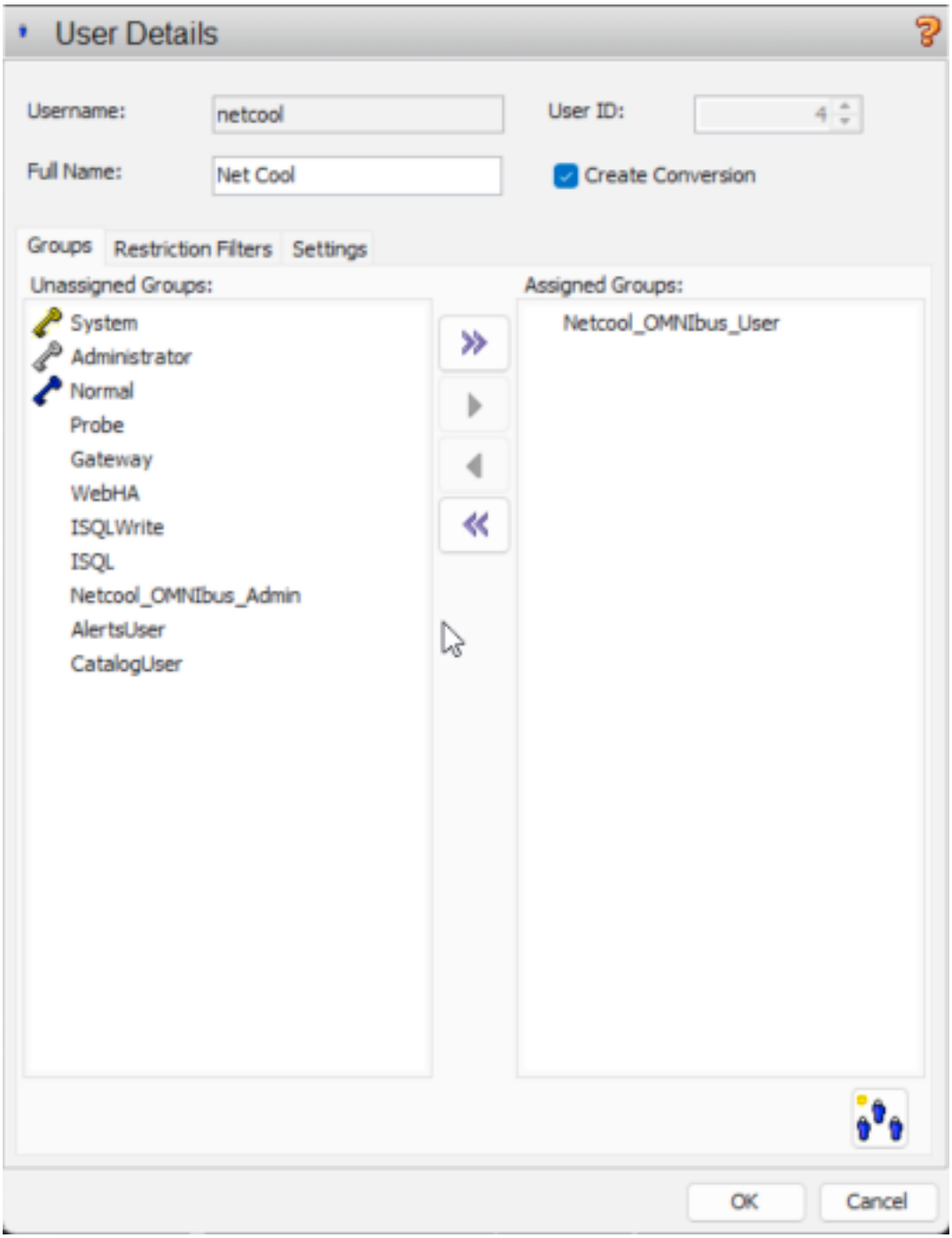


Figure 31: Netcool user group assignment dialog

7. Select the **Settings** tab and enter the password `netcool` in both the **Password** and **Verify** fields, then click **OK** to create the user. (Do **not** click the "Change" button as this will simply clear out the password you have entered.)

   Note: The **External Authentication** checkbox is used when the user account should authenticate through an external identity source such as PAM (Unix OS authentication) or LDAP. When enabled, the ObjectServer delegates authentication to the external system instead of validating the password against the internal ObjectServer user database.

8. We are now going to assign Web GUI application roles to the `netcool` user. These Web GUI roles are distinct from the ObjectServer group membership assigned in the previous steps — ObjectServer groups control database and event access, while Web GUI roles control what functionality the user can access inside the browser-based interface.

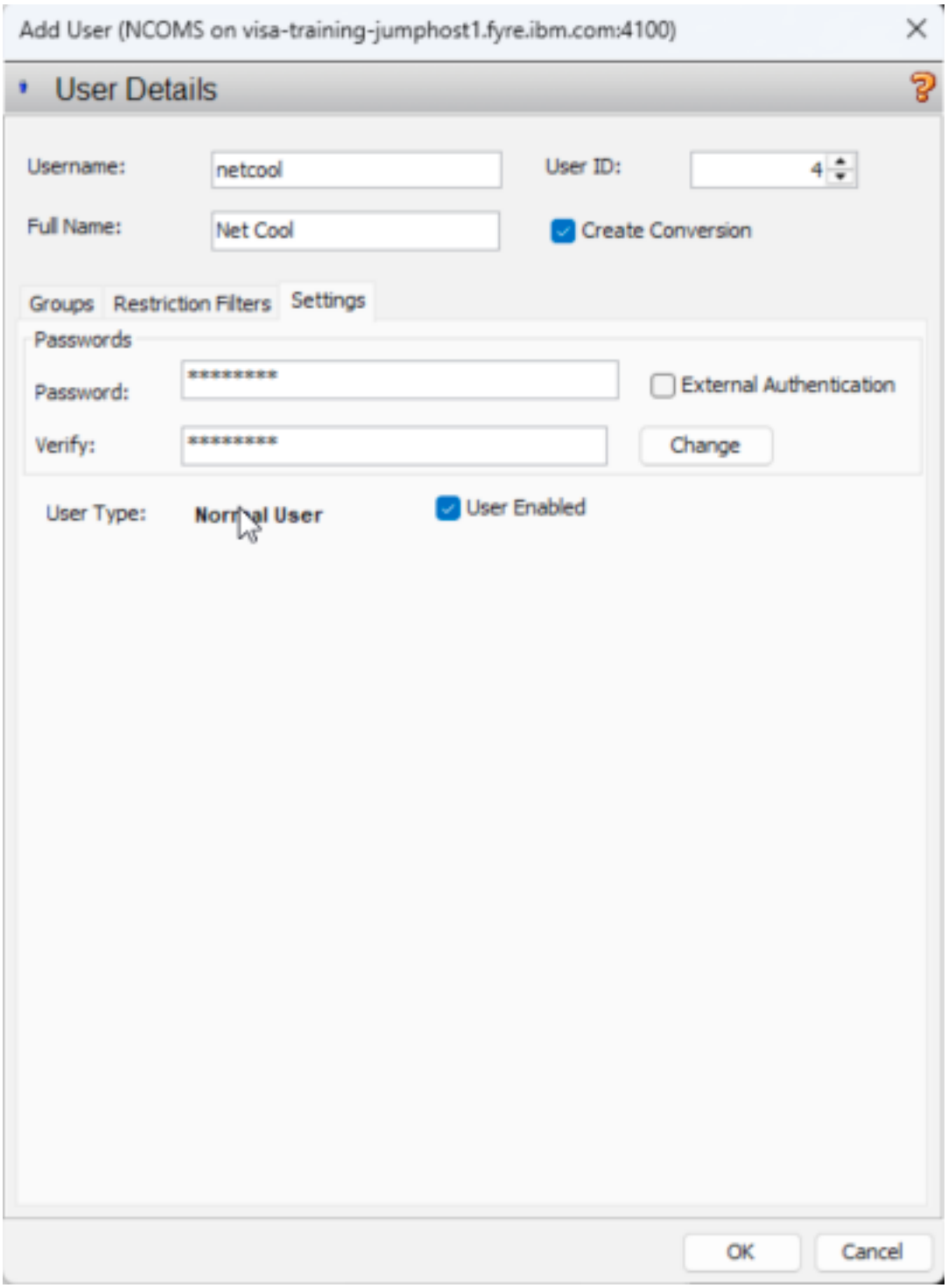


Figure 32: User password settings in Netcool/OMNIbus Administrator

Return to your browser and open the Web GUI at: `https://<your-server>:16311/ibm/console` If your session has timed out, log in again as `ncoadmin` with password `netcool`.

9. Click the gear icon in the lower-left corner and open **User Roles**.

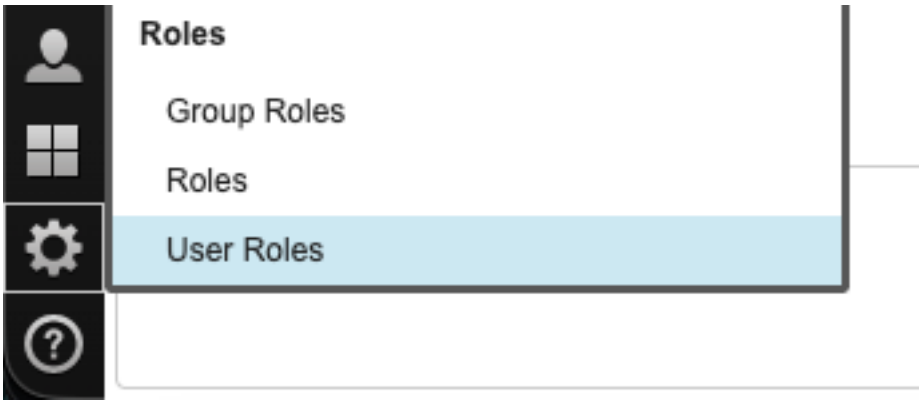


Figure 33: Open User Roles in Web GUI

11. Leave the existing entry boxes blank and click **Search** to search and display **all** users.

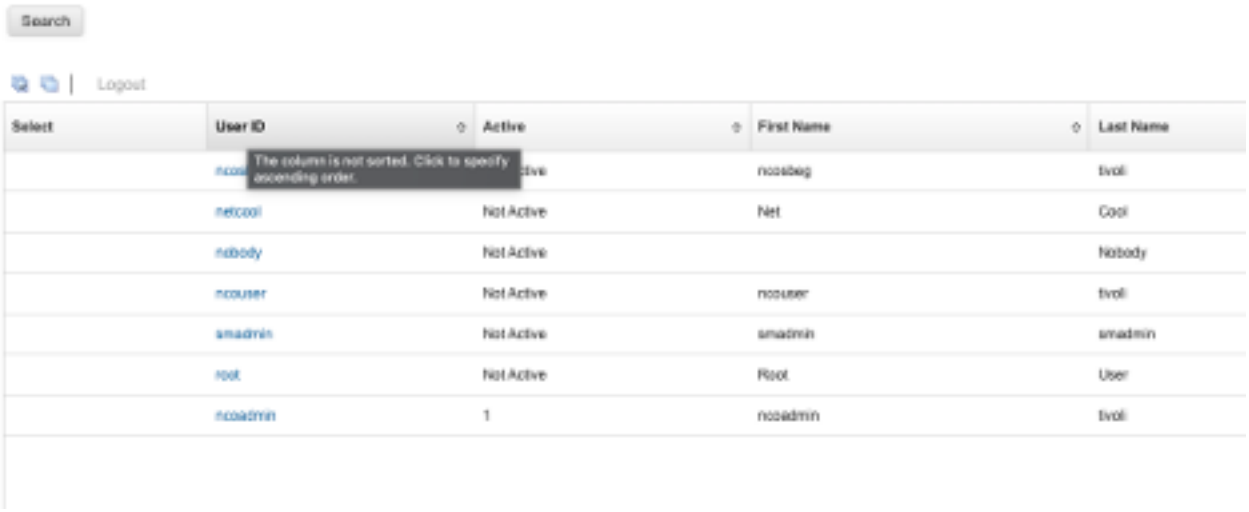


Figure 34: Search User Roles in Web GUI

Note: performing a search with an empty search string returns all ObjectServer users in the Web GUI search results. There are also Web GUI users stored in the federated WebSphere repository, such as `smadmin`, which is the WebSphere/JazzSM administrative account used for managing the application server and the JazzSM-based Web GUI environment. These users are not replicated down to the ObjectServer database, which is why you see more users here than you saw in the OMNIbus Administrator Client.

- `smadmin` is used to administer the underlying JazzSM/WebSphere stack that hosts Web GUI, including LDAP and SSO configuration, certificate and trust management, application/server tuning, profile and deployment maintenance, and other platform-level integration tasks.

12. Select the `netcool` user and review the available Web GUI roles.

Web GUI roles control what the user can do inside the Web GUI application: log in, view event lists, run event tools, edit dashboards, manage gauges, administer Web GUI settings, and so on. ObjectServer roles, conversely, control what the user can do in the ObjectServer database itself: read/write alert tables, manage users/groups/roles, create tools, modify desktop objects, run SQL, and administer server-side security.

How they relate:

- Web GUI roles are application-layer permissions. They decide whether the Web GUI shows menus, widgets, dashboards, event actions, and admin pages.

- ObjectServer roles are data/database permissions. They decide whether the backend account can read or update `alerts.status`, manipulate security objects, run SQL, or create tools and desktop artifacts.

Practical example:

- A typical read-only operator might have `ncw_user` to use the Web GUI and `netcool_ro` to view events but not change them.
- A backend ObjectServer administration account might have `CatalogUser` plus `SecurityAdmin` or `AlertsUser`, depending on what it needs to do in the database.

That means a user can appear to be an admin in the Web GUI while still having limited ObjectServer rights, or vice versa. The two role systems overlap in purpose, but they operate at different layers.

Short rule of thumb:

- **Web GUI roles** = UI and feature access.
- **ObjectServer roles** = backend data and security access.

Common default Web GUI roles include:

- **ncw_user** – Base Web GUI role: makes the account a Web GUI user. By itself it lets the user log in and see the UI, but you pair it with other roles to define access level.
- **netcool_ro** – Read-only event management: can view event lists and events, but cannot run tools or change event data. Must be combined with `ncw_user`.
- **netcool_rw** – Read/write event management: can run event list tools and modify event data (ack, clear, update fields, etc.). Also combined with `ncw_user`.
- **operator** – Typically used as an ObjectServer-side role (combined with CatalogUser/AlertsUser, etc.) to give basic operator permissions; in many deployments this is mapped in LDAP or user groups and then combined with the Web GUI roles above.

Other Web GUI roles include:

- **ncw_admin** – Web GUI admin: full admin access to Web GUI configuration (views, filters, tools, widgets) plus everything from `ncw_user`.
- **chartAdministrator / chartCreator / chartViewer** – Roles for managing and viewing chart/gauge resources and dashboards. Admin/Creator can define charts and gauges; Viewer can only see and use gauges.
- **ncw_dashboard_editor** – Allows editing of Event Dashboard layouts and widgets.
- **ncw_gauges_editor / ncw_gauges_viewer** – Editor can administer gauge definitions; Viewer can only see and use gauges.
- **ncw_sbeg_editor / ncw_sbeg_view** – Roles tied to scope-based event grouping configuration and viewing grouped events. Editor can manage grouping policies; View can see the results.
- **administrator** – Often a legacy or general admin role depending on version; many shops rely on `ncw_admin` instead and leave this unused.
- **monitor, iscadmns, samples, suppressmonitor** – Environment-specific or sample/demo roles; IBM ships some for example dashboards or integrations, and many teams add custom ones.

13. Scroll down, locate, and assign the following Web GUI roles to the user, then confirm the selections using the screenshot below:

    - `ncw_user`
    - `netcool_rw`
    - `operator`

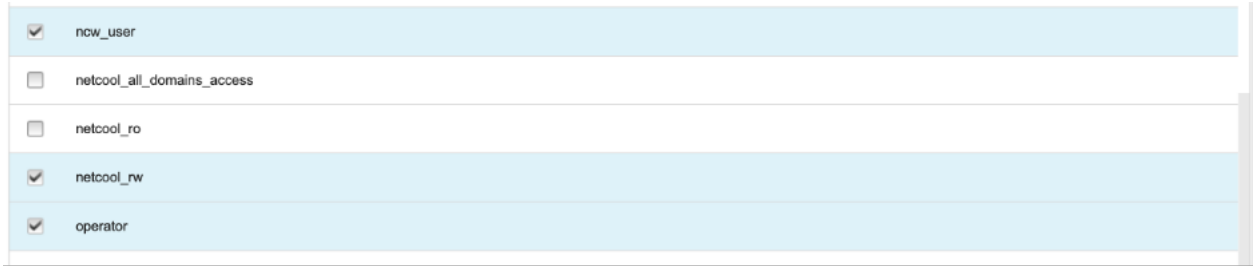


Figure 35: Assign Web GUI roles to the netcool user

14. Click **Save** at the bottom-left of the window to apply the Web GUI role assignments.

15. Log out of the Web GUI:
    - Find the **User** menu icon in the lower-left corner of the Web GUI.
    - Click the **User** menu icon and select **Logout** from the popup menu.

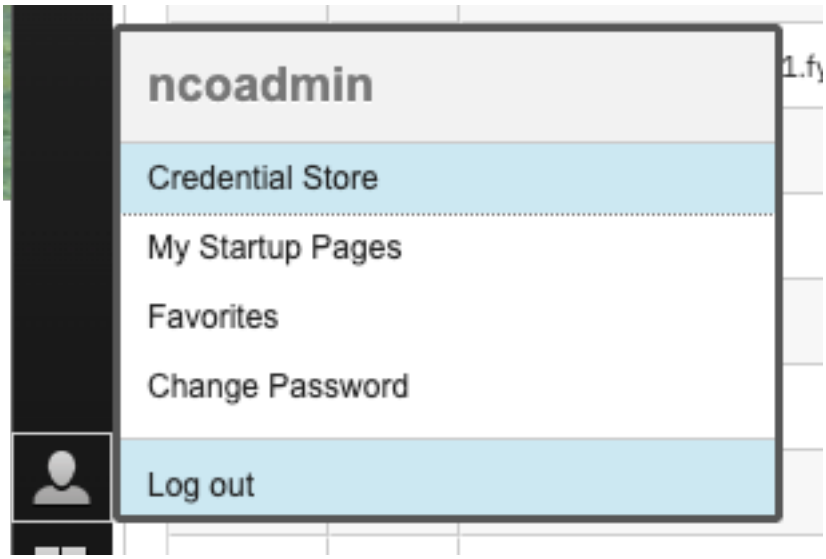


Figure 36: Web GUI logout via User menu

16. Log back in as user `netcool` with password `netcool`.
17. As before, click the flag icon to open the event list and verify that the user can access events, filters, views, and right-click tools.
18. Log out of the Web GUI by using the logout control in the lower-left corner, then log back in again as the `netcool` user to confirm that access persists after a fresh login.
19. Users can also be added and managed from the command line by using the nco_sql utility. This utility provides an interactive SQL interface to the OMNIbus ObjectServer, allowing administrators to query and modify ObjectServer data and configuration, including users, groups, roles, and permissions. The nco_sql utility can also be used for automating object server administrative and query tasks in scripts as well.

    Open a terminal on the workstation or lab server and connect to the EVENTS ObjectServer as the root administrator account:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool
```

20. At the `nco_sql` prompt (**1>**), inspect the ObjectServer security tables to confirm their structure and column names:

```
DESCRIBE security.users;
go
```

    The table **security.users** contains the relevant user configuration data such as `UserName`, encrypted `Password`, and other relevant fields we saw when configuring the user with the OMNIbus Administrator client.

```
select * from security.users where UserName='netcool';
go
```

    Here you can see the full relevant user details for the user we created in the Administrator client.

```
DESCRIBE security.groups;
go
```

    The **security.groups** table contains the relevant group information including `GroupID`, `GroupName`, whether or not the group is considered a `SystemGroup`, and other information.

```
select * from security.groups;
go
```

Here you can see the actual group records currently defined in the EVENTS ObjectServer. The result contains 13 groups. The first nine are standard OMNIbus groups created when the ObjectServer database is initialized, while the remaining groups—such as Netcool_OMNIbus_User, Netcool_OMNIbus_Admin, Network_Manager_IP_Admin, and Network_Manager_User—were added by installed products, integrations, or local configuration. IBM documents the standard groups, including Normal, Probe, Gateway, WebHA, ISQLWrite, and ISQL.

It is important to note that this table does not show which users belong to each group or what permissions the group grants. Membership is stored in the security.group_members table, which maps UserID values to GroupID values. IBM identifies security.groups as the table containing the group name and ID, and security.group_members as the mapping table between users and groups.

21. Create a new user `jdoe` and make him a User as well as an Administrator by assigning the two relevant ObjectServer groups:

    Create the user: `sql  CREATE USER 'jdoe' FULL NAME 'John Doe' PASSWORD 'SecretPass123';  go`

    Note: When you execute this command, you will see a response indicating `(0 rows affected)`. This is expected and does not indicate that the command failed. Commands like `CREATE USER` and `ALTER GROUP` modify the ObjectServer configuration data but do not return rows from a table query. The "(0 rows affected)" message simply indicates that no rows were returned from a SELECT operation, which is normal for data modification commands. If the command were unsuccessful, you would instead receive an error message describing what went wrong, such as a duplicate user name or an invalid group name. As long as you do not see an error message, the command was executed successfully.

    Add `jdoe` to the `Netcool_OMNIbus_User` group:

    ```
    ALTER GROUP 'Netcool_OMNIbus_User' ASSIGN MEMBERS 'jdoe';
    go
    ```

    And finally, add him to the `Netcool_OMNIbus_Admin` group:

    ```
    ALTER GROUP 'Netcool_OMNIbus_Admin' ASSIGN MEMBERS 'jdoe';
    go
    ```

22. Verify that the new user was created successfully, and validate group membership. Start by checking the *security.users* table:

    ```
    SELECT * FROM security.users WHERE UserName = 'jdoe';
    go
    ```

    Review the user data that was returned from your SELECT statement.

    Next, query the security.group_members table to check membership. This query checks membership across three related security tables. It first locates the GroupID associated with the Netcool_OMNIbus_Admin group in security.groups. It then uses that group ID to find the user IDs assigned to the group in security.group_members. Finally, it checks whether the user named jdoe has a matching user ID in the security.users table.

    ```
    SELECT UserName FROM security.users WHERE UserName = 'jdoe' AND UserID IN (SELECT UserID
     ↪  FROM security.group_members WHERE GroupID IN (SELECT GroupID FROM security.groups
     ↪  WHERE GroupName = 'Netcool_OMNIbus_Admin'));
    go
    ```

    If jdoe is a member of the group, the query returns the username:

    ```
    UserName
    --------
    jdoe
    ```

Validation The exercise is successful when all of the following are true:

- You can view existing users in Netcool/OMNIbus Administrator.
- The `netcool` user is created successfully.
- The `netcool` user is assigned to the `Netcool_OMNIbus_User` group.
- The `netcool` user is found in the Web GUI **User Roles** interface.
- The Web GUI roles are saved successfully for the `netcool` user.
- The `netcool` user can log in to the Web GUI and open the event list.
- The `netcool` user has access to filters, views, and right-click event tools.
- The `jdoe` user has been created using `nco_sql` command line utility, and group membership for the user assigned

Expected result At the end of this exercise, you understand that ObjectServer access is controlled through users, groups, and roles, and that Web GUI capabilities are controlled through separate Web GUI role assignments. You also have a working non-administrative user account that can be used in later labs to validate operator-level access and event interaction.

3.2 The alerts.status table and important event fields

Objective In this exercise, you will examine the `alerts.status` table in the ObjectServer, review the most important event fields used in everyday OMNIbus event handling, and observe how the table can be extended with custom fields. By the end of this exercise, you will understand where active alert data is stored, what key fields mean, and how additional columns can be added when custom event data must be provided.

Prerequisites

- Completion of section 3.1.
- The ObjectServer is running.
- Netcool/OMNIbus Administrator is available on Windows (or `nco_config` on Unix/Linux with X-Windows).

Background The `alerts.status` table is the primary active-alert table in the ObjectServer and the central focus of this exercise. It is one of several tables that make up the `alerts` database. The other key tables in that database are `alerts.details`, which stores supplementary event text that does not fit in the main alert row; `alerts.journal`, which holds operator and automated journal entries associated with individual alerts; and `alerts.col`, which is used to record parent-child relationships between alerts for correlation purposes.

Because the ObjectServer is an in-memory database, event data in tables such as `alerts.status` can be inserted, updated, deduplicated, correlated, and queried with very low latency. This design is particularly well suited to large IT and service management environments, where probes and gateways may deliver large volumes of alerts and operators need the event list to update immediately. The ObjectServer maintains its active data in memory for performance, but it also persists the database to disk through checkpoint files and transaction logs. When an ObjectServer restarts, it reloads the most recent checkpoint and replays subsequent transaction-log entries to restore the database to its most recent consistent state.

Each row in the `alerts.status` table represents one active alert. The schema is defined at startup and is consistent across all alerts stored in the table. Administrators can extend the schema by adding custom columns to add additional data to the alerts, such as business context. This exercise focuses on the fields most commonly used to identify the event source, summarize the condition, categorize the event, and support deduplication and event handling.

Tasks

1. Launch Netcool/OMNIbus Administrator on the Windows workstation and log in to the EVENTS ObjectServer as `root` with password `netcool`.
2. In Netcool/OMNIbus Administrator, open the **System** menu drawer on the left, and select the **Databases** icon. In the database list, open the **alerts** database by clicking on the “plus” sign (+), then select the **status** table to open the definition of the `alerts.status` table:
3. Review the columns that make up the active alert schema. For each column, note the column name, data type, and defined length or precision. The data type determines what kind of values the column can store, such as text, numeric, or timestamp values. The length or precision defines the storage limit for that field. Most data types have a fixed size, while `VarChar` is variable-length and must be defined with a maximum size up front.

   The following fields are some of the most important in the ObjectServer event schema because they are commonly used for identification, grouping, deduplication, and operational handling. Review the purpose of each field as part of the table definition:

   - **Node**: Identifies the host, device, or source associated with the event.
   - **Summary**: Provides a short text description of the event.
   - **AlertGroup**: Identifies the general event category or class.
   - **AlertKey**: Typically identifies the specific resource, sub-component, or object associated with the event (interface, disk, cpu, etc)
   - **Severity**: Indicates the event severity level and supports operational prioritization.
   - **Type**: Identifies the event type and is commonly used in event handling and correlation logic.
   - **Identifier**: Uniquely identifies the event condition and is used for deduplication behavior.

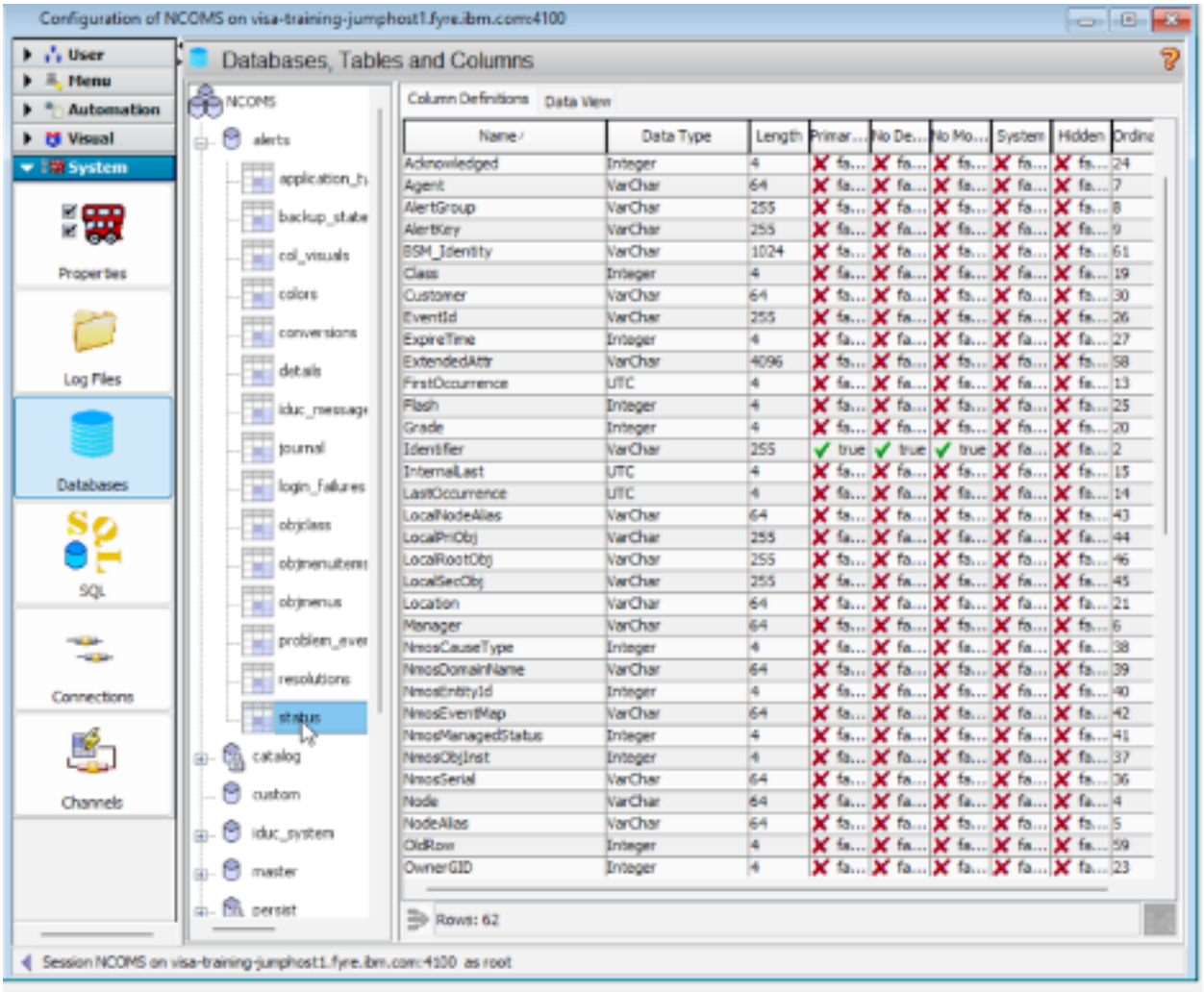


Figure 37: alerts.status table definition screenshot

- **Server**: Assigned and maintained by the ObjectServer to identify the source or server context of the record; it is not typically set by the user.
- **ServerSerial**: Assigned and maintained by the ObjectServer as the unique record identifier within that server; it is not typically set by the user.

4. Next, return to the Web GUI event list and compare the visible event columns with the field names observed in the `alerts.status` table. As noted earlier, the event list view typically shows only a subset of all available alert fields. Select an alert and **double-click** to open the full event details and confirm that values displayed in the event list correspond to fields stored in the `alerts.status` table, such as `Node`, `Summary`, and `Severity`.

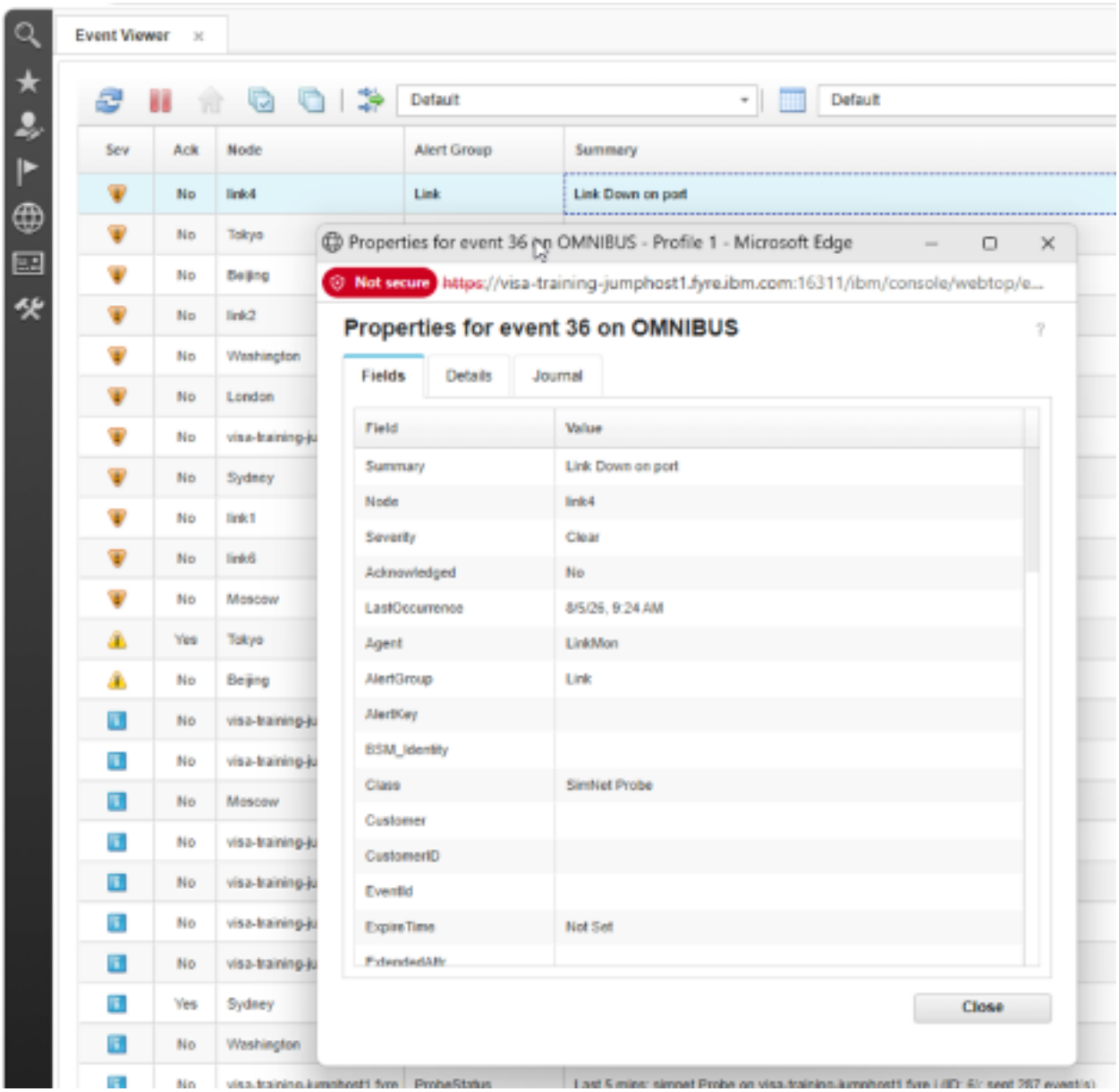


Figure 38: Event list fields and full alert details

5. Next, we will add a custom field to the alert schema. We will use this field in upcoming tasks to add business context to our alerts.

   Return to the Netcool/OMNIbus Administrator client, **Column Definitions** view. Add the new field by right-clicking in the **Column Definitions** data view and selecting **Add Column**.

   Give the new column the name `CustomerID`, set the size of the text to 64, and click **OK**.

   > **Note:** ObjectServer column names are **case-sensitive**. When writing queries or update statements, you must use the exact column-name capitalization defined in the table schema.

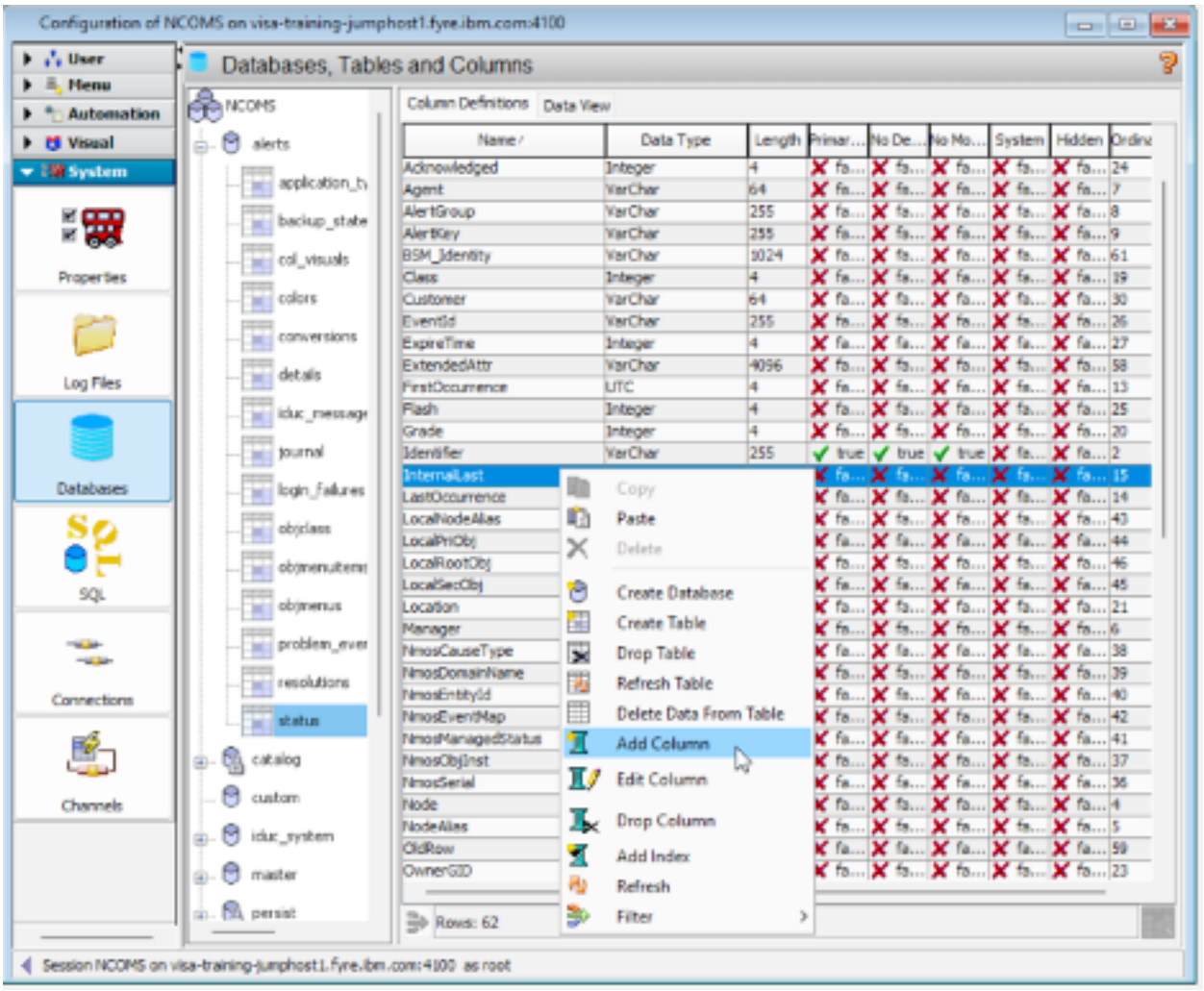


Figure 39: Add custom CustomerID field to Column Definitions

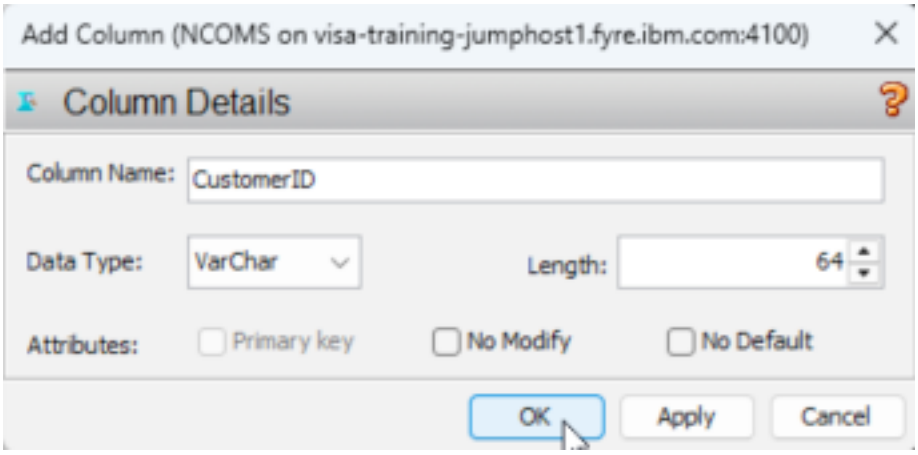


Figure 40: CustomerID column size and properties

6. Validate that the new `CustomerID` column appears in the **Column Definitions** view of the Administrator client:

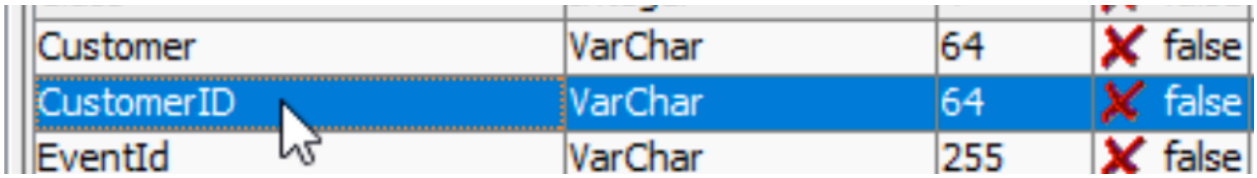


Figure 41: CustomerID column size and properties

7. Open the Web GUI event list and double-click on an alert to bring up the alert properties. Note that the new `CustomerID` field is now available in the event data shown by Web GUI. You may need to click the "refresh" icon at the top left of the event viewer for the new field to show.
8. As with adding users, you can also use the command line equivalent tool to add custom colums. Open a terminal on your Netcool/OMNIbus lab server and run the following command:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool
```

At the `nco_sql` prompt, run the following commands to describe the `alerts.status` schema to inspect the current the column definitions and their data types:

```
DESCRIBE alerts.status;
go
```

Issue the following command to add another custom column called `CircuitID` to the alerts.status schema. It will also be a variable character length data type with a maximum length of 64 characters (`varchar(64)`): `sql ALTER TABLE alerts.status ADD COLUMN CircuitID varchar(64); go` Finally, check the `alerts.status` schema to ensure your new field exists: `sql DESCRIBE alerts.status; go`

Note: Custom fields in `alerts.status` are commonly used in production environments to carry business-specific or integration-specific data that has no equivalent in the default schema. A telecommunications provider might add a `CircuitID` field to tie alerts directly to provisioned network circuits. A financial services organization might add a `CostCenter` or `ApplicationOwner` field to support automatic ticket routing. Managed service providers often add a `CustomerID` or `TenantID` field so that a shared ObjectServer can store alerts from multiple clients while keeping event ownership clear. In each case, the custom field is populated by the probe rules file or a gateway at ingestion time and flows through the ObjectServer alongside the standard alert data. We will use the CustomerID field to enrich events in upcoming labs.

Validation The exercise is successful when all of the following are true:

- You can navigate through **System -> Databases -> alerts -> status** and open the `alerts.status` table definition.
- You can identify `Node`, `Summary`, `AlertGroup`, `AlertKey`, `Severity`, `Type`, and `Identifier` in the table.
- You can explain the operational purpose of the key event fields.
- You can correlate event-list columns with fields in the `alerts.status` table.
- You can add a custom field to `alerts.status` by using Netcool/OMNIbus Administrator.
- You can confirm that the new `CustomerID` column appears in the **Column Definitions** view.
- You can confirm that the new `CustomerID` field is available in the Web GUI event data or event list configuration.

Expected result At the end of this exercise, you understand that the `alerts.status` table is the central active-alert table in the ObjectServer and that its fields define how OMNIbus stores and presents event information. You also understand that the ObjectServer schema can be extended with custom columns when additional event attributes are required.

3.3 Reviewing ObjectServer triggers

Objective In this exercise, you will review standard ObjectServer triggers in Netcool/OMNIbus Administrator and examine how they support deduplication, event clearing, and event expiration. By the end of this exercise, you will understand how triggers are grouped, where to inspect their configuration, and what the `deduplication`, `generic_clear`, and `expire` triggers do in a default OMNIbus environment.

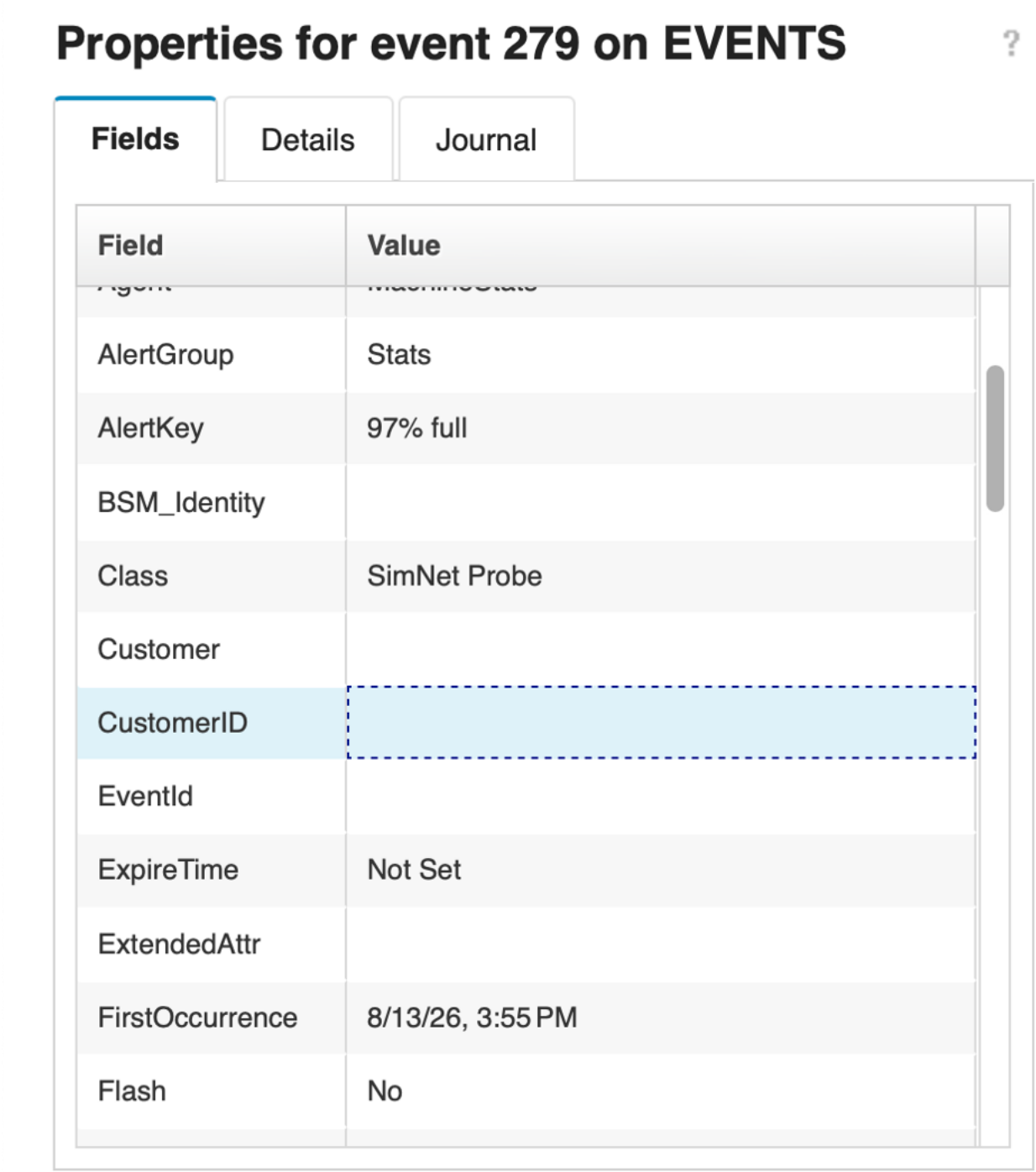


Figure 42: CustomerID column size and properties

Prerequisites

- Completion of section 3.2.
- The ObjectServer is running.
- `Netcool/OMNIbus Administrator` is accessible on the Windows workstation.

Background The ObjectServer automation framework is built around triggers, which are units of SQL logic that run automatically in response to defined conditions. There are three types of triggers in OMNIbus: database triggers, temporal triggers, and signal triggers.

Database triggers are bound to a specific table and a specific database action. The supported actions are insert, update, reinsert, and delete. A database trigger can be configured to run either before or after the database modification takes place, giving it the ability to inspect, modify, or cancel the operation depending on the trigger logic. The `deduplication` trigger is an example of a database trigger — it is bound to the `alerts.status` table and fires as a pre-action on reinsert, meaning it runs before the ObjectServer processes an incoming event that matches an existing alert key (@Identifier).

Temporal triggers run on a repeating schedule rather than in response to a database event. The interval between executions is defined as part of the trigger configuration. Temporal triggers are typically used for housekeeping tasks such as scanning the alert table for conditions that need to be acted on periodically. The `generic_clear` trigger is an example — it runs on a schedule and looks for resolution events that can be matched against open problem events in order to clear both.

Signal triggers run when a named signal is raised within the ObjectServer, either by another trigger, a procedure, or an external process. They are used to coordinate automation across components or to respond to custom-defined events that do not map naturally to a database action or a schedule.

Trigger groups are used to organize related triggers so they can be enabled or disabled together as a single unit. This makes it straightforward to activate or deactivate a set of related automation behaviors without having to manage each trigger individually.

Tasks

1. Launch `Netcool/OMNIbus Administrator` on the Windows workstation and log in to the EVENTS ObjectServer as `root` with password `netcool`.
2. In Netcool/OMNIbus Administrator, open the **Automation** drawer and select **Triggers**:
3. Review the list of triggers that are provided out of the box. Note that many of these are internal ObjectServer maintenance or automation triggers.
4. Observe that the trigger list includes both the trigger name and the trigger group. Trigger groups are used to organize related triggers so they can be enabled or disabled together as one unit. The list also shows other key configuration details for each trigger, including the trigger kind (such as Temporal, Signal, or Database), the trigger priority, and whether debug mode is active. Finally, the Enabled and Group Enabled columns indicate if the individual trigger is currently running and whether its trigger group is enabled, which together determine if the trigger actually executes.
5. Locate the `deduplication` trigger and double-click it to open its configuration. Note the tabs in the trigger configuration dialog: **Settings / When / Action / Comment**.
6. On the **Settings** tab, note that the trigger is configured against the `alerts.status` table as a **pre-database action** on `reinsert`.

   When a probe sends an event to the ObjectServer, the ObjectServer checks whether an alert with the same value for the `Identifier` field already exists in `alerts.status`.

   - If no matching alert is found, the event is treated as a new `insert`.
   - If a matching alert already exists, the event is treated as a `reinsert`, which means the event is another occurrence of an existing condition rather than a brand-new alert.

   The deduplication trigger fires before the ObjectServer processes the reinsert, allowing it to update the existing alert appropriately. Deduplication of alerts ensure that the event list is not cluttered with duplicate and redundant events, and helps keep operators focused on the current state of the environment.
7. Select the **When** tab and note that the field is empty. The **When** clause is an optional SQL condition that can be added to a trigger to restrict when it fires. If a When clause is present, the trigger will only execute when both the trigger condition and the When clause evaluate to true. An empty When clause means the trigger fires unconditionally whenever its defined database action or schedule occurs.

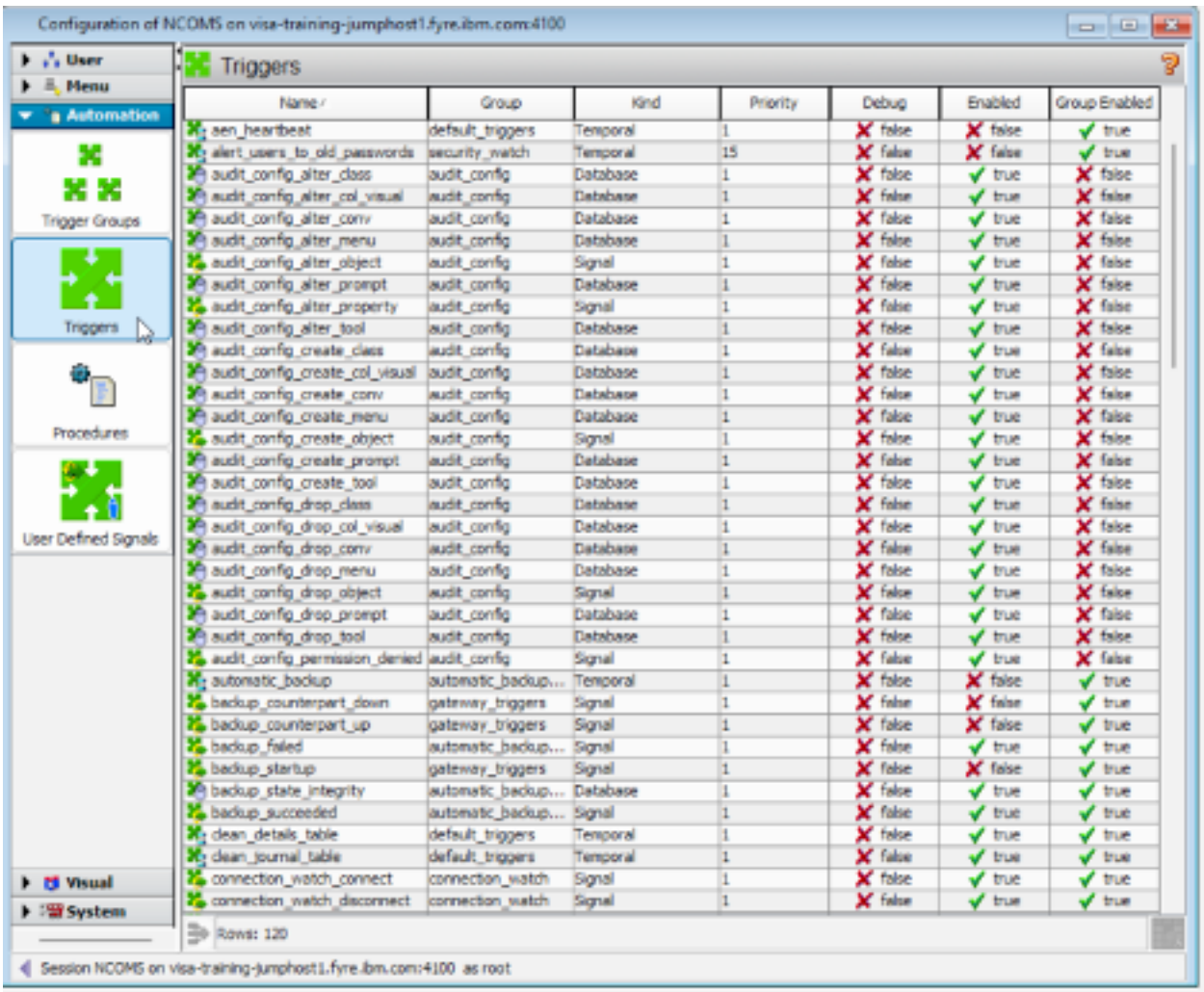


Figure 43: Netcool/OMNIbus Administrator triggers list screenshot

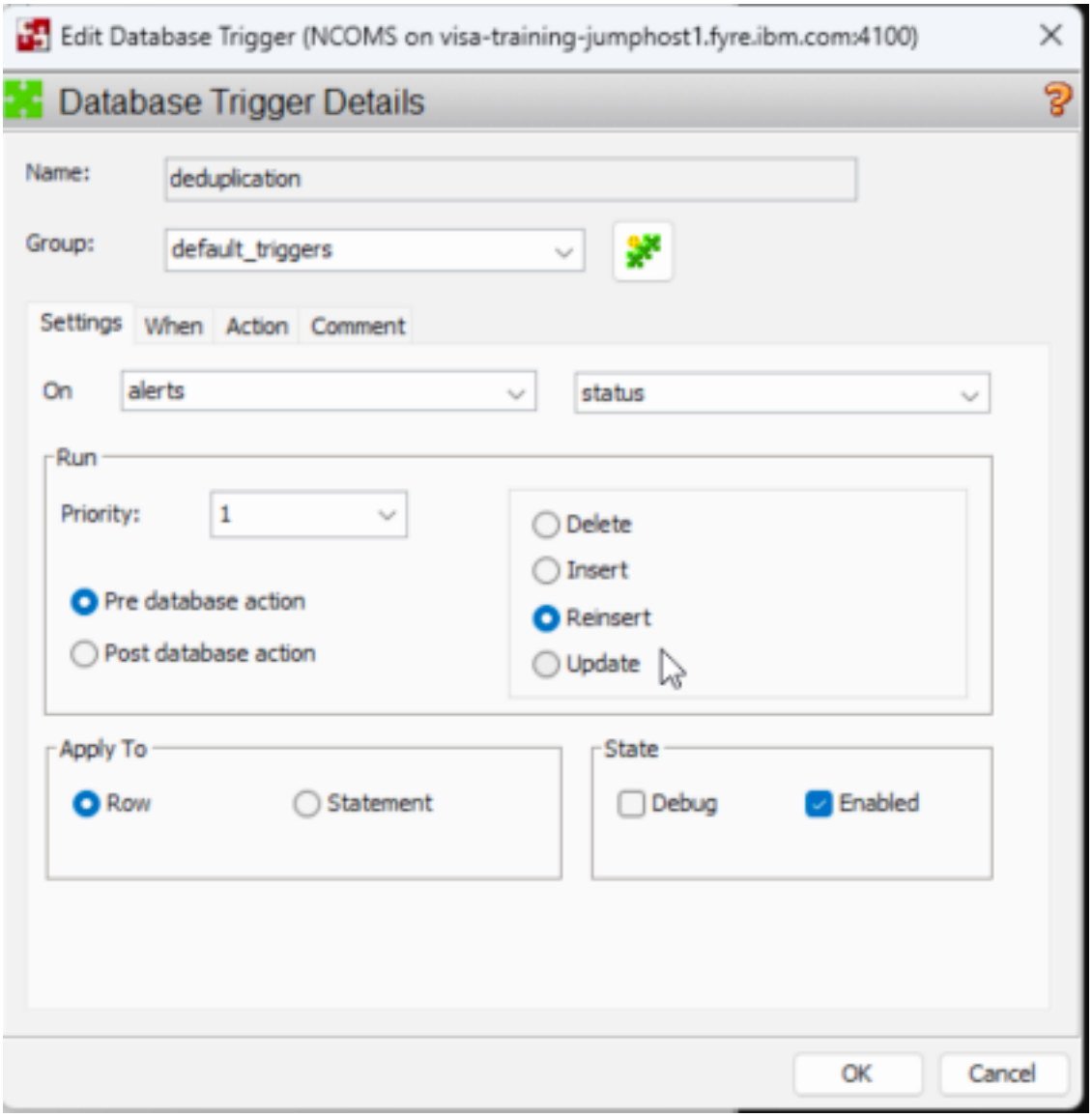


Figure 44: Open deduplication trigger configuration

8. Select the **Action** tab and review the SQL trigger code. This is the SQL code that is executed when the criter of this trigger is met (re-insert of an existing event):

   At the top of the trigger, there is a check on the application name:

```
if (%user_app_name = 'PROBE')
then
    if ( (old.LastOccurrence >= new.LastOccurrence) or
         ((old.ProbeSubSecondId >= new.ProbeSubSecondId) and
          (old.LastOccurrence = new.LastOccurrence)) )
    then
        cancel;
    end if;
end if;
```

   - This logic only applies when the event comes from a **probe**.
   - It compares the **existing** alert's time (`old.LastOccurrence` and `old.ProbeSubSecondId`) with the **incoming** event's time (`new.LastOccurrence` and `new.ProbeSubSecondId`).
   - If the existing alert is **newer or equal** to the incoming event, the trigger calls `cancel;`, which stops the reinsert from updating the alert. In other words, older or duplicate events from the probe are ignored. (rare)

   Next, the trigger records the current time, assigning it to the variable `now`, and updates key fields on the existing alert:

```
set now = getdate();
set old.Tally = old.Tally + 1;

set old.StateChange   = now;
set old.InternalLast  = now;
set old.Summary       = new.Summary;
set old.AlertKey      = new.AlertKey;
set old.ProbeSubSecondId = new.ProbeSubSecondId;
```

   - `old.Tally` is incremented to indicate the existing alert has been seen again.
   - `StateChange` and `InternalLast` fields are refreshed to the current time.
   - Text and identification fields such as `Summary` and `AlertKey` are updated from the incoming event.
   - `ProbeSubSecondId` is updated so future comparisons use the latest precise timestamp.

   The trigger then updates the `LastOccurrence` time based on the incoming event:

```
if (new.LastOccurrence = 0)
then
    set old.LastOccurrence = now;
else
    set old.LastOccurrence = new.LastOccurrence;
end if;
```

   - If the incoming event doesn't provide a `LastOccurrence` time (zero), the trigger uses the current time.
   - Otherwise, it copies the event's `LastOccurrence` into the alert.

     Note: in general, the time of an event is not set or managed by the source or the rules, but rather the time that the event was received in the ObjectServer is used. However, there are cases where you may want or need to reflect the time that the source generated the alert, rather than when the ObjectServer eventually received it. In these cases you can explicitly set LastOccurrence in the probe rules (described in the next section). Otherwise, if it is not explicitly set, it will be '0'.

   Finally, the trigger updates the severity if it has changed:

```
if ((old.Severity = 0) and (new.Severity > 0))
then
    set old.Severity = new.Severity;
end if;
```

   - If the existing alert has a severity of 0 and the incoming event has a non-zero severity, the alert's severity is raised to match the new event.

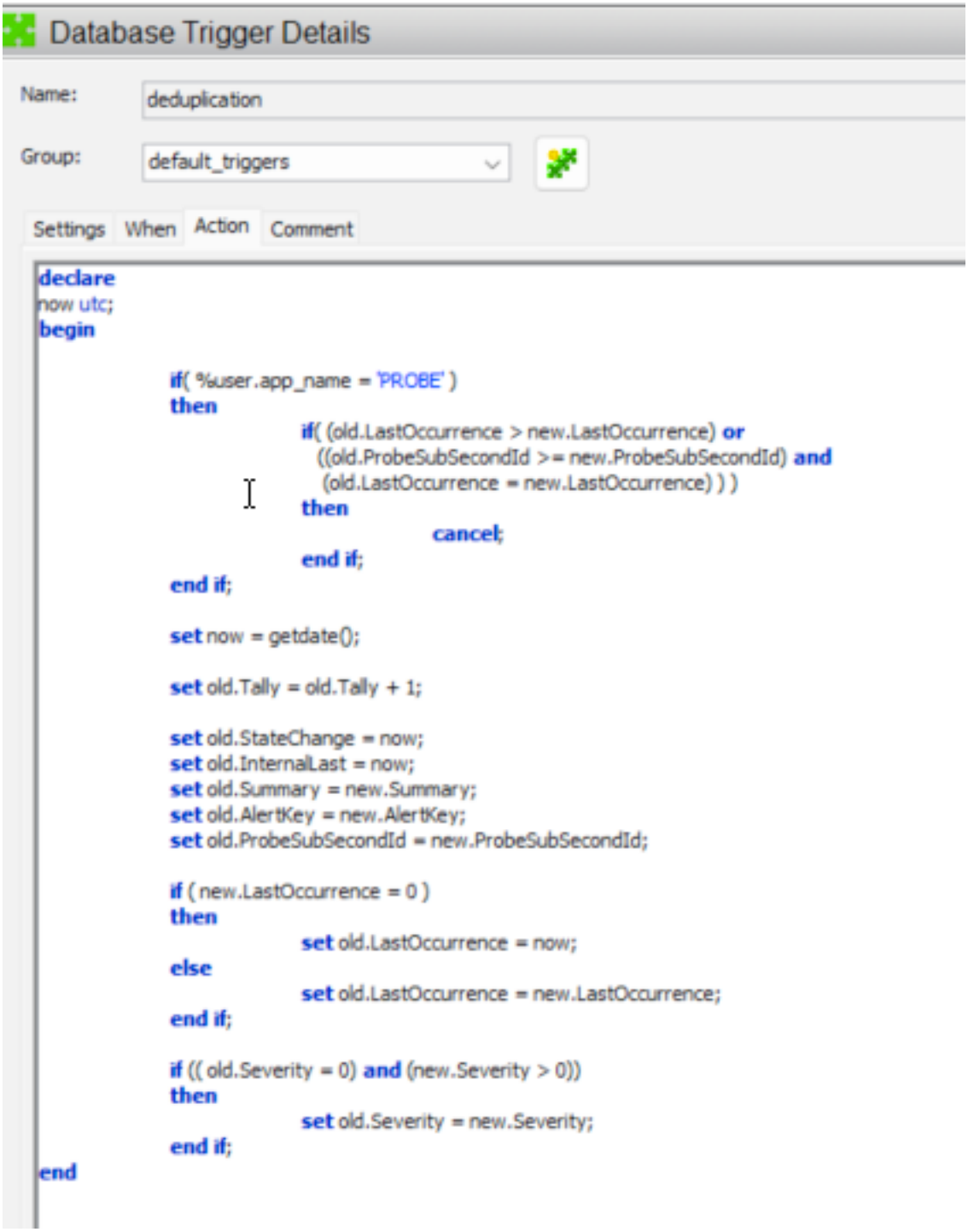


Figure 45: Deduplication trigger action tab

`old.<column>` vs `new.<column>`

In this trigger:

- `old.<column>` refers to the values already stored in the alert row **before** the reinsert is processed.
- `new.<column>` refers to the values from the **incoming event** that caused the reinsert.

The trigger uses `old.` values to read and update the existing alert, and uses `new.` values to bring in information from the latest event. This distinction is important because it lets the ObjectServer compare the current alert state to the new event and decide whether to ignore the event, update a few fields, or change severity, all without creating a new row.

9. Close the `deduplication` trigger, locate the `generic_clear` trigger, and review its configuration on the **Settings** tab. Note that it is a **temporal** trigger and that its repeat interval is set to 5 seconds, meaning the ObjectServer runs this trigger continuously every 5 seconds as long as the ObjectServer is running.
10. Select the **Action** tab and review the trigger code.

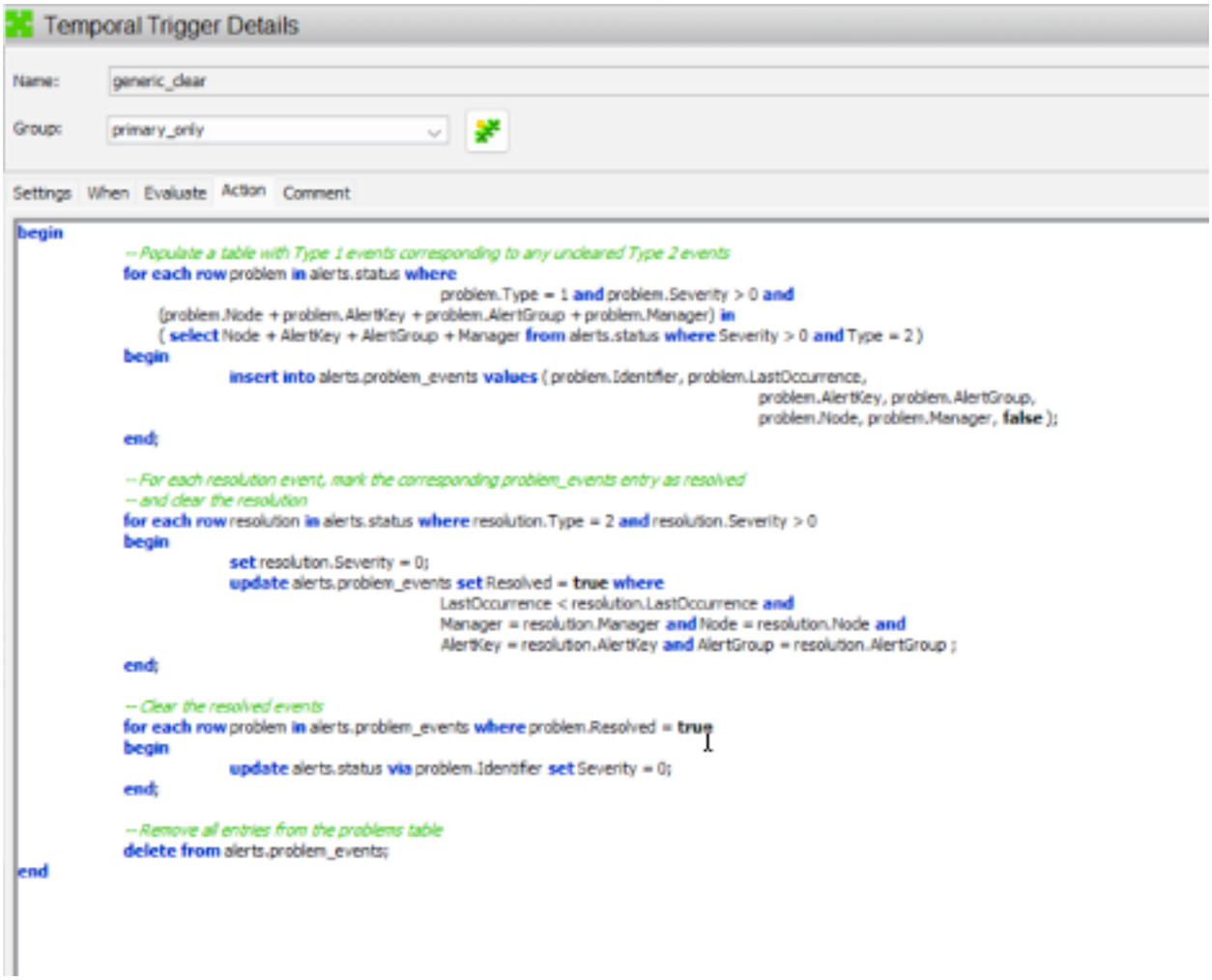


Figure 46: generic_clear trigger action tab

The **generic_clear** trigger is a temporal trigger that periodically scans for related events and clears them when their associated resolution events have arrived. It uses a helper table called `alerts.problem_events` to track which problem alerts should be cleared.

For example, when a router interface goes down, OMNIbus receives and stores an interface-down problem event. The generic_clear trigger records that event in the alerts.problem_events helper table so it can be tracked as an active condition. When the monitoring system later sends a matching interface-up resolution event for the same router interface, the trigger identifies the related interface-down event and clears it automatically. This keeps the event list focused on unresolved conditions while preserving the original event history.

Inspect the **Action** for the `deduplication` trigger. At the start of the trigger, a loop populates the `alerts.problem_events` ObjectServer table:

```
-- Populate a table with Type 1 events corresponding to any uncleared Type 2 events
for each row problem in alerts.status where
    problem.Type = 1 and problem.Severity > 0 and
    (problem.Node + problem.AlertKey + problem.AlertGroup + problem.Manager) in
        (select Node + AlertKey + AlertGroup + Manager from alerts.status where Severity > 0
 ↪  and Type = 2)
begin
    insert into alerts.problem_events values (
        problem.Identifier,
        problem.LastOccurrence,
        problem.AlertKey,
        problem.AlertGroup,
        problem.Node,
        problem.Manager,
        false
    );
end;
```

- This loop finds **Type 1 problem events** with non-zero severity that have a matching **Type 2 resolution event** (same node, key, group, and manager).
- For each matching problem event, it inserts a row into `alerts.problem_events`, storing key fields like `Identifier`, `LastOccurrence`, and ownership information, plus a `Resolved` flag initially set to `false`.
- This step builds a working list of problem alerts that are candidates to be cleared when their resolution appears.

Next, the trigger marks problem events as resolved and clears the resolution alerts:

```
-- For each resolution event, mark the corresponding problem_events entry as resolved and
 ↪  clear the resolution
for each row resolution in alerts.status where resolution.Type = 2 and resolution.Severity >
 ↪  0
begin
    set resolution.Severity = 0;
    update alerts.problem_events set Resolved = true where
        LastOccurrence < resolution.LastOccurrence and
        Manager      = resolution.Manager and Node  = resolution.Node and
        AlertKey     = resolution.AlertKey and AlertGroup = resolution.AlertGroup;
end;
```

- This loop scans **Type 2 resolution events**.
- It immediately clears the resolution alert itself by setting `resolution.Severity = 0`.
- It then updates matching rows in `alerts.problem_events`, setting `Resolved = true` for problem alerts that occurred **before** the resolution and share the same node, manager, key, and group.
- At this point, `alerts.problem_events` knows which problem alerts should be cleared.

The trigger then clears the resolved problem alerts:

```
-- Clear the resolved events
for each row problem in alerts.problem_events where problem.Resolved = true
begin
    update alerts.status via problem.Identifier set Severity = 0;
end;
```

- This loop walks the `alerts.problem_events` table and finds entries marked as resolved.
- For each one, it locates the corresponding alert in `alerts.status` by `Identifier` and sets its `Severity = 0`, effectively clearing the original problem alert.

Finally, the trigger cleans up the helper table:

```
-- Remove all entries from the problems table
delete from alerts.problem_events;
```

- After processing, all rows are removed from `alerts.problem_events` so the next run starts with a clean slate.

This trigger reduces noise by automatically clearing problem alerts when a matching resolution event arrives. In our previous example of a router interface down/up, a **Type 1** problem event might describe `Node='router01'`, `AlertGroup='LinkStatus'`, `Manager='MTTrapd Probe'` and `AlertKey='GigabitEthernet0/0'` when that interface goes down. When the corresponding **Type 2** resolution event arrives with the same `Node`, `AlertGroup`, `AlertKey`, and `Manager`, the trigger clears the original problem alert.

Another example would be a server disk issue such as `Node='server12'`, `AlertGroup='DiskError'`, `Manager='Instana'`, and `AlertKey='/dev/sda1'` when that drive or partition reports errors. When a matching **Type 2** resolution event arrives for the same disk partition after the error condition has cleared, the trigger clears the original problem alert as well. In both cases, the ObjectServer keeps the event list current so operators can focus on issues that are still active rather than manually clearing related alerts one by one.

11. Close the `generic_clear` trigger, locate the `expire` trigger, and review its configuration on the **Settings** tab. Then select the **Action** tab and review the trigger code.

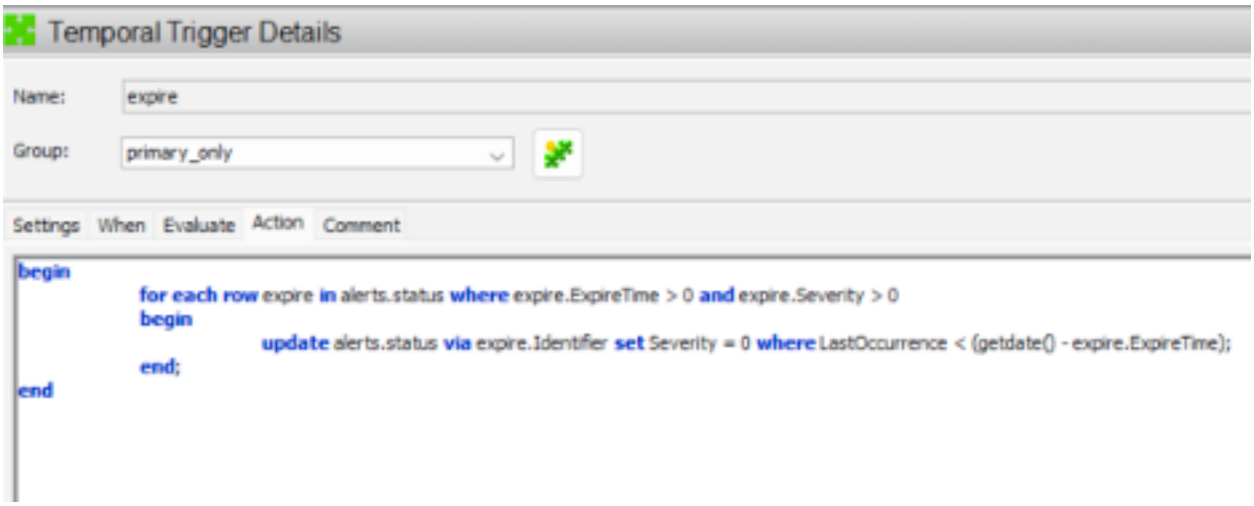


Figure 47: expire trigger settings tab

The **expire** trigger is also a **temporal** trigger that periodically clears alerts that have passed their configured expiry time. Compared to the deduplication and generic_clear triggers, its logic is much simpler.

The trigger loops over alerts in `alerts.status` that have both an expiry time and a non-zero severity:

```
for each row expire in alerts.status where expire.ExpireTime > 0 and expire.Severity > 0
begin
    update alerts.status via expire.Identifier set Severity = 0
    where LastOccurrence < (getdate() - expire.ExpireTime);
end;
```

- The loop selects alerts where `ExpireTime` is greater than zero (meaning an expiry period has been set) and `Severity` is greater than zero (the alert is currently active).
- For each such alert, it checks whether `LastOccurrence` is **older than** the current time minus the `ExpireTime` value.
- If the alert is older than its expiry window, the trigger updates the alert in `alerts.status` using its `Identifier` and sets `Severity = 0`, effectively clearing the alert.

Because this trigger doesn't track problem/resolution pairs or update multiple fields, it is much easier to follow: it simply enforces time-based expiry for alerts that have been quiet for longer than their configured `ExpireTime`.

The expire trigger is important because it automatically clears stale alerts that have not been updated for a defined period. This is especially useful for alerts that do **not** have a matching resolution alert, such as some status, threshold, or informational alerts that simply age out over time rather than being explicitly cleared by another event. For example, if an interface or disk alert has an `ExpireTime` of 2 hours and no new events occur in that window, the trigger will clear the alert so it no longer appears as an active problem. This prevents long-forgotten issues from cluttering the event list and helps operators focus on current conditions rather than alerts that should have aged out of view. Ultimately, the cleared alert is then removed by the `delete_clears` trigger, which completes the cleanup process.

12. Compare the purposes of the three triggers:

- `deduplication` handles repeated inserts of the same event by updating an existing alert instead of creating a duplicate row. It uses the event's identifying fields, such as Identifier, to decide whether the incoming event is a new problem or another occurrence of one that is already being tracked.
- `generic_clear` runs every 5 seconds and looks for Type 2 resolution events that match Type 1 problem events. When it finds a match, it clears the related problem alert and the resolution event so the event list reflects that the issue has been resolved.
- `expire` clears events when their ExpireTime is reached. This is useful for alerts that do not have a matching resolution event, because it allows the ObjectServer to age out alerts that are no longer relevant and prevent stale entries from remaining active indefinitely.
- `delete_clears` deletes cleared alerts from alerts.status (where Severity = 0 and StateChange is older than 120 seconds), which keeps the ObjectServer database from growing without bound and ensures that only recent and active alert history is retained.

Validation The exercise is successful when all of the following are true:

- You can navigate through **Automation -> Triggers** in Netcool/OMNIbus Administrator.
- You can identify that trigger groups are used to organize related triggers for grouped management.
- You can open and review the `deduplication` trigger.
- You can explain why the `deduplication` trigger is configured as a **pre database action** on **reinsert**.
- You can describe the main behavior of the deduplication trigger action code.
- You can open and review the `generic_clear` trigger, identify it as a temporal trigger running every 5 seconds, and explain the four-step logic it uses to match and clear problem and resolution events.
- You can open and review the `expire` trigger, explain that it acts on events where `ExpireTime` is set, and describe how it uses `LastOccurrence` and `ExpireTime` to determine when to clear an alert.

Expected result At the end of this exercise, you understand how to inspect ObjectServer triggers in the Administrator client and can explain the operational role of three important default triggers: `deduplication`, `generic_clear`, and `expire`. You also have a working understanding of how the ObjectServer automation framework responds to database events, scheduled intervals, and signals — a foundation that can be applied directly when designing or modifying automation logic to meet specific business requirements.

3.4 ObjectServer properties

Objective In this exercise, you will examine ObjectServer properties using Netcool/OMNIbus Administrator, understand how properties control component behavior, and enable the ObjectServer REST interface by modifying the relevant properties. By the end of this exercise, you will be able to navigate the properties list, identify editable and immediate properties, apply a configuration change, and verify the result using a curl test against the REST API.

Prerequisites

- Completion of section 3.3.
- The ObjectServer is running.
- `Netcool/OMNIbus Administrator` is accessible on the Windows workstation.
- SSH access to the lab server.

Background ObjectServer properties are named configuration settings that control the runtime behavior of the ObjectServer. Each property has a name, a current value, and a description. The properties list in Netcool/OMNIbus Administrator also shows two important flags for each property: **Editable** and **Immediate**.

The **Editable** flag indicates whether the property can be changed at runtime through `nco_config` (unix/Linux), `Netcool/OMNIbus Administrator` (Windows), or the `nco_sql` command line tool. Properties that are not editable cannot be changed while the ObjectServer is running and must be set in the properties file before startup.

The **Immediate** flag indicates whether a change to the property takes effect right away or only after the ObjectServer is restarted. Properties that are not immediate can still be changed, but the new value will not take effect until after a restart. Properties that are immediate take effect as soon as they are changed. For example, changing the ObjectServer MessageLevel from `warn` to `debug` takes effect immediately, while changing a property such as MessageLog (the ObjectServer log file path) typically requires a restart before the new location is used.

Properties can be set in two ways. The ObjectServer properties file — typically named `EVENTS.props` and located in the `/opt/IBM/tivoli/netcool/omnibus/etc/` directory — is only read at startup and sets the initial values for all properties. Properties can also be modified at runtime through `nco_config` or `Netcool/OMNIbus Administrator` for properties that are marked as editable, with the caveat that non-immediate changes will not take effect until the next restart.

The ObjectServer includes an embedded HTTP server and REST API that can be used to query alert data programmatically. This interface is controlled by the `NHttpd.*` and `NRestOS.*` ObjectServer properties. By default, `NRestOS.Enable` is set to `false` and `NHttpd.ListeningPort` is set to `0`, meaning the REST interface is disabled. Enabling it requires setting `NRestOS.Enable` to `true`, `NHttpd.EnableHTTP` to `true`, and `NHttpd.ListeningPort` to a valid port number, then restarting the ObjectServer.

To illustrate managing OMNIbus ObjectServer properties, we will enable the REST API and test it.

Tasks

1. Launch `Netcool/OMNIbus Administrator` on the Windows workstation and log in to the EVENTS ObjectServer as `root` with password `netcool`.
2. In Netcool/OMNIbus Administrator, open the **System** drawer item on the left menue and select **Properties**.

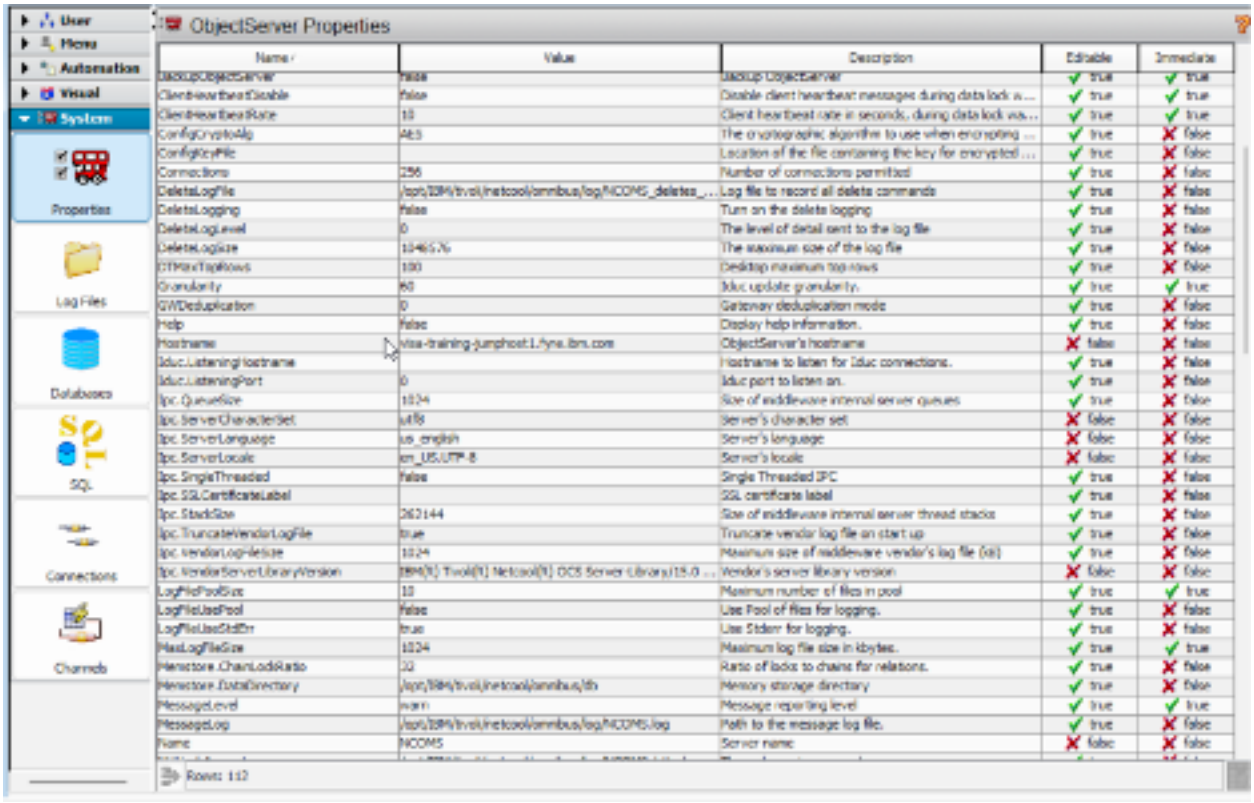


Figure 48: ObjectServer properties list

3. Review the properties list. Note that it is a flat list of all ObjectServer properties, each with a **Name**, **Value**, **Description**, **Editable**, and **Immediate** column.
4. Scroll through the list and observe that the majority of properties show `false` in the **Immediate** column, confirming that most configuration changes require an ObjectServer restart to take effect.
5. Locate the property `NRestOS.Enable` and confirm that its current value is `false`. Note that this property controls whether the embedded REST API is active.
6. Locate the property `NHttpd.EnableHTTP` and confirm that its current value is `false`. Note that this property controls whether the embedded HTTP server is active.
7. Locate the property `NHttpd.ListeningPort` and confirm that its current value is `0`. This value of `0` indicates that no HTTP port is assigned because the HTTP server is currently disabled.
8. Double-click `NRestOS.Enable` to edit it. Change the value to `true` and click **OK**.

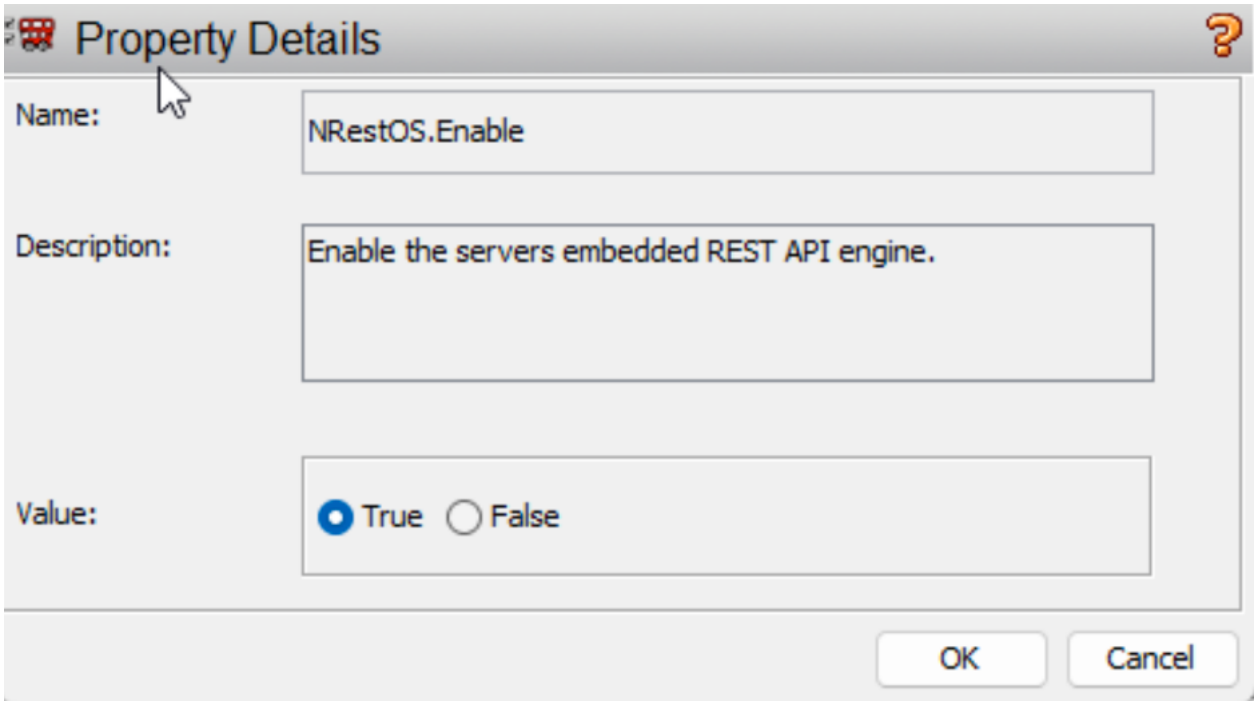


Figure 49: ObjectServer properties list

9. Double-click `NHttpd.EnableHTTP` to edit it. Change the value to `true` and click **OK**.
10. Double-click `NHttpd.ListeningPort` to edit it. Change the value to 8888 and click **OK**. > Note: take care to select `NHttpd.ListeningPort` and not `NHttpd.SSLListeningPort` as we are just working with HTTP for the lab.
11. Note that none of these properties are marked as **Immediate**, so the changes will not take effect until the ObjectServer is restarted. Connect to the lab server by using SSH and stop the ObjectServer, and run the `nco_sql` tool:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password netcool
```

Once connected to the EVENTS ObjectServer `nco_sql` at the 1> prompt, issue the following commands to gracefully shut down the ObjectServer:

```
1> alter system shutdown
2> go
(0 rows affected)
1> go
```

The expected output is:

```
CT-LIBRARY error:
    ct_results(): network packet layer: internal net library error: Net-Library operation
 ↪  terminated due to disconnect...
```

This output confirms that `nco_sql` lost its connection because the ObjectServer has shut down successfully.

12. In the SSH session on the lab server, wait a moment for the ObjectServer process to terminate. The Netcool Process Agent (PA) will automatically restart the ObjectServer. Validate that it shows running by executing the following command:

```
nco_pa_status
```

Expected output:

```
-------------------------------------------------------------------------------------------
 ↪  ------------------
Service Name         Process Name         Hostname                          User        Status
 ↪  PID
-------------------------------------------------------------------------------------------
 ↪  ------------------
```

```
Core                 MasterObjectServer   ibm-training-server1.dcimusic.com  root
 ↪  RUNNING       463880
-------------------------------------------------------------------------------------------
 ↪  ------------------
```

13. Verify that the ObjectServer is accepting connections with the following command:

    ```
    $OMNIHOME/bin/nco_ping EVENTS
    ```

    Confirm that the output includes: `NCO_PING: Server available.`

14. Return to `Netcool/OMNIbus Administrator` on your Windows workstation. Note that the connection to EVENTS will have dropped during the restart — reconnect by logging in again as `root` with password `netcool`. Navigate to **System → Properties** and confirm that `NRestOS.Enable`, `NHttpd.EnableHTTP`, and `NHttpd.ListeningPort` now show the updated values.

15. From the SSH session on the lab server, run the following curl command to test the REST interface. `curl` is a command-line tool used to send HTTP requests — the `-u` flag supplies the username and password for basic authentication:

    ```
    curl -u root:netcool http://localhost:8888/objectserver/restapi/alerts/status
    ```

16. Confirm that the command returns a JSON response containing alert data from the `alerts.status` table. The response will include a list of current alerts in JSON format.

Validation The exercise is successful when all of the following are true:

- You can navigate to **System → Properties** in Netcool/OMNIbus Administrator and describe the purpose of the **Editable** and **Immediate** columns.
- You can identify `NRestOS.Enable`, `NHttpd.EnableHTTP`, `NHttpd.ListeningPort`, and `NHttpd.BasicAuth` in the properties list and explain what each one controls.
- The properties `NRestOS.Enable`, `NHttpd.EnableHTTP`, `NHttpd.ListeningPort`, and `NHttpd.BasicAuth` are updated to `true`, `true`, 8888, and `true` respectively.
- The ObjectServer restarts successfully and `nco_ping EVENTS` returns `NCO_PING: Server available.`
- The curl command against `http://localhost:8888/objectserver/restapi/alerts/status` returns a valid JSON response containing alert data.

Expected result At the end of this exercise, you understand how ObjectServer properties control component behavior, know the difference between editable and immediate properties, and have successfully enabled and tested the ObjectServer REST interface. You also understand that the REST API provides a programmatic way to access alert data that can be used in integrations and custom tooling to meet specific business requirements.

4. Probes

Probes are the event collection layer that receives information from external sources and turns that information into alerts that the ObjectServer can store and manage. Depending on the probe type, a probe may **listen** for incoming events on a network port, **connect** to an external system and poll or receive events, or **watch** files and create events from new log entries.

Although probes differ in the protocols and sources they support, they follow a common processing model. A probe receives raw event data, breaks that data into tokens, applies rules to determine whether the event should be kept and how it should be mapped, and then forwards the resulting alert to the ObjectServer, where it is stored in the `alerts.status` table and displayed to operators.

4.0 Lab setup — event filter and view configuration

Objective Before you start the probe exercises, you'll set yourself up with a dedicated event filter and a custom event list view in the Web GUI. The filter will show only alerts generated by the syslogd probe, and the view will focus on the fields you need to check that your rules are working in sections 4.1 through 4.3.

Doing this first means that every time you change a rule, you can immediately see the impact in a clean, focused event list, without scrolling through unrelated alerts or constantly adding and removing columns as you go.

Prerequisites

- Completion of section 3.4.
- The ObjectServer is running.
- The Web GUI application server is running.
- The student can log in to the Web GUI and open an event list.

Background The Web GUI event list supports **filters** and **views** that can be configured independently and saved for reuse. A filter controls which alerts are displayed — it is essentially a SQL `WHERE` clause applied against `alerts.status`. A view controls which columns are displayed and in what order.

For the probe labs, a filter that shows only events where `Manager` contains the probe name will keep the event list focused on the alerts you are generating. A custom view with the fields used throughout the rules exercises will make it straightforward to confirm that each field is being populated correctly as the rules file is built up.

Note that the `Tally` field in `alerts.status` is the underlying field name for what the Web GUI typically displays as **Count**. When adding this field to the view, select `Tally` as the field name.

Tasks

1. Open a browser and log in to the Web GUI at: `https://<your-server>:16311/ibm/console` Log in as `ncoadmin` with password `netcool`.
2. Select the flag icon in the upper-left and open the event viewer.
3. Open the filter editor by selecting the filter editor icon (three green/blue arrows) at the top of the event list and choosing **Edit Filters**.



Figure 50: Edit Filters dialog

4. Create a new filter by clicking on the “New Filter” icon in the filter editor window:

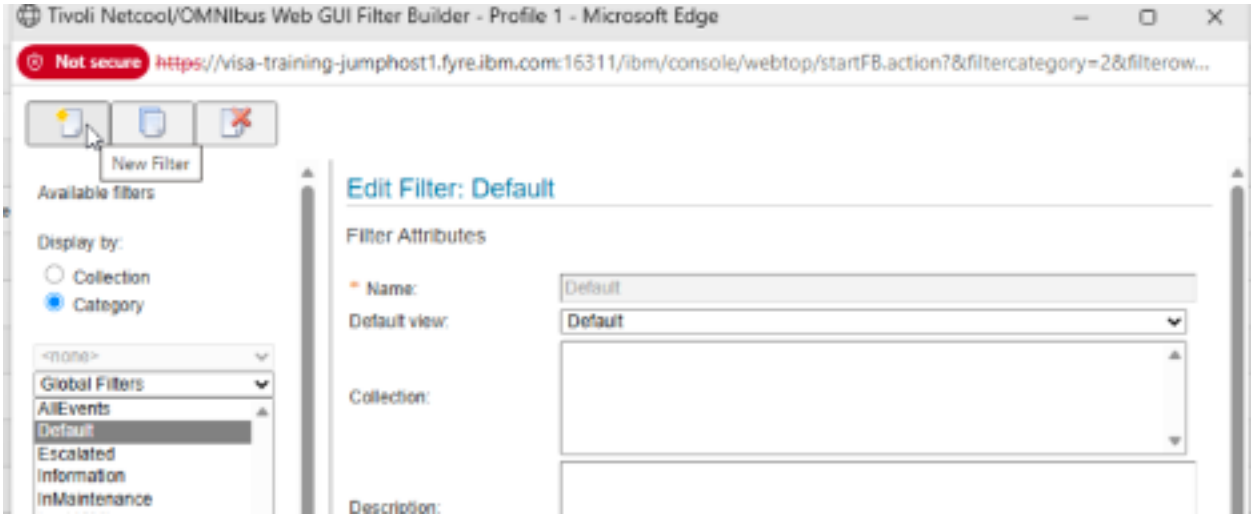


Figure 51: Create new filter dialog

5. You are first asked to choose who can use this filter. You can make it **global** (available to all users), **system** (available only to administrators), assign it to specific **users**, or assign it to one or more **groups**. For now, assign it to Netcool_OMNIbus_Admin and Netcool_OMNIbus_User, and click **OK**:
6. The Web GUI filter builder lets you define exactly which alerts appear in an event list, from very simple filters to quite complex ones.

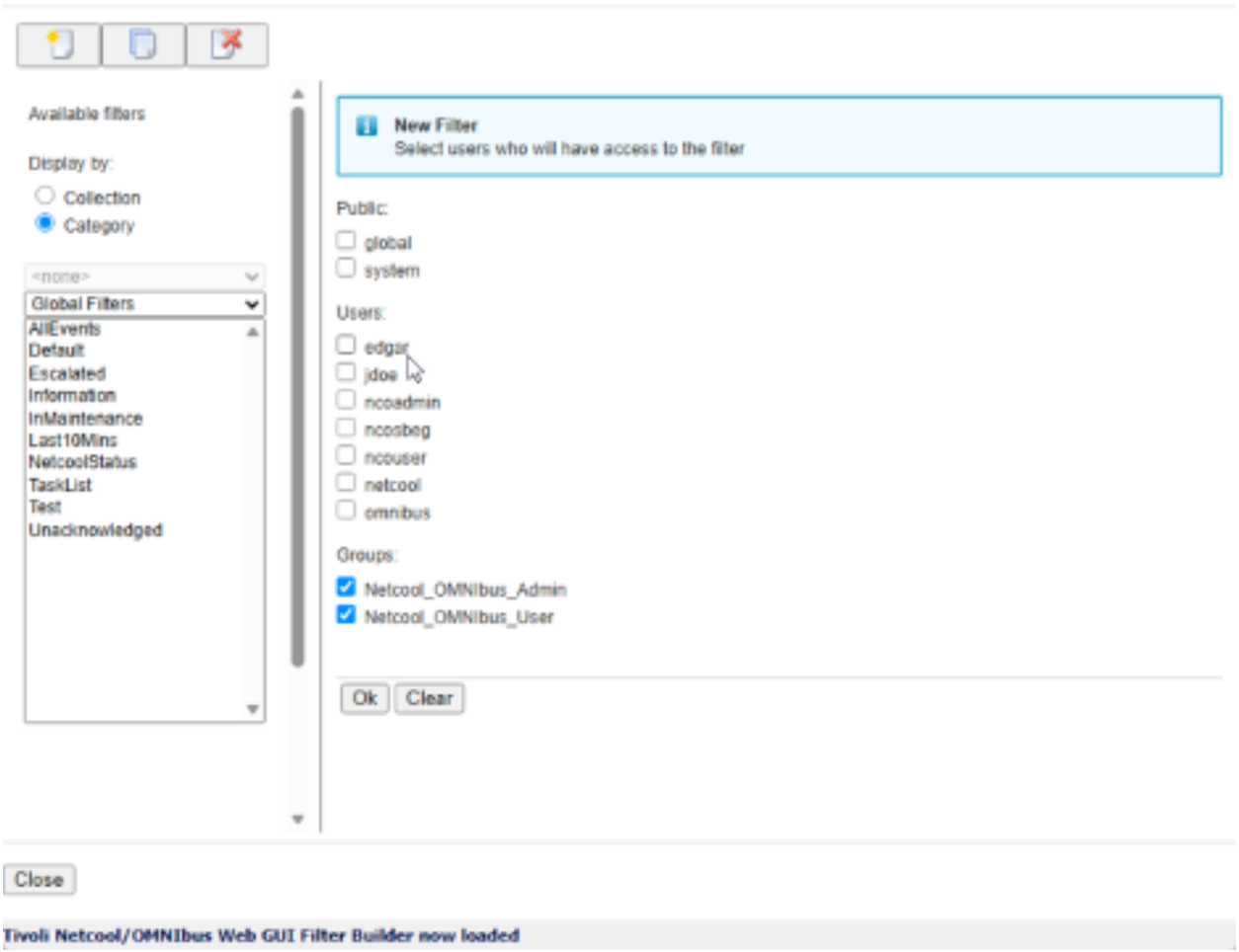


Figure 52: Filter scope selection dialog

At the top, you set the **filter attributes** — the filter **Name**, the **Default view** it uses, and an optional **Description** so others know what it's for.

Below that, in **Filter Conditions**, you build the actual logic. On the **Basic** tab you add simple conditions by choosing a **Field**, a **Comparator** (such as =, `like`, >, <), and a **Value'. You can add multiple rows and choose whether** All** conditions must match (AND) or **Any** condition can match (OR).

The other tabs (**Advanced**, **Dependent**, **Metric**) allow more sophisticated filters, such as expressions, filters that depend on other filters, or conditions based on metrics, so you can create complex, reusable filters tailored to different operational needs.

7. Configure your new filter with the following settings:
   - **Name**: `SyslogdProbe`
   - **Filter condition**: `Manager like 'MyProbe'` This filter will display only alerts where the `Manager` field contains `MyProbe` — the value we will set in the `@Manager` assignment in upcoming section 4.3.1. Alerts from other probes or sources will not appear while this filter is active.

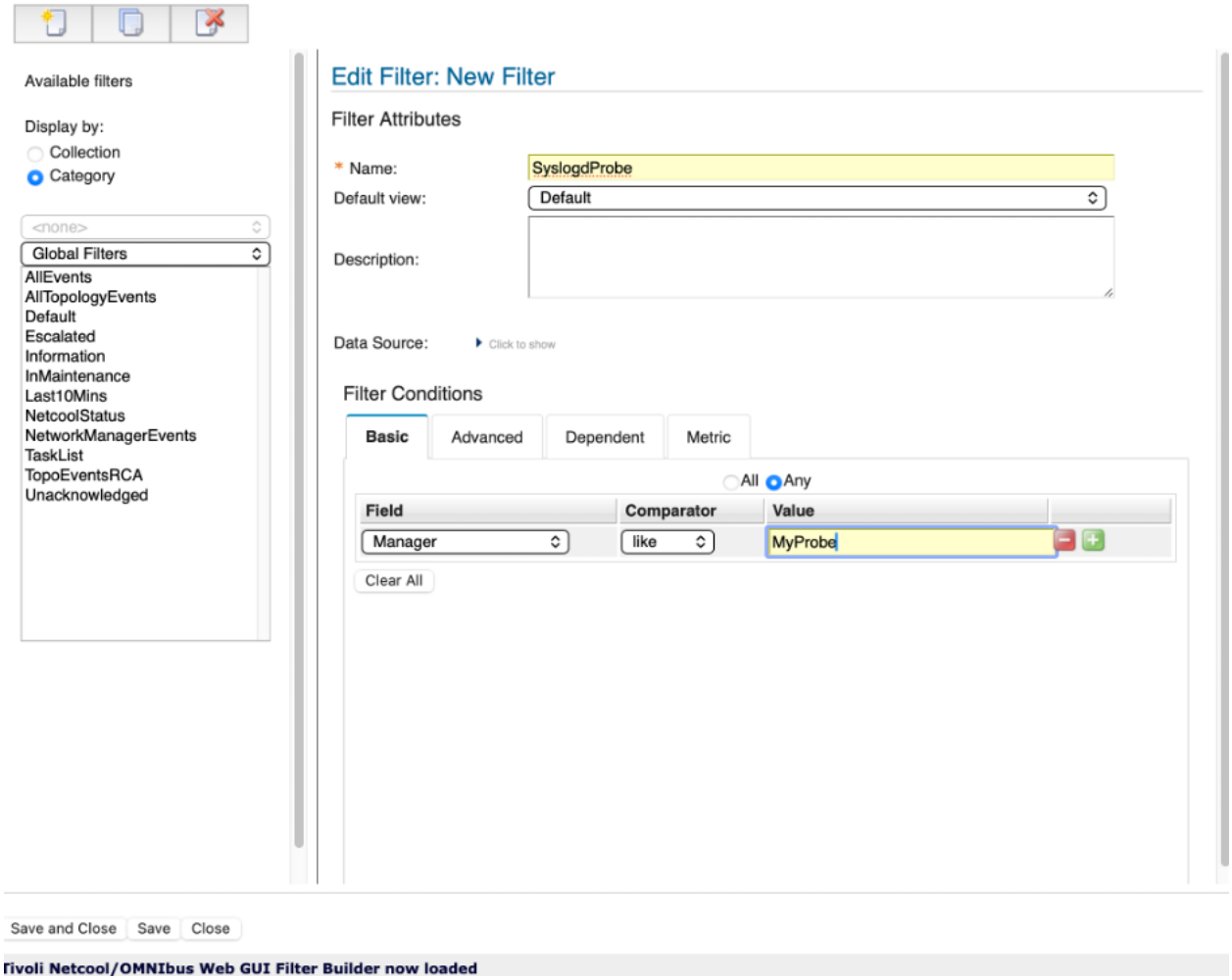


Figure 53: Filter builder configuration

8. Save and close the filter by clicking on the **Save and Close** button at the lower left.
9. Next we will create a custom view. Open the view editor by clicking on the **Edit Views** icon to the left of the view dropdown:
10. Click the "New View" icon at the upper-left of the window. Again, you will be asked to grant access to this view, for our purposes select **Global**, and click "Ok".
11. The View Builder lets you control how alerts are displayed in the event list — which columns you see, their order, and some additional custom layout options.

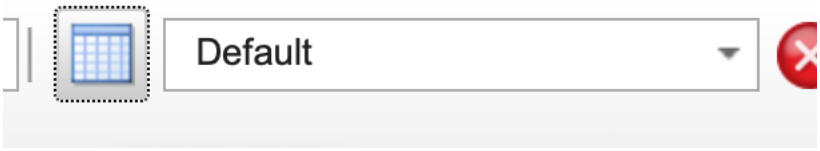


Figure 54: Open view editor

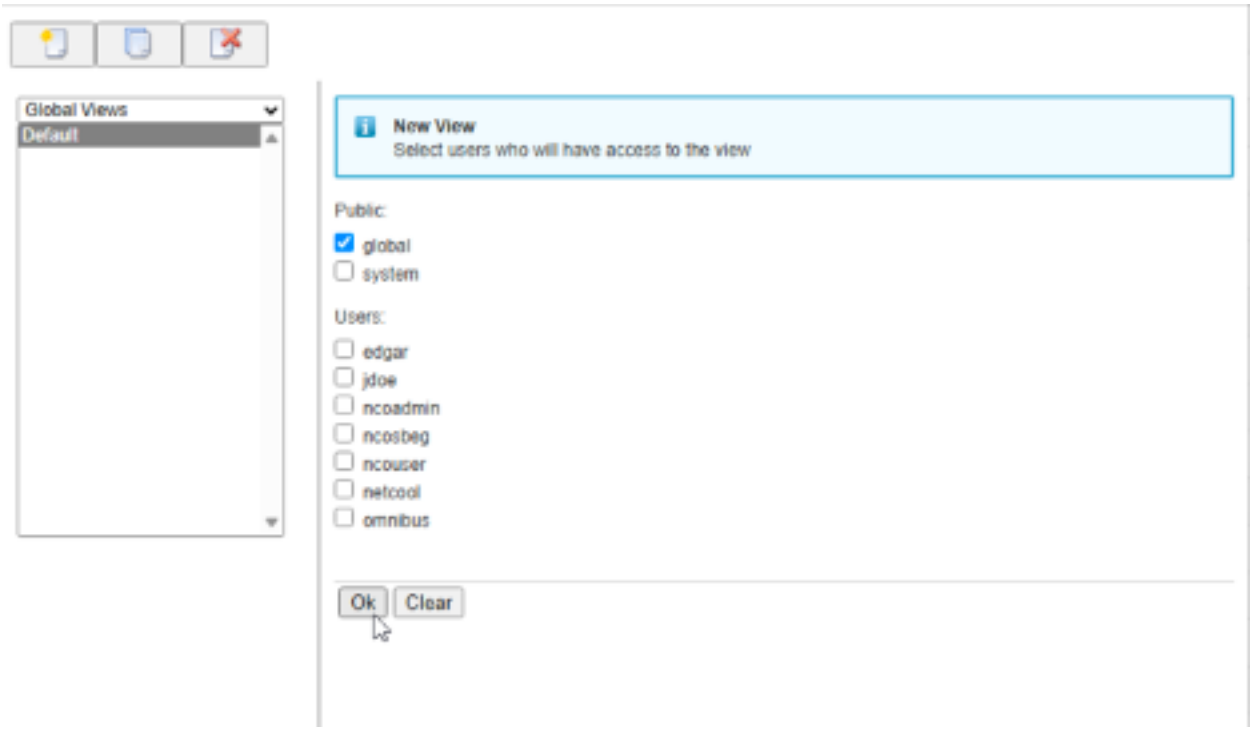


Figure 55: View access selection

At the top, you give the view a **Name** and optionally specify a **Data Source** (the data source can be used for multiple ObjectServers, not relevant for these labs).

The main area is split into tabs:

- **Display Columns**
  On this tab, you choose which fields will appear as columns in the event list. The **Available fields** list on the left shows all fields from `alerts.status`. The **Event list view** list on the right shows the columns currently in the view.
  Use the arrow buttons between the lists to add or remove fields and to change their order. You can also:
  - Tick **Lock column** to prevent a column from being moved or resized in the event list.
  - Set the **Column title** (what the user sees in the header, overriding the ObjectServer column name).
  - Choose text alignment for the displayed column with **Justify title** and **Justify data**.
  - Adjust **Column width** if needed.
- **Sort Columns**
  This tab controls the default sort order for the view. Select one or more fields and specify whether each field is sorted in ascending or descending order so that the most relevant alerts appear first. The initial row order in the Event Viewer is determined by the applied view, although users may be able to perform temporary custom sorting by selecting column headers if that feature is enabled for their account.

  Common sorting choices include:
  - Severity descending, so Critical and Major events appear before lower-priority events.
  - LastOccurrence descending, so the most recently received or updated events appear first.
  - FirstOccurrence ascending, so the oldest unresolved events are prioritized.
  - Count descending, to highlight frequently recurring or repeatedly received events.
  - Node, Location, Customer, or Service ascending, to keep events from the same device, site, customer, or service together.
  - Acknowledged ascending, so unacknowledged events appear before events already being worked.
  - OwnerUID or OwnerGID ascending, to group alerts by assigned operator or support team.

  A common operational order is Severity descending followed by LastOccurrence descending. This places the highest-priority alerts at the top of the view, with the most recently updated alerts shown first within each severity level. Note that some fields, including Acknowledged and OwnerUID, may be sorted by their displayed conversion value rather than their underlying numeric ObjectServer value.
- **Group Columns**
  Here you define how alerts are grouped in the event list (for example, by `Node` or `AlertGroup`). Grouping can make it easier to see related alerts together. For example, if you add the “Location” field to this, all alerts will be grouped by the unique value of the Location field in all of the alerts that match the filter of this view (For example, “New York” and “Los Angeles”).
- **Relationships**
  This tab allows you to enable alert grouping and rollup of alerts that are correlated by the IBM Tivoli Network Manager (ITNM) Root Cause Analysis engine.

12. Add the following fields to the view in the order listed by selecting the field under “Available sort fields”, and clicking the single “Arrow” icon. Change the Column Widths for each field as noted. This order is arranged by operational relevance — the most actionable fields appear first, followed by temporal fields, categorisation fields, and finally technical fields that are useful for validating rules behavior:

Order	Field name	Notes	Column Width
1	`Severity`	Alert severity level	5
2	`Acknowledged`	Acknowledgement state	5
3	`CustomerID`	Custom field added in section 3.2	10
4	`Service`	Service level from lookup table	10
5	`Node`	Source device	10
6	`AlertKey`	Specific resource or condition	10

Order	Field name	Notes	Column Width
7	`Summary`	Event description	40
8	`EventId`	Extracted event type identifier	10
9	`FirstOccurrence`	When the alert was first seen	10
10	`LastOccurrence`	When the alert was most recently updated	10
11	`Tally`	Number of occurrences — displayed as **Count** in the event list	5
12	`AlertGroup`	Event category	10
13	`ExpireTime`	Expiry timer value in seconds	5
14	`Manager`	Probe or management system that generated the alert	10
15	`Identifier`	Deduplication key — (useful for validating rules, less relevant in production operations views)	20

Note: First, move all required fields from Available fields to Event list view using the left/right buttons to move them back and forth. Then use the move controls beside the Event list view list to arrange the fields in the desired order. If the arrangement controls are not visible, enlarge the editor window or adjust the browser zoom level. After arranging the fields, set each field's justification and width.

13. Name your new view `SyslogdView` and save the view by clicking on the **Save and Close** button at the bottom-left of the view builder window.
14. Before applying the training filter, try switching the view independently of the filter. Switch to the **Default** or **AllEvents** filter while applying the **SyslogdView** view. If the Simnet probe is running or other events are present, they will now appear with the new column layout — confirming that views and filters are independent controls that can be mixed and matched freely.
15. Apply the **Syslogd Training** filter and the **Syslogd Training** view to the event list. The event list will be empty at this stage — this is expected. The filter condition `Manager like 'MyProbe'` will only match alerts generated once the training rules file is in place from section 4.3.1 onwards. The **SyslogdView** view will be visible and correctly configured even before any matching events exist.
16. Leave the **SyslogdProbe** filter and **SyslogdView** view active for all exercises in sections 4.1 through 4.3. The filter and view can be switched back to the defaults at the end of the probe labs.

Validation The exercise is successful when all of the following are true:

- You have created the **SyslogdProbe** filter with the condition `Manager like 'MyProbe'`.
- You have created the **SyslogdView** view containing all fifteen fields in the specified order.
- You can confirm that the view can be applied independently of the filter by switching to the **Default** or **AllEvents** filter while the **SyslogdView** view is active.
- The event list shows the **SyslogdProbe** filter and view as active, and you understand why it is empty at this stage.
- You can explain the purpose of having a dedicated filter and view before beginning the rules exercises.

Expected result Filters and views help operators focus on the right event information in the OMNIbus event list. A filter determines which alerts are returned from the ObjectServer—for example, only Critical and Major events, events for a specific customer or location, or alerts received within the last 10 minutes. A view determines how those returned alerts are presented, including the displayed columns, their order, sorting, grouping, and formatting. Together, filters reduce event noise while views present the fields most useful for a specific task, helping operators assess impact, prioritize work, and investigate issues more efficiently.

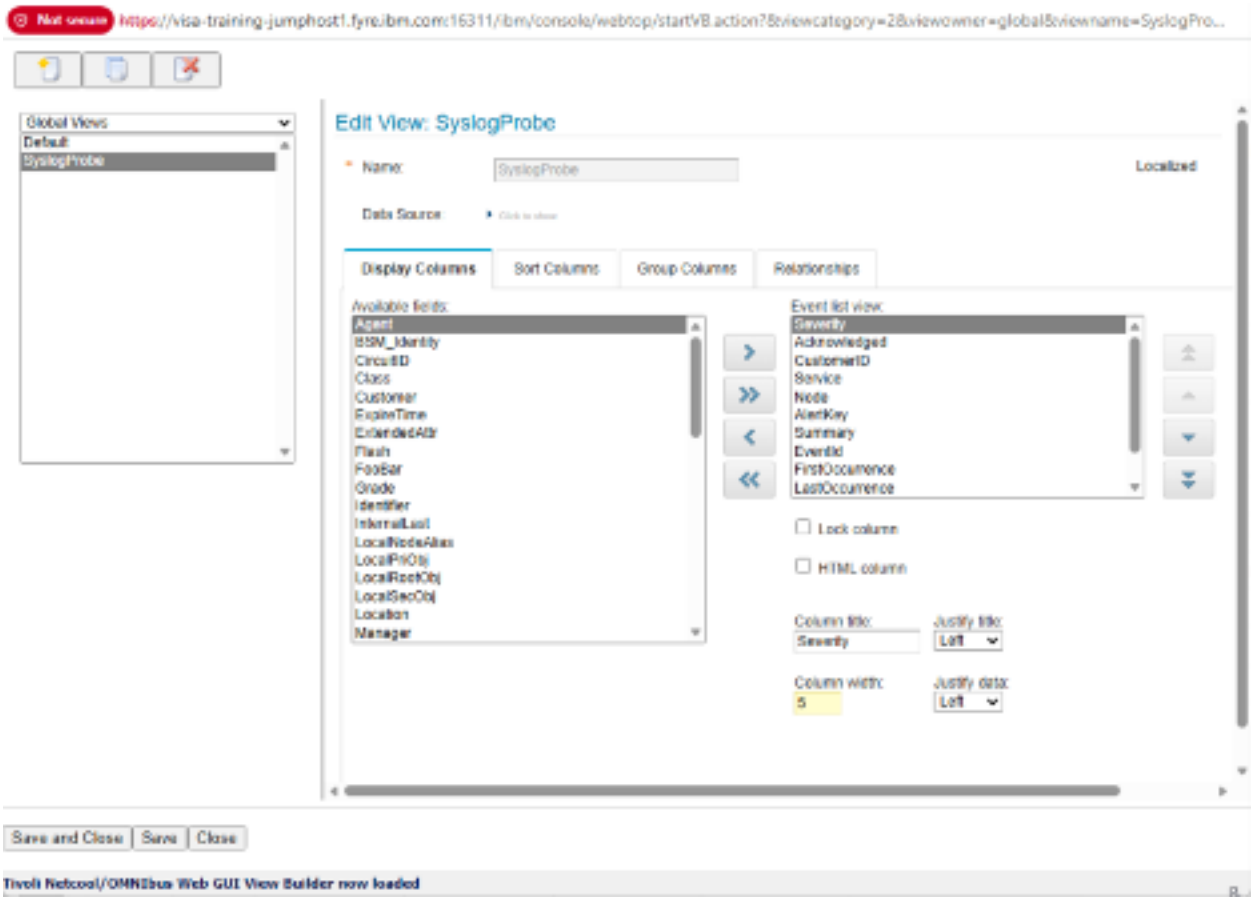


Figure 56: View Builder overview

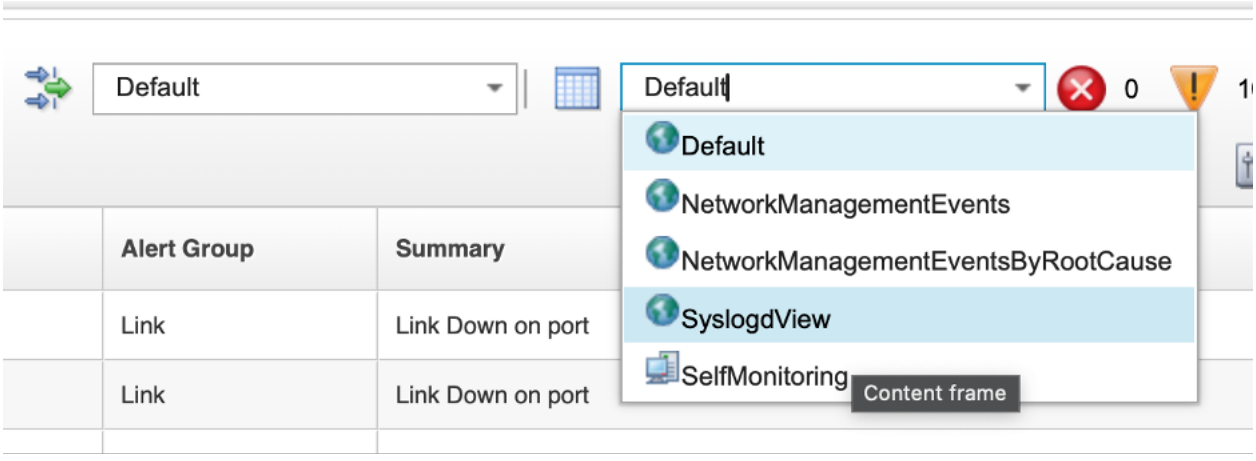


Figure 57: SyslogdView applied with Default filter

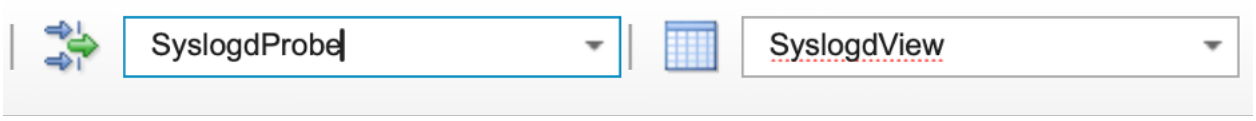


Figure 58: SyslogdProbe filter and SyslogdView active - empty event list

By completing this exercise, you have created two essential tools that will support your work throughout sections 4.1, 4.2, and 4.3:

- **SyslogdProbe filter**: You created a reusable filter that displays only alerts where the `Manager` field contains `MyProbe`. This filter isolates the events you will generate with the syslogd probe, keeping unrelated alerts (such as Simnet probe events) out of view. This focused view allows you to immediately see the impact of each rules file change without scrolling through noise or constantly toggling filters.
- **SyslogdView** view: You created a custom event list view with fifteen specific fields in a deliberate order. This view is tailored to the fields you will be working with in the rules exercises — Node, Summary, AlertKey, Manager, and others — making it easy to validate that each field is being populated correctly by your rules.
- **Filter and view independence**: You discovered that filters and views operate independently. You can apply a view to any filter and vice versa, giving you flexibility when exploring and troubleshooting event data.

The combination of a focused filter and a tailored view creates an ideal development environment where you can iterate quickly on probe rules, see results immediately, and understand exactly which field assignments are working and which need adjustment.

4.1 Probe overview and architecture

Objective In this section, you will be introduced to the role of probes in the OMNIbus architecture and how they communicate with the ObjectServer. By the end of this section, you will understand the common probe processing flow, the major types of probes, and the relationship between tokens, rules, and ObjectServer fields.

Prerequisites

- Completion of section 3.4.
- Completion of Lab 4 setup.
- The ObjectServer is running.
- SSH access to the lab server.

Background Probes can obtain data in several ways. Some probes listen for events on a network port, some connect to external services or message buses, and others open or tail files and convert new log entries into events. Because all probes share the same general architecture and rule-processing model, learning how one probe creates tokens and maps fields provides a strong foundation for working with many other probes.

The probe processing pipeline works as follows:

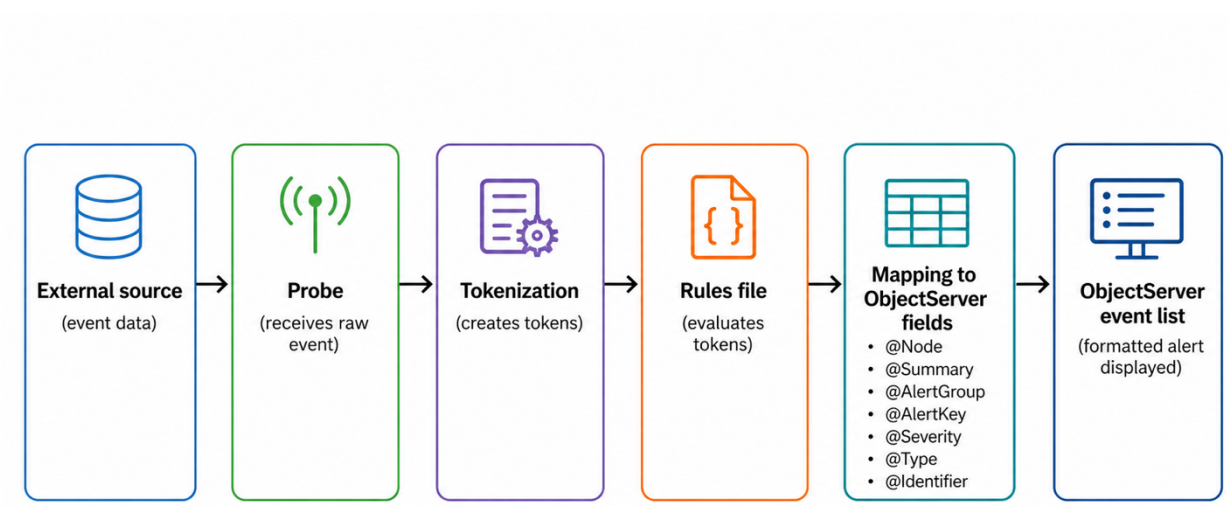


Figure 59: Probe processing pipeline diagram

1. An external source sends or exposes event data.
2. The probe receives the raw event.

3. The probe creates tokens from the incoming data.
4. The rules file evaluates those tokens.
5. The rules map selected values into ObjectServer fields such as `@Node`, `@Summary`, `@AlertGroup`, `@AlertKey`, `@Severity`, `@Type`, and `@Identifier`.
6. The formatted event is sent to the ObjectServer and appears in the event list.

Each probe is controlled by two key files: a **properties file** and a **rules file**. The properties file controls runtime behavior such as which ObjectServer to connect to, the log level, and whether raw event capture is enabled. The rules file controls how incoming event data is interpreted, which fields are populated, and whether an event is kept or discarded.

Probes also share a common set of properties regardless of the source they monitor. Properties such as `Server`, `MessageLevel`, `RulesFile`, and `RawCapture` appear across virtually all probe types and behave consistently. This makes it straightforward to apply knowledge of one probe's configuration to another.

Tasks *Environment variables for the lab are already defined earlier in this guide; proceed with the exercises below.*

1. On the lab server, list the contents of the main probes directory to review the probe binaries available in the environment:

```
ls $OMNIHOME/probes/
```

   Note: The probes in this directory are often symlinks to `nco_probe`, a wrapper script that sets up the probe environment before invoking the actual binary under `$OMNIHOME/probes/linux2x86`.

2. Note the `nco_p_syslogd` binary in probes directory. This is the syslogd probe that will be used throughout Section 4.
3. List the platform-specific probe configuration files to locate the syslogd properties and rules files:

```
ls $OMNIHOME/probes/linux2x86/syslogd*
```

4. Note that the syslogd probe has two key files in the `linux2x86` directory:
   - `syslogd.props` — the properties file that controls probe runtime behavior and ObjectServer connectivity.
   - `syslogd.rules` — the rules file that controls how incoming syslog events are parsed and mapped into OMNIbus alert fields.
5. Review the contents of the rules file:

```
cat $OMNIHOME/probes/linux2x86/syslogd.rules
```

6. Note that the rules file contains field assignments that map parsed syslog tokens into ObjectServer fields. At this stage, review the structure rather than the detail — the rules development labs in section 4.3 will cover syntax and behavior in depth.
7. Review the contents of the properties file:

```
cat $OMNIHOME/probes/linux2x86/syslogd.props
```

8. Locate and note the following key **commented** properties in the properties file:
   - `Server` — specifies the ObjectServer the probe will connect to. The default is `NCOMS`.
   - `MessageLevel` — controls the verbosity of the probe log output.
   - `RawCapture` — controls whether incoming raw event data is written to a capture file.
   - `RulesFile` — specifies the path to the rules file the probe will use.

Validation The exercise is successful when all of the following are true:

- You can identify the `nco_p_syslogd` binary and its location.
- You can locate the `syslogd.props` and `syslogd.rules` files under the `linux2x86` directory.
- You can explain the role of the properties file and the rules file.
- You can identify the `Server`, `MessageLevel`, `RawCapture`, and `RulesFile` properties in the properties file and explain what each one controls.

Expected result By the end of this exercise, you will understand the two-file structure used by OMNIbus probes, know how to navigate to the relevant probe files on the lab server, and have a clear mental model of the probe processing pipeline before you start hands-on probe configuration and rules development.

4.2 Syslogd probe configuration and testing

Objective In this exercise, you will configure and start the `nco_p_syslogd` probe, send a test event with the `logger` command, verify the resulting alert in the event list, and review the raw and debug output produced by the probe. The goal is for you to see how a probe receives and forwards events before any custom rules-based formatting has been added, and to establish the debug and raw capture workflow that you'll use later to support rules development.

Prerequisites

- Completion of section 4.1.
- The ObjectServer is running.
- SSH access to the lab server.
- Basic familiarity with a shell text editor such as `vi` or `nano`.

Background Each probe reads its configuration from a properties file at startup. The properties file controls how the probe connects to the ObjectServer, where it writes its log output, how verbose that output is, and whether raw event capture is active. For the syslogd probe, the properties file also controls the UDP port on which the probe listens for incoming syslog messages.

When a probe starts, it connects to the ObjectServer specified in the `Server` property, loads the rules file specified in the `RulesFile` property, and begins listening for or collecting events. Events that pass through the rules file without being discarded are forwarded to the ObjectServer as alerts.

Debug logging and raw capture are two important diagnostic tools for probe administration and rules development. Increasing `MessageLevel` to debug causes the probe to write detailed processing information to its log file. Enabling `RawCapture` causes the probe to write the original raw event data and the tokens it generates to a capture file, which is stored under `$OMNIHOME/var/`. This token output is the primary input used when writing and testing rules, since it shows exactly what values are available for extraction and field mapping.

Tasks

1. On the lab server, run the syslogd probe with the `-dumpprops` flag to display all available properties and their default values:

   ```
   $OMNIHOME/probes/nco_p_syslogd -dumpprops
   ```

2. Review the output and note properties such as `Server`, `MessageLevel`, `RawCapture`, `RulesFile`, and `Port`. These are the primary properties used to control probe behavior in this lab. When running -dumpprops, the probe will read its property file, and then dump out the value of each property here. Any property that is not explicitly set (or commented out) in the properties file shows the default value for the probe.

3. Open the syslogd properties file in a text editor:

   ```
   vi $OMNIHOME/probes/linux2x86/syslogd.props
   ```

   Note that all properties are commented out initially. Properties are listed and commented out in the properties file for reference, your configured properties should be added to the end of the file, after the section indicating "Add your settings here". Add the following line to the end of the file to direct the probe to connect to the EVENTS object server:

   ```
   Server :        'EVENTS'
   ```

   Write and close the file.

4. Validate that the configuration change has taken effect by running the probe with the `-dumpprops` flag and grepping for the `Server` property:

   ```
   $OMNIHOME/probes/nco_p_syslogd -dumpprops 2>&1 | grep Server
   ```

   Expected output:

   ```
   PollServer: 0 # INTEGER (No description available)
   Server: 'EVENTS' # STRING (No description available)
   ServerBackup: '' # STRING (No description available)
   SSLServerCommonName: '' # STRING (No description available)
   ```

The key line to verify is `Server: 'EVENTS'`, which confirms that your property file change is now active. This validates that the probe will connect to the EVENTS ObjectServer when started.

5. Start the syslogd probe using the default properties file:

```
$OMNIHOME/probes/nco_p_syslogd -propsfile $OMNIHOME/probes/linux2x86/syslogd.props &
```

> **Note:** Specifying an explicit `-propsfile` is optional when using the default properties file, but it is good practice and becomes important when running multiple instances of the same probe. This applies to virtually all OMNIbus probes — by pointing each instance at a different properties file, it is possible to run several copies of the same probe binary simultaneously with different ports, different rules files, or connections to different ObjectServers — for example, to collect events from separate network segments or customers.

6. Verify that the probe process is running:

```
ps -ef | grep nco_p_syslogd | grep -v grep
```

Confirm that the output shows a running `nco_p_syslogd` process with a PID. For example:

```
root     1464458 1463499  1 16:21 pts/3    00:00:00
 ↪  /opt/IBM/tivoli/netcool/omnibus/platform/linux2x86/probes64/nco_p_syslogd -propsfile
 ↪  /opt/IBM/tivoli/netcool/omnibus/probes/linux2x86/syslogd.props
```

7. Generate a test syslog event using the `logger` command:

```
logger -n 127.0.0.1 "Test event from syslogd probe lab"
```

The `logger` command sends a syslog message using the local system's syslog facility, which delivers the message to the host's configured syslog destination. In this lab, the `nco_p_syslogd` probe is acting as the syslog receiver and is listening directly on UDP port 514. The probe receives the incoming syslog message, tokenizes it, applies the rules file, and forwards the resulting alert to the ObjectServer. This chain — `logger` → syslog facility → syslogd probe → ObjectServer — is how syslog-sourced events flow into OMNIbus in this lab environment.

8. Open the Web GUI event viewer and change the **filter** to `AllEvents`. Then, in the quick filter box in the upper-right portion of the event viewer window, type "Test event" and press the **Enter** key. This will create a quick filter that matches any field that contains the text "Test event". You should see the test event that you just sent in:

   Note that the event appears with minimal formatting — the field values will reflect whatever default mappings exist in the current rules file rather than a fully developed field set.

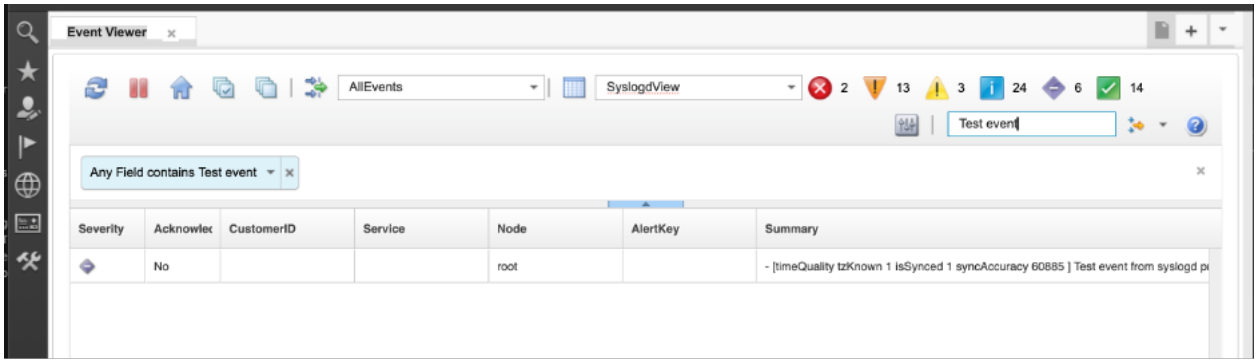


Figure 60: Locate the resulting alert in the Web GUI event list

9. Review the alert fields, particularly `Node`, `Summary`, `Severity`, and `Identifier`. Note the default or incomplete field presentation and observe that without custom rules, the event data is not yet in a form that would be useful to an operator.
10. Stop the probe: `bash  kill $(pgrep nco_p_syslogd)`
11. Open the syslogd properties file again in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd.props
```

Scroll to the end of the file and add the following lines. The convention in OMNIbus properties files is to place custom settings after the server entry for EVENTS at the end of the file, as shown below:

```
MessageLevel            : 'debug'
RawCapture              : 1
```

- Setting the probe `MessageLevel` to debug causes the probe to log *everything* it is doing — transport activity, tokenization, rules evaluation, and any errors. This is ideal when you are building or troubleshooting rules, but it generates a lot of output, so you should turn it back down (for example, to `warn`) in production.
- Most probes support a **raw capture mode** in addition to debug logging. Raw capture saves the *unprocessed* event stream to a file, exactly as it was received from the source, without applying the rules file. You can then replay that file later using the Standard Input probe (`nco_p_stdin`) to test new rules without waiting for real events.
- When a rules file contains syntax errors or logic problems, the probe's debug log will usually show:
  - The **first error encountered**.
  - The **line number** or section of the rules file where it occurred.
  - The event data that triggered the problem.

This makes debug mode one of the primary tools for fixing rules and understanding how they behave.

- Debug logs can also be used for **performance tuning**. By looking at how many events are processed over a period and how long certain rules sections take, you can identify expensive logic (for example, unnecessary database lookups) and optimize your rules for higher event rates.

  Remember: use debug mode while you are developing and testing, and then reduce the logging level once your probe and rules are stable.

12. Now that we have enabled debug mode and raw capture, restart the probe:

```
$OMNIHOME/probes/nco_p_syslogd -propsfile $OMNIHOME/probes/linux2x86/syslogd.props &
```

13. Generate another test event:

```
logger -n 127.0.0.1 "Test event from syslogd probe lab"
```

14. Review the probe log file to observe the debug output:

```
cat $OMNIHOME/log/syslogd.log
```

In debug mode, the syslogd probe logs much more detail than at the default message level. In particular, it shows how the incoming syslog message is broken down into **tokens**.

```
2026-08-13T15:51:11: Debug: D-P_S-105-000: Processing   -->
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Priority:  13
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Facility:  1
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Severity:  5
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Raw:       1 2026-08-13T15:51:11.399650-05:00 ibm-training-server1.dcimusic.com root - - [timeQuality tzKnown="
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token1:    1
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token2:    2026-08-13T15:51:11.399650-05:00
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token3:    ibm-training-server1.dcimusic.com
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token4:    root
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token5:    -
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token6:    -
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token7:    [timeQuality
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token8:    tzKnown
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token9:    1
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token10:   isSynced
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token11:   1
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token12:   syncAccuracy
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token13:   21205
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token14:   ]
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token15:   Test
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token16:   event
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token17:   from
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token18:   syslogd
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token19:   probe
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Token20:   lab
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Details:   - [timeQuality tzKnown 1 isSynced 1 syncAccuracy 21205 ] Test event from syslogd probe lab
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] TokenCount:        20
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] EventCount:        1
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Time:      1786654271
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] ArrivalTime:       Aug 13 15:51:11
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] RemoteHostInfo:    (localhost) 127.0.0.1 53177
2026-08-13T15:51:11: Debug: D-UNK-000-000: [Event Processor] Processing alert (0 remaining)
2026-08-13T15:51:11: Debug: D-UNK-000-000: Rules file processing took 137 usec.
2026-08-13T15:51:11: Information: I-UNK-000-000: Event processed by rules. discard=false
2026-08-13T15:51:11: Debug: D-UNK-000-000: Write 2424 bytes to raw capture file.
2026-08-13T15:51:11: Debug: D-UNK-000-000: Flushing events to object servers
2026-08-13T15:51:11: Debug: D-UNK-000-000: 1 buffered alerts
```

Figure 61: Probe debug output example

From the debug output above, you can see the probe reporting:

- A **Facility** (1), **Priority** (13), and **Severity** (5), derived from the syslog header.

- A **Raw** token that contains the entire syslog message as received (timestamp, host, program, structured data, and the text `Test event from syslogd probe lab`).
- A series of **TokenX** entries (`Token1` through `Token20`). For this probe, the message body is split on whitespace, so each word or structured element becomes a separate token (for example, `Token3` is the host name, `Token15`–`Token20` are the words of your test message).
- Additional tokens such as **`Details`** (a reconstructed message body), **`TokenCount`** (total number of tokens created), **`Time`**, **`ArrivalTime`**, and **`RemoteHostInfo`** (source host and port information).

These tokens are what you will reference in your rules file to populate ObjectServer fields (such as `@Node`, `@Summary`, `@AlertGroup`, `@AlertKey`, `@Severity`, and `@Type`) for each event.

15. Review the raw capture file to inspect the original event data and the tokens generated by the probe:

```
cat $OMNIHOME/var/syslogd.cap
```

16. In the capture file output, identify the following:

    - The original raw syslog message as received by the probe.
    - The individual tokens generated from that message, such as $1, $2, and $* (the full raw message).



Figure 62: Capture output example

17. Compare the token values in the capture file with the field values shown in the event list alert properties. Note which tokens were mapped to which fields by the current rules, and identify where the mapping is incomplete or missing.

Validation The exercise is successful when all of the following are true:

- You can use `-dumpprops` to display available probe properties.
- You can verify and confirm the `Server` property in the syslogd properties file.
- You can start and verify the `nco_p_syslogd` process.
- You can generate a syslog event with `logger` and locate the resulting alert in the event list.
- You can explain why the initial event formatting is incomplete without custom rules.
- You can enable `MessageLevel` debug and `RawCapture` by editing the properties file.
- You can identify the raw message and generated tokens in the raw capture file.
- You can relate the token output in the capture file to the field values visible in the event list.

Expected result At the end of this exercise, you understand the basic files and properties used by the syslogd probe, can start and test the probe, and can use debug logging and raw capture to inspect incoming event data. You also understand the gap between raw token output and a fully formatted OMNIbus alert, which establishes the need for the rules development work in section 4.3.

4.3 Probe rules by example

Objective In this section, you'll learn how probe rules transform raw event data into useful OMNIbus alerts. You'll start with a conceptual explanation of rules structure and field mapping, then work through a series of hands-on labs that incrementally build a working rules file. Each lab introduces one concept — field mapping, token extraction, switch/case logic, deduplication, correlation, conditional severity, and more — and you'll test each one immediately against live events that you generate with the `logger` command.

Prerequisites

- Completion of section 4.2.
- The ObjectServer is running.
- SSH access to the lab server.
- Basic familiarity with a shell text editor such as `vi` or `nano`.

Background As we have learned, when a probe receives an event, it creates tokens from the raw input. The rules file evaluates those tokens and determines two things: whether the event should be discarded or forwarded to the ObjectServer, and if it is to be forwarded, how it should be mapped into ObjectServer alert fields.

Some of the key ObjectServer fields that rules are responsible for populating are:

- **`@Identifier`** — controls deduplication. Must be designed carefully so that only truly matching events deduplicate against each other.
- **`@Type`** — distinguishes problem events (1) from resolution events (2) and drives the `generic_clear` correlation behavior reviewed in section 3.3.
- **`@Node`** — the host or device the event relates to.
- **`@AlertGroup`** — the event category or class.
- **`@AlertKey`** — the specific resource or condition within the node.
- **`@Summary`** — the operator-facing description of the event.
- **`@Severity`** — the operational priority, using the 0–5 scale covered in section 2.2.
- **`@Manager`** — identifies the probe or management system that generated the alert.
- **`@Agent`** — identifies the probe agent name.

Raw capture is the primary tool for rules development. It provides the original token stream — including the `$Raw` token which contains the full original message — that can be used to design extraction patterns and validate rule changes. For our tasks using the syslogd probe, working with the `$Raw` token is more reliable than depending on positional tokens such as $1 and $2, because syslog messages are tokenized on whitespace and the position of any given value can vary between message types. For probes that receive structured data such as SNMP traps with well-defined varbinds, positional tokens will be more predictable — but for free-form text-based sources like syslog, regular expression extraction from `$Raw` is the recommended approach.

Rules files also support a `log()` function that writes diagnostic output to the probe log file. This is invaluable during development for confirming which code path an event is taking and what values are being set.

4.3.1 Basic rules and first mapped fields Objective

Create a new blank rules file for the syslogd probe, add the standard ProbeWatch stanza, set basic fields, and verify that events reach the ObjectServer correctly.

Background

Rather than modifying the default syslogd rules file, this lab creates a new training-specific rules file at: `$OMNIHOME/probes/linux2x8 training.rules`

This approach preserves the default `syslogd probe` rules file and makes it clear which content belongs to this training exercise.

All OMNIbus probes generate internal self-monitoring events known as ProbeWatch events. These are sent by the probe itself to the ObjectServer to indicate that the probe has started, is running, or is shutting down. The `@Manager` field of these events is set to `"ProbeWatch"` by the probe, which makes it straightforward to identify and handle them separately from real events. The ProbeWatch stanza must always be present in a rules file, irrespective of the probe type.

Tasks

1. On the lab server, create the new training rules file using a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Enter the following content as the starting rules file. This is the minimum viable rules file — it handles ProbeWatch events and provides an empty `else` block where all real event processing will be added:

```
if( match( @Manager, "ProbeWatch" ) )
{
        switch(@Summary)
        {
```

```
        case "Running ...":
                @Severity = 1
                @AlertGroup = "probestat"
                @Type = 2
        case "Going Down ...":
                @Severity = 5
                @AlertGroup = "probestat"
                @Type = 1
        default:
                @Severity = 1
        }
        @AlertKey = @Agent
        @Summary = @Agent + " probe on " + @Node + ": " + @Summary
}
else
{
   # event processing begins here
}
```

Review the structure of the rules file and identify the two main blocks: the ProbeWatch block, which handles the probe's own self-monitoring events, and the empty `else` block, which is where all real incoming event processing will be added in the labs that follow.

Save and close the file.

3. Stop the syslogd probe if it is still running:

```
kill $(pgrep nco_p_syslogd)
```

4. Use the probe rules syntax checker to validate your new rules file.

   The `nco_p_syntax` utility is a standalone **rules syntax checker** for Netcool probes. It reads a rules file, checks the syntax, connects to the requested ObjectServer to validate the existence of fields referenced in the rules, and reports any errors without having to start the actual probe.

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

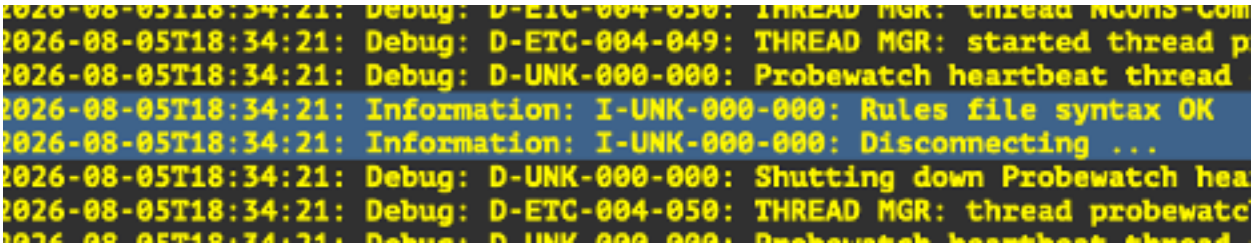


Figure 63: Probe syntax checker command

If the syntax is correct, the command exits without error and prints no syntax error messages. You will see the entry "Rules file syntax OK".

If errors are reported, note the line numbers that are shown to be in error, open the rules file, and correct the problems before proceeding.

5. Start the probe with the training rules file, using the `-rulesfile` command line switch to specify our custom rules file:

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  \
-messagelog $OMNIHOME/log/syslogd-training.log &
```

6. Verify that the probe is running:

```
ps -ef | grep nco_p_syslogd
```

If the probe is not running, you can check the log file for errors:

```
cat $OMNIHOME/log/syslogd-training.log
```

7. Next, re-open our `syslogd-training.rules` file and add the following lines to the `else` block in the rules file. Open the file in a text editor and place these lines between the `else` block's { and }:

```
        log(DEBUG, "===== Entering my rules file section")
        @Manager = "MyProbe running on " + hostname()
        @AlertGroup = "Network"
        log(DEBUG, "===== Exiting my rules file section")
```

The `log(DEBUG, ...)` statements write diagnostic output to the probe log file. The `hostname()` function returns the hostname of the server the probe is running on. The full `else` block should now look like this:

```
else
{
      # event processing begins here

        log(DEBUG, "===== Entering my rules file section")
        @Manager = "MyProbe running on " + hostname()
        @AlertGroup = "Network"
        log(DEBUG, "===== Exiting my rules file section")
}
```

In this case, we are setting the Manager field of the alert to the value of the hostname() function, and setting the AlertGroup field to the literal string "Network". Note that event fields are referenced with the @ symbol preceding the name. Tokens are referenced with the $ symbol.

Save and close the file.

8. Stop the probe, syntax-check the updated rules file, and restart the probe:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

9. Next, generate a test event using the following `logger` command: `bash  logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface GigabitEthernet0/0 : unknown reason code'`

10. Open the Web GUI event list, change your filter and views to the ones you created in the previous lab (SyslogdProbe) and locate the resulting alert. Note that `@Manager` and `@AlertGroup` are now populated, but other fields such as `@Node`, `@Summary`, `@AlertKey`, and `@Identifier` are not yet set by the rules. `@Severity` is also **indeterminate**.

11. Examine the raw capture file to review the tokens generated from the incoming syslog message:

```
cat $OMNIHOME/var/syslogd.cap
```

Note the $Raw token, which contains the full original message text, and the positional tokens such as $1, $2, and so on, which are generated by splitting the message on whitespace. Note that because syslog messages vary in structure, positional tokens are unreliable for extracting specific values — the same piece of information may appear at a different position in a different message type. For this reason, the labs that follow will use regular expression extraction from $Raw rather than positional tokens.

Validation

- The `syslogd-training.rules` file exists and passes syntax check.
- The probe starts successfully using the training rules file.
- A test event generated with `logger` appears in the event list with `@Manager` and `@AlertGroup` populated.

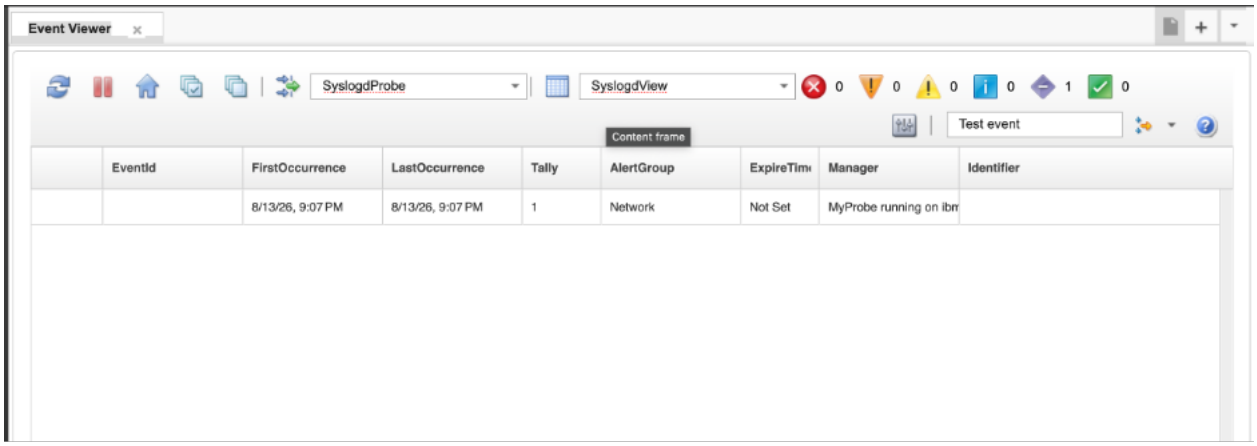


Figure 64: Raw capture output view

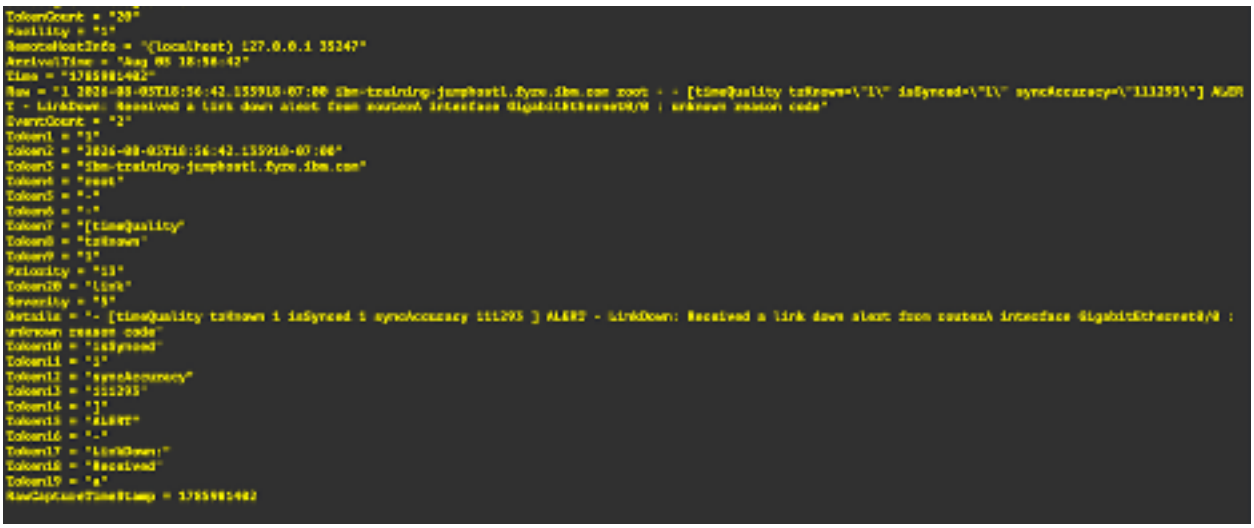


Figure 65: Raw capture output view

- The raw capture file shows the $Raw token containing the full original syslog message.

Expected result

At the end of this lab, the student has a working training rules file, understands the role of the ProbeWatch stanza, and has observed the difference between what raw token output looks like and what a fully mapped alert should look like. The student also understands why $Raw and regular expression extraction are preferred over positional tokens for syslog-based event processing.

4.3.2 Extracting EventId Objective

Use the `extract()` function to populate the `@EventId` field from the raw syslog message, and understand the difference between greedy and non-greedy regular expression matching.

Background

The `extract()` function applies a regular expression to a string and returns the content of the first capture group `()`. It is the primary tool for pulling specific values out of unstructured text such as syslog messages. Regular expressions are used heavily in processing events and alerts in rules.

In this lab example alert for our purposes, the convention is that the event **type** always follows the text `ALERT -` in the message, and ends at the first colon (`:`). A regular expression is used to extract whatever appears between those two boundaries and assign it to `@EventId`, which will then be used in subsequent labs to drive a switch/case structure.

> **Note** Event types can be used to classify alerts according to the condition they report. Common examples that indicate issues might be LINK DOWN, INTERFACE UP, DISK FULL, HIGH CPU, MEMORY THRESHOLD, and APPLICATION UNAVAILABLE. Classifying events in this way makes it easier to apply consistent processing rules. In later labs, the extracted @EventId value will be used in a switch or case structure to apply event-specific logic, such as assigning a severity, setting an alert group, identifying a resolution event, or adding enrichment fields.
>
> **Note:** Understanding the difference between greedy and non-greedy matching is important when writing extraction patterns. A greedy quantifier such as `.*` will match as much text as possible — so `.*:` would match all the way to the *last* colon in the string. A non-greedy quantifier such as `.*?` will match as little text as possible — stopping at the *first* colon. In a message like `ALERT - LinkDown: Received a link down alert from routerA interface GigabitEthernet0/0 : unknown reason code`, using a greedy match to the colon would extract `LinkDown: Received a link down alert from routerA interface GigabitEthernet0/0` — far more than intended. The non-greedy `.*?` stops at the first colon and correctly extracts `LinkDown`.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Add the following line to the `else` block, after the `@AlertGroup` assignment:

```
        @EventId = extract($Raw, ".*ALERT - (.*?):.*")
```

   The full `else` block should now look like this:

```
else
{
   # event processing begins here

        log(DEBUG, "===== Entering my rules file section")
        @Manager = "MyProbe running on " + hostname()
        @AlertGroup = "Network"
        @EventId = extract($Raw, ".*ALERT - (.*?):.*")
        log(DEBUG, "===== Exiting my rules file section")
}
```

   Save and close the file.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  \
 -messagelog $OMNIHOME/log/syslogd-training.log &
```

If the syntax check reports errors, open the log file for details:

```
cat $OMNIHOME/log/syslogd-training.log
```

4. Delete the previous alert from the event list so the result is clear before testing. Right-click on the alert and select the `Delete` tool:

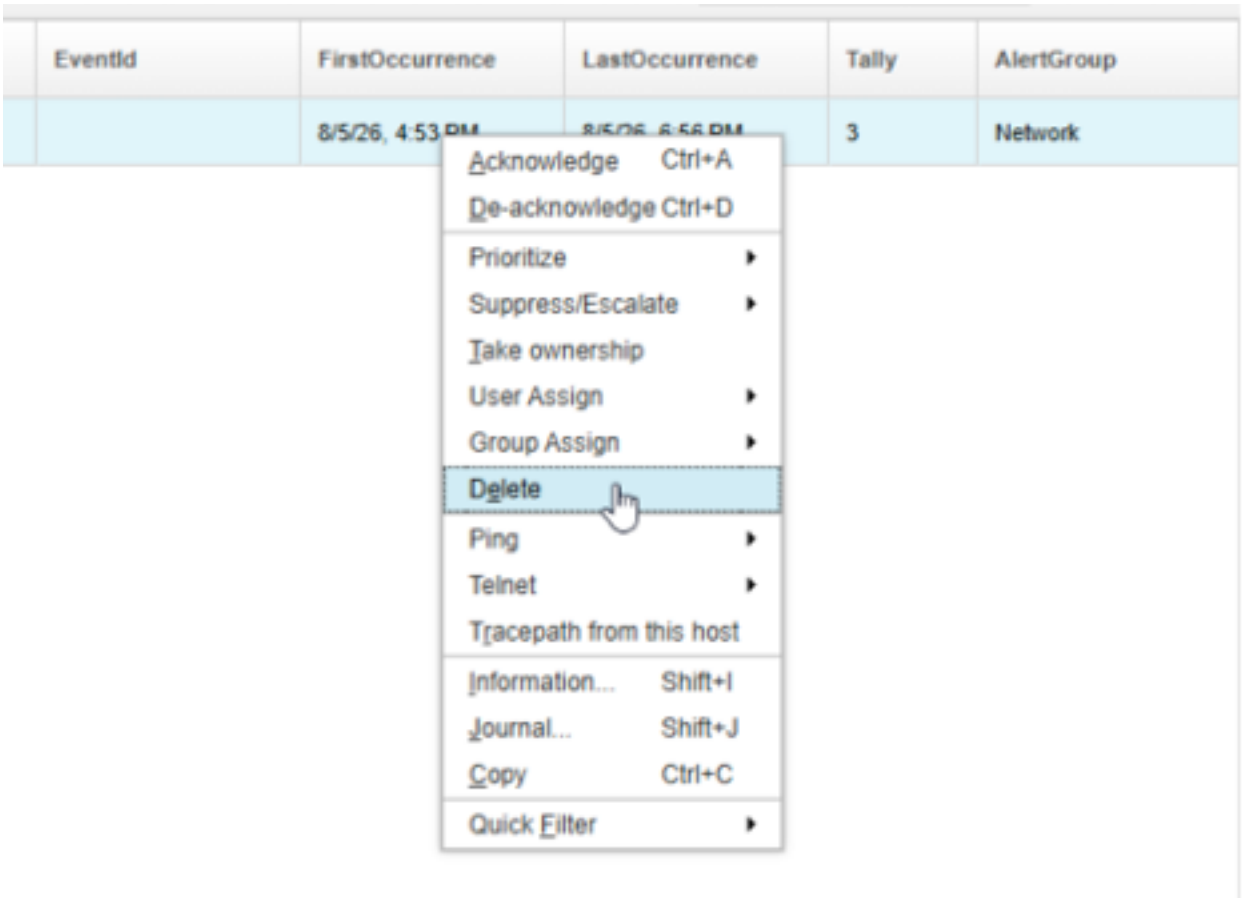


Figure 66: Delete previous alert in the event list

5. Generate the test event again:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
 ↪  GigabitEthernet0/0 : unknown reason code'
```

6. Locate the alert in the event list and confirm that the `@EventId` field now shows `LinkDown`. This confirms that the non-greedy extraction correctly stopped at the first colon rather than continuing to the last.

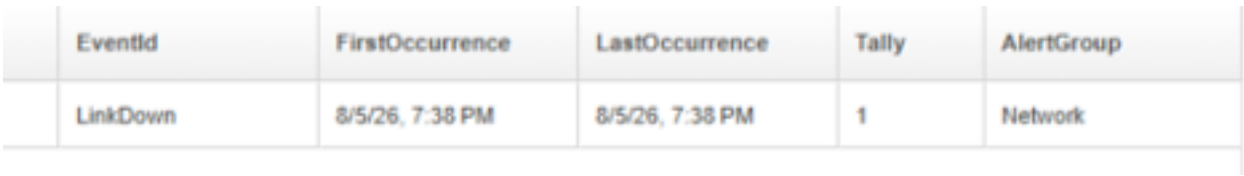


Figure 67: EventId displayed in the event list

Validation

- The rules file passes syntax check.
- The `@EventId` field in the resulting alert shows `LinkDown`.
- The student can explain why `.*?` is used instead of `.*` in this extraction pattern.

Expected result

At the end of this lab, the student can use `extract()` to pull specific values from unstructured text and understands the practical difference between greedy and non-greedy matching.

4.3.3 Switch/case field mapping Objective

Add a `switch/case` structure to the rules file to handle multiple event types, map additional alert fields for the `LinkDown` event type, and provide a default handler for unrecognized events.

Background

Now that `@EventId` is populated, a `switch/case` structure can be used to branch the rules logic based on the event type. Each `case` handles one specific event type and maps the fields appropriate for that type. The `default` case handles any event type that does not match a defined case, ensuring that unrecognized events are still forwarded to the ObjectServer with a minimal but informative field set rather than being silently dropped or left unmapped.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Replace the content of the `else` block with the following. The `switch` is placed after the `@EventId` extraction so that the switch can branch on the extracted value:

```
else
{
        log(DEBUG, "===== Entering my rules file section")
        @Manager = "MyProbe running on " + hostname()
        @AlertGroup = "Network"
        @EventId = extract($Raw, ".*ALERT - (.*?):.*")

        switch(@EventId)
        {
                case "LinkDown":
                        log(INFO, "===== Entering LinkDown segment")
                        @Node = extract($Raw, ".* alert from (.*?) .*")
                        @Summary = extract($Raw, ".* ALERT - LinkDown: (.*)")
                        @AlertKey = extract($Raw, ".* interface (.*?) .*")
                        @Type = 1
                        @Severity = 5
                default:
                        log(INFO, "===== Found an unknown/unhandled event")
                        @Summary = "Unhandled event: " + $Raw
                        @Severity = 1
        }

        log(DEBUG, "===== Exiting my rules file section")
}
```

Here we perform the following extractions and OMNIbus field assignments when we receive an event where the @EventId is `LinkDown`:

- `@Node = extract($Raw, ".* alert from (.*?) .*")`
  - Searches for the phrase `alert from`.
  - Captures the shortest sequence of characters after `alert from` up to the next space using parentheses for the capture group `(.*?)`.
  - In our example alert, this yields `routerA`, so `@Node` is set to the router name.

- `@Summary = extract($Raw, ".* ALERT - LinkDown: (.*)")`
  - Matches everything up to `ALERT - LinkDown:`.
  - Captures everything after that prefix on the line using `(.*)`.
  - For our example alert, it captures `Received a link down alert from routerA interface GigabitEthernet0/0 : unknown reason code`, so `@Summary` becomes the full descriptive text.
- `@AlertKey = extract($Raw, ".* interface (.*?) .*")`
  - Finds the word `interface`.
  - Captures the shortest sequence of characters immediately after `interface` up to the next space using `(.*?)`.
  - In the example alert, this is `GigabitEthernet0/0`, so `@AlertKey` uniquely identifies the interface that went down.
- `@Type = 1`
  - Assigns a numeric type code of 1.
  - This is used to classify the alert as a `problem` alert for correlating pairs (more on this later).
- `@Severity = 5`
  - Sets the numeric severity level to 5 for any `LinkDown` alert.

  Save and close the file.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

   If the syntax check fails, examine the log file:

```
cat $OMNIHOME/log/syslogd-training.log
```

4. Return to the WebGUI event viewer and delete the previous alert from the event list.
5. Generate the test event:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
 ↪  GigabitEthernet0/0 : unknown reason code'
```

6. Locate the alert in the event list and confirm that `@Node`, `@Summary`, `@AlertKey`, `@Type`, and `@Severity` are now populated correctly from the `LinkDown` case statement.

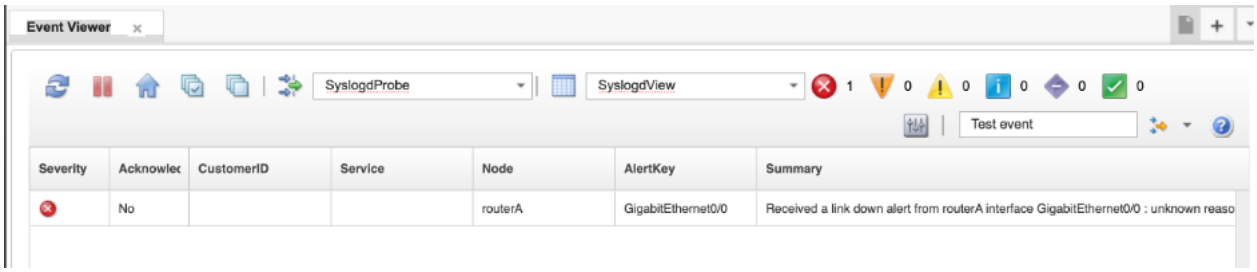


Figure 68: Mapped LinkDown alert fields in the event list

7. Now test the default case by generating an event that does not match any defined case:

```
logger -n 127.0.0.1 'ALERT - LinkUp: Received a link up alert from routerA interface
 ↪  GigabitEthernet0/0 : peer re-established'
```

   Confirm that this event appears in the event list with a summary of `Unhandled event:` followed by the full raw message text, and a severity of 1. Also have a look at the `Count` field and see that it has incremented. This shows how the `default:` case acts as a catch-all for any event that does not match one of the specific patterns. However, it deduplicated with the existing LinkDown alert. This is due to the fact we have not yet set the `@Identifier` for our alerts in the rules.

Validation

- The rules file passes syntax check.

- The `LinkDown` event populates `@Node`, `@Summary`, `@AlertKey`, `@Type = 1`, and `@Severity = 5`.
- An unrecognized event type hits the `default` case and appears with `@Severity = 1` and a summary beginning with `Unhandled event:`.

Expected result

At the end of this lab, the student understands how to use `switch/case` to branch rules logic by event type, and has a working structure that can be extended with additional cases in subsequent labs.

4.3.4 Deduplication with Identifier Objective

Add an `@Identifier` value to the `LinkDown` case and observe deduplication behavior in the ObjectServer.

Background

As covered in section 3.3, the `deduplication` trigger fires when an incoming event matches an existing alert by `@Identifier`. Without an `@Identifier` set in the rules, all alerts with no @Identifier would simply deduplicate into one row. Conversely, with an @Identifer that is too unique (for example, containing a time stamp), each occurrence of the same event will be inserted as a new row in `alerts.status`, quicly filling up the view with redundant alarms. Once an appropriate `@Identifier` is set, repeated occurrences of the same condition update the existing alert rather than creating duplicates — incrementing the `Count` field and updating `LastOccurrence`.

The `@Identifier` should be composed of fields that together uniquely identify the specific condition: typically `@Node`, `@AlertKey`, `@EventId`, and `@Manager`. Including `@Manager` ensures that the same condition reported by different probes or management systems does not incorrectly deduplicate.

> **Note:** In this lab the `LinkDown` `@Identifier` does not include `@Type`. This is intentional at this stage — the next lab (4.3.5) introduces the `LinkUp` resolution event, which uses a distinct `@EventId` value (`"LinkUp"` vs `"LinkDown"`), meaning the two events will never share the same `@Identifier` regardless of whether `@Type` is included. You could choose to append `@Type` to the `LinkDown` identifier for consistency, and it would make no difference to the outcome — the problem and resolution events would still not deduplicate against each other.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Add the `@Identifier` assignment statement to the case statement for the `LinkDown` alert:

```
                case "LinkDown":
                        log(INFO, "===== Entering LinkDown segment")
                        @Node = extract($Raw, ".* alert from (.*?) .*")
                        @Summary = extract($Raw, ".* ALERT - LinkDown: (.*)")
                        @AlertKey = extract($Raw, ".* interface (.*?) .*")
                        @Type = 1
                        @Severity = 5
                        @Identifier = @Node + " : " + @AlertKey + ":" + @EventId + ":" +
 ↪  @Manager
```

   This identifier combines the most important pieces of context for the alert: the device (`@Node`), the specific resource on that device (`@AlertKey`), the event type (`@EventId`), and the source probe (`@Manager`). Putting these together into one string makes it much easier to deduplicate repeated events for the same interface and device, while still distinguishing different event types and probes. It also produces an identifier that is human-readable, which helps when troubleshooting in the event list.

   Save and close the file.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

4. Return to the Web GUI event viewer and delete the previous alert from the event list.
5. Generate the LinkDown event:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
 ↪  GigabitEthernet0/0 : unknown reason code'
```

6. Locate the alert in the event list. Note the values of FirstOccurrence, LastOccurrence, and Tally.
7. Generate the same event two or three more times:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
 ↪  GigabitEthernet0/0 : unknown reason code'
```

8. Refresh the event list and observe that no new rows were created. Instead, the existing alert has been updated — LastOccurrence has advanced and Count has incremented for each repeated occurrence. This is the deduplication behavior driven by the @Identifier field and the deduplication trigger reviewed in section 3.3.
9. Now send a second LinkDown event from a different device and interface to confirm that a different @Identifier produces a separate alert row:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerB interface
 ↪  GigabitEthernet1/11 : unknown reason code'
```

   Refresh the event list again and confirm that the new alert appears as a separate row because the value of @Identifier is different for routerB and GigabitEthernet1/11.

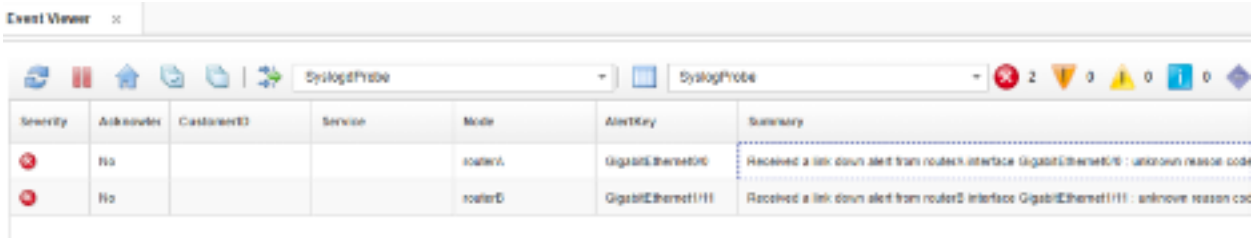


Figure 69: Separate alert row for routerB and GigabitEthernet1/11

Validation

- The rules file passes syntax check.
- Sending the same event multiple times results in a single alert in the event list, not multiple rows.
- The Count field increments and LastOccurrence updates with each repeated occurrence.
- The student can explain how @Identifier drives this behavior through the ObjectServer deduplication trigger.

Expected result

By the end of this lab, you understood how to design a meaningful @Identifier value and why it matters. You saw how the ObjectServer uses @Identifier to **deduplicate** events, so repeated alerts for the same device and interface updated a single record instead of creating multiple separate ones. This gave you a concrete example of how good identifier design improves both event quality and operator usability.

4.3.5 Down/Up correlation Objective

Add a LinkUp case to the rules file and observe how a resolution event correlates with and clears the earlier LinkDown problem event through the generic_clear ObjectServer automation.

Background

As covered in section 3.3, the generic_clear trigger matches Type 2 resolution events against Type 1 problem events using the combination of @Node, @AlertGroup, @AlertKey, and @Manager. For correlation to work, all four of these fields must match between the problem and resolution events, and the resolution event must have a later LastOccurrence than the problem event.

The `@Identifier` for the resolution event should be distinct from the problem event's `@Identifier` to prevent the two from deduplicating against each other. Appending `@Type` to the `@Identifier` achieves this — the problem event (`@Type = 1`) and resolution event (`@Type = 2`) will have different `@Identifier` values and will not deduplicate, but they will still correlate and clear through `generic_clear`.

Tasks

1. Open the training rules file in a text editor:

   ```
   vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
   ```

2. Add a new case statement for our `LinkUp` case to the `switch` block, immediately after the `LinkDown` case and before the `default` case statement:

   ```
                   case "LinkUp":
                           log(INFO, "===== Entering LinkUp segment")
                           @Node = extract($Raw, ".* alert from (.*?) .*")
                           @Summary = extract($Raw, ".* ALERT - LinkUp: (.*)")
                           @AlertKey = extract($Raw, ".* interface (.*?) .*")
                           @Type = 2
                           @Severity = 1
                           @Identifier = @Node + " : " + @AlertKey + ":" + @EventId + ":" +
    ↪  @Manager + ":" + @Type
   ```

   Note that we have set the `@Type` field of this alert to 2, which indicates that it is a `resolution` alert, and we have set the `@Severity` to 1, which is in turn set to 0 by the `generic clear` automation once the original problem alert has been set to clear. We also append the `@Type` field to the end of the `@Identifier`.

   Save and close the file.

3. Stop the probe, syntax-check, and restart:

   ```
   kill $(pgrep nco_p_syslogd)
   ```

   ```
   $OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
    ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
   ```

   ```
   $OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
    ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
   ```

4. Return to the WebGUI event viewer and delete any existing alerts from your event list.

5. Generate the `LinkDown` **problem** event:

   ```
   logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
    ↪  GigabitEthernet0/0 : unknown reason code'
   ```

6. Confirm that the `LinkDown` alert appears in the event list with `@Severity = 5` and `@Type = 1`. If it does not appear immediately, refresh the event list periodically until it does.

7. Generate the `LinkUp` resolution event:

   ```
   logger -n 127.0.0.1 'ALERT - LinkUp: Received a link up alert from routerA interface
    ↪  GigabitEthernet0/0 : unknown reason code'
   ```

8. Observe the event list. Within the next `generic_clear` cycle (which runs every 5 seconds), both the `LinkDown` and `LinkUp` events will be cleared — their severity will be set to 0. If they are not visible after clearing, confirm that the event list filter is set to show cleared events, or note that they have been removed from the active view as expected.

   This clearing behavior is entirely driven by the ObjectServer `generic_clear` trigger reviewed in section 3.3 — no additional probe rules or ObjectServer configuration was required beyond setting `@Type`, `@Node`, `@AlertGroup`, `@AlertKey`, and `@Manager` correctly and consistently between the two events.

Validation

- The `LinkDown` event appears in the event list with `@Severity = 5` and `@Type = 1`.
- The `LinkUp` event appears in the event list with `@Severity = 1` and `@Type = 2`.

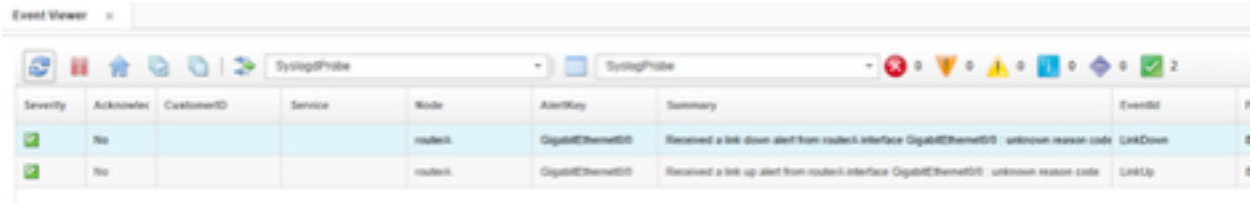


Figure 70: LinkDown and LinkUp events cleared by generic_clear

- Both events are cleared within seconds of the LinkUp event arriving, driven by the generic_clear trigger.
- You understand which fields must match between the problem and resolution events for generic_clear to work.

Expected result

At the end of this lab, the student understands how to design problem and resolution event pairs in probe rules so that they correlate and clear automatically through the ObjectServer generic_clear automation.

4.3.6 Conditional logic Objective

Add a LinkUtilization case that uses if / else if / else logic to set event severity based on the extracted utilization value, and understand how conditional rules logic enables dynamic severity assignment.

Background

Not all events have a fixed severity based on the event time — some may carry a value in the event text that should determine how the event is classified. Probe rules support if / else if / else conditional logic to handle these cases. In this lab, a utilization percentage is extracted from the event text and used to set @Severity and @Type according to defined thresholds.

Note that when comparing extracted values numerically, they must be cast to int() before the comparison. Extracted values are strings by default, and using <, >, or <= on strings will cause a parse error.

Also note that in this scenario the @Identifier includes @Type. Unlike LinkDown and LinkUp which are distinct event types, LinkUtilization has no separate and distinct clearing event — the same event type serves as both the problem and the resolution depending on the utilization value. If @Type were not included in the @Identifier, a resolution event (Type 2) would deduplicate against a problem event (Type 1) and the existing alert would not be cleared correctly. Appending @Type to the @Identifier keeps problem and resolution events separate in the ObjectServer.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Add the LinkUtilization case to the switch block, after the LinkUp case and before the default case:

```
                case "LinkUtilization":
                        log(INFO, "===== Entering LinkUtilization segment")
                        @Node = extract($Raw, ".* link utilization for (.*?) .*")
                        @Summary = extract($Raw, ".* ALERT - LinkUtilization: (.*)")
                        @AlertKey = extract($Raw, ".* ALERT.* interface (.*?) is.*")
                        # extract the utilization value from the alert into a token named
↪  $utilval, which will be used for the conditional
                        $utilval = extract($Raw, ".* ALERT.* is (.*)%.*")
                        log(INFO, "$utilval is " + $utilval)
                        if (int($utilval) < 70)
                        {
                                # less than 70% is a resolution — clears previous alerts
                                @Severity = 1
                                @Type = 2
                        }
                        else if (int($utilval) <= 80)
                        {
                                # 70–79% is a warning
                                @Severity = 3
```

```
                        @Type = 1
                }
                else if (int($utilval) <= 95)
                {
                        # 80-94% is major
                        @Severity = 4
                        @Type = 1
                }
                else if (int($utilval) <= 100)
                {
                        # 95-100% is critical
                        @Severity = 5
                        @Type = 1
                }
                else
                {
                        # unknown or non-integer value - flag as critical
                        @Summary = @Summary + " (unknown or non-integer utilization
 ↪  value)"
                        @Type = 1
                        @Severity = 5
                }
                @Identifier = @Node + " : " + @AlertKey + ":" + @EventId + ":" +
 ↪  @Manager + @Type
                update(@Severity)
```

Save and close the file.

> **Note:** The `update(@Severity)` statement explicitly instructs the ObjectServer to overwrite the `@Severity` field of the existing alert with the value from the incoming event when deduplication occurs. Without this, only `@Summary` and `@AlertKey` are updated automatically on deduplication (refer back to the deduplication trigger) — all other fields retain their original values. For `LinkUtilization`, where severity changes with each new reading, this statement is essential to ensure the alert @Summary reflects the current condition.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

If the syntax check fails, examine the log file for clues:

```
cat $OMNIHOME/log/syslogd-training.log
```

Restart the probe when the sytax check succeeds:

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

4. Return to the Web GUI and delete any existing alerts from the event list.
5. Generate a warning-level utilization event (77% — falls in the 70–79% range, Severity 3):

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 77%'
```

Confirm the alert appears in the event list with `@Severity = 3`, and make note of the @Summary. (You may need to refresh the list a few times for it to show up)

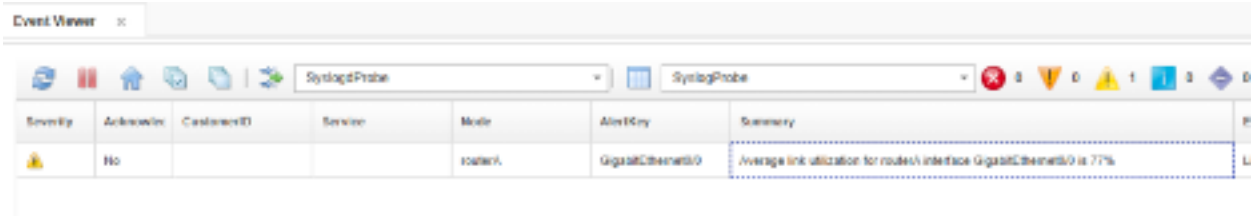


Figure 71: Warning-level utilization alert with Severity 3

6. Generate a major-level utilization event (84% — falls in the 80–94% range, Severity 4):

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 84%'
```

   Because both the 77% and 84% events produce `@Type = 1` and share the same `@Node`, `@AlertKey`, `@EventId`, and `@Manager`, their `@Identifier` values are identical. This means the 84% event deduplicates against the 77% alert rather than creating a new row. Because `update(@Severity)` is explicitly set in the rules, the existing alert's severity is updated from 3 to 4, reflecting the worsening condition. This is intentional — the alert maintains state as the utilization level changes over time, rather than creating a new alert for each new reading.

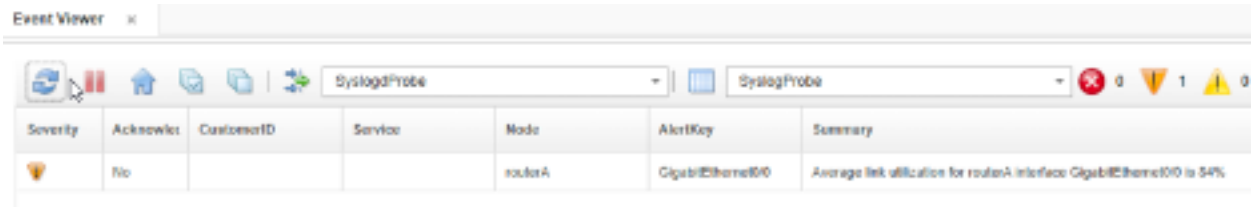


Figure 72: Utilization alert updated to Severity 4 with deduplication

7. Generate a critical-level event at 98% again:

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 98%'
```

   Confirm that the alert @Summary and @Severity change to reflect our rules criteria.

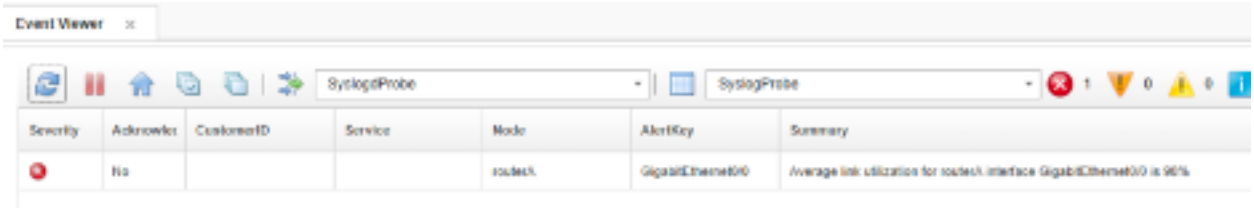


Figure 73: LinkUtilization alert deduplicated with updated Count and LastOccurrence

8. Finally, generate a resolution event (25% — below 70%, Severity 1, Type 2):

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 25%'
```

   Observe that the resolution event is forwarded to the ObjectServer, indicated by falling below our threshold, and within the next `generic_clear` cycle, the problem event is cleared. Subsequently the delete_clears will reap the event if it doesn't occur again (within 120 seconds of being cleared):

Validation

- The rules file passes syntax check.
- A 77% utilization event produces `@Severity = 3`.
- An 84% utilization event produces `@Severity = 4`.
- An 98% utilization event produces `@Severity = 5`.
- A 25% utilization event triggers clearing of the earlier problem alert via `generic_clear`.

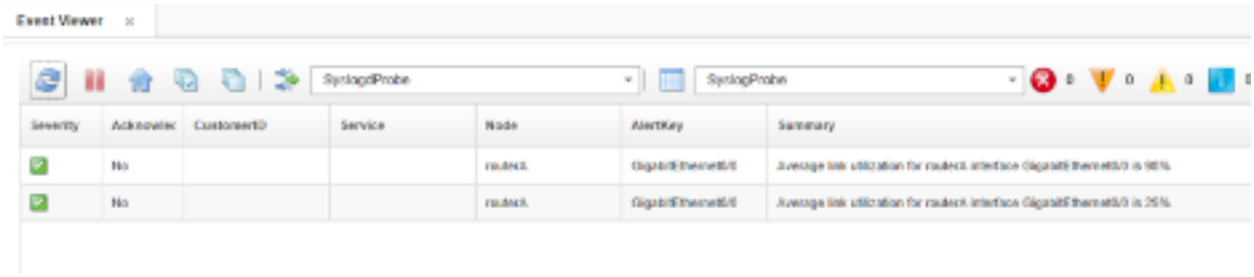


Figure 74: Resolution event cleared by generic_clear

- You can explain why `int()` is required when comparing extracted values.
- Youcan explain why `@Type` is included in the `@Identifier` for this event type.

Expected result

At the end of this lab, the student can write conditional rules logic to assign severity dynamically based on extracted values, understands the importance of `int()` casting, and understands how `@Type` in the `@Identifier` prevents incorrect deduplication between problem and resolution events of the same type.

4.3.7 Lookup tables Objective

Create a lookup file that maps device names to customer data, declare the table in the rules file, and use it to enrich incoming alerts with the `@CustomerID` field populated from external data.

Background

Lookup tables allow probe rules to enrich events with data that is not present in the original event message. Rather than embedding customer, location, or service data in the rules file itself, a lookup table stores that data in an external file and makes it available to the rules at processing time. The rules file declares the table and then performs a lookup using a key value extracted from the event — typically `@Node`, `@AlertKey`, or another identifying field — to retrieve the associated data and assign it to one or more alert fields.

This pattern is widely used in production OMNIbus environments. Common examples include:

- **Customer enrichment** — mapping device hostnames to customer names, account numbers, or service tiers in a managed services environment.
- **Location data** — mapping node names to physical site, building, or data centre location so operators can immediately see where a fault is occurring.
- **Contact information** — mapping nodes or alert groups to on-call team names or escalation contacts.
- **Service classification** — mapping devices or alert groups to business service names so events can be correlated to the services they affect.
- **Severity overrides** — mapping specific node and alert group combinations to a predefined severity, allowing business rules to be applied at ingestion time without modifying the probe rules logic.

In this lab, a lookup table maps router names to customer names and service levels. The `@CustomerID` field — added to `alerts.status` in section 3.2 — will be populated from the lookup result.

Prerequisites

- Completion of section 4.3.6.
- The `CustomerID` custom column must exist in `alerts.status`. If section 3.2 has not been completed, return to that section and add the `CustomerID` column before proceeding.

Tasks

1. On the lab server, create the lookup file:

   ```
   vi $OMNIHOME/probes/linux2x86/CUSTOMERS.lookup
   ```

   Enter the following content. Each line consists of three fields separated by a single tab character: the node name, the customer name, and the service level. Take care not to leave any extraneous lines in the file, only these three lines. Blank or any lines that do not match three tab-separated lines will cause the syntax checker to fail:

```
routerA  ACME, Inc.  Gold
routerB  Candy's Candies Platinum
routerC  Mad Martin's Menagerie  Silver
```

Save and close the file.

> **Note:** The separators between fields must be literal tab characters, not spaces. If spaces are used instead, the lookup will fail silently and the fields will not be populated. You can use the `set list` directive in the `vi` editor to validate that the separaters are actual tabs, showing as `^I` when set list is enabled:

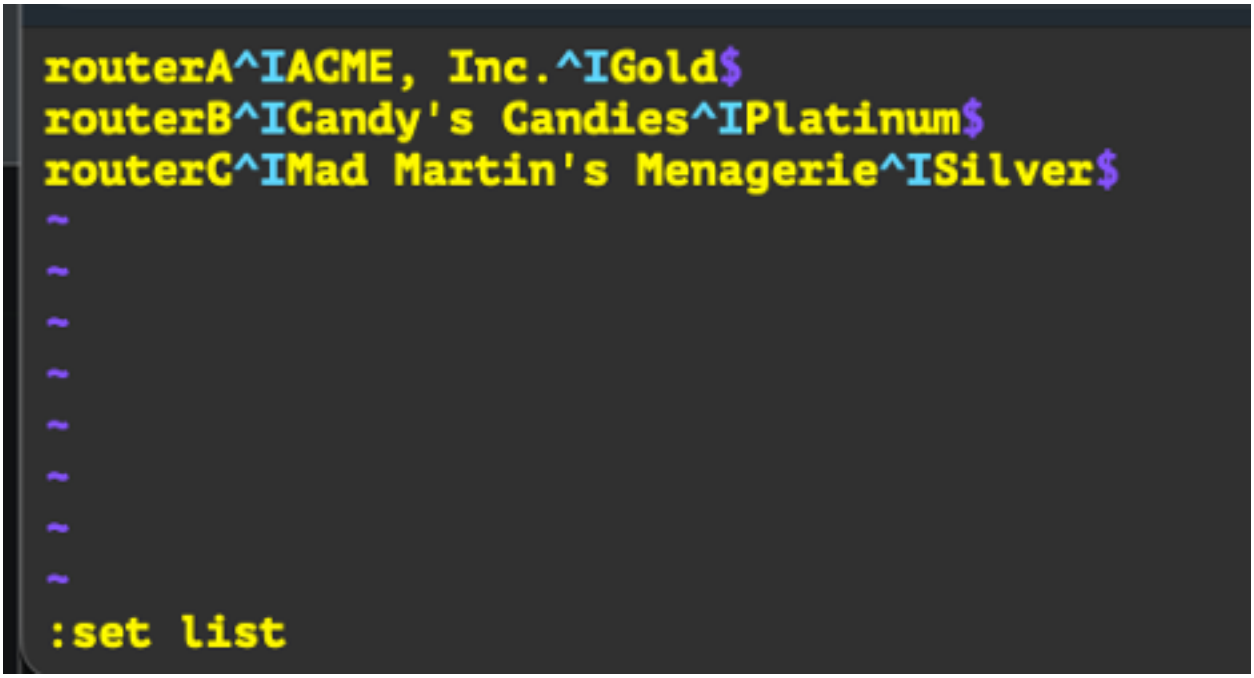


Figure 75: CUSTOMERS.lookup file with tab-separated fields

> **Note:** Also be aware of any extraneous blank lines and remove them, or your syntax checker will fail with an **"incorrect column count"** error when validating the lookup file.

2. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

3. Add the following table declaration at the very top of the rules file, before any other content. The table must be declared before any event processing occurs:

```
table custdata = "/opt/IBM/tivoli/netcool/omnibus/probes/linux2x86/CUSTOMERS.lookup"
```

The top few lines the rules file should now look like this:

```
table custdata = "/opt/IBM/tivoli/netcool/omnibus/probes/linux2x86/CUSTOMERS.lookup"

if( match( @Manager, "ProbeWatch" ) )
{
...
```

> **Note:** The path inside the `table` declaration must be the full absolute path. The `$OMNIHOME` shell variable is not expanded inside rules file string literals without extra configuration — the probe reads the path as written, not through the shell environment.

4. Add the following lookup statement to the `else` block, after the `switch` statement and before the closing `log(DEBUG, ...)` line. This performs the lookup using `@Node` as the key and assigns the results to `@CustomerID` and `@Service`, and runs on every event since it is outside of the switch/case statement and is the last directive in the rules:

```
        [ @CustomerID, @Service ] = lookup(@Node, custdata)
```

The end of the `else` block should now look like this: “‘text }

```
    [ @CustomerID, @Service ] = lookup(@Node, custdata)
    log(DEBUG, "===== Exiting my rules file section")
```

} “‘ Save and close the file.

> **Note:** `@CustomerID` maps to the custom column added to `alerts.status` in section 3.2. `@Service` is a standard string field in the default `alerts.status` schema and does not require a custom column. The lookup assigns both values in a single operation using the multi-field assignment syntax `[ field1, field2 ] = lookup(key, table)`.

5. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

If the syntax check fails, examine the log file: `cat $OMNIHOME/log/syslogd-training.log`

6. Return to the Web GUI and delete any existing alerts from the event list.
7. Generate test events for each of the three nodes defined in the lookup file:

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 77%'
```

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerB interface
 ↪  GigabitEthernet0/0 is 84%'
```

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerC interface
 ↪  GigabitEthernet0/0 is 98%'
```

8. Open the event list and locate the three resulting alerts. Confirm that the `@CustomerID` and `@Service` fields are populated for each event:
   - `routerA` — `CustomerID: ACME, Inc.` / `Service: Gold`
   - `routerB` — `CustomerID: Candy's Candies` / `Service: Platinum`
   - `routerC` — `CustomerID: Mad Martin's Menagerie` / `Service: Silver`

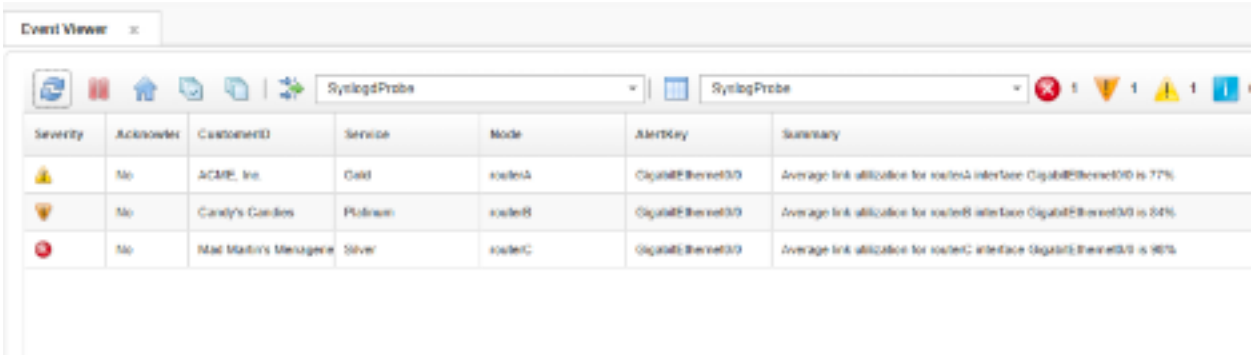


Figure 76: Lookup results in event list showing CustomerID and Service columns

9. Generate a test event for a node that does not exist in the lookup file:

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerX interface
 ↪  GigabitEthernet0/0 is 77%'
```

Observe that the resulting alert has empty `@CustomerID` and `@Service` fields. This is expected behavior — when a lookup key has no matching entry in the table, the target fields are left empty rather than causing an error.

Validation

- The `CUSTOMERS.lookup` file exists with three entries separated by tab characters.
- The `table` declaration appears at the top of the rules file before any event processing.
- The rules file passes syntax check.
- Events from `routerA`, `routerB`, and `routerC` produce alerts with correctly populated `@CustomerID` and `@Service` fields.
- An event from an unknown node produces an alert with empty `@CustomerID` and `@Service` fields.
- You are able to explain the business value of lookup table enrichment and give examples of other data that could be stored in a lookup file.

Expected result

You now understand how to create and declare a lookup table, use an event field as the key to perform a lookup, and enrich alerts with external data at ingestion time. You also see how this pattern can be applied to a wide range of business enrichment use cases in a production OMNIbus environment.

4.3.8 Discarding events Objective

Add a `discard` statement to the rules file to suppress unwanted events and verify that discarded events do not appear in the ObjectServer event list.

Background

Not every event that reaches a probe needs to be forwarded to the ObjectServer. Some events are informational noise, low-value status messages, or event types that are simply not relevant to the operations team. Rather than letting these events flow through to the event list and contribute to alert noise, probe rules can use the `discard` statement to suppress them at the probe level before they are ever sent to the ObjectServer.

When a `discard` statement is encountered, the probe marks the event for suppression. However, it is important to understand that the probe continues to process the event through the remainder of the rules file — the event is not immediately dropped at the point the `discard` statement is reached. This means it is possible to recover a discarded event later in the rules using the `recover` statement if subsequent logic determines the event should be kept after all. If `discard` has been called and `recover` is not called before the end of the rules file, the event is silently dropped and never forwarded to the ObjectServer.

Common uses for discard in production environments include suppressing keepalive or heartbeat messages, dropping low-severity informational events that have no operational value, and filtering out event types that are handled by a different probe or management system.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Add a `WirelessAssociation` case to the `switch` block to handle inactionable WAP association events. It can be placed in any position within the switch, but by convention discard cases are often grouped together. Add the following after the `LinkUtilization` case and before the `default` case:

```
                case "WirelessAssociation":
                        log(INFO, "===== Discarding wireless association alert")
                        discard
```

   Save and close the file.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

If the syntax check fails, examine the log file for clues:

```
cat $OMNIHOME/log/syslogd-training.log
```

4. Generate the following test event:

```
logger -n 127.0.0.1 'ALERT - WirelessAssociation: 802.11 association for SSID abc123 from
 ↪  client 00:00:00:00:00'
```

5. Open the event list and confirm that no new alert has appeared for this event. The `WirelessAssociation` event should not be present in the ObjectServer.
6. To confirm the event was received and processed by the probe, check the probe log file and verify that the `log(INFO, ...)` message appears, showing that the rules did execute for this event before it was discarded:

```
cat $OMNIHOME/log/syslogd-training.log
```

Look for the line `===== Discarding wireless association alert` in the log output. This confirms the event was received, the correct case was matched, and the `discard` statement was applied.

Validation

- The rules file passes syntax check.
- A `WirelessAssociation` event generated with `logger` does not appear in the ObjectServer event list.
- The probe log file shows the `===== Discarding wireless association alert` log message, confirming the event was received and the discard case was matched.
- The student can explain the difference between `discard` and `recover` and describe when each would be used.

Expected result

At the end of this lab, you understand how to suppress unwanted events at the probe level using `discard`, know that the rules file continues to process after a `discard` statement, and understand that `recover` can be used to reinstate an event if later logic determines it should be kept.

4.3.9 Expire timers Objective

Add an `@ExpireTime` value to the `LinkUtilization` case and observe how the ObjectServer automatically clears the alert after the specified time has elapsed.

Background

The `@ExpireTime` field is a special ObjectServer field that controls how long an alert remains active before it is automatically cleared, irrespective of the event `@Severity` or any other criteria. When `@ExpireTime` is set to a non-zero value, the ObjectServer `expire` trigger — reviewed in section 3.3 — periodically scans `alerts.status` for alerts whose `LastOccurrence` is older than the current time minus the `@ExpireTime` value. When that condition is met, the trigger sets the alert's `@Severity` to 0, effectively clearing it without any operator or probe intervention.

Setting `@ExpireTime` to `0` disables the expiry behavior — the alert will remain active indefinitely unless it is cleared by another mechanism such as a resolution event, operator action, or another trigger. Understand that if an `@ExpireTime` is not set in the rules explicitly, it defaults to `0`.

This is particularly useful for event types that are known to be transient or self-resolving, where a distinct clearing event is not always guaranteed to arrive. Rather than leaving stale alerts in the event list, `@ExpireTime` ensures they are cleaned up automatically after a defined window. In a production environment, typical values range from a few minutes to several hours depending on the nature of the event. In this lab a value of 30 seconds is used so the behavior can be observed quickly.

Tasks

1. Open the training rules file in a text editor:

```
vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

2. Add `@ExpireTime = 30` as the first line of the `LinkUtilization` case, before any other field assignments. The beginning of the case should now look like this:

```
                case "LinkUtilization":
                        @ExpireTime = 30
```

```
                        log(INFO, "===== Entering LinkUtilization segment")
                        @Node = extract($Raw, ".* link utilization for (.*?) .*")
                        @Summary = extract($Raw, ".* ALERT - LinkUtilization: (.*)")
                        @AlertKey = extract($Raw, ".* ALERT.* interface (.*?) is.*")
                        $utilval = extract($Raw, ".* ALERT.* is (.*)%.*")
                        log(INFO, "$utilval is " + $utilval)
                        ...
```

The rest of the `LinkUtilization` case remains unchanged. Save and close the file.

3. Stop the probe, syntax-check, and restart:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -messagelog $OMNIHOME/log/syslogd-training.log &
```

If the syntax check fails, examine the log file: `cat $OMNIHOME/log/syslogd-training.log`

4. Return to the WebGUI and delete any existing alerts from the event list so we don't deduplicate with the previous alert.
5. Generate the following critical-level utilization event:

```
logger -n 127.0.0.1 'ALERT - LinkUtilization: Average link utilization for routerA interface
 ↪  GigabitEthernet0/0 is 99%'
```

6. Locate the alert in the event list and confirm that it appears with `@Severity = 5` and that the `@ExpireTime` field is set to `30`. (You may need to refresh the event list a few times before it shows up)
7. Wait approximately 30 seconds without generating any further events for this alert. Observe that the alert is automatically cleared by the `expire` trigger — its severity will be set to 0. If the event list does not show cleared events by default, adjust the filter to show all severities including 0 to confirm the clearing has occurred.
8. Note that this clearing behavior is driven entirely by the `expire` trigger reviewed in section 3.3, which runs on a schedule and checks `LastOccurrence` against the current time minus `@ExpireTime`. No probe action, operator action, or resolution event was needed.
9. Generate the same event again and observe that the 30-second timer resets each time a new occurrence arrives, because the `deduplication` trigger updates `LastOccurrence` with each reinsert. This means an alert that continues to be refreshed by incoming events will not be cleared while the condition is still active.

Validation

- The rules file passes syntax check.
- A `LinkUtilization` event at 99% appears in the event list with `@Severity = 5` and `@ExpireTime = 30`.
- The alert is automatically cleared approximately 30 seconds after the last occurrence, without any operator or probe intervention.
- The student can explain how the `expire` trigger uses `LastOccurrence` and `@ExpireTime` to determine when to clear an alert.
- The student can explain why generating a new occurrence of the same event resets the expiry timer.

Expected result

At the end of this lab, the student understands how `@ExpireTime` works in conjunction with the ObjectServer `expire` trigger to automatically clear transient alerts, and can apply this pattern to event types that do not have a guaranteed resolution event.

4.3.10 Additional rules functions and capabilities Objective

This section provides a reference overview of additional probe rules functions and capabilities that are commonly used in production OMNIbus environments. No hands-on tasks are included — the purpose is to make the student aware of what is available so they can apply these tools when building or maintaining rules files beyond the training exercises.

Include files

Probe rules files support `include` statements that load a secondary rules file into the current rules file at processing time. This is useful for organizing large rulesets into smaller, maintainable files — for example, separating different event source types into individual include files that are all called from a single master rules file.

When an event reaches an include statement in the main rules file, processing jumps into the included rules at that exact point, and those rules are executed for the current event. After the included rules finish, control returns to the main rules file and processing continues from the line after the include, unless the event is discarded in the included rules.

The format of an include statement is:

```
include "/path/to/rulesfile.rules"
```

For example, to include a secondary rules file from the same platform directory:

```
include "/opt/IBM/tivoli/netcool/omnibus/probes/linux2x86/my-secondary.rules"
```

The path in an include statement can point to a rules file on a web server (a base URL) or to a file on disk using an absolute or relative directory path, depending on how you've set up your environment. Include files are commonly used with the Netcool Knowledge Library (NcKL), which ships pre-built master rules and include files for a wide range of vendor-specific SNMP traps. You'll see this pattern in more detail later, when you work through the section on SNMP trap processing and NcKL-based trap rules.

`regreplace()`

The `regreplace()` function applies a regular expression substitution to a string, replacing matches with a specified replacement value. It is useful for cleaning up or normalising string tokens and field values before they are stored in the ObjectServer.

Replace multiple consecutive spaces with a single space:

```
$x = regreplace($x, " +", " ")
```

Strip all occurrences of specific characters (in this example $, `!`, and `'`):

```
$x = regreplace($x, "[$!']", "")
```

Date and time functions

The following date and time functions are available in probe rules:

- **`datetotime()`** — converts a textual representation of a timestamp into a UNIX epoch time integer. Useful when the source event contains a human-readable timestamp that needs to be stored as a numeric value.
- **`timetodate()`** — converts a UNIX epoch timestamp into a human-readable string. Useful for populating summary or detail fields with a formatted date.
- **`getdate()`** — takes no arguments and returns the current time as a UNIX epoch integer. Note that `@FirstOccurrence` and `@LastOccurrence` are set automatically by the probe and ObjectServer if not explicitly set in the rules — there is no requirement to manage these manually. `getdate()` is more commonly used in conditional processing, for example to compare the current time against a value in the event data, or to calculate a relative time offset for use in other fields.

Full reference: IBM Documentation — Date and time functions

`ltrim()` and `rtrim()`

`ltrim()` removes leading whitespace from the left side of a string. `rtrim()` removes trailing whitespace from the right side of a string. Both are useful for cleaning up extracted values that may have surrounding spaces due to the structure of the source message.

`printable()`

The `printable()` function replaces all non-printable characters in a string with a space. This is useful when processing event data from sources that may include control characters or binary content in their messages, which would otherwise cause display or storage issues in the ObjectServer.

`gethostname()` and `gethostaddr()`

`gethostname()` performs a reverse DNS lookup and returns the hostname for a given IP address. `gethostaddr()` performs a forward DNS lookup and returns the IP address for a given hostname.

> **Note:** DNS lookups can take an appreciable amount of time to complete, and performing them on every incoming event can severely impact probe performance — particularly at high event rates. While the mttrapd probe can do dns caching, it

may be prudent in your enviornment to use a lookup table in the rules file instead, and only fall back to `gethostaddr()` or `gethostname()` when the host is not found in the table.

`length()`

The `length()` function returns the number of bytes in a string. This can be useful for validating that an extracted value is within an expected range before assigning it to an alert field, or for conditional logic that depends on the size of a token.

`upper()` and `lower()`

`upper()` converts all characters in a string to uppercase. `lower()` converts all characters to lowercase. These are particularly useful for normalising field values across event sources that may use inconsistent casing — for example, ensuring that `@Node` values from different probes are stored in a consistent case so that deduplication and correlation work correctly.

`exists()`

The `exists()` function tests whether a token exists in the current event. It returns TRUE if the token is present and FALSE if it is not. This is useful when dealing with event sources that produce variable token sets — some events may include a token that others do not, and `exists()` allows the rules to handle both cases gracefully:

```
if (exists($alarmcode))
{
        @EventId = $alarmcode
}
```

Full function reference

A complete list of probe rules functions and operators is available in the IBM Documentation: IBM Netcool/OMNIbus 8.1 — Rules file functions and operators

---

4.3.11 Dynamic RawCapture control Objective

Use the `%RawCapture` rules variable to selectively enable raw event capture for unhandled or unknown events, without capturing every event that passes through the probe.

Background

In section 4.2 you set `RawCapture : 1` in the syslogd properties file to enable raw capture globally — every incoming event was written to the capture file. This is useful during initial development, but in a production probe processing thousands of events per minute it would produce a very large capture file very quickly and provide little diagnostic value.

The probe rules engine provides a way to control raw capture dynamically on a per-event basis using the `%RawCapture` variable. Setting `%RawCapture = 0` in the rules file turns capture off for the current event. Setting `%RawCapture = 1` turns it back on. This means you can disable capture globally in the properties file and then selectively enable it only for the specific events you want to capture — such as events that hit the `default` case of a `switch` block, meaning they arrived with an event type your rules do not yet know how to handle.

This is a valuable production pattern: it gives you automatic capture of novel or unexpected events without the overhead of capturing everything.

> **Note:** The `%RawCapture` assignment near the top of the rules file (after tables and arrays are defined) sets the default for all events. Assignments within `switch/case` or `if/else` blocks then override that default selectively for specific event types. The last assignment in the rules processing chain wins for each event.

Tasks

1. Open the training rules file:

   ```
   vi $OMNIHOME/probes/linux2x86/syslogd-training.rules
   ```

2. Locate the section after your `table` declaration but before the main processing logic begins. Add the following lines to turn raw capture off by default for all events:

   ```
   # Turn %RawCapture property off dynamically
   # Note that this will override the setting in the properties file or command line
   %RawCapture = 0
   ```

3. Now locate the `default:` case inside your `switch(@EventId)` block. Add `%RawCapture = 1` to selectively re-enable capture for any event type that your rules do not recognize:

```
default:
        log(INFO, "==== Found an unknown/unhandled event")
        @Summary = "Unhandled event: " + $Raw
        @Severity = 1
        @Type = 1
        %RawCapture = 1
```

4. Save the file and run a syntax check:

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules
```

5. Restart the probe:

```
kill $(pgrep nco_p_syslogd)
```

```
$OMNIHOME/probes/nco_p_syslogd -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules
 ↪  -capturefile $OMNIHOME/var/syslogd-training.cap -messagelog
 ↪  $OMNIHOME/log/syslogd-training.log &
```

6. Clear the existing capture file so you can see only new captures:

```
cat /dev/null > $OMNIHOME/var/syslogd-training.cap
```

7. Generate a known event type that your rules handle — for example a `LinkDown` event:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Port Gi0/1 on router-01 is down'
```

   Now check the capture file:

```
cat $OMNIHOME/var/syslogd-training.cap
```

   The capture file should be **empty** — the `LinkDown` event matched a defined case and `%RawCapture` remained `0`.

8. Now generate an event with an unknown type that will hit the `default` case:

```
logger -n 127.0.0.1 'ALERT - UnknownEvent: This is an unrecognized event type'
```

   Check the capture file again:

```
cat $OMNIHOME/var/syslogd-training.cap
```

   This time the raw event **should be present** in the capture file — the `default` case set `%RawCapture = 1`, so the probe wrote the raw event data before forwarding it to the ObjectServer.

Validation

- Known event types (LinkDown, LinkUp, etc.) do not appear in the capture file.
- Unknown event types hitting the `default` case do appear in the capture file.
- The probe log shows `==== Found an unknown/unhandled event` for the unknown event type.
- The event appears in the Web GUI with `@Severity = 1` and a `Summary` beginning with `Unhandled event:`.

Expected result

At the end of this lab, you now understand how to use `%RawCapture` as a targeted diagnostic tool rather than a blunt instrument. This pattern — capture off by default, selectively enabled for unhandled events — is directly applicable in production probe deployments where blanket raw capture is impractical.

---

5. SNMP Trap Processing with MTTrapd and NcKL

Network devices — routers, switches, servers, storage arrays, and countless other managed systems — communicate fault conditions using the **Simple Network Management Protocol (SNMP)**. When a device detects a problem, it does not wait for a management system to poll it. Instead, it sends an unsolicited notification called a **trap** to a pre-configured management address. Traps are the backbone of network event management, and processing them reliably and meaningfully is one of the most important functions Netcool/OMNIbus performs in a real environment.

In this section you will work with two components that together form the OMNIbus SNMP processing pipeline: the **MTTrapd probe**, which receives and decodes SNMP trap PDUs, and **NcKL (Netcool Knowledge Library)**, which provides an enterprise-grade rules framework that already understands thousands of device-specific trap definitions. You will also build and load a custom MIB using the **MIB Manager** tool, then write rules to process its traps — giving you end-to-end understanding of the complete SNMP ingestion pipeline.

5.0 Lab Setup

Before starting the SNMP labs, you will create a dedicated event filter and custom view in the Web GUI so that trap events are easy to identify and work with independently of the syslogd events from Section 4.

Objective: Create a *SNMP Traps* filter and a matching event list view to isolate trap events during the labs.

Prerequisites: Web GUI accessible at `https://<your-server>:16311/ibm/console`, and you are logged into as `ncoadmin`. ObjectServer running.

Create the SNMP Traps filter

1. In the Web GUI, open the **Event List** and click the filter dropdown at the top of the view.
2. Select **New Filter**.
3. Name the filter **SNMP Traps**.
4. Set the condition to:

Field	Operator	Value
Manager	like	MTTrapd

5. Set the default view to the view we created prevously: **SyslogdProbe**
6. Click **Save and Close**.

This filter matches events whose `Manager` field contains the string `MTTrapd`. The MTTrapd probe sets `@Manager` to include its own process name by default, so this will cleanly separate trap events from syslogd events.

> **Note:** The `Manager` field will show the MTTrapd process name, not `MyProbe` — so your existing syslogd filter will remain clean and will not show trap events. The two filters give you independent visibility into each event source.

5.1 SNMP and MTTrapd Overview

What is SNMP? SNMP is a standards-based protocol defined by the IETF for managing and monitoring networked devices. It operates over UDP (User Datagram Protocol), traditionally on port **161** for polling (GET/SET operations) and port **162** for unsolicited trap notifications.

The three most common versions in production environments are:

Version	Key characteristics
SNMPv1	Original standard. Simple community-string authentication. Limited trap PDU structure.

Version	Key characteristics
SNMPv2c	Adds 64-bit counters, bulk operations, and an improved trap PDU format. Community-string auth.
SNMPv3	Adds authentication (MD5/SHA) and privacy (DES/AES) encryption. Most secure.

For most lab and production trap processing use cases you will encounter SNMPv1 and SNMPv2c. SNMPv3 is used in high-security environments and requires additional configuration.

Traps, OIDs, and MIBs Every SNMP trap is identified by an **Object Identifier (OID)** — a hierarchical dotted-decimal number such as `1.3.6.1.2.1.11.5`. OIDs are globally unique and form a tree structure maintained by standards bodies and individual vendors.

A **Management Information Base (MIB)** is a text file written in a formal language called **SMI (Structure of Management Information)** that provides human-readable names, descriptions, and types for OIDs. Without a MIB, a trap arrives as a string of dotted numbers. With a MIB loaded, those same numbers are translated into meaningful names and values.

When a device sends a trap it may also include **variable bindings (varbinds)** — additional name-value pairs that carry supplementary data about the event. For example, a `linkDown` trap from a router may include varbinds containing the interface index (ifIndex), interface administrative status (ifAdminStatus), and interface operational status (ifOperStatus).

The MTTrapd probe The **MTTrapd probe** (`nco_p_mttrapd`) is the component that receives SNMP traps and feeds them into the OMNIbus event pipeline. Its responsibilities are:

1. **Listen** on UDP port 162 (or TCP 162) for incoming trap PDUs.
2. **Decode** the raw PDU using loaded MIB definitions to resolve OIDs to names.
3. **Map** the decoded trap fields into OMNIbus `alerts.status` column tokens.
4. **Process** the mapped tokens through a rules file to produce a normalized alert.
5. **Forward** the finalized alert to the ObjectServer.

The MTTrapd probe binary is located at `$OMNIHOME/probes/nco_p_mttrapd`. Its properties file is `$OMNIHOME/probes/linux2x86` and its default rules file is `$OMNIHOME/probes/linux2x86/mttrapd.rules`.

NcKL — Netcool Knowledge Library **NcKL** (Netcool Knowledge Library) is an IBM-provided add-on package installed at `/opt/IBM/tivoli/NcKL`. It ships with:

- **Pre-built rules files for 39 vendors and approximately 2,300 MIB definitions**, covering IETF/IEEE RFC traps as well as vendor-specific traps from Cisco, Juniper, HP, IBM, and many others.
- **A master rules file** at `/opt/IBM/tivoli/NcKL/rules/snmptrap.rules` that acts as the top-level dispatcher — it determines the trap version, routes enterprise-specific traps to their vendor rules file, and handles generic traps via a common path.
- **Per-MIB rules and lookup files** — for each exported MIB, NcKL creates several files: a main rules file (`.include.snmptrap.rule` a severity lookup file (`.sev.snmptrap.lookup`) that controls `@Severity`, `@Type`, and `@ExpireTime`, and a user include file (`.user.include.snmptrap.rules`) for site-specific customizations. There may also be an enumeration lookup file (`.include.snmptrap.lookup`) for varbind enumeration mappings.
- **A `$NC_RULES_HOME` environment variable** that points to the NcKL rules directory root — this is used throughout NcKL path references.

  Note: The severity lookup file and user include file are key NcKL concepts. Rather than hardcoding `@Severity`, `@Type`, and `@ExpireTime` in the rules, NcKL stores these as lookup table entries. Likewise, any custom rules logic for specific SNMP traps can be applied iin the `.user.include.snmptrap.rules`, which is included at the end of each MIB include file. This means you can tune severity and expiry for any trap by editing the lookup file without touching the rules code itself, and perform custom logic without editing or modifying the base NcKL rules. This makes upgrades to the NcKL easier to maintain.

NcKL high-level process flow When a trap arrives, the NcKL rules process it as follows:

1. Determine the SNMP version (v1/v2) and normalize the tokens to a common format.
2. Check whether it is an enterprise-specific trap (generic trap type 6) or a generic trap (types 0–5).
3. **For generic traps:** switch on generic trap type (0–5), create the appropriate event, then look up `@Severity`, `@Type`, and `@ExpireTime` in the `snmptrap.sev.lookup` file.

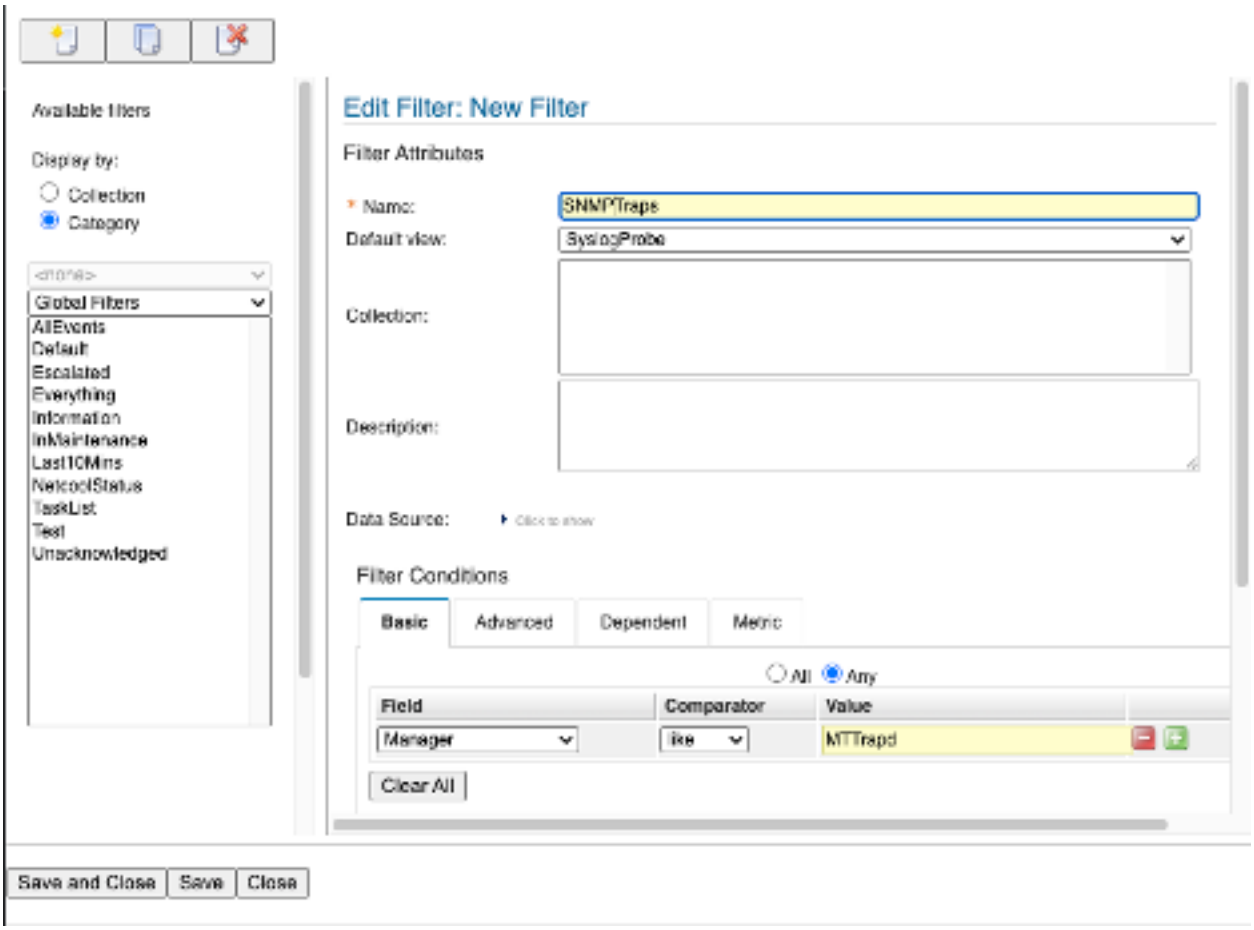


Figure 77: SNMP Traps filter configuration

4. **For enterprise-specific traps:** switch on enterprise ID. If a matching case is found, switch on the specific trap number, build the event, look up severity/type/expiry in the appropriate per-MIB lookup file, then run through the per-MIB user include file for any site-specific customizations. If no matching enterprise case is found, NcKL creates a Severity 1 “No Rules Found” event identifying the enterprise ID.

NcKL process flow:

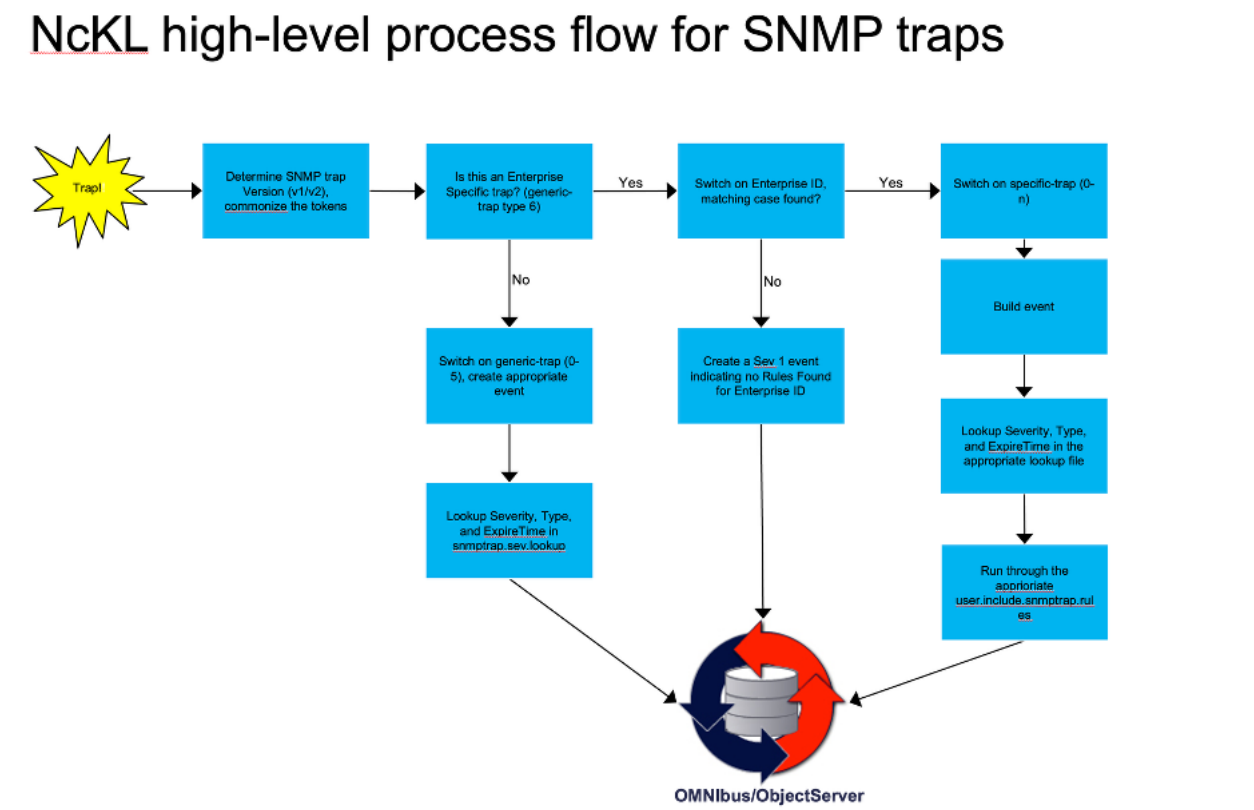


Figure 78: MTTrapd and NcKL event flow diagram

5.2 Configure MTTrapd with NcKL

In this lab you will configure MTTrapd to use the NcKL rules framework, start the probe for the first time, and verify that it is connected to the ObjectServer and ready to receive traps.

Objective: Start MTTrapd, verify connectivity to the ObjectServer, and point it at the NcKL rules framework.

Prerequisites: ObjectServer running. You are logged in to the lab VM as `root`. `$OMNIHOME` and `$NCHOME` are exported in your shell.

Step 1 — Inspect the MTTrapd properties file Open the MTTrapd properties file and review the key settings before making any changes:

```
cat $OMNIHOME/probes/linux2x86/mttrapd.props
```

The properties you will work with are:

Property	Default value	Purpose
`Server`	`'NCOMS'`	Default ObjectServer to connect to
`Port`	`4100`	Default ObjectServer listener port

Property	Default value	Purpose
`MessageLevel`	`'warn'`	Probe log verbosity
`RulesFile`	`'$OMNIHOME/probes/linux2x86/mttrapd.rules'`	Default rules file to process traps

Note: Like all probe properties files, MTTrapd stores paths as string literals. The `$OMNIHOME` variable notation in the default value is resolved at startup by the probe binary itself — but when you add entries that reference NcKL, you must use the full absolute path because the NcKL installation is outside `$OMNIHOME`.

Step 2 — Setting up the NcKL environment Before updating MTTrapd, prepare the ObjectServer schema for NcKL and configure the NcKL environment variable.

1. Run the NcKL database schema correction script in the lab environment to add the custom fields NcKL requires in `alerts.status`, and the addition of triggers supporting NcKL:

```
$OMNIHOME/bin/nco_sql -server ALERTS -user root -password 'netcool' <
 ↪  /opt/IBM/tivoli/NcKL/advcorr.sql
```

Expected output is similar to:

```
ERROR=Object not found on line 114 of statement
'--##############################################################################################...', at
 ↪  or near 'AdvCorr_SetCauseType'
ERROR=Object not found on line 1 of statement 'drop trigger AdvCorr_LPC_RC;...',
at or near 'AdvCorr_LPC_RC'
ERROR=Object not found on line 1 of statement 'drop trigger
AdvCorr_LPC_Sym;...', at or near 'AdvCorr_LPC_Sym'
ERROR=Object not found on line 4 of statement '-- Drop tables in case they
already exists from a previous installation...', at or near 'AdvCorrLpcSymCand'
ERROR=Object not found on line 1 of statement 'drop table
alerts.AdvCorrLpcRcCand;...', at or near 'AdvCorrLpcRcCand'
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(10 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
(0 rows affected)
```

The errors are expected: the script drops any existing triggers before recreating them. The important indicators are the final row counts, especially the `10 rows affected` line.

2. Persist `NC_RULES_HOME` in your shell profile and reload it, then verify the value:

```
echo 'export NC_RULES_HOME="/opt/IBM/tivoli/NcKL/rules"' >> ~/.bash_profile
source ~/.bash_profile
echo $NC_RULES_HOME
```

The final command should output `/opt/IBM/tivoli/NcKL/rules`.

3. Add the same environment variable to MTTrapd's probe environment file so the probe inherits it when started by Process Agent:

```
echo 'export NC_RULES_HOME="/opt/IBM/tivoli/NcKL/rules"' >>
 ↪  $OMNIHOME/probes/linux2x86/nco_p_mttrapd.env
```

Step 3 — Update the properties file to send alerts to our lab ObjectServer, and to use NcKL You need to tell MTTrapd to use: - Our ObjectServer **EVENTS** - The NcKL master rules file as its rules engine. - The NcKL MIB directory so that OIDs in incoming traps are resolved against NcKL's bundled MIB library.

Add the following lines at the end of `$OMNIHOME/probes/linux2x86/mttrapd.props`:

```
Server : 'EVENTS'
MessageLevel : 'debug'
RulesFile : '/opt/IBM/tivoli/NcKL/rules/snmptrap.rules'
MessageLog  : '/opt/IBM/tivoli/netcool/omnibus/log/mttrapd-training.log'
RawCapture : 1
```

Save the file. The `MessageLevel` change to `debug` will give you verbose output in the probe log during the labs, making it easy to trace exactly how each trap is decoded and processed.

Step 4 — Syntax check the NcKL rules, and start the MTTrapd probe Run the nco_p_syntax checker to validate the NcKL rules. This will evaluate the top level snmptrap.rules file and any included `include` files that are uncommented in the snmptrap.rules. You will note that the output of the syntax check in this case is much more extensive (more on this later):

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile
 ↪  /opt/IBM/tivoli/NcKL/rules/snmptrap.rules
```

Start MTTrapd in the background, sending its log output to a dedicated log file:

```
$OMNIHOME/probes/nco_p_mttrapd &
```

> **Note:** Unlike the syslogd probe in Section 4, we will not be using the `-rulesfile` on the command line here to explicitly set a custom rules file — the rules file path is already set in the properties file pointing to the NcKL location. You could override it on the command line if needed, but for NcKL it is cleaner to manage it in the props file.

Step 5 — Confirm the probe started Verify that the MTTrapd probe process is running and that it is listening on UDP port 162:

```
netstat -anlp | grep udp | grep 162
```

You should see a line similar to:

```
udp        0      0 0.0.0.0:162             0.0.0.0:*                           1890598/nco_p_mttrapd
```

Then verify the process name directly:

```
pgrep -a nco_p_mttrapd
```

You should see the process listed with its PID. If the probe does not appear to be running or listening on port 162, consult our log file and inspect it for startup errors:

```
vi $OMNIHOME/log/mttrapd-training.log
```

Step 6 — Verify probe operation with the MTTrapd heartbeat event Switch back to the Web GUI and open the Event Viewer.

1. Select the **NetcoolStatus** filter.
2. Locate the `Probe Heartbeat Message` event for `mttrapd`.

The `Probe Heartbeat Message` is a periodic status event that confirms the probe is alive and connected to the ObjectServer. The MTTrapd sends the heartbeat alarm every minute to the ObjectServer, so its presence in the NetcoolStatus view is a strong validation that the probe is running normally.

Use this event to verify probe operation as follows: - If the heartbeat is present and recent, MTTrapd is active and able to communicate with the ObjectServer. - If the heartbeat is missing or stale, the probe may have stopped, failed to start, or lost ObjectServer

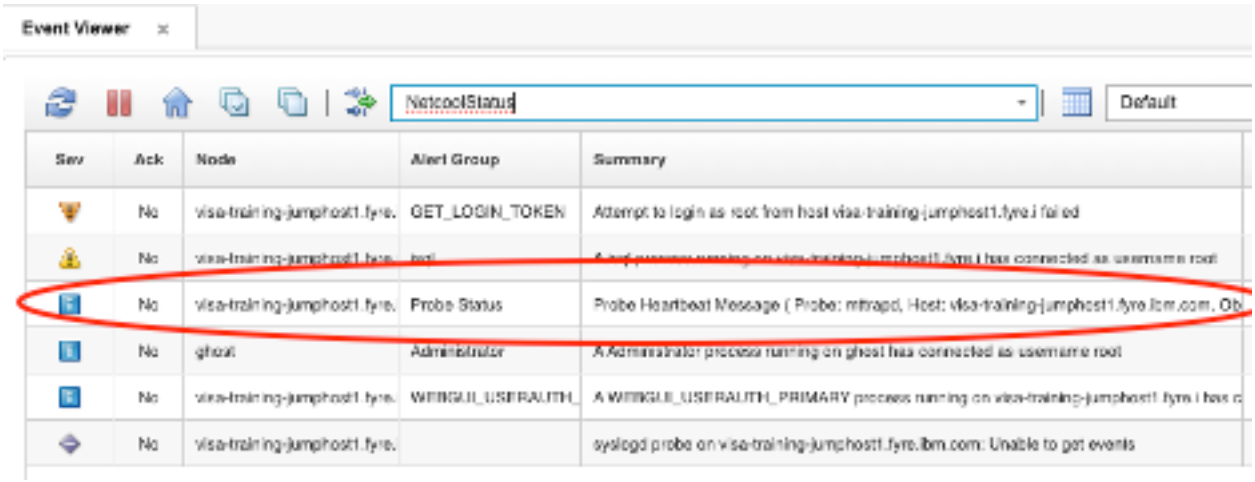


Figure 79: MTTrapd Probe Heartbeat Message

connectivity. - If you see repeated heartbeat failures, review the probe configuration and check for port conflicts or environment variable issues.

Only consult the probe log if the heartbeat does not appear or if the probe fails to start.

5.3 Test with Standard IF-MIB Traps

Before working with a custom MIB, you will send standard SNMP traps defined in the **IF-MIB** (Interfaces MIB, RFC 2863). This confirms that the MTTrapd / NcKL pipeline is working end to end, and gives you a reference point for the trap structure you will see in subsequent labs.

Objective: Generate `linkDown` and `linkUp` traps using `snmptrap` and observe how NcKL processes them into events in the ObjectServer.

Prerequisites: MTTrapd running and connected. `net-snmp-utils` installed (`snmptrap` command available). ObjectServer running.

Background — linkDown and linkUp `linkDown` and `linkUp` are two of the most fundamental SNMP traps defined in the original IF-MIB standard. Every SNMP-capable network device is expected to generate these traps when a network interface changes state.

- **linkDown** OID: `1.3.6.1.6.3.1.1.5.3` (SNMPv2c notification)
- **linkUp** OID: `1.3.6.1.6.3.1.1.5.4` (SNMPv2c notification)

Both carry a standard varbind — `ifIndex` — that identifies which interface changed state. In a real network event, the management system would look up `ifIndex` against the device's interface table to determine exactly which physical port or logical circuit is affected.

Step 1 — Send a linkDown trap Use the `snmptrap` utility to send a simulated `linkDown` trap to `localhost`:

```
ssnmptrap -v 2c -c public localhost '' 1.3.6.1.6.3.1.1.5.3 \
  1.3.6.1.2.1.2.2.1.1 i 1 \
  1.3.6.1.2.1.2.2.1.7 i 1 \
  1.3.6.1.2.1.2.2.1.8 i 2
```

Breaking down the command:

Argument	Meaning
`-v 2c`	Use SNMPv2c format
`-c public`	Community string (like a password)
`localhost`	Send to the local machine (where MTTrapd is listening)
`''`	Empty sysUpTime — probe will substitute current time

Argument	Meaning
`1.3.6.1.6.3.1.1.5.3`	The linkDown notification OID
`1.3.6.1.2.1.2.2.1.1`	The ifIndex OID (the varbind name)
`i 1`	Integer value 1 (interface index 1)
`1.3.6.1.2.1.2.2.1.7`	the ifAdminStatus for ifIndex 1
`i 1`	Integer value 1 (1=up, 2=down, 3=testing) - interface is admin UP
`1.3.6.1.2.1.2.2.1.8`	the ifOperStatus for ifIndex 1
`i 2`	Integer value 2 (2=down, 1=up, 3=testing, 4=unknown, 5=dormant, 6=notPresent, 7=lowerLayerDown)

Step 2 — Observe the event in the Web GUI Switch to the Web GUI and apply the **SNMP Traps** filter. You should see a new event in the **SNMP Traps View** with:

- Node set to `127.0.0.1` (or `localhost`, depending on NcKL hostname resolution)
- `AlertGroup` reflecting the IF-MIB or the trap category
- `Summary` containing a human-readable description of the linkDown trap
- Type set to 1 (problem event)

  Note: The exact field values depend on the NcKL rules for the IF-MIB. NcKL maps OID components to `@AlertGroup`, `@AlertKey`, and `@Summary` according to its built-in knowledge of the IF-MIB. Reviewing the event fields tells you how NcKL has classified the trap.

Step 3 — Examine the MTTrapd rawcapture output Open the MTTrapd rawcapture output and review the tokens generated for the `linkDown` trap:

```
cat $OMNIHOME/var/mttrapd.cap
```

Note the tokens that are created by the mttrapd probe. The table below shows the various tokens that were generated by the probe for this particular trap:

Aspect	rawcapture Tokens	Meaning
SNMP version	`SNMP_Version = "2"`	SNMPv2c trap
Community	`community = "public"`	Community string `public`
Source IP	`IPaddress = "127.0.0.1"`, `PeerIPaddress = "127.0.0.1"`, `PeerAddress = "127.0.0.1"`, `Node = "127.0.0.1"`	Trap came from localhost
Protocol/port	`Protocol = "UDP"`, `ReceivedPort = "162"`	UDP on standard trap port 162
Trap type OID	`notify = ".1.3.6.1.6.3.1.1.5.3"`, `.1.3.6.1.6.3.1.1.4.1.0 = ".1.3.6.1.6.3.1.1.5.3"`	IF-MIB `linkDown` notification
Device uptime	`.1.3.6.1.2.1.1.3.0 = "(35172831) 4 days, 1:42:08.31"`, `UpTime = "35172831"`, `Uptime = "1:42:08.31"`	Device had been up ~4 days when trap sent
Capture time	`ReceivedTime = "1786042759"`, `RawCaptureTimeStamp = 1786042759`	Time probe received the trap (UNIX timestamp)
Request ID	`ReqId = "1388912577"`	SNMP request ID for this trap packet
Event count	`EventCount = "6"`	Internal count of parsed tokens/varbinds

Varbinds (OID1, OID2, OID3)

Varbind index	OID	Name	rawcapture Tokens	Meaning
`OID1`	`.1.3.6.1.2.1.2.2.1.1`	`ifIndex`	`1 = "1"`, `1_raw = "1"`, `1_text = "1"`, `1_hex = "1"`; `.1.3.6.1.2.1.2.2.1.1 = "1"`	Interface index = **1**
`OID2`	`.1.3.6.1.2.1.2.2.1.7`	`ifAdminStatus`	`2 = "1"`, `2_raw = "1"`, `2_text = "1"`, `2_hex = "1"`; `.1.3.6.1.2.1.2.2.1.7 = "1"`	Admin status = **1 (up)**

Varbind index	OID	Name	rawcapture Tokens	Meaning
OID3	.1.3.6.1.2.1.2.2.1.8	ifOperStatus	3 = "2", 3_raw = "2", 3_text = "2", 3_hex = "2"; .1.3.6.1.2.1.2.2.1.8 = "2"	Oper status = **2 (down)**

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.6.3.1.1.5.4 \
  1.3.6.1.2.1.2.2.1.1 i 1 \
  1.3.6.1.2.1.2.2.1.7 i 1 \
  1.3.6.1.2.1.2.2.1.8 i 1
```

Step 4 — Send a linkUp trap (resolution) The only differences is the OID `1.3.6.1.6.3.1.1.5.4` which is the `linkUp` trap, and the varbind `1.3.6.1.2.1.2.2.1.8` which is `ifOperStatus` of "UP". NcKL includes built-in rules for the IF-MIB and does correlate `linkUp` against `linkDown` — it sets `@Type = 2` on the `linkUp` event with a matching `@AlertKey`, so the `generic_clear` trigger in the ObjectServer will clear the original `linkDown` problem event automatically.

Observe the event list — the `linkDown` event should go to clear once `generic_clear` runs (within a few seconds). If it does not clear, check that both events have the same `Node`, `AlertGroup`, `AlertKey`, and `Manager` values — all four must match for `generic_clear` to fire.

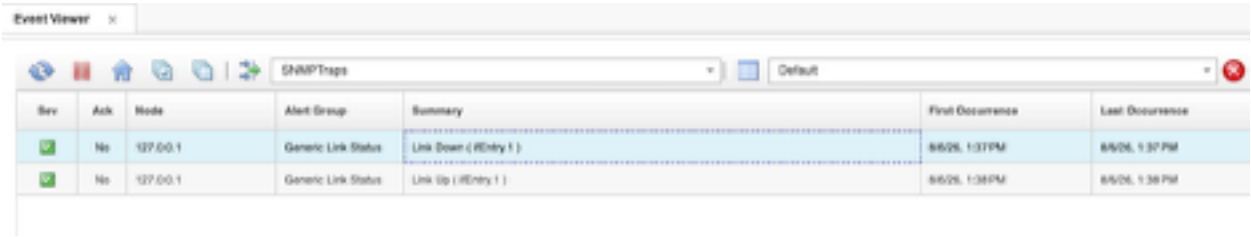


Figure 80: Send linkUp trap

5.4 Build and Load a Custom MIB with MIB Manager

In production environments you will regularly encounter network devices whose MIBs are not part of the Netcool Knowledge Library. Vendor-specific MIBs must be loaded into the NcKL before the probe can resolve the OIDs in the traps they generate.

In this lab you will: 1. Review the **IBM-TRAINING-MIB** — a purpose-built custom MIB that you will use throughout the remaining labs. 2. Load it into the **MIB Manager** tool on your Windows workstation, and export it as NcKL-compatible rules files. 3. Verify that the MIB is correctly loaded and that OIDs are now resolved.

Objective: Load the IBM-TRAINING-MIB into MIB Manager, export it for use with NcKL, integrate the exported rules into the NcKL rules framework, and verify end-to-end OID resolution and event generation.

Prerequisites: MTTrapd running on the lab VM. `$OMNIHOME` and `$NC_RULES_HOME` exported in your shell. ObjectServer running.

First, set the `$NC_RULES_HOME` variable if it is not already set:

```
export NC_RULES_HOME=/opt/IBM/tivoli/NcKL/rules
```

5.4.1 The IBM-TRAINING-MIB The IBM-TRAINING-MIB defines three notification types that model realistic IT infrastructure events:

Trap name	OID suffix	Severity intent	Type
`serverDown`	`.1`	Critical	Problem (Type 1)
`serverUp`	`.2`	Clear	Resolution (Type 2)

Trap name	OID suffix	Severity intent	Type
diskUtilization	.3	Warning or Critical (depends on value)	Problem (Type 1)

The MIB also defines supporting objects (varbinds) carried in each trap:

Object	OID suffix	Syntax	Description
ibmTrainingServerName	.4	DisplayString	Hostname of the affected server
ibmTrainingServerIP	.5	IpAddress	IP address of the affected server
ibmTrainingDiskMount	.6	DisplayString	Mount point of the disk being reported
ibmTrainingDiskPercent	.7	Integer32 (0–100)	Disk utilization percentage
ibmTrainingEventSeverity	.8	Integer32	Severity hint from device (maps to OMNIbus scale)

The full MIB text is below. You will save this to a file on the lab VM for loading into MIB Manager:

```
IBM-TRAINING-MIB DEFINITIONS ::= BEGIN

IMPORTS
    MODULE-IDENTITY, OBJECT-TYPE, NOTIFICATION-TYPE, Integer32,
    enterprises
        FROM SNMPv2-SMI
    DisplayString
        FROM SNMPv2-TC;

ibmTraining MODULE-IDENTITY
    LAST-UPDATED "202601010000Z"
    ORGANIZATION "IBM Training"
    CONTACT-INFO "IBM Training Lab Environment"
    DESCRIPTION  "Custom MIB for OMNIbus training exercises."
    REVISION     "202601010000Z"
    DESCRIPTION  "Initial version."
    ::= { enterprises 99999 }

-- Object definitions

ibmTrainingObjects OBJECT IDENTIFIER ::= { ibmTraining 1 }

ibmTrainingServerName OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "The hostname of the server generating the event."
    ::= { ibmTrainingObjects 4 }

ibmTrainingServerIP OBJECT-TYPE
    SYNTAX      IpAddress
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "The IP address of the server generating the event."
    ::= { ibmTrainingObjects 5 }

ibmTrainingDiskMount OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "The filesystem mount point being reported."
```

```
    ::= { ibmTrainingObjects 6 }

ibmTrainingDiskPercent OBJECT-TYPE
    SYNTAX      Integer32 (0..100)
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "Disk utilization as a percentage (0-100)."
    ::= { ibmTrainingObjects 7 }

ibmTrainingEventSeverity OBJECT-TYPE
    SYNTAX      Integer32 (0..5)
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "Severity of the event, mapped to OMNIbus scale:
         0=Clear, 1=Indeterminate, 2=Warning, 3=Minor, 4=Major, 5=Critical."
    ::= { ibmTrainingObjects 8 }

-- Notification definitions

ibmTrainingNotifications OBJECT IDENTIFIER ::= { ibmTraining 2 }

serverDown NOTIFICATION-TYPE
    OBJECTS {
        ibmTrainingServerName,
        ibmTrainingServerIP,
        ibmTrainingEventSeverity
    }
    STATUS  current
    DESCRIPTION
        "A server has gone down or become unreachable."
    ::= { ibmTrainingNotifications 1 }

serverUp NOTIFICATION-TYPE
    OBJECTS {
        ibmTrainingServerName,
        ibmTrainingServerIP
    }
    STATUS  current
    DESCRIPTION
        "A server has recovered and is reachable."
    ::= { ibmTrainingNotifications 2 }

diskUtilization NOTIFICATION-TYPE
    OBJECTS {
        ibmTrainingServerName,
        ibmTrainingServerIP,
        ibmTrainingDiskMount,
        ibmTrainingDiskPercent,
        ibmTrainingEventSeverity
    }
    STATUS  current
    DESCRIPTION
        "Disk utilization on a server has crossed a threshold."
    ::= { ibmTrainingNotifications 3 }

END
```

Copy this text and paste it into notepad on your Windows workstation as `IBM-TRAINING-MIB.txt` (e.g. on the Desktop).

5.4.2 Load and export the MIB using MIB Manager **MIB Manager** is the Windows and Linux tool used to load MIB files, browse the OID tree, and export trap definitions as NcKL-compatible rules and lookup files. It is installed as part of the OMNIbus core package which is installed on your Windows workstation.

MIB files that you add are stored locally on the workstation under the OMNIbus installation's MIB directory — typically `%OMNIHOME%\var\mibmanager\mibs`. Exported rules and supporting files are placed in `%OMNIHOME%\var\mibmanager\exports`.

Launch MIB Manager On Windows, the MIB Manager is launched from a .bat file located at `%OMNIHOME%\bin\nco_mibmanager.ba`
The default %OMNIHOME% is `C:\IBM\tivoli\netcool`. If you installed OMNIbus core package in another location, use that path instead.

Locate the nco_mibmanager.bat in the file explorer, and double-click to start:

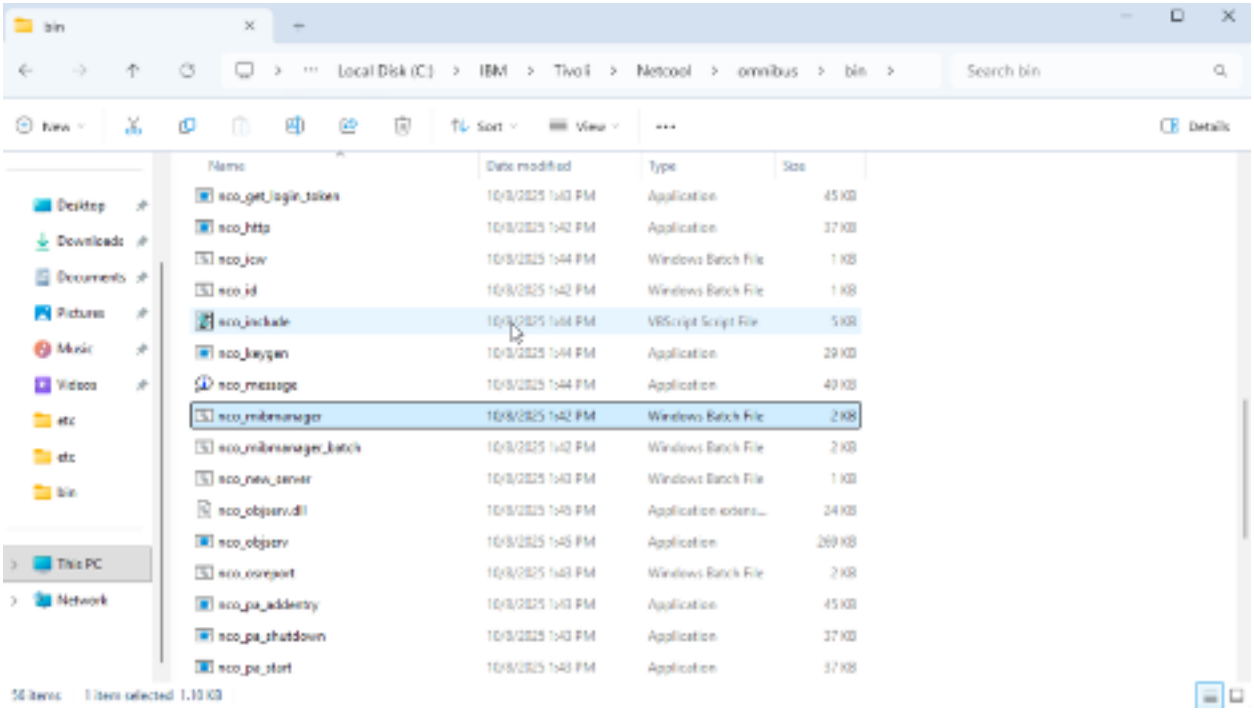


Figure 81: Launch MIB Manager

The MIB Manager window opens showing three key panels: a MIB module list on the left, an OID tree on the right, and a details/console panel at the bottom. There is also a Device panel that allows you to define and manage "devices" as collections of MIB modules, so you can work in a device-centric way.

Import the IBM-TRAINING-MIB

1. Copy the `IBM-TRAINING-MIB.txt` file from the Desktop to the MIB Manager mibs directory: `%OMNIHOME%\var\mibmanager\m`
2. Click the **Import** button in the upper left of the MIB Manager window.
3. The import dialog shows a directory field. Leave the default path pointing to the mibs directory and click **Import**.

   MIB Manager will parse all MIB files in that directory. If there are import errors for missing dependent MIBs, they will be listed in the Import Status dialog. The IBM-TRAINING-MIB only imports from standard SNMPv2-SMI and SNMPv2-TC, which are already present — you should see a clean import.
4. After import, locate `IBM-TRAINING-MIB` in the MIB module list, and select it.
5. Expand it in the OID tree on the right and confirm you can see:
   - `ibmTrainingObjects` with the five object definitions `ibmTrainingServerName`, `ibmTrainingServerIP`, `ibmTrainingDiskMount`, `ibmTraininingDiskPercent`, and `ibmTrainingEventSeverity`
   - `ibmTrainingNotifications` with `serverDown`, `serverUp`, and `diskUtilization` - these are our trap definitions as indicated by the `!` exclamation point.

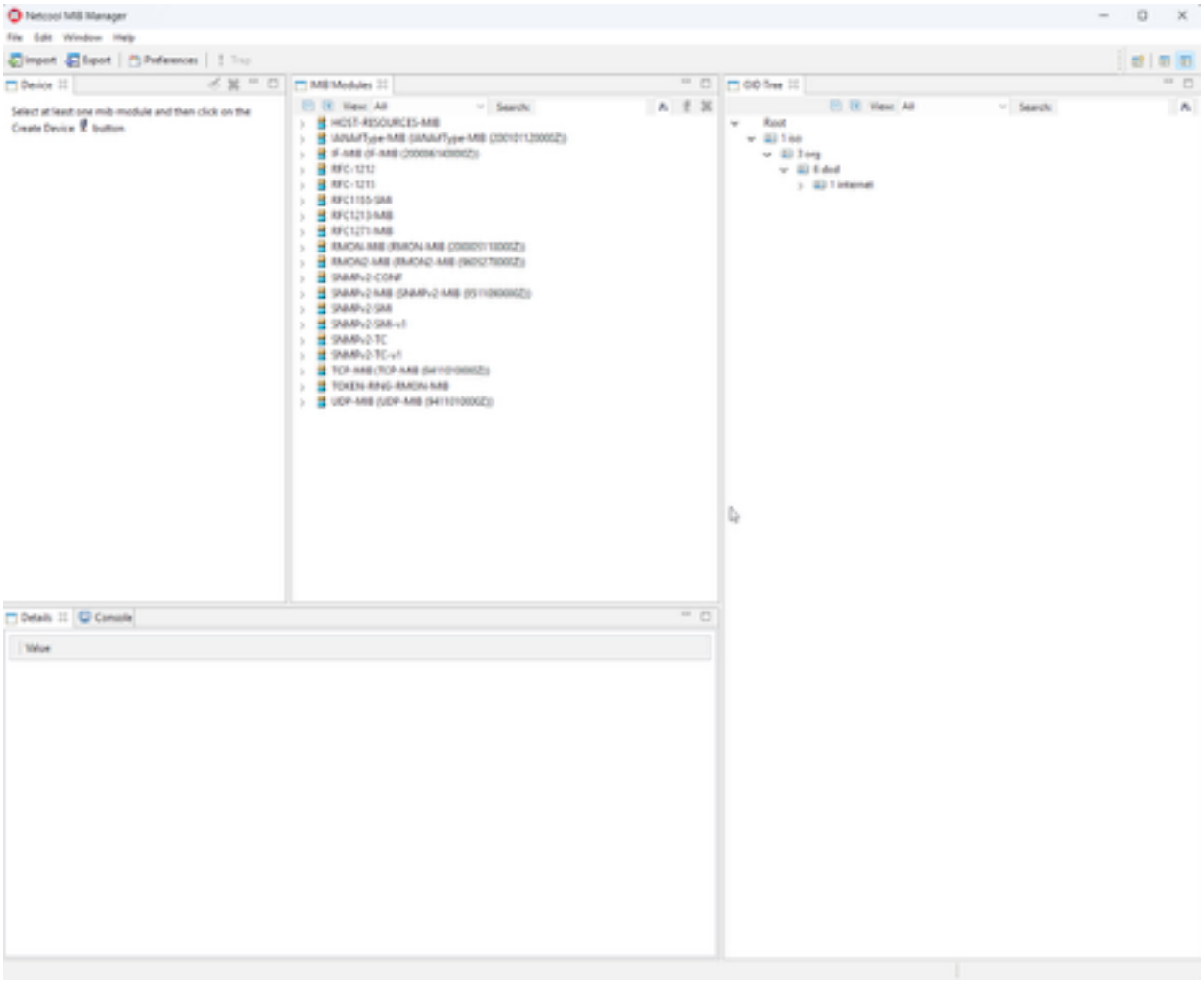


Figure 82: MIB Manager UI

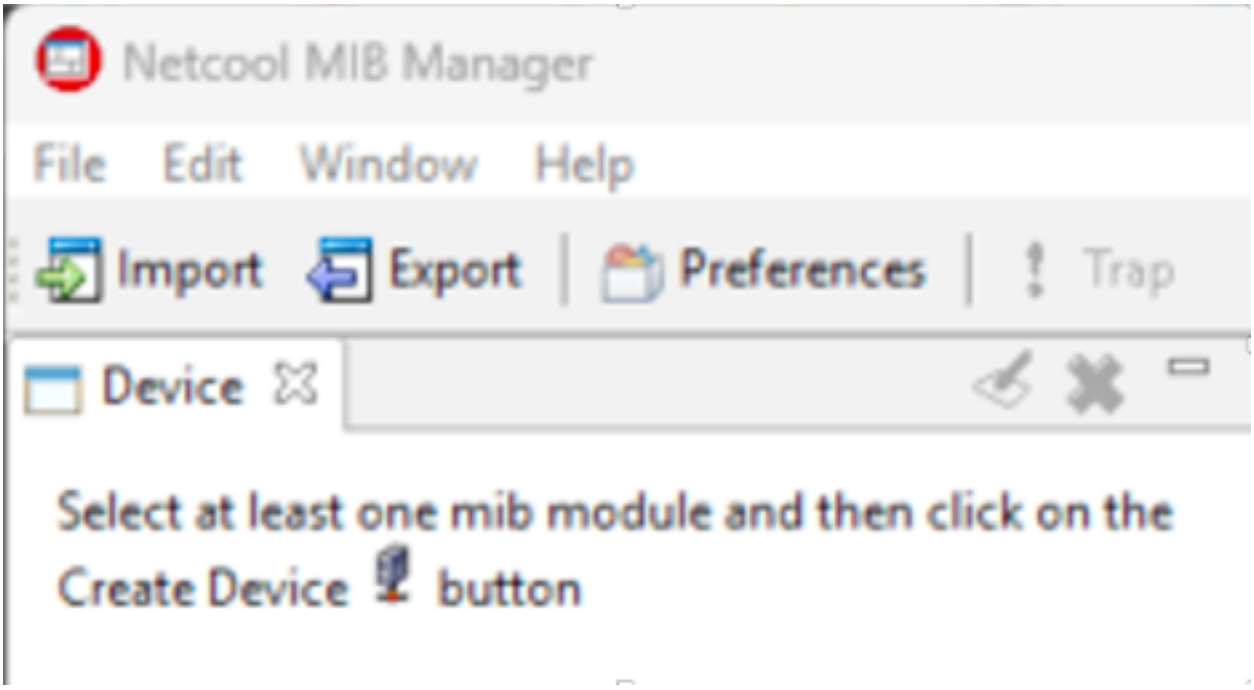


Figure 83: MIB Manager Import dialog

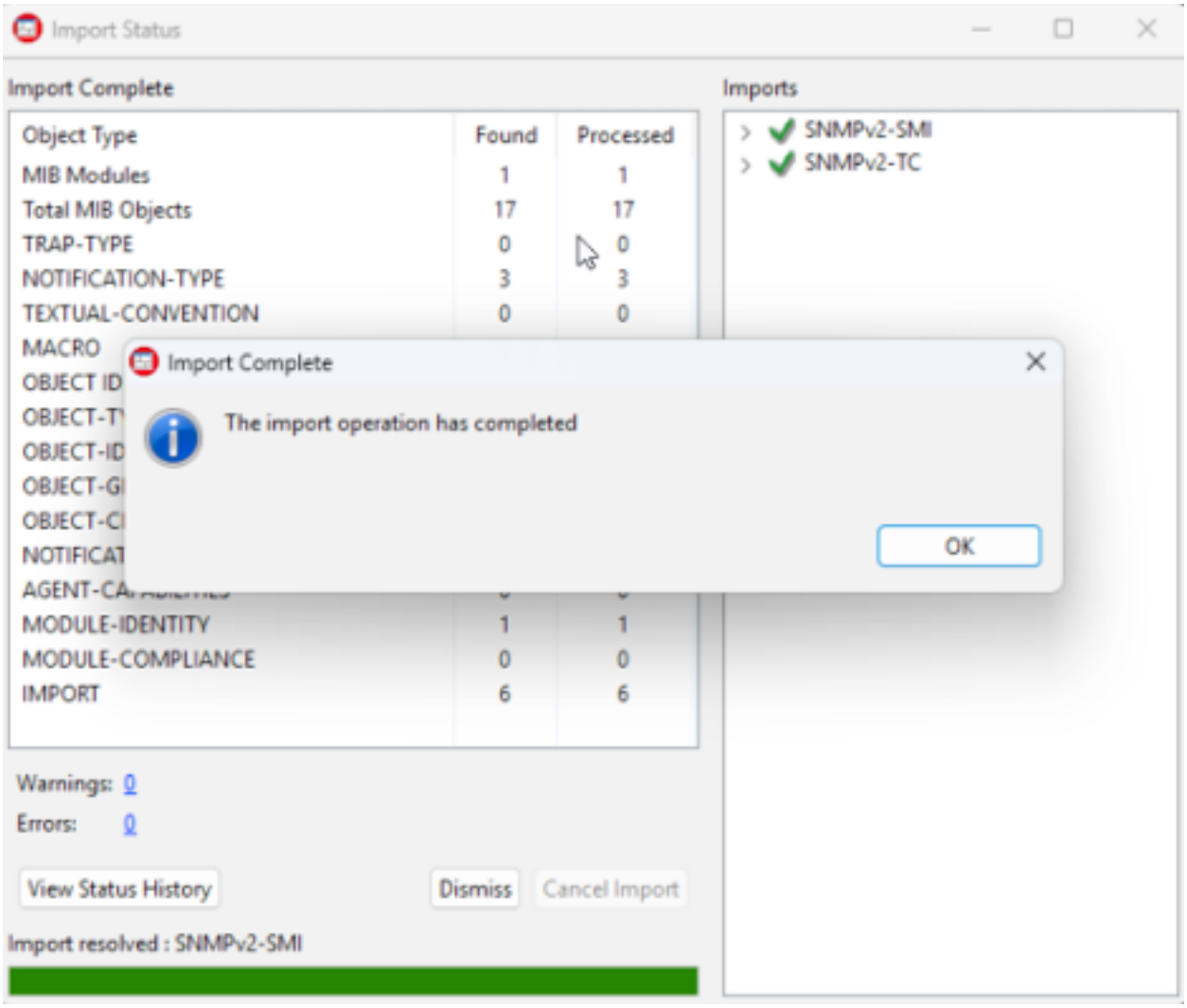


Figure 84: MIB Manager Import status

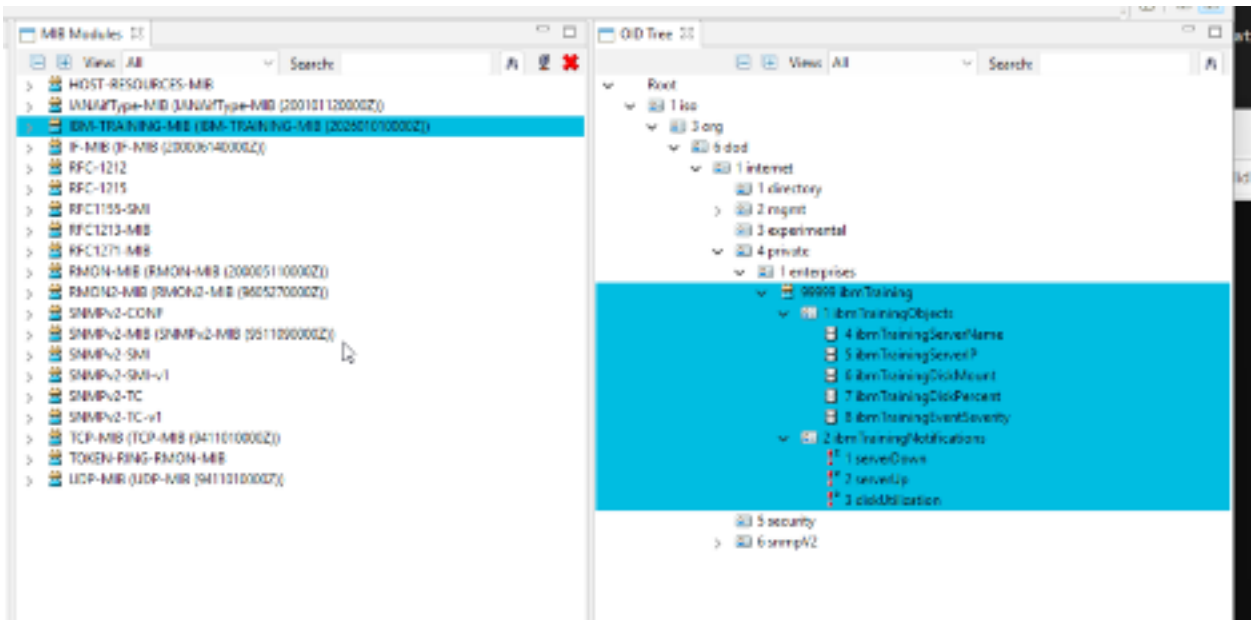


Figure 85: Training MIB tree structure

6. Click on `serverDown` in the tree. The details panel at the bottom should show its full OID (`1.3.6.1.4.1.99999.2.1`), object type, and the description text from the MIB.

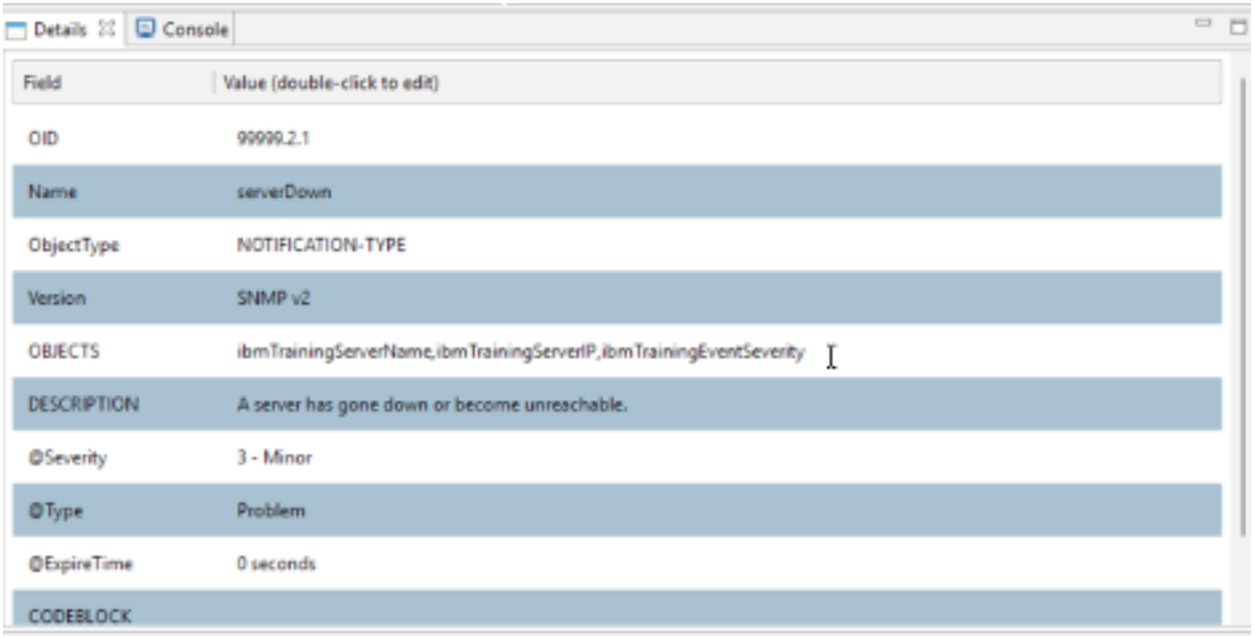


Figure 86: Training MIB trap details

Export the MIB for NcKL

7. Highlight (select) the **IBM-TRAINING-MIB** entry in the MIB module list to ensure the entire MIB tree is selected:

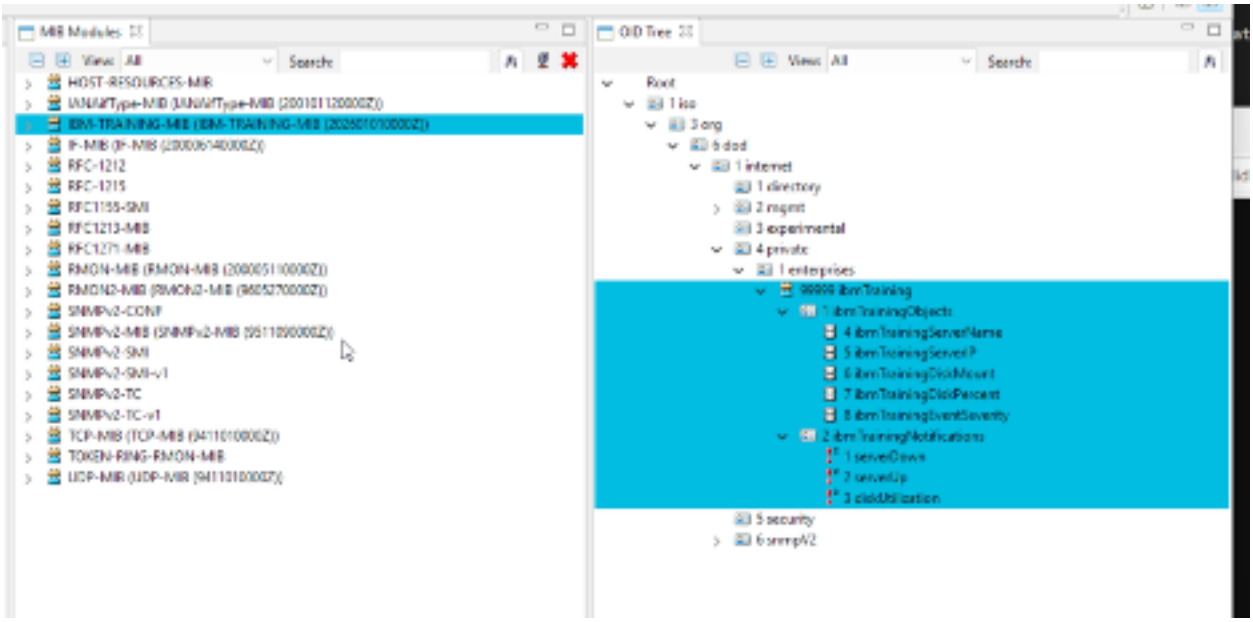


Figure 87: Training MIB tree structure

8. Click the **Export** button in the upper left of the window.
9. In the Export dialog, set the following options:
   - **Directory:** leave as the default exports directory
   - **File Type:** `Netcool Knowledge Library version 3.x`
   - **Scope:** `Traps only`

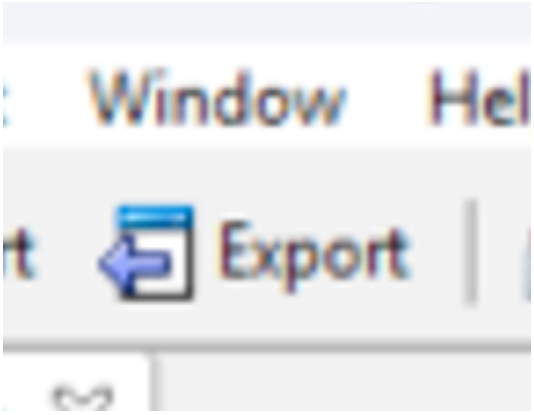


Figure 88: Export

- Check **Selected Subtree(s) only**

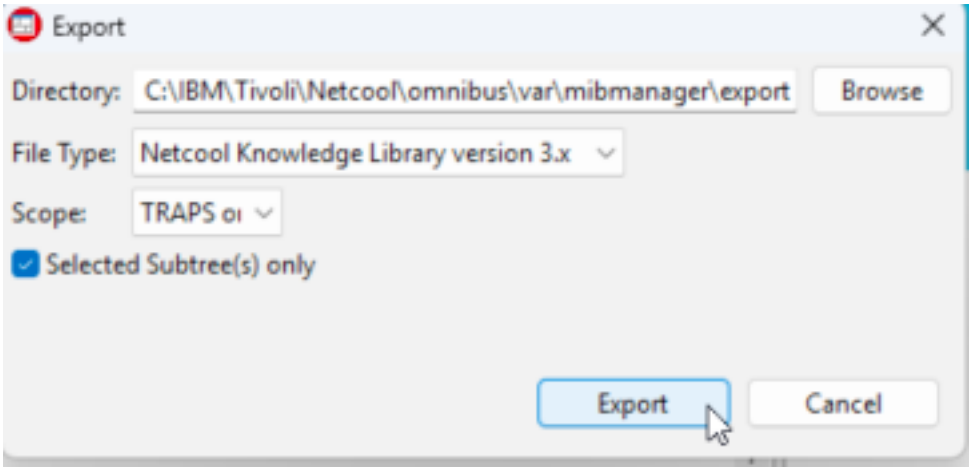


Figure 89: Export

10. Click **Export**. A confirmation dialog will report the number of objects exported (should be 3 — one for each trap type).

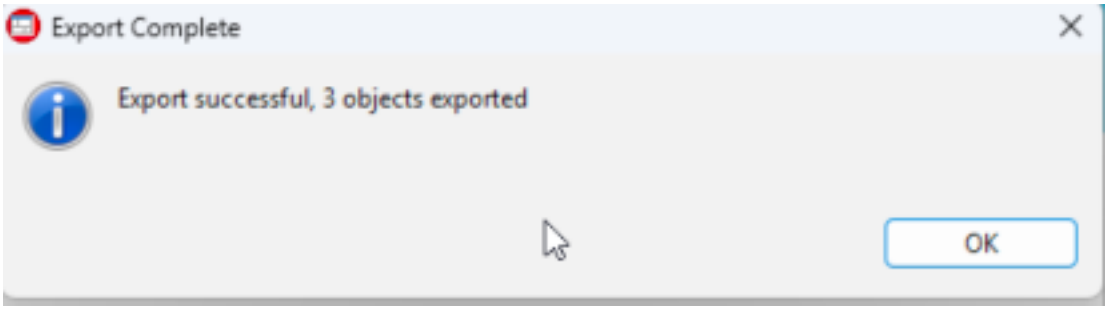


Figure 90: Export

Transfer the exported files to the lab VM

11. On the Windows workstation, open the exports directory: `%OMNIHOME%\var\mibmanager\exports\`

    You will see a directory named: <timestamp>_nckl_3_0, where <timestamp> is the time that you exported the traps. Open it, then open the `ibmtraining` subdirectory. It contains the following files:

File name	Purpose
ibmTraining-IBM-TRAINING-MIB.adv.include.snmptrap.rules	Advanced/optional SNMP trap rules that add extra behavior on top of the base IBM-TRAINING-MIB mapping.
ibmTraining-IBM-TRAINING-MIB.include.snmptrap.rules	Main SNMP trap rules that recognize IBM-TRAINING-MIB trap OIDs and map varbinds into OMNIbus event fields.
ibmTraining-IBM-TRAINING-MIB.sev.snmptrap.lookup	Lookup table that assigns severity, event type, and expiry to IBM-TRAINING-MIB events.
ibmTraining-IBM-TRAINING-MIB.user.include.snmptrap.rules	User customization rules, used to override or extend the vendor's base IBM-TRAINING-MIB rules.
ibmTraining-preclass.include.snmptrap.rules	Pre-classification rules that run before main mapping, doing early generic classification and setup.
ibmTraining-preclass.snmptrap.lookup	Lookup table used by the pre-classification rules (e.g., initial alert group/type/category).
ibmTraining.m2r.master.include.lookup	Master lookup include file that pulls together all IBM-training lookup tables (sev, preclass, etc.).
ibmTraining.m2r.master.include.rules	Master rules include file that pulls together all IBM-training rules files in the correct order.

12. Copy the entire `ibmtraining` directory to the lab VM using SCP. Open a PowerShell or Command Prompt window and run:

```
scp -r "%OMNIHOME%\var\mibmanager\export\<timestamp>_nckl_3_0\ibmTraining"
 ↪  root@<your-server>:/tmp/
```

Replace `<timestamp>` with the actual directory name containing the timestamp of when you exported the traps, and `<your-server>` with your lab VM hostname or IP.

13. Switch to your SSH session on the lab VM and confirm the files arrived:

```
ls /tmp/ibmTraining/
```

You should see all five files listed in the table above.

5.4.3 Integrate the exported rules into NcKL Now that MIB Manager has generated the rules and lookup files, you need to integrate them into the NcKL rules framework. This is a five-step process.

Step 1 — Move the ibmTrianing include files to the NcKL rules include directory

```
mv /tmp/ibmTraining $NC_RULES_HOME/include-snmptrap/
```

Step 2 — Add the lookup reference to the NcKL snmptrap.rules

Cat the following file to obtain the include file location for our ibmTraining rules:

```
cat $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining.m2r.master.include.lookup
```

copy the last line from the output of that command:

```
##############################################################################################
# ibmTraining.m2r.master.include.lookup
# Automatically created by MIB Manager
# Aug 6, 2026 1:34:33 PM
##############################################################################################

include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-preclass.snmptrap.lookup"

include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.sev.snmptrap.
 ↪  lookup"
```

Using vi, open the top-level snmptrap.rules file:

```
vi $NC_RULES_HOME/snmptrap.rules
```

Locate the vendor "lookup includes" section in the top level rule:

```
##############################################################################################
# Enter lookup table Includes below with the following syntax:
#
# include "<$NCHOME>/etc/rules/include-snmptrap/<vendor>/<vendor>.master.include
# .lookup"
##############################################################################################
```

In this section, you will see an alphabetized list of vendor “include.lookup” files. The lookup files provide mappings of OIDs to human readable variables as mapped in the relevant MIB files. As such, the operator will see the actual OID names (e.g. ifIndex, LinkDown, etc) instead of an OID number (1.3.6.1.4.1.x.x.x).

The best practice is to paste the lookup include in the appropriate alphabetized location as such:

```
...
...
include "$NC_RULES_HOME/include-snmptrap/huawei/huawei.master.include.lookup"
include "$NC_RULES_HOME/include-snmptrap/IANA/IANA.master.include.lookup"
include "$NC_RULES_HOME/include-snmptrap/ibm/ibm.master.include.lookup"
include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining.m2r.master.include.lookup"
include "$NC_RULES_HOME/include-snmptrap/IEEE/IEEE.master.include.lookup"
include "$NC_RULES_HOME/include-snmptrap/IETF/IETF.master.include.lookup"
...
...
```

Key line above: `include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining.m2r.master.include.look`

Save the file and exit.

Step 3 — Add the “rules include” to the NcKL snmptrap.rules file

In the same snmptrap.rules file, locate the vendor “rules includes” section:

```
        switch($enterprise)
        {
            case "dummy case statement": ### This will prevent syntax errors in case no includes
↪  are added below.

            ##############################################################################################
            # Enter rules file Includes below with the following syntax:
            #
            # include "<$NCHOME>/etc/rules/include-snmptrap/<vendor>/<vendor>.master.include
            # .rules"
            ##############################################################################################
```

In this section, the **top-level rules file** uses the trap’s **Enterprise ID** to decide *which* rules to apply when the trap is an **enterprise-specific** trap (generic-trap **6**).

Typically, what comes next is a list of enterprise **“master” include files**. Each master file:

- corresponds to one **vendor/enterprise**, and
- in turn **includes many MIB-specific include files** that contain the actual **trap OIDs to be processed**.

For example, **Cisco** has a `master.include.rules` file that pulls in **hundreds of individual MIB include files**.

In our lab:

- We created our own enterprise with **Enterprise ID 99999**.
- For that enterprise, we currently have **only one MIB** referenced in `ibmTraining.m2r.master.include.rules`.
- Conceptually, if we later add more MIBs for the **`ibmTraining`** enterprise, we would **add them to `ibmTraining.m2r.master.in`** *not* directly into the top-level rules file.

The key idea is **modularity**:

- The **top-level rules** only choose the right **enterprise master file**.
- Each **enterprise master file** then manages its own set of **MIB-specific includes**.

This keeps the configuration **clean, scalable, and easy to maintain** as more vendors or MIBs are added.

Much like the **lookup include section**, this section is an **alphabetized list of vendor MIB rules include files**.

Each line includes a vendor's **master rules file**, which in turn pulls in that vendor's **MIB-specific trap definitions**.

For our lab, we need to include the **ibmTraining master rules file**, because it contains the include for the **IBM-TRAINING-MIB** trap definitions:

```
include "$NC_RULES_HOME/include-snmptrap/hatteras/hatteras.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/huawei/huawei.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/IANA/IANA.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/ibm/ibm.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining.m2r.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/IEEE/IEEE.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/IETF/IETF.master.include.rules"
include "$NC_RULES_HOME/include-snmptrap/infinera/infinera.master.include.rules"
```

Key line above: `include "$NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining.m2r.master.include.rule`

Once you have added our master.include.rules file as directed above, save and quit your editor.

Step 4 — Run a syntax check on the top-level NcKL rules file

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile $NC_RULES_HOME/snmptrap.rules
```

When you run the **syntax checker**, it traverses the entire **NcKL include structure**, checking for:

- **Syntax errors** in all referenced rules files.
- **`@Column` assignments** to make sure they match the columns defined in the configured **ObjectServer**.

Any problems are reported in the syntax checker **output**.

If everything is valid, you'll see output similar to:

```
2026-08-06T15:15:18: Information: I-UNK-000-000: Rules file syntax OK
2026-08-06T15:15:18: Information: I-UNK-000-000: Disconnecting ...
```

If the syntax check fails, look in the output for the **filename** and **line number** of the error, then correct the issue in that rules file and run the checker again.

Step 5 — Restart MTTrapd

Stop the probe:

```
 kill $(pgrep nco_p_mttrapd)
```

Start the probe:

```
$OMNIHOME/probes/nco_p_mttrapd &
```

Validate that the **mttrapd** probe is listening on UDP port **162**:

```
netstat -anlp | grep udp | grep 162
```

Expected output:

```
udp        0      0 0.0.0.0:162             0.0.0.0:*                   1991508/nco_p_mttra
```

5.4.4 Verify OID resolution Now we will send a test trap using the raw OID of the `serverDown` notification, as defined in our **IBM-TRAINING-MIB** mib file:

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.1 \
  1.3.6.1.4.1.99999.1.4 s "web-server-01" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.10 \
```

```
  1.3.6.1.4.1.99999.1.8 i 5
```

Breaking down the command:

Argument	Meaning
`1.3.6.1.4.1.99999.2.1`	The `serverDown` notification OID
`1.3.6.1.4.1.99999.1.4 s "web-server-01"`	`ibmTrainingServerName` varbind, string value
`1.3.6.1.4.1.99999.1.5 a 192.168.1.10`	`ibmTrainingServerIP` varbind, IP address value
`1.3.6.1.4.1.99999.1.8 i 5`	`ibmTrainingEventSeverity` varbind, integer 5 (Critical)

Return to the Web GUI and observe your server down trap:



Figure 91: Observed server down trap in Web GUI

Check the MTTrapd RawCapture immediately after sending the trap:

```
tail -30 $OMNIHOME/var/mttrapd.cap
```

With the MIB loaded, you should see the token names resolved — `$ibmTrainingServerName`, `$ibmTrainingServerIP`, and `$ibmTrainingEventSeverity` — rather than raw OID strings. This confirms the MIB is correctly loaded and the probe is decoding the varbinds using your MIB definitions.

5.5 Customizing the NcKL-Generated Rules

When MIB Manager exports a MIB for NcKL, it generates a generic but functional set of rules. The generated rules build basic field mappings from the trap's OID components and varbind names, and they work out of the box — but the `@Summary` text is typically generic, and deduplication may not behave as expected until you examine and tune the `@Identifier` field. This lab walks through exactly the kind of customization you will do in production.

Objective: Review the NcKL-generated rules for the IBM-TRAINING-MIB, customize `@Summary` to include varbind values, fix the `@Identifier` to enable correct deduplication, and tune severity and expiry via the NcKL severity lookup file.

Prerequisites: IBM-TRAINING-MIB exported and integrated into NcKL (Section 5.4 complete). MTTrapd restarted and running. ObjectServer running.

5.5.1 Review the generated rules file Open the generated rules file and review what MIB Manager produced:

```
vi $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.include.snmptrap.ru
↪ les
```

At the top of this rules file you'll see a **`case` statement** that uses the **OID** for our training MIB:

- `.1.3.6.1.4.1.99999.2`

This OID matches the **`switch($enterprise)`** statement in the top-level **`snmptrap.rules`** file.

The flow is:

1. The **top-level `snmptrap.rules`** file receives a trap and evaluates `switch($enterprise)`.
2. It hits the **include** for our **enterprise master include file**, which in turn includes **this** rules file.
3. Because the trap's **enterprise ID** matches `.1.3.6.1.4.1.99999.2`, the **`case` block** in this file is selected and **processing continues here**.

So this MIB-specific rules file only runs when the trap belongs to our training enterprise **99999.2**, keeping the logic clean and vendor-scoped. In this rules file you'll see a **switch/case structure** — there is **one case block for each notification type** that MIB Manager exported from the MIB.

Inside each case, the rules typically:

- Set **@AlertGroup** – the high-level category of the event.
- Set **@AlertKey** – the key that identifies the specific object instance (interface, disk, etc.).
- Set **@EventId** – a string ID for the event (for example, SNMPTRAP-serverDown).
- Build a generic **@Summary** using the MIB's **DESCRIPTION** text.

The serverDown case

Look specifically at the **serverDown** case block.

In this case, the rules set the **@Identifier** field using the NcKL default format:

```
@Identifier = @Node + " " + @AlertKey + " " + @AlertGroup + " " +
              $DEFAULT_Type + " " + @Agent + " " + @Manager + " " +
              $specific-trap
```

This is the **NcKL default** way of constructing the identifier.

Key point: **@AlertKey is part of @Identifier**. That matters when you look at **deduplication** later, because events with the **same @Identifier** are treated as repeats of the **same problem**.

User include file (where you customize)

Open the **user include file**, which MIB Manager creates with a basic switch/case structure but no custom logic yet:

```
vi $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.user.include.snmptr
↪  ap.rules
```

This file provides a **switch on the specific-trap identifier**, giving you an empty case block for each notification type.

It is **recommended** that all rule **customizations** are done here, rather than in the MIB-generated rules file. In the user include file you can maintain changes such as:

- Adjusting @Summary, @AlertGroup, @AlertKey, or @Identifier.
- Adding extra fields or additional logic for lab exercises or site-specific behavior.

Important:

- **Do not edit** the main generated rules file (…IBM-TRAINING-MIB.include.snmptrap.rules) directly — MIB Manager will **overwrite it** whenever you re-export the MIB.
- Always put your changes in the **.user.include.snmptrap.rules** file so they **persist across re-exports** and remain clearly separated from the auto-generated content.

5.5.2 Generate initial test events and check deduplication In this step, send two diskUtilization traps with different utilization values:

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3 \
  1.3.6.1.4.1.99999.1.4 s "db-server-02" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.20 \
  1.3.6.1.4.1.99999.1.6 s "/var" \
  1.3.6.1.4.1.99999.1.7 i 64 \
  1.3.6.1.4.1.99999.1.8 i 4

snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3 \
  1.3.6.1.4.1.99999.1.4 s "db-server-02" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.20 \
  1.3.6.1.4.1.99999.1.6 s "/var" \
  1.3.6.1.4.1.99999.1.7 i 77 \
  1.3.6.1.4.1.99999.1.8 i 4
```

Look at the **event list**:

- You see **two separate events**, not one event with `Tally = 2`.
- This means the events are **not being deduplicated**.

The reason is in the **NcKL-generated `@Identifier`**:

- NcKL has included one of the **varbinds that carries the percent utilization** (from OID …`1.7`) in either `@AlertKey` or directly in `@Identifier`.
- Because `@Identifier` contains the **current utilization value**, each trap with a different percentage produces a **different identifier**.

So:

- First trap: utilization 64 → one `@Identifier`.
- Second trap: utilization 77 → a different `@Identifier`.

Since deduplication uses **`@Identifier`** as the key, those two events **cannot dedup** — they are treated as two distinct problems instead of the same disk filling up.

Examine the `Identifier` field on the events (add it to your view if needed). If the identifiers differ between the two events, note what component is changing. In the next step of the lab, you'll adjust **`@AlertKey` / `@Identifier`** so they only use **stable identifiers** (host, disk path, etc.) and **exclude** the changing utilization value, allowing all disk-utilization traps for `/var` on `db-server-02` to deduplicate into a single event.

5.5.3 Customize the @Summary and fix @Identifier Open the user include file for the IBM-TRAINING-MIB:

```
vi $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.user.include.snmptr
↪ ap.rules
```

The first thing we will do in this file is add the following line to add all of the tokens generated by the probe to the @ExtendedAttr field of the alert. This differs from the RawCapture file because the NcKL has the useful feature of creating tokens from the actual varbind names as defined in the MIB, rather than the raw OIDs that you only see in the RawCapture file. Add this line immediately after the log(DEBUG…) statement and before the switch($specific-trap) statement:

```
@ExtendedAttr = nvp_add($*)
```

The nvp_add function is a special probe rules function that simply adds all of the current tokens (either generated by the probe or generated within the rules) to the @ExtendedAttr field of the alert.

Stop the mttrapd probe:

```
kill $(pgrep nco_p_mttrapd)
```

Perform a syntax check of the rules:

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile $NC_RULES_HOME/snmptrap.rules
```

If you don't see the following lines in the syntax check output, scroll back to find the file and the lines in the file that the syntax checker is complaining about, and address any errors before proceeding.

```
2026-08-06T17:43:39: Information: I-UNK-000-000: Rules file syntax OK
2026-08-06T17:43:39: Information: I-UNK-000-000: Disconnecting ...
```

And start the probe:

```
$OMNIHOME/probes/nco_p_mttrapd &
```

Validate that the **mttrapd** probe is listening on UDP port **162**:

```
netstat -anlp | grep udp | grep 162
```

Expected output:

```
udp        0      0 0.0.0.0:162             0.0.0.0:*                   1991508/nco_p_mttra
```

Return to the Web GUI and delete any existing events from your SNMPTraps view to ensure we don't deduplicate our test alert with an existing alert.

Next, send a test event in again using `snmptrap`:

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3   1.3.6.1.4.1.99999.1.4 s
↪  "db-server-02"   1.3.6.1.4.1.99999.1.5 a 192.168.1.20   1.3.6.1.4.1.99999.1.6 s "/var"
↪  1.3.6.1.4.1.99999.1.7 i 77   1.3.6.1.4.1.99999.1.8 i 4
```

Find the new **diskUtilization** alert in the event list, and double-click on the alert to bring up the properties view. Scroll down to the ExtendedAttr field and note the tokens not only include the OIDs, but the human-readable MIB representation of the OID as well:

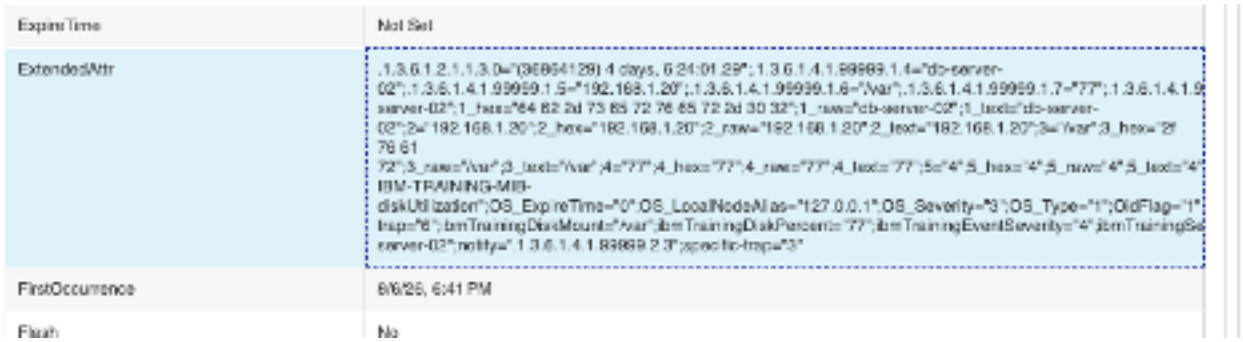


Figure 92: diskUtilization alert properties view

There are some items we shall fix here. First, given that the node name is coming from the host that ran the `logger` command (localhost), but the actual server name is in a varbind and token called **ibmTrainingServerName**, we will set the `@Node` field of the alert to the appropriate token. Second, we see that the disk that this alarm is associated with is in a token called **ibmTrainingDiskMount**, we will set the `@AlertKey` to the value of this token. To to fix deduplication, we will re-work the `@Identifier` field to include only the relevant fields and build from the following tokens: **ibmTrainingServerName**, **ibmTrainingDiskMount**, and the field that contains the alert identifier **@AlertGroup**. Finally, we will rebuild the `@Summary` field to be more useful to operation staff.

Open the user include file for our custom MIB:

```
vi $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.user.include.snmptr
↪  ap.rules
```

Since we know our MIB sets the varbind to the server name for all traps in this MIB, we'll set the `@Node` field to reflect the actual node that is reporting the problem at the top of the user includes, outside of the switch/case statement. Set the @Node field immediately before the switch/case statement:

```
@Node = $ibmTrainingServerName
```

Locate the switch/case for the traps, and find the case statement for the **diskUtilization** trap. Add the following line to that case statement to build a more appropriate `@Identifier` field:

```
@Identifier = $ibmTrainingServerName + " : " + $ibmTrainingDiskMount + " : " + @AlertGroup
```

note: we use the + symbol here to concatenate text, substituting tokens which are preceded with $, and the current value of `@AlertGroup`. We use literal text `:` to separate each value for legibility when examining the `@Identifier` field in the event list.

Key point: excluding the **$ibmTrainingDiskMount** from the identifier would result in any **diskUtilization** alarms from the same server deduplicating with each other. Likewise, excluding the `@Node` or **ibmTrainingServerName** token would result in any **diskUtilization** alarms from the */var* filesystem on *any server* to deduplicate with each other.

Next, we'll set the `@Node` field to reflect the actual node that is reporting the problem, which is in the varbind/token `$ibmTrainingServerName`:

```
@Node = $ibmTrainingServerName
```

Next, we will modify the `@Summary` field to be more human friendly and to also include the drive and the actual disk utilization percentage value:

```
@Summary = "Disk utilization alert for drive " + $ibmTrainingDiskMount + ", utilization is at " +
 ↪  $ibmTrainingDiskPercent
```

Next, as this alert does define a severity in the varbind/token **ibmTrainingEventSeverity**, let's set it here as well:

```
@Severity = $ibmTrainingEventSeverity
```

note: depending on varying severity levels between sources, you may need to use additional logic to map the source severity to one of the five severities in the ObjectServer

While we are editing this file, let's set the @AlertKey appropriately for all event types. Since the **serverDown** and **serverUp** alerts are node/chassis level alarms, and have no associated subcomponent, set the `@AlertKey` for both of these alarms to `@Node` in the appropriate case statements:

```
@AlertKey = @Node
```

Since the **diskUtilization** trap does have an associated subcomponent (the disk), let's set the `@AlertKey` to the **ibmTrainingDiskMount** varbind/token:

```
@AlertKey = $ibmTrainingDiskMount
```

Finally, ensure that the **serverDown** and **serverUp** alerts share the same `@AlertGroup`. The generic_clear enforces the following fields to match between problem/resolution pairs: `@Node`, `@AlertGroup`, `@AlertKey`, `@Manager`. Set the following for each:

```
@AlertGroup = "serverDownUp"
```

When finished, your `user.include.rules` file should look like this:

```
##############################################################################################################
# ibmTraining-IBM-TRAINING-MIB
# Automatically created by MIB Manager
# Aug 6, 2026 1:34:33 PM
##############################################################################################################

log(DEBUG, "<<<<< Entering... ibmTraining-IBM-TRAINING-MIB.user.include.snmptrap.rules >>>>>")

@ExtendedAttr = nvp_add($*)
@Node = $ibmTrainingServerName

        switch($specific-trap) {
        case "1": ### - serverDown
                        @AlertKey = @Node
                        @AlertGroup = "serverDownUp"
        case "2": ### - serverUp
                        @AlertKey = @Node
                        @AlertGroup = "serverDownUp"
        case "3": ### - diskUtilization
                       @Identifier = $ibmTrainingServerName + " : " + $ibmTrainingDiskMount + " :
 ↪  " + @AlertGroup
                        @Summary = "Disk utilization alert for drive " + $ibmTrainingDiskMount +
 ↪  ", utilization is at " + $ibmTrainingDiskPercent + "% utilization"
                        @Severity = $ibmTrainingEventSeverity
                        @AlertKey = $ibmTrainingDiskMount
                default:
}

log(DEBUG, "<<<<< Leaving... ibmTraining-IBM-TRAINING-MIB.user.include.snmptrap.rules >>>>>")
```

Save the file and exit.

Stop the mttrapd probe:

```
kill $(pgrep nco_p_mttrapd)
```

Perform a syntax check of the rules:

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -server EVENTS -rulesfile
 ↪  $NC_RULES_HOME/snmptrap.rules
```

If you don't see the following lines in the syntax check output, scroll back to find the file and the lines in the file that the syntax checker is complaining about, and address any errors before proceeding:

```
2026-08-06T18:05:39: Information: I-UNK-000-000: Rules file syntax OK
2026-08-06T18:05:39: Information: I-UNK-000-000: Disconnecting ...
```

Start the probe:

```
$OMNIHOME/probes/nco_p_mttrapd &
```

Validate that the **mttrapd** probe is listening on UDP port **162**:

```
netstat -anlp | grep udp | grep 162
```

Expected output:

```
udp        0      0 0.0.0.0:162             0.0.0.0:*                   1991508/nco_p_mttra
```

Return to the Web GUI and delete any existing events from your SNMPTraps view to ensure we don't deduplicate our test alert with an existing alert

Next, send our diskUtilization test event in a few times with varying values for utilizaiton using `snmptrap`:

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3   1.3.6.1.4.1.99999.1.4 s
 ↪  "db-server-02"   1.3.6.1.4.1.99999.1.5 a 192.168.1.20   1.3.6.1.4.1.99999.1.6 s "/var"
 ↪  1.3.6.1.4.1.99999.1.7 i 77   1.3.6.1.4.1.99999.1.8 i 4
```

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3   1.3.6.1.4.1.99999.1.4 s
 ↪  "db-server-02"   1.3.6.1.4.1.99999.1.5 a 192.168.1.20   1.3.6.1.4.1.99999.1.6 s "/var"
 ↪  1.3.6.1.4.1.99999.1.7 i 84   1.3.6.1.4.1.99999.1.8 i 4
```

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3   1.3.6.1.4.1.99999.1.4 s
 ↪  "db-server-02"   1.3.6.1.4.1.99999.1.5 a 192.168.1.20   1.3.6.1.4.1.99999.1.6 s "/var"
 ↪  1.3.6.1.4.1.99999.1.7 i 65   1.3.6.1.4.1.99999.1.8 i 4
```

5.5.4 Tune @Type, @Severity and @ExpireTime in the NcKL lookup file In addition to the `*.user.include.snmptrap.rules` file for our MIB, NcKL also creates a **severity lookup file**.

This lookup file lets you maintain **`@Severity`**, **`@Type`**, and **`@ExpireTime`** for each alert in one **simple table**, instead of hard-coding them in the rules. The file is:

```
vi $NC_RULES_HOME/include-snmptrap/ibmTraining/ibmTraining-IBM-TRAINING-MIB.sev.snmptrap.lookup
```

Severity lookup file format The file contains one entry per trap that MIB Manager exported, in this format:

```
{"SNMPTRAP-ibmTraining-IBM-TRAINING-MIB-<trapname>","<severity>","<type>","<expiretime>"}
```

- `severity` → numeric **`@Severity`** (0=Clear, 1=Indeterminate, 2=Warning, 3=Minor, 4=Major, 5=Critical, etc.).
- `type` → numeric **`@Type`** (1=problem, 2=resolution).
- `expiretime`→ **`@ExpireTime`** in seconds (0 = no auto-clear).

** note: setting the severity column to 6 will cause the rules to drop the alert if it is unwanted/inactionable.

By default, MIB Manager sets all entries to:

- `Severity = 3` (Minor)
- `Type = 1` (Problem)

- `ExpireTime = 0` (no expiry)

Lab changes: serverDown, serverUp, diskUtilization For our lab, we want:

- **`serverDown`** to be more important (Critical problem).
- **`serverUp`** to act as a **clear/resolution**.
- **`diskUtilization`** to auto-expire after **30 minutes** if no new traps arrive.

Update the entries as follows.

serverDown Set `serverDown` to **Critical problem**:

```
{"SNMPTRAP-IBM-TRAINING-MIB-serverDown","5","1","0"}
```

- Severity 5 → Critical
- Type 1 → Problem
- Expire 0 → No auto-clear

serverUp Set `serverUp` to a **resolution/clear**:

```
{"SNMPTRAP-IBM-TRAINING-MIB-serverUp","1","2","0"}
```

note: for the generic_clear automation to work, the type 2 alert must have a severity of '1', which will then be set to '0' along with the original problem alert.

- Severity 1 → Indeterminate
- Type 2 → Resolution
- Expire 0 → No auto-clear

diskUtilization Set `diskUtilization` to **Major** with a **30-minute expiry**:

```
{"SNMPTRAP-IBM-TRAINING-MIB-diskUtilization","4","1","1800"}
```

- Severity 4 → Major
- Type 1 → Problem
- Expire 1800 → 1800 seconds = 30 minutes

Final table When you're done, the table in the lookup file should look like:

```
table ibmTraining-IBM-TRAINING-MIB_sev =
{
        {"SNMPTRAP-IBM-TRAINING-MIB-serverDown","5","1","0"},
        {"SNMPTRAP-IBM-TRAINING-MIB-diskUtilization","4","1","1800"},
        {"SNMPTRAP-IBM-TRAINING-MIB-serverUp","1","2","0"}
}
```

> **Note:**
> Adjusting severity and expiry in the **lookup file** (instead of in rules code) is the **NcKL-recommended approach**. It keeps your rule logic clean and makes these operational parameters easy to change. You still need to **restart mttrapd** to pick up changes, but you don't have to modify or re-syntax-check the rules themselves.

Save the file and exit the editor.

Stop the mttrapd probe:

```
kill $(pgrep nco_p_mttrapd)
```

Perform a syntax check of the rules:

```
$OMNIHOME/probes/nco_p_syntax -server EVENTS -rulesfile $NC_RULES_HOME/snmptrap.rules
```

If you don't see the following lines in the syntax check output, scroll back to find the file and the lines in the file that the syntax checker is complaining about, and address any errors before proceeding.

```
2026-08-06T17:43:39: Information: I-UNK-000-000: Rules file syntax OK
2026-08-06T17:43:39: Information: I-UNK-000-000: Disconnecting ...
```

Start the probe:

```
$OMNIHOME/probes/nco_p_mttrapd &
```

Validate that the **mttrapd** probe is listening on UDP port **162**:

```
netstat -anlp | grep udp | grep 162
```

Expected output:

```
udp        0      0 0.0.0.0:162             0.0.0.0:*                       1991508/nco_p_mttra
```

Return to the Web GUI and delete any existing events from your SNMPTraps view before proceeding to the final tests.

5.5.6 Test all trap types

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.1 \
  1.3.6.1.4.1.99999.1.4 s "db-server-02" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.20 \
  1.3.6.1.4.1.99999.1.8 i 5
```

Test 1 — serverDown In the Web GUI you should see a **Critical** (red) event with: - Node = `db-server-02` - AlertGroup = `IBM-TRAINING-MIB` - AlertKey = `serverDown` - Summary = `Server db-server-02 (192.168.1.20) is DOWN` - Type = `1`

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3 \
  1.3.6.1.4.1.99999.1.4 s "db-server-01" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.22 \
  1.3.6.1.4.1.99999.1.6 s "/var" \
  1.3.6.1.4.1.99999.1.7 i 87 \
  1.3.6.1.4.1.99999.1.8 i 4
```

Test 2 — diskUtilization (with ExpireTime) Expected event: - Node = `db-server-01` - AlertKey = `diskUtilization:/var` - Severity = 4 (Major) - Summary = `Disk /var on db-server-02 is at 87% utilization` - ExpireTime = `1800`

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.3 \
  1.3.6.1.4.1.99999.1.4 s "db-server-01" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.22 \
  1.3.6.1.4.1.99999.1.6 s "/home" \
  1.3.6.1.4.1.99999.1.7 i 92 \
  1.3.6.1.4.1.99999.1.8 i 5
```

Test 3 — Second diskUtilization (different mount point) This should produce a **separate** Critical event for `/home` rather than deduplicating with the `/var` event — confirming that the mount point in `@AlertKey` is working correctly.

```
snmptrap -v 2c -c public localhost '' 1.3.6.1.4.1.99999.2.2 \
  1.3.6.1.4.1.99999.1.4 s "db-server-02" \
  1.3.6.1.4.1.99999.1.5 a 192.168.1.20
```

Test 4 — serverUp (clears serverDown) Watch the Web GUI — the `serverDown` Critical event for `db-server-02` should clear within a few seconds as `generic_clear` matches the Type 2 `serverUp` event against the Type 1 `serverDown` event.

Validation: After all four tests: - Two disk utilization events remain (one for `/var`, one for `/home`) — `ExpireTime` of 1800 seconds is counting down. - The `serverDown` event for `db-server-02` is cleared and no longer visible. - No `serverDown` event exists for `db-server-02` in the event list.

5.6 Summary

In this section you have built a complete SNMP trap processing pipeline from the ground up. You have:

- Configured the **MTTrapd probe** to receive SNMP traps on UDP port 162 and forward them to the ObjectServer.
- Integrated MTTrapd with the **NcKL knowledge library**, providing immediate out-of-the-box handling for hundreds of vendor MIBs.
- Used **MIB Manager** to load a custom `IBM-TRAINING-MIB`, enabling MTTrapd to decode OIDs from a device type it had no prior knowledge of.
- Written probe **rules** that map decoded trap tokens to `alerts.status` fields, apply severity mapping from varbind values, implement `serverDown`/`serverUp` correlation using OMNIbus Type 1/Type 2 event pairs, and set expire timers for threshold conditions.
- Validated the end-to-end pipeline by generating test traps with `snmptrap` and observing correct event creation, deduplication, and auto-clear behavior in the Web GUI.

The patterns used here — MIB loading, varbind-driven field mapping, severity mapping via `switch`, Type 1/Type 2 correlation, and `@ExpireTime` for threshold events — are the fundamental building blocks of all production SNMP event management. Every vendor-specific MIB you encounter will follow the same workflow.

> **Note:** In a production environment, a single Netcool/OMNIbus deployment may process millions of SNMP traps per day from thousands of devices across hundreds of different MIBs. NcKL provides the breadth to handle vendor-specific traps out of the box, while the custom rules framework you have practiced here provides the flexibility to handle anything NcKL does not cover.

6. Managing OMNIbus Processes with Process Agent and nco_pad

The Process Agent (`nco_pad`) is the Netcool/OMNIbus process-control component. It provides a centralized mechanism for defining, starting, stopping, monitoring, and—when configured—restarting ObjectServers, probes, gateways, and supporting scripts. Administrators can manage an individual process or operate on a logical group of processes called a *service*.

Process Agent reads its configuration from `nco_pa.conf`. That file defines:

- **Processes** — named commands that Process Agent can launch and supervise, including the command line, host, operating-system account, restart behavior, and process type.
- **Services** — logical groups of related processes that can be started or stopped together.
- **Startup behavior** — whether a service starts automatically when `nco_pad` starts or requires manual intervention.
- **Process monitoring and recovery** — whether Process Agent monitors a child process and how it responds when that process exits.
- **Routing** — the Process Agent instance and host information used by process-control clients and PA-aware components.

Process Agent administration is controlled by the authentication mode configured for the daemon. In a normally authenticated environment, an administrator must be a member of the system `ncoadmin` group and uses their Unix account password when running commands such as `nco_pa_status`, `nco_pa_start`, `nco_pa_stop`, and `nco_pa_shutdown`. This lab starts `nco_pad` with `-authenticate NONE` so that authentication is disabled for the training environment; therefore, press Enter at the login or password prompt instead of entering Unix credentials.

A process definition does not start a component by itself. The process must also be assigned to a service. When a service is configured with `ServiceStart = Auto`, Process Agent starts that service when the daemon starts. Administrators can also use process-control commands and tools to start, stop, or check the status of a named process or service.

Netcool/OMNIbus distinguishes between PA-aware and non-PA-aware processes. Components such as the ObjectServer can register with and communicate directly with Process Agent, allowing Process Agent to track their lifecycle and use dependency relationships. Most probes are non-PA-aware processes: Process Agent can launch and monitor them, but the probe does not communicate directly with Process Agent or participate in PA-aware process dependencies. IBM defines a `PaNOT_PA_AWARE` process as one that cannot communicate with the Process Agent and is unaware that it was started by it.ibm

In this lab, the ObjectServer is already managed by Process Agent as the `MasterObjectServer` process in the `Core` service. You will extend `nco_pa.conf` to define the `nco_p_syslogd` and `nco_p_mttrapd` probes as managed, non-PA-aware processes. You will then group them in a new `Probes` service and configure that service for automatic startup.

After updating the configuration, you will restart the Process Agent and validate:

- The `Core` service starts and the ObjectServer is running.
- The `Probes` service starts both probe processes.
- `nco_pa_status` reports the expected process and service status.
- Individual probes can be stopped and started without stopping the ObjectServer.
- The complete `Probes` service can be stopped and started as a single operational unit.

This arrangement provides a simple operational model: the ObjectServer remains the core event-management service, while the Syslogd and MTTrapd probes are managed together as event-ingestion processes.

What Process Agent manages

The Process Agent daemon is `nco_pad`. In this lab, it runs as the `NCO_PA` server and reads its configuration from:

```
$OMNIHOME/etc/nco_pa.conf
```

The configuration contains three related kinds of entries:

- A **process definition** describes how to start one managed process.
- A **service definition** groups processes for coordinated startup and shutdown.
- A **routing table** identifies the Process Agent server available on the host.

The initial configuration contains one managed process, `MasterObjectServer`, in the `Core` service. The probe definitions added in this lab are managed by Process Agent but are not Process Agent-aware, so they use `PaNOT_PA_AWARE`.

6.0 Inspecting the existing Process Agent configuration

Objective Confirm the current Process Agent and ObjectServer state before adding the probe processes.

Prerequisites

- The `EVENTS` ObjectServer is running under Process Agent.
- The `NCO_PA` Process Agent server is configured.
- SSH access to the lab server.
- The `OMNIHOME` environment variable is set.
- The student can edit files with `vi` or another shell editor.

Tasks

1. Confirm the Process Agent and ObjectServer processes are present:

   ```
   ps -ef | grep nco_ | grep -v grep
   ```

   The existing setup should include an `nco_pad` process and an `nco_objserv` process similar to:

   ```
   root  1093023  1  1 16:08 ? 00:00:00 .../nco_pad -authenticate NONE
   root  1093051 1093023 11 16:08 ? 00:00:00 .../nco_objserv -name EVENTS -pa NCO_PA
   ```

2. Display the current Process Agent status:

   ```
   nco_pa_status
   ```

Because this lab runs Process Agent with `-authenticate NONE`, press Enter at the login or password prompt. Do not enter a username or password. The initial status should show the `Core` service and the `MasterObjectServer` process in `RUNNING` status. Use the same approach at every subsequent `nco_pa_*` command prompt in this lab.

3. Display the current Process Agent configuration:

```
cat $OMNIHOME/etc/nco_pa.conf
```

4. Confirm that the original configuration contains the `MasterObjectServer` process and the `Core` service.

Validation The exercise is successful when:

- `nco_pad` is running.
- `nco_objserv` is running with the `EVENTS` name and `NCO_PA` Process Agent.
- `nco_pa_status` shows `MasterObjectServer` as `RUNNING` in the `Core` service.
- The original configuration does not yet contain the probe process or service definitions.

Expected result You have confirmed the starting state before changing the Process Agent configuration.

6.1 Adding probe process definitions and the Probes service

Objective Add the syslogd and MTTrapd probes as managed Process Agent processes and group them in an automatically started `Probes` service.

Background A Process Agent process definition supplies the command and operating context used to start a process. The `Managed` attribute tells Process Agent to monitor the process. With `RetryCount = 0`, Process Agent does not automatically retry a process after it exits; the lab will explicitly demonstrate stop and start operations later.

The `Probes` service is a non-master service. Its processes are independent of the `Core` master service, so stopping `Probes` does not stop the ObjectServer.

Tasks

1. Create a backup of the current Process Agent configuration before editing it:

```
cp $OMNIHOME/etc/nco_pa.conf $OMNIHOME/etc/nco_pa.conf.before-lab6
```

2. Open the Process Agent configuration:

```
vi $OMNIHOME/etc/nco_pa.conf
```

3. Add the following process definitions after the existing `MasterObjectServer` process definition:

```
nco_process 'SyslogdProbe'
{
        Command '$OMNIHOME/probes/nco_p_syslogd -server EVENTS -rulesfile
 ↪  $OMNIHOME/probes/linux2x86/syslogd-training.rules' run as 0
        Host            =       'ibm-training-server1.ibmtraining.org'
        Managed         =       True
        RestartMsg      =       '${NAME} running as ${EUID} has been restored on ${HOST}.'
        AlertMsg        =       '${NAME} running as ${EUID} has died on ${HOST}.'
        RetryCount      =       0
        ProcessType     =       PaNOT_PA_AWARE
}

nco_process 'MttrapdProbe'
{
        Command '$OMNIHOME/probes/nco_p_mttrapd -server EVENTS -rulesfile
 ↪  $NC_RULES_HOME/snmptrap.rules' run as 0
        Host            =       'ibm-training-server1.ibmtraining.org'
```

```
        Managed         =       True
        RestartMsg      =       '${NAME} running as ${EUID} has been restored on ${HOST}.'
        AlertMsg        =       '${NAME} running as ${EUID} has died on ${HOST}.'
        RetryCount      =       0
        ProcessType     =       PaNOT_PA_AWARE
}
```

Use the actual fully qualified hostname of the lab server instead of the example shown here. In this example, the fully qualified hostname is `ibm-training-server1.ibmtraining.org`; replace it with the server name assigned to your lab environment.

4. Add the following service definition after the existing `Core` service definition:

```
nco_service 'Probes'
{
        ServiceType     =       Non-Master
        ServiceStart    =       Auto
        process 'SyslogdProbe' NONE
        process 'MttrapdProbe' NONE
}
```

5. Save and close the file.
6. Review the edited configuration and confirm that:
   - `SyslogdProbe` uses the training syslogd rules file.
   - `MttrapdProbe` uses `$NC_RULES_HOME/snmptrap.rules`.
   - Both processes use the correct host and `PaNOT_PA_AWARE`.
   - The `Probes` service is `Non-Master` and `Auto`.
   - Both probe processes are listed in the `Probes` service.

Validation The exercise is successful when:

- A backup of the original configuration exists.
- Both process definitions are present in `$OMNIHOME/etc/nco_pa.conf`.
- The `Probes` service contains both probe processes.
- The original `Core` service and `MasterObjectServer` definition remain unchanged.

Expected result The Process Agent configuration contains the ObjectServer, two managed probes, the existing `Core` service, and the new automatically started `Probes` service.

6.2 Restarting Process Agent and validating automatic service startup

Objective Restart Process Agent using the updated configuration and confirm that the `Core` and `Probes` services start with their expected processes.

Tasks

1. Shut down Process Agent and stop all managed processes:

```
nco_pa_shutdown
```

2. At the shutdown prompt, select option 2:

```
2) Shutdown Server and stop all managed processes.
```

   Confirm that the shutdown completes.

3. Confirm that the Process Agent is no longer available:

```
nco_pa_status
```

The command should report that it failed to connect to NCO_PA. This is expected because Process Agent was shut down.

4. Start the Process Agent with authentication disabled, as in the existing lab environment:

```
nco_pad -authenticate NONE
```

Process Agent reads $OMNIHOME/etc/nco_pa.conf and automatically starts services whose ServiceStart is Auto.

5. Display Process Agent status:

```
nco_pa_status
```

6. Confirm that the status includes all three managed processes:

```
Service Name  Process Name       Status   PID
Core          MasterObjectServer RUNNING  <pid>
Probes        SyslogdProbe       RUNNING  <pid>
              MttrapdProbe       RUNNING  <pid>
```

The exact PIDs will differ on each run.

Validation The exercise is successful when:

- Process Agent shuts down and stops its managed processes.
- nco_pa_status cannot connect while Process Agent is stopped.
- nco_pad -authenticate NONE starts successfully.
- The Core and Probes services start automatically.
- MasterObjectServer, SyslogdProbe, and MttrapdProbe all show RUNNING status.

Expected result Process Agent loads the updated configuration and automatically starts both probe processes in the Probes service.

6.3 Controlling an individual managed process

Objective Stop and start one managed probe without affecting the other probe or the ObjectServer.

Tasks

1. Stop only the syslogd process:

```
nco_pa_stop -process SyslogdProbe
```

2. Display status:

```
nco_pa_status
```

3. Confirm that SyslogdProbe is DEAD with PID 0, while MttrapdProbe and MasterObjectServer remain RUNNING.
4. Start the syslogd process again:

```
nco_pa_start -process SyslogdProbe
```

5. Display status again:

```
nco_pa_status
```

6. Confirm that SyslogdProbe returns to RUNNING with a new PID, while the other managed processes remain running.

Validation The exercise is successful when:

- Stopping SyslogdProbe changes only that process to DEAD.
- MttrapdProbe and MasterObjectServer remain RUNNING.
- Starting SyslogdProbe returns it to RUNNING.
- The restarted process receives a new PID.

Expected result A single managed process can be controlled independently through Process Agent.

6.4 Controlling the Probes service

Objective Stop and start the complete Probes service and observe how Process Agent controls its member processes.

Tasks

1. Stop the Probes service:
   ```
   nco_pa_stop -service Probes
   ```
2. Display status:
   ```
   nco_pa_status
   ```
3. Confirm that both SyslogdProbe and MttrapdProbe are DEAD, while MasterObjectServer remains RUNNING in the Core service.
4. Start the Probes service:
   ```
   nco_pa_start -service Probes
   ```
5. Display status again:
   ```
   nco_pa_status
   ```
6. Confirm that both probe processes return to RUNNING status.

Validation The exercise is successful when:

- Stopping Probes stops both member processes.
- The ObjectServer remains running.
- Starting Probes starts both probe processes again.
- nco_pa_status reports the expected service and process states.

Expected result A Process Agent service provides coordinated control over a group of related probes.

6.5 Reviewing the completed configuration

Objective Relate the Process Agent configuration to the observed process and service behavior.

Tasks

1. Review the final configuration:
   ```
   cat $OMNIHOME/etc/nco_pa.conf
   ```
2. Confirm that the file contains:
   - MasterObjectServer with ProcessType = PaPA_AWARE.
   - SyslogdProbe with ProcessType = PaNOT_PA_AWARE.
   - MttrapdProbe with ProcessType = PaNOT_PA_AWARE.

- The Core master service.
- The Probes non-master service.
- The existing NCO_PA routing entry.

3. Display the final runtime state:

```
nco_pa_status
```

4. Confirm that all expected processes are RUNNING:

```
Core    MasterObjectServer  RUNNING
Probes  SyslogdProbe        RUNNING
        MttrapdProbe        RUNNING
```

Validation The exercise is successful when the final configuration and runtime status agree: the processes listed in the Probes service are the processes shown as running by nco_pa_status.

Expected result You can read a Process Agent configuration, explain how process definitions map to services, and control individual processes or complete services with the Process Agent command-line utilities.

Process Agent routing table

The nco_routing section identifies the Process Agent instances that can be contacted for process-control operations. It maps a host name to the name of the Process Agent running on that host.

```
nco_routing
{
   host 'ibm-training-server1.ibmtraining.org' 'NCO_PA'
}
```

In this example:

- ibm-training-server1.ibmtraining.org is the host on which the Process Agent is running.
- NCO_PA is the Process Agent's configured server name.
- The entry tells Netcool/OMNIbus process-control clients where to send requests for that Process Agent.

For example, when an administrator uses a process-control command or GUI to request the status of SyslogdProbe, start the Probes service, or stop MasterObjectServer, the client uses this routing entry to locate the NCO_PA Process Agent on ibm-training-server1.ibmtraining.org.

Local and remote control

Even on a single-host lab system, the routing table is useful because it gives the Process Agent a named, consistent control endpoint. In a distributed deployment, the same concept allows centralized process control across multiple hosts.

For example:

```
nco_routing
{
   host 'omnibus-core-01.example.net' 'NCO_PA_CORE'
   host 'probe-host-01.example.net'   'NCO_PA_PROBES_01'
   host 'probe-host-02.example.net'   'NCO_PA_PROBES_02'
   host 'gateway-host-01.example.net' 'NCO_PA_GATEWAYS'
}
```

A process-control client can then connect to the appropriate Process Agent and manage processes on that host without requiring a separate login to every server.

Secure-mode entries

The optional user and password values are used only when the **target** Process Agent runs in secure mode. The named account must belong to the UNIX ncoadmin group on the target host.

Conceptually, a secure-mode route can look like this:

```
nco_routing
{
   host 'probe-host-01.example.net' 'NCO_PA_PROBES_01' 'netcooladmin' 'encrypted-password'
}
```

Do not store clear-text passwords in `nco_pa.conf`. Use `nco_pa_crypt` to generate the encrypted form required by the configuration.

What it does not control

The routing table does not define process commands, services, startup order, or restart behavior. Those are defined elsewhere in `nco_pa.conf`:

- `nco_process` defines what command Process Agent starts.
- `nco_service` groups processes for operational control.
- `nco_routing` identifies where a Process Agent can be reached for control requests.

For this lab, the routing section simply declares that the local Process Agent named `NCO_PA` is running on `ibm-training-server1.ibmtraining.org`.

6.6 Summary

In this lab you:

- Inspected the existing Process Agent and ObjectServer setup.
- Added `SyslogdProbe` and `MttrapdProbe` process definitions.
- Created an automatically started non-master `Probes` service.
- Restarted `nco_pad` and validated automatic service startup.
- Stopped and started an individual managed process.
- Stopped and started the complete `Probes` service.
- Used `nco_pa_status` to verify service, process, status, and PID changes.

The key concept is that Process Agent manages process lifecycle through declarative process and service definitions. Individual processes can be controlled independently, while a service provides coordinated management for a related group of processes.

7. Cloning an ObjectServer

Objective

Create a separate ObjectServer named `EVENTS_BACKUP` by exporting the configuration of the existing `EVENTS` ObjectServer and using that export to initialize a new instance.

By the end of this lab, you will be able to:

- Export an ObjectServer configuration with `nco_osreport -dbinit`.
- Identify which exported configuration content is appropriate for a training or test clone.
- Initialize a new ObjectServer with `nco_dbinit`.
- Add a unique communications entry for the new ObjectServer.
- Start and verify the clone independently of the source ObjectServer.

Scenario

`EVENTS` is the existing ObjectServer used by the lab environment. You will create `EVENTS_BACKUP` as a separate ObjectServer on the same host.

The clone is intended for controlled testing and training. It will contain the source ObjectServer's schema and selected configuration, but it will **not** include historical event rows from `alerts.status`.

```
EVENTS
  │
  ├─ nco_osreport -dbinit
  │
  ├─ Reviewed SQL export
  │
  └─ nco_dbinit
       │
       └─ EVENTS_BACKUP
            ├─ Separate ObjectServer database
            ├─ Separate omni.dat endpoint
            ├─ Separate Process Agent process
            └─ No historical alerts.status data
```

How cloning works

`nco_dbinit` creates and initializes an ObjectServer from SQL definition files. It does not connect to an existing ObjectServer and copy its configuration automatically.

The clone workflow is therefore:

1. Export the source ObjectServer configuration.
2. Review the export and decide what should be replicated.
3. Initialize a new ObjectServer using selected SQL files.
4. Configure communications and process control for the new instance.
5. Start and validate the clone.

The export can contain more than tables and columns. Depending on the source configuration, it can include custom schema, ObjectServer triggers, automations, users, groups, roles, desktop configuration, and historical alert data.

> **Important:** Treat exported SQL files as configuration artifacts. They may contain credentials, user definitions, automation actions, script paths, gateway configuration, hostnames, and other environment-specific details.

Prerequisites

Before beginning, confirm:

- The `EVENTS` ObjectServer is running.
- You have authorized ObjectServer administrator credentials.
- `OMNIHOME` is defined.
- You can write to a temporary working directory.
- `EVENTS_BACKUP` is not already configured or running.
- You know the fully qualified host name and an unused port for `EVENTS_BACKUP`.

Confirm the source ObjectServer is reachable:

```
$OMNIHOME/bin/nco_ping EVENTS
```

Confirm that `EVENTS_BACKUP` is not already running:

```
ps -efww | grep '[n]co_objserv'
```

7.1 Export the source configuration

Objective

Export the EVENTS ObjectServer configuration into SQL files for review and reuse.

1. Create and enter a dedicated export directory:

```
mkdir -p /tmp/events-backup-export
cd /tmp/events-backup-export
```

2. Export the source ObjectServer:

```
$OMNIHOME/bin/nco_osreport \
  -server EVENTS \
  -user root \
  -password 'netcool' \
  -dbinit
```

3. Confirm that the export completed and that the expected SQL files were generated:

```
ls -1 *.sql
```

The output includes the following files:

File	Purpose
`system.sql`	Creates the core ObjectServer system database structure, including system tables and default security foundations: system users, groups, roles, and permissions. IBM advises that this file should not be edited.
`application.sql`	Creates the initial alert and tools database tables—the standard structures used for `alerts.*` and related application data. Use it when creating a clean ObjectServer without importing the source system's table contents.
`alertsdata.sql`	Contains exported row data from non-system tables, potentially including `alerts.status`, `alerts.details`, `alerts.journal`, tools, conversions, and other configured data. Import it only when a clone requires the source data. When used with `nco_dbinit`, specify both `-alertsdata` and `-alertsdatafile alertsdata.sql`; otherwise, many application tables can remain empty.
`desktop.sql`	Defines initial desktop configuration, including colors, conversions, tools, and menus used by classic desktop clients. It is often empty in modern environments, but `nco_dbinit` expects it as part of the standard initialization set.
`automation.sql`	Defines ObjectServer automations: trigger groups, database triggers, and internal procedures. Review this file carefully before cloning across environments because its content may invoke scripts, gateways, notifications, or other source-environment-specific actions.
`security.sql`	Adds security configuration beyond the basic system security created by `system.sql`, including additional users, groups, roles, role assignments, and grants. Ownership of exported entities transfers to `root` in the destination ObjectServer; original owners receive permissions on the entities they owned rather than retaining ownership. IBM documentation

Validation

You should have a set of SQL files in `/tmp/events-backup-export`.

Do not continue if `nco_osreport` reports an authentication, communications, permission, or SQL export error.

7.2 Select the clone content

Objective

Choose the configuration that belongs in a clean training/test ObjectServer.

For this lab, initialize EVENTS_BACKUP with:

```
system.sql
application.sql
desktop.sql
automation.sql
security.sql
```

Do **not** import:

```
alertsdata.sql
```

`alertsdata.sql` contains exported event data. Omitting it creates a clean ObjectServer without the source system's existing `alerts.status` rows.

Review before use

Review the following files sufficiently to understand what is being carried into the clone:

```
less system.sql
less automation.sql
less security.sql
```

Pay particular attention to:

- Custom `alerts.status` columns and database objects.
- Automations that invoke scripts, commands, gateways, email, or external integrations.
- Hostnames, filesystem paths, Process Agent names, and external endpoint references.
- User, group, role, and security definitions.

For this training environment, the exported configuration is presumed approved for use. In a production-to-test clone, you would normally sanitize or omit source-specific automations and security content before initialization.

Validation

You should be able to explain the difference between:

- A **configuration clone**, which copies selected schema/configuration.
- A **data clone**, which additionally imports historical alert rows.

7.3 Initialize EVENTS_BACKUP

Objective

Create a new ObjectServer named EVENTS_BACKUP from the selected export files.

From the export directory, run:

```
$OMNIHOME/bin/nco_dbinit \
  -server EVENTS_BACKUP \
  -systemfile system.sql \
  -applicationfile application.sql \
  -desktopfile desktop.sql \
  -automationfile automation.sql \
  -securityfile security.sql
```

Do not add the `-alertsdata` or `-alertsdatafile` options. This lab intentionally creates a clean clone.

`nco_dbinit` creates the ObjectServer database and imports the selected SQL definitions. It does not start the ObjectServer.

> On Linux and UNIX systems, `nco_dbinit` can appear as a symbolic link to `nco_run`. This is expected. `nco_run` is a Netcool/OMNIbus launcher that prepares the required runtime environment and invokes the utility associated with the command name. Continue to run `nco_dbinit`; do not invoke `nco_run` directly.

Validation

Confirm that a properties file was created for the new ObjectServer:

```
ls -l $OMNIHOME/etc/EVENTS_BACKUP.props
```

> Note: nco_osreport -dbinit does not clone the ObjectServer .props file, and that is intentional. It exports database-resident ObjectServer configuration into SQL, while an ObjectServer properties file is a local, file-based runtime configuration artifact.

7.4 Configure communications

Objective

Create a unique client communications endpoint for EVENTS_BACKUP.

Every ObjectServer requires its own name, host, and port. EVENTS_BACKUP must not reuse the communications port already assigned to EVENTS.

`omni.dat` is the Netcool/OMNIbus connections data file. It maps component names—such as **ObjectServers**, **Process Agents**, **gateways**, and **proxies** to the hostnames and ports where those components listen.

On UNIX and Linux, OMNIbus components do not normally read `omni.dat` directly. It is the source file used to generate the platform-specific interfaces file with nco_igen or the Server Editor (nco_xigen - the XWindows equivalent tool). After changing omni.dat, regenerate the interfaces file and restart any affected components so they use the updated communications information.

The `omni.dat` file is located under `$NCHOME/etc/omni.dat`, rather than `$OMNIHOME/etc` because it is a shared Netcool communications configuration, not an OMNIbus-component-specific configuration file.

1. Back up `omni.dat`:

```
cp $NCHOME/etc/omni.dat \
  $NCHOME/etc/omni.dat.before-events-backup
```

2. Edit the communications file:

```
vi $NCHOME/etc/omni.dat
```

3. Add an entry for EVENTS_BACKUP.

Use the hostname and an unused port. For example:

```
[EVENTS_BACKUP]
{
        Primary: <your-fqdn> 4101
}
```

4. Regenerate the UNIX interfaces file:

```
$NCHOME/bin/nco_igen
```

5. Confirm the new ObjectServer is present in the generated interfaces:

```
grep -A4 -B2 EVENTS_BACKUP $NCHOME/etc/interfaces.linux2x86
```

Validation

The new ObjectServer name should resolve in the communications configuration, even though the ObjectServer is not yet running:

```
grep -n EVENTS_BACKUP $NCHOME/etc/omni.dat
```

7.5 Add the clone to Process Agent

Objective

Allow Process Agent to start and monitor EVENTS_BACKUP independently of EVENTS.

The new ObjectServer is PA-aware. Its process definition must include `-pa NCO_PA` and use `ProcessType = PaPA_AWARE`.

1. Back up the Process Agent configuration:

   ```
   cp $OMNIHOME/etc/nco_pa.conf \
   $OMNIHOME/etc/nco_pa.conf.before-events-backup
   ```

2. Edit the configuration:

   ```
   vi $OMNIHOME/etc/nco_pa.conf
   ```

3. Add another process definition to the `nco_process` section for our backup object server.

   Replace **<YOUR-FQDN>** with the fully qualified name for your system:

   ```
   nco_process 'BackupObjectServer'
   {
       Command '$OMNIHOME/bin/nco_objserv -name EVENTS_BACKUP -pa NCO_PA' run as 0
       Host            =       '<YOUR-FQDN>'
       Managed         =       True
       RestartMsg      =       '${NAME} running as ${EUID} has been restored on ${HOST}.'
       AlertMsg        =       '${NAME} running as ${EUID} has died on ${HOST}.'
       RetryCount      =       0
       ProcessType     =       PaPA_AWARE
   }
   ```

4. Add an additional service definition:

   ```
   nco_service 'BackupObjectServerService'
   {
       ServiceType = Non-Master
       ServiceStart = Non-Auto
       process 'BackupObjectServer' NONE
   }
   ```

   For illustration, we set the `BackupObjectServerService` to `Non-Auto` so the training clone does not start automatically each time Process Agent starts. We must explicitly start it upon starting up `Process Control`.

Validation

Before restarting Process Agent, confirm:

- The existing `MasterObjectServer` definition still uses `-name EVENTS`.
- `BackupObjectServer` uses `-name EVENTS_BACKUP`.
- The clone has its own service.
- The clone is `PaPA_AWARE`.
- The service uses `ServiceStart = Non-Auto`.

7.6 Start and validate the clone

Objective

Start EVENTS_BACKUP, confirm that it is running under Process Agent, and validate its schema and clean alert table.

1. Shut down and restart the Process Agent so it reads the updated configuration:

   ```
   nco_pa_shutdown
   ```

   At the shutdown prompt, select option 2 to shut down Process Agent and stop all managed processes. Because this lab runs Process Agent with `-authenticate NONE`, press Enter at any login or password prompt.

   Then restart Process Agent:

   ```
   nco_pad -authenticate NONE
   ```

   Confirm that Process Agent has restarted before continuing. The `Core` service and the existing `MasterObjectServer` should be available again; the backup service remains DEAD because it is configured with `ServiceStart = Non-Auto`.

2. Start the clone service:

   ```
   nco_pa_start -service BackupObjectServerService
   ```

3. Check status:

   ```
   nco_pa_status
   ```

   Both of your ObjectServers should now be running:

   ```
   Core                        MasterObjectServer      RUNNING
   BackupObjectServerService   BackupObjectServer      RUNNING
   ```

4. Confirm that the new ObjectServer responds and connect to it with `nco_sql`:

   ```
   $OMNIHOME/bin/nco_ping EVENTS_BACKUP
   ```

   ```
   $OMNIHOME/bin/nco_sql \
   -server EVENTS_BACKUP \
   -user root \
   -password 'netcool'
   ```

5. Confirm that the expected `alerts.status` schema exists:

   ```
   DESCRIBE alerts.status;
   go
   ```

   Confirm that the custom fields added in the previous ObjectServer database lab, including `CustomerID` and `CircuitID`, exist in the cloned `alerts.status` table.

6. Confirm that the user created in the previous ObjectServer database lab also exists in the cloned ObjectServer:

   ```
   SELECT * FROM security.users WHERE UserName = 'jdoe';
   go
   ```

   Confirm that the query returns the `jdoe` user.

7. Exit `nco_sql`:

   ```
   quit
   ```

Validation

The lab is successful when:

- EVENTS remains running.
- EVENTS_BACKUP is independently running under Process Agent.

- `nco_ping EVENTS_BACKUP` succeeds.
- The clone contains the expected schema and custom fields.
- `alerts.status` contains no source historical events.

Lab summary

In this lab, you:

- Exported the EVENTS ObjectServer configuration with `nco_osreport -dbinit`.
- Selected configuration SQL files suitable for a clean training clone.
- Initialized EVENTS_BACKUP with `nco_dbinit`.
- Deliberately omitted `alertsdata.sql`.
- Added a unique EVENTS_BACKUP communications endpoint.
- Added the clone as an independently controlled PA-aware ObjectServer process.
- Started and validated the clone without affecting EVENTS.

The important operational model is:

```
Export source configuration
  → review and select content
  → initialize a separate ObjectServer
  → configure communications
  → add Process Agent control
  → start and validate independently
```

IBM documents `nco_osreport -dbinit` exports as input for `nco_dbinit` when cloning an ObjectServer, including the option to explicitly include or omit exported alert data. [web:118]

8. Configuring a Bidirectional ObjectServer Gateway

Objective

Configure a bidirectional ObjectServer Gateway between EVENTS and EVENTS_BACKUP, verify that required custom alert fields are mapped, place the gateway under Process Agent control, and validate alert replication in both directions.

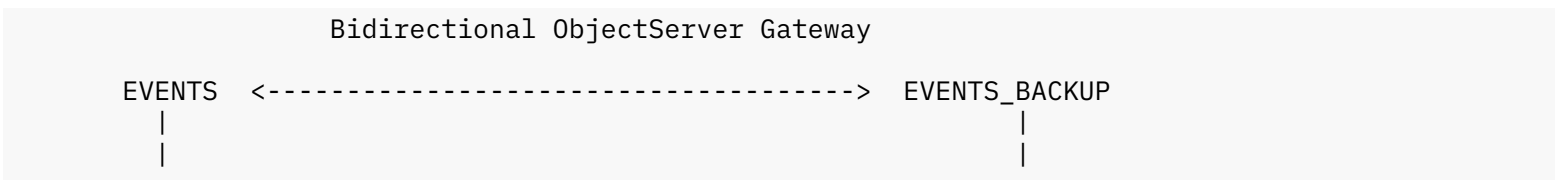
At the end of this lab, you will be able to:

- Explain the role of a bidirectional ObjectServer Gateway.
- Confirm that both ObjectServers have compatible schemas.
- Configure the gateway to connect EVENTS and EVENTS_BACKUP.
- Verify that custom `alerts.status` fields are mapped.
- Add the gateway to Process Agent as a separately managed service.
- Validate inserts and updates in both replication directions.

Scope: This lab uses EVENTS and EVENTS_BACKUP on the same host. It demonstrates ObjectServer replication and gateway process control. It does not provide host-level high availability. Probe peer-to-peer high availability is intentionally out of scope for this lab.

Scenario

The previous lab created EVENTS_BACKUP as a separate ObjectServer. In this lab, you will configure a bidirectional ObjectServer Gateway so alerts and selected updates can flow between the two ObjectServers.



```
   Primary training                                    Backup/training
   ObjectServer                                        ObjectServer
```

A bidirectional gateway synchronizes configured ObjectServer tables in both directions. The gateway uses a map definition file to map source fields to target fields. For `alerts.status`, any custom fields required by the lab must:

1. Exist in the source ObjectServer.
2. Exist in the destination ObjectServer.
3. Be compatible in type and length.
4. Be explicitly mapped by the gateway.

A gateway map cannot create a missing ObjectServer column. Resolve schema differences before starting the gateway.

How bidirectional replication works

The ObjectServer Gateway listens for changes in configured source tables, transforms those changes according to its map definition, and applies the corresponding changes to the destination ObjectServer. In bidirectional mode, the gateway is designed to replicate eligible changes in both directions while preventing the replicated update from being endlessly returned to its origin.

For this lab, the primary replication target is:

```
alerts.status
```

Depending on deployment requirements, the gateway may also map:

```
alerts.details
alerts.journal
```

Replicating configuration, security, or automation tables is outside the scope of this exercise.

Prerequisites

Before beginning, verify:

- EVENTS is running.
- EVENTS_BACKUP is running.
- Both ObjectServers are reachable through `omni.dat` and the generated interfaces file.
- The ObjectServer Gateway package is installed.
- You have authorized ObjectServer administrator credentials.
- The custom alert fields required by the lab are known.
- Process Agent NCO_PA is running on the lab host.

Confirm connectivity:

```
$OMNIHOME/bin/nco_ping EVENTS
$OMNIHOME/bin/nco_ping EVENTS_BACKUP
```

Confirm the gateway installation and identify the supplied templates and executable:

```
find $OMNIHOME/gates -type f \( -name 'nco_g_objserv*' -o -name '*objserv*.props' -o -name
 ↪  '*objserv*.map' \) -print
```

8.1 Verify schema compatibility

Objective

Confirm that EVENTS and EVENTS_BACKUP have compatible `alerts.status` schemas before configuring replication.

A bidirectional gateway can replicate a custom field only when the field exists on both ObjectServers and has a compatible definition. Compare field names, data types, and string lengths.

1. Export the EVENTS schema description to a raw output file:

```
printf 'DESCRIBE alerts.status;\ngo\n' \
  | $OMNIHOME/bin/nco_sql -server EVENTS -user root -password 'netcool' \
  > /tmp/EVENTS-schema-raw.txt
```

2. Export the EVENTS_BACKUP schema description to a raw output file:

```
printf 'DESCRIBE alerts.status;\ngo\n' \
  | $OMNIHOME/bin/nco_sql -server EVENTS_BACKUP -user root -password 'netcool' \
  > /tmp/EVENTS_BACKUP-schema-raw.txt
```

3. Identify the schema differences:

```
diff -u /tmp/EVENTS-schema-raw.txt /tmp/EVENTS_BACKUP-schema-raw.txt
```

If the `diff` command produces no output, the schema descriptions already match. If it produces output, review the differences in field names, data types, and string lengths, especially for the custom fields used by this lab.

Validation

Before continuing, confirm that every required custom field:

- Exists in both ObjectServers.
- Uses the same or compatible type.
- Uses the same or sufficient string length.

If a field is missing from EVENTS_BACKUP, add it through the approved schema-management procedure before continuing. Do not attempt to solve a schema mismatch only by editing the gateway map.

8.2 Create a gateway working area

Objective

Create a working directory for our bi-directional gateway, and copy the installed bi-directional gateway template map and configuration files.

1. Create the working directories:

```
mkdir -p $OMNIHOME/gates/events_backup
```

2. Copy the template properties and mapping files:

```
cp -R $OMNIHOME/gates/objserv_bi/* $OMNIHOME/gates/events_backup
```

3. Restrict access to the configuration directory because gateway properties can contain connection credentials:

```
chmod 750 $OMNIHOME/gates/events_backup
chmod 640 $OMNIHOME/gates/events_backup/*
```

Validation

You should have a dedicated gateway configuration area that is separate from the installed defaults and from any other gateway instances.

8.3 Configure the gateway properties

Objective

Configure a dedicated bidirectional gateway instance that connects EVENTS and EVENTS_BACKUP.

1. Open the copied properties file:

   ```
   vi $OMNIHOME/gates/events_backup/objserv_bi.props
   ```

   As we saw with the **probe** property files, in this property file you can see all of the default property settings, and note that they are commented out.

2. At the bottom of the properties file, configure the source and destination ObjectServer connection settings, as well as our custom mapping and startup command file locations:

   ```
   Name                                  : 'EVENTS_GATE'
   Gate.ObjectServerA.Server             : 'EVENTS'
   Gate.ObjectServerA.Username           : 'root'
   Gate.ObjectServerA.Password           : 'netcool'
   Gate.ObjectServerB.Server             : 'EVENTS_BACKUP'
   Gate.ObjectServerB.Username           : 'root'
   Gate.ObjectServerB.Password           : 'netcool'
   Gate.MapFile                          : '$OMNIHOME/gates/events_backup/objserv_bi.map'
   Gate.StartupCmdFile                   :
    ↪  '$OMNIHOME/gates/events_backup/objserv_bi.startup.cmd'
   ```

3. Save the properties file and exit the editor.

Validation

Before starting the gateway, verify that the properties identify both ObjectServers and reference the dedicated map file.

8.4 Map standard and custom alert fields

Objective

The gateway mapping file, also called a **map definition file**, tells the ObjectServer Gateway how to translate data between the ObjectServers it connects. It defines which source table columns are copied to which destination columns and can also apply transformations, lookups, default values, or conditional mapping logic where required. The gateway properties file identifies the map file through the Gate.MapFile setting; when the gateway processes an alert, it uses the map definitions to determine what data is replicated and how it is represented at the destination.

The `CREATE MAPPING StatusMap` section defines how fields from an ObjectServer alerts.status row are mapped to fields in the gateway's destination. Each mapping entry identifies a destination field on the left and the source ObjectServer alert field on the right.

For a direct field-to-field mapping, the general format is:

```
'DestinationField' = '@SourceField'
```

The @ prefix means "read this field from the current ObjectServer alert." Gateway map entries are comma-separated, and the final entry is followed by the closing);

In this section we will confirm that the map definition includes the required standard fields and all custom `alerts.status` fields used by the lab.

1. Open the copied map file:

   ```
   vi $OMNIHOME/gates/events_backup/objserv_bi.map
   ```

2. Locate the `CREATE MAPPING StatusMap` mapping section near the top of the file.

3. Confirm that the standard alert fields required for normal event synchronization are mapped. Typical examples include:

```
Identifier
Node
Summary
Severity
Class
Type
AlertGroup
Manager
Agent
FirstOccurrence
LastOccurrence
Tally
Acknowledged
OwnerUID
StateChange
```

4. Add mappings for our custom fields.

   At the end of the mapping section, you will see a mapping for the column `BSM_Identity`. Add a comma to the end of that line, then add another line and map the two custom fields as such:

```
        'BSM_Identity'    =   '@BSM_Identity',
        'CustomerID'      =   '@CustomerID',
        'CircuitID'       =   '@CircuitID

  );
```

   Do not end the `CircuitID` mapping with a comma, it will cause a parse error.

5. Save the map file and exit.

Validation

For every custom field required by the lab, verify:

- The field exists in EVENTS.
- The field exists in EVENTS_BACKUP.
- The field is present in the gateway's `alerts.status` map.
- The mapping preserves the field in both replication directions.

  A useful test is not merely that a replicated event appears on the other ObjectServer. You must create or update a custom field and confirm that its value also replicates.

8.5 Start the gateway manually and validate operation

Objective

Before we start the gateway, we must add an entry to the $NCHOME/etc/omni.dat connections data file so OMNIbus components can resolve the gateway's logical name to its host and listening port. The bidirectional gateway is itself a network service: each ObjectServer connects to the gateway by using its configured gateway name, rather than a hard-coded host and port. After adding the entry, run nco_igen or use the Server Editor to regenerate the platform-specific interfaces file that OMNIbus components actually read at runtime.

1. Back up `omni.dat`:

```
cp $NCHOME/etc/omni.dat \
  $NCHOME/etc/omni.dat.before-events-backup-gateway
```

2. Edit the communications file:

```
vi $NCHOME/etc/omni.dat
```

3. Add an entry for the gateway configured as EVENTS_GATE in Section 8.3. Use the actual fully qualified hostname and an unused port assigned for this lab. For example:

```
[EVENTS_GATE]
{
      Primary: <your-fqdn> 4102
}
```

Use an unused port and the actual fully qualified hostname for your lab environment. Do not reuse the port assigned to EVENTS or EVENTS_BACKUP.

4. Regenerate the UNIX interfaces file:

```
$NCHOME/bin/nco_igen
```

5. Confirm that the gateway is present in the generated interfaces:

```
grep -A4 -B2 EVENTS_GATE $NCHOME/etc/interfaces.linux2x86
```

6. Start the gateway with the command:

```
$OMNIHOME/bin/nco_g_objserv_bi -propsfile $OMNIHOME/gates/events_backup/objserv_bi.props &
```

7. Confirm that the gateway displays the expected startup output:

```
Netcool/OMNIbus Bi-Directional ObjectServer Gateway - Version 8.1.0 64-bit

(C) Copyright IBM Corp. 1994, 2012

Server 'EVENTS_GATE' initialised - entering RUN state.
```

8. We are going to test synchronization using the syslogd probe and the `logger` command. First, check whether the syslogd probe is running:

```
ps -ef | grep nco_p_syslogd | grep -v grep
```

If no `nco_p_syslogd` process appears, start it with the training rules file:

```
$OMNIHOME/probes/nco_p_syslogd \
  -rulesfile $OMNIHOME/probes/linux2x86/syslogd-training.rules &
```

9. To make the synchronization result unambiguous, remove the existing events from the EVENTS ObjectServer before sending the new test event. This leaves a clean starting point so the `LinkDown` event generated in the next task can be identified clearly in both ObjectServers.

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password 'netcool'
```

At the `nco_sql` prompt, run:

```
DELETE FROM alerts.status;
go
```

Exit `nco_sql` before continuing to the next task:

```
quit
```

This is a destructive command. For this lab, it provides a clean event list for validating the gateway replication flow.

10. Send the `LinkDown` event used in the earlier probe labs:

```
logger -n 127.0.0.1 'ALERT - LinkDown: Received a link down alert from routerA interface
 ↪  GigabitEthernet0/0 : unknown reason code'
```

11. Connect to EVENTS and confirm that the `LinkDown` event arrived:

```
$OMNIHOME/bin/nco_sql -server EVENTS -user root -password 'netcool'
```

At the nco_sql prompt, run:

```
SELECT Identifier, Node, Summary, Severity, AlertGroup, Manager
FROM alerts.status
WHERE Node = 'routerA';
go
```

Exit nco_sql before continuing to the next task:

```
quit
```

12. Connect to EVENTS_BACKUP and confirm that the same LinkDown event replicated:

```
$OMNIHOME/bin/nco_sql -server EVENTS_BACKUP -user root -password 'netcool'
```

At the nco_sql prompt, run the same query:

```
SELECT Identifier, Node, Summary, Severity, AlertGroup, Manager
FROM alerts.status
WHERE Node = 'routerA';
go
```

Confirm that the event appears in both ObjectServers with the expected mapped fields.

13. Stop the manually started gateway before continuing to Section 8.6. This prevents the Process Agent-managed gateway from starting while the manually launched gateway is still using the same configuration and communications endpoint:

```sh
pkill -f nco_g_objserv_bi
```

Confirm that the manually started gateway process has stopped:

```sh
ps -ef | grep nco_g_objserv_bi | grep -v grep
```

The process check should produce no gateway process before you add the gateway to Process Agent.

Validation

The exercise is successful when:

- The syslogd probe is running and accepts the logger event.
- The LinkDown event appears in EVENTS.
- The same event appears in EVENTS_BACKUP.
- The expected standard and custom field mappings are present in the replicated event.

8.6 Add the gateway to Process Agent

Objective

As we did earlier with the backup ObjectServer and the probes, we will now add the gateway to process control to ensure the process becomes part of the Netcool/OMNIbus installation ecosystem.

1. Back up the Process Agent configuration:

```
cp $OMNIHOME/etc/nco_pa.conf \
  $OMNIHOME/etc/nco_pa.conf.before-events-backup-gateway
```

2. Edit nco_pa.conf:

```
vi $OMNIHOME/etc/nco_pa.conf
```

3. Add a process definition for our gateway. Be sure to update the Host entry to your server FQDN:

```
nco_process 'EventsBackupBidirectionalGateway'
{
        Command '$OMNIHOME/bin/nco_g_objserv_bi -propsfile
         ↪  $OMNIHOME/gates/events_backup/objserv_bi.props' run as 0
        Host = '<YOUR-FQDN>'
        Managed = True
        RestartMsg = '${NAME} running as ${EUID} has been restored on ${HOST}.'
        AlertMsg = '${NAME} running as ${EUID} has died on ${HOST}.'
        RetryCount = 0
        ProcessType = PaNOT_PA_AWARE
}
```

4. Add the gateway to the existing BackupObjectServerService:

```
nco_service 'BackupObjectServerService'
{
        ServiceType = Non-Master
        ServiceStart = Non-Auto
        process 'BackupObjectServer' NONE
        process 'EventsBackupBidirectionalGateway' 'BackupObjectServer'
}
```

   The gateway depends on BackupObjectServer because it must connect to EVENTS_BACKUP when it starts. Declaring the dependency lets Process Agent start the backup ObjectServer before the gateway and manage the gateway in the same operational service.

5. Save the configuration.

6. Restart Process Agent so it reads the revised configuration:

```
nco_pa_shutdown
```

   At the shutdown prompt, select option 2 to shut down Process Agent and stop all managed processes. Because this lab uses -authenticate NONE, press Enter at any login or password prompt.

   Restart Process Agent:

```
nco_pad -authenticate NONE
```

   Confirm that Process Agent is available again:

```
nco_pa_status
```

   Expected output:

```
[root@ibm-training-server1 omnibus]# nco_pa_status

Login Password:

------------------------------------------------------------------------------------------
 ↪  ------------------------------------

Service Name               Process Name                    Hostname
 ↪  User        Status           PID

------------------------------------------------------------------------------------------
 ↪  ------------------------------------

Core                     MasterObjectServer              ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1141965
```

```
BackupObjectServerService  BackupObjectServer
 ↪  ibm-training-server1.dcimusic.com  root        DEAD               0

                         EventsBackupBidirectionalGateway  ibm-training-server1.dcimusic.com
 ↪  root        DEAD               0

Probes                    SyslogdProbe                     ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1142072

                         MttrapdProbe                     ibm-training-server1.dcimusic.com
 ↪  root        RUNNING       1142073

------------------------------------------------------------------------------------------
 ↪  ------------------------------------
```

7. Start the backup ObjectServer service, which now includes the gateway:

```
nco_pa_start -service BackupObjectServerService
```

8. Check Process Agent status:

```
nco_pa_status
```

Expected output:

```
[root@ibm-training-server1 omnibus]# nco_pa_status

Login Password:

------------------------------------------------------------------------------------------
 ↪  ------------------------------------

Service Name               Process Name                     Hostname
 ↪  User        Status           PID

------------------------------------------------------------------------------------------
 ↪  ------------------------------------

Core                      MasterObjectServer               ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1144846

BackupObjectServerService  BackupObjectServer
 ↪  ibm-training-server1.dcimusic.com  root        RUNNING      1145176

                         EventsBackupBidirectionalGateway  ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1145215

Probes                    SyslogdProbe                     ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1144966

                         MttrapdProbe                     ibm-training-server1.dcimusic.com
 ↪  root        RUNNING      1144848

------------------------------------------------------------------------------------------
 ↪  ------------------------------------
```

Run `nco_pa_status` a couple of times and compare the PIDs for `BackupObjectServer` and `EventsBackupBidirectionalGateway`. The PIDs should remain stable. If either PID changes, Process Agent is restarting that process, which indicates that the process is exiting and there is likely a configuration or runtime error. Inspect the Process Agent and gateway/ObjectServer logs before continuing.

Validation

The `BackupObjectServerService` should show both `BackupObjectServer` and `EventsBackupBidirectionalGateway` as `RUNNING`. If the gateway is `DEAD`, inspect the gateway logfile first, then inspect the Process Agent log.

8.7 Validate bidirectional replication

Objective

Prove that alert inserts and custom-field updates replicate from `EVENTS` to `EVENTS_BACKUP` and from `EVENTS_BACKUP` to `EVENTS`.

Use a unique identifier for each test.

Test A: EVENTS to EVENTS_BACKUP

1. Connect to `EVENTS`:

```
$OMNIHOME/bin/nco_sql \
  -server EVENTS \
  -user root \
  -password 'netcool'
```

2. Create a controlled test event.

```
INSERT INTO alerts.status
(
  Identifier,
  Node,
  Summary,
  Severity,
  AlertGroup,
  Manager,
  CustomerID
)
VALUES
(
  'LAB-GW-<initials>-A',
  'gateway-lab',
  'EVENTS to EVENTS_BACKUP replication test',
  1,
  'GatewayLab',
  'LabGateway',
  'custom value from EVENTS'
);
go
```

3. Connect to `EVENTS_BACKUP` and verify the event and custom field:

```
SELECT Identifier, Node, Summary, Severity, AlertGroup, Manager, CustomerID
FROM alerts.status
WHERE Identifier = 'LAB-GW-<initials>-A';
go
```

4. On `EVENTS`, update the custom field:

```
UPDATE alerts.status
SET CustomerID = 'updated value from EVENTS'
WHERE Identifier = 'LAB-GW-<initials>-A';
go
```

5. On EVENTS_BACKUP, verify the updated custom field value.

Test B: EVENTS_BACKUP to EVENTS

1. Connect to EVENTS_BACKUP.
2. Create a second controlled test event:

```
INSERT INTO alerts.status
(
  Identifier,
  Node,
  Summary,
  Severity,
  AlertGroup,
  Manager,
  CustomerID
)
VALUES
(
  'LAB-GW-<initials>-B',
  'gateway-lab',
  'EVENTS_BACKUP to EVENTS replication test',
  1,
  'GatewayLab',
  'LabGateway',
  'custom value from EVENTS_BACKUP'
);
go
```

3. Connect to EVENTS and verify the event and custom field:

```
SELECT Identifier, Node, Summary, Severity, AlertGroup, Manager, CustomerID
FROM alerts.status
WHERE Identifier = 'LAB-GW-<initials>-B';
go
```

4. On EVENTS_BACKUP, update CustomerID and confirm the updated value appears on EVENTS.

Validation

The lab is successful when:

- An event inserted in EVENTS appears in EVENTS_BACKUP.
- An event inserted in EVENTS_BACKUP appears in EVENTS.
- Updates to the selected custom field replicate in both directions.
- The gateway process remains RUNNING in nco_pa_status.
- The gateway log contains no repeated mapping, connection, or replication errors.

8.8 Clean up test events

Objective

Remove the controlled test events after replication has been validated.

Delete the test events from either ObjectServer, then verify that the deletion replicates to the other ObjectServer.

For example:

```
DELETE FROM alerts.status
WHERE Identifier IN ('LAB-GW-<initials>-A', 'LAB-GW-<initials>-B');
go
```

Confirm that neither test identifier remains on either ObjectServer.

Validation

The test alerts are removed from both EVENTS and EVENTS_BACKUP.

Lab summary

In this lab, you:

- Verified that EVENTS and EVENTS_BACKUP have compatible alert schemas.
- Created a dedicated configuration area for a bidirectional gateway.
- Configured gateway properties to connect both ObjectServers.
- Verified mappings for standard and custom `alerts.status` fields.
- Tested the gateway manually and reviewed its log.
- Added the gateway to Process Agent as a non-PA-aware managed process.
- Validated event creation and custom-field updates in both replication directions.
- Removed the controlled test alerts.

The key operational principle is:

```
Compatible schema
  + explicit field mapping
  + controlled gateway configuration
  + bidirectional validation
  = reliable ObjectServer synchronization
```

A gateway is not a substitute for schema management. Keep both ObjectServers structurally compatible, then use the map file to determine which values are synchronized and how they are transformed.

OMNIbus Lab — Completion Summary

Congratulations on completing the Netcool/OMNIbus lab guide. Across eight sections you have built a fully working Netcool/OMNIbus event management environment from scratch — from establishing the lab foundation on day one to connecting two ObjectServers with a bidirectional gateway on the last. This page summarises what you covered in each section and the skills you have developed along the way.

Section 1 — OMNIbus Fundamentals

You learned what IBM Tivoli Netcool/OMNIbus is and how it fits into an IT operations environment. You explored the core pipeline — **source → probe → ObjectServer → automation → operator** — and studied the key components: probes, the ObjectServer, gateways, the Event List, and the Web GUI. You established the terminology — events, alerts, deduplication, correlation, triggers, rules files, and journals — that carries through the rest of the guide.

Section 2 — ObjectServer and Event Model

You connected to the lab server via SSH, set the required OMNIbus environment variables, and started the ObjectServer and JazzSM Web GUI. You configured the Windows Servers Editor, validated connectivity from the native Event List client and Netcool/OMNIbus Administrator, and logged in to the browser-based Web GUI. You then worked with events as an operator — prioritizing severity,

acknowledging alerts, adding journal entries, escalating events, and switching between filters and views. You started the Simnet probe and observed deduplication in real time through the `FirstOccurrence`, `LastOccurrence`, and `Count` fields.

Section 3 — The OMNIbus ObjectServer Database

You used Netcool/OMNIbus Administrator to create the `netcool` user, assign ObjectServer group membership, and assign Web GUI roles — understanding the two-layer security model that separates database access from UI feature access. You used `nco_sql` to create a second user, `jdoe`, and assign group membership via SQL. You inspected the `alerts.status` schema, added the custom columns `CustomerID` and `CircuitID`, and examined the three default ObjectServer triggers — `deduplication`, `generic_clear`, and `expire` — that govern event lifecycle automatically. Finally, you enabled the ObjectServer REST API by modifying properties, restarting the ObjectServer, and validating with `curl`.

Section 4 — Probes and Rules Development

You configured and started the `nco_p_syslogd` probe, inspected its properties and rules files, and observed what an unmapped event looks like before rules processing. You then built a complete probe rules file from scratch across eleven progressive labs: the ProbeWatch stanza, basic field mapping, `extract()` with regular expressions for `@EventId`, `switch/case` branching, `@Identifier` design for deduplication, Type 1 / Type 2 correlation for `LinkDown` / `LinkUp` auto-clearing, conditional severity logic with `if/else if`, lookup table enrichment populating `@CustomerID` and `@Service`, event discarding with `discard`, `@ExpireTime` for automatic alert expiry, and selective raw capture control with `%RawCapture`.

Section 5 — SNMP Trap Processing with MTTrapd and NcKL

You configured the `nco_p_mttrapd` probe with the NcKL rules framework, ran the NcKL schema setup script, and set the `NC_RULES_HOME` environment variable. You tested the pipeline end-to-end with standard IF-MIB `linkDown` / `linkUp` traps and inspected the rawcapture token structure. You then built and loaded the **IBM-TRAINING-MIB** — defining three custom trap types (`serverDown`, `serverUp`, `diskUtilization`) — using MIB Manager on Windows, exported it as NcKL-compatible rules files, transferred and integrated them into the NcKL include structure, and customized the user include file to fix `@Summary`, `@Identifier`, `@AlertKey`, and `@AlertGroup`. You tuned `@Severity`, `@Type`, and `@ExpireTime` in the severity lookup file and validated all four trap types.

Section 6 — Managing OMNIbus Processes with Process Agent

You extended `nco_pa.conf` to define the syslogd and MTTrapd probes as managed `PaNOT_PA_AWARE` processes, grouped them in a new `Probes` service with `ServiceStart = Auto`, and restarted `nco_pad` to validate automatic startup. You used `nco_pa_stop` and `nco_pa_start` to control an individual probe without affecting the ObjectServer, then stopped and started the entire `Probes` service as a single operational unit. You reviewed the completed configuration and understood how process definitions, service definitions, and the routing table relate to the observed runtime state reported by `nco_pa_status`.

Section 7 — Cloning an ObjectServer with nco_osreport and nco_dbinit

You exported the EVENTS ObjectServer configuration with `nco_osreport -dbinit`, reviewed the six generated SQL files, and deliberately omitted `alertsdata.sql` to create a clean configuration clone. You initialized `EVENTS_BACKUP` with `nco_dbinit`, added a unique communications endpoint to `omni.dat` on port 4101, and regenerated the interfaces file with `nco_igen`. You added `BackupObjectServer` as a `PaPA_AWARE` process in a `Non-Auto` service, restarted Process Agent, and started the clone on demand. You validated that the clone carries the full schema — including `CustomerID`, `CircuitID`, and the `jdoe` user — but contains zero historical alert rows.

Section 8 — Configuring a Bidirectional ObjectServer Gateway

You confirmed schema compatibility between EVENTS and EVENTS_BACKUP, created a dedicated gateway configuration directory from the `objserv_bi` template, and configured the gateway properties to connect both ObjectServers as EVENTS_GATE. You added explicit mappings for the `CustomerID` and `CircuitID` custom fields to the map file, registered the gateway in `omni.dat`, and started it manually to validate one-direction replication before adding it to Process Agent. You placed the gateway inside the `BackupObjectServerService` with a `BackupObjectServer` dependency, then ran bidirectional validation tests — inserting from each ObjectServer, updating the `CustomerID` field, and confirming that inserts, updates, and deletes all propagated in both directions.

What you built

Over the course of the lab you assembled a complete, layered Netcool/OMNIbus environment running on a single lab server:

Component	Purpose
EVENTS ObjectServer	Primary in-memory alert database; managed by Process Agent in the `Core` service
EVENTS_BACKUP ObjectServer	Schema-identical clone; started on demand via the `BackupObjectServerService`
EVENTS_GATE (bidirectional gateway)	Replicates inserts, updates, and deletes between both ObjectServers in both directions
`nco_p_syslogd` + `syslogd-training.rules`	Custom syslogd probe with full rules pipeline: extraction, switch/case, deduplication, correlation, lookup enrichment, discard, expiry, and selective raw capture
`nco_p_mttrapd` + NcKL + IBM-TRAINING-MIB	MTTrapd probe using the NcKL framework with a custom MIB, user include customizations, and severity lookup tuning
`Probes` service (Process Agent)	Both probes managed together as an auto-started, independently controllable service group
Web GUI (JazzSM)	Browser-based event management with custom `SyslogdProbe` filter, `SyslogdView` view, and `SNMP Traps` filter
`alerts.status` custom schema	`CustomerID` and `CircuitID` columns added to the default schema and replicated to the clone

---

Key files created

File	Purpose
`$OMNIHOME/probes/linux2x86/syslogd-training.rules`	Complete syslogd probe rules file with all lab concepts applied
`$OMNIHOME/probes/linux2x86/CUSTOMERS.lookup`	Lookup table mapping router names to customer names and service tiers
`$NC_RULES_HOME/include-snmptrap/ibmTraining/`	NcKL rules, severity lookup, and user include files for the IBM-TRAINING-MIB
`$NCHOME/etc/omni.dat`	Communications endpoints for EVENTS (4100), EVENTS_BACKUP (4101), EVENTS_GATE (4102)
`$OMNIHOME/etc/nco_pa.conf`	Process Agent configuration managing all five processes across three services
`$OMNIHOME/gates/events_backup/objserv_bi.props`	Gateway instance properties: names, credentials, map file and startup command paths
`$OMNIHOME/gates/events_backup/objserv_bi.map`	Gateway field mapping including standard alert fields and custom `CustomerID` / `CircuitID` columns

---

Where to go next — Netcool/Impact

The OMNIbus environment you have built is the foundation for the Netcool/Impact training that follows. Impact connects to the EVENTS ObjectServer you configured here, queries the same `alerts.status` fields you know intimately, and uses the same `CustomerID` and `CircuitID` columns you added. The patterns you practised here translate directly.

- The **ObjectServer DSA** in Impact connects to EVENTS — the same server you have been managing throughout this lab.
- Impact policies read and write the same `alerts.status` fields you mapped in probe rules — `Node`, `AlertGroup`, `AlertKey`, `Severity`, `Identifier`, and your custom columns.
- The **deduplication** and **generic_clear** triggers you studied in Section 3 interact directly with Impact-written events — understanding them makes Impact automation easier to reason about.
- The **NcKL-enriched SNMP traps** from Section 5 are the kind of real-world events Impact is built to act on — escalate, enrich, correlate, and dispatch.
- Process Agent from Section 6 can also manage the Impact server process in production environments, using the same `nco_pa.conf` model you configured here.